FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Multi-Agent Reinforcement Learning Control of Distributed Mixed Cooperative–Competitive Real-World Cyber-Physical Systems

Case Studies in Smart Energy Management

Tiago Fonseca

WORKING VERSION



Programa Doutoral em Engenharia Informática

Supervisor: Prof. Luís Miguel Moreira Lino Ferreira

Co-Supervisor: Prof. Armando Sousa

December 2026

"For the things we have to learn before we can do them, we learn by doing them."

Aristotle

Nicomachean Ethics

© Tiago Fonseca, 2026

Declaration of Honour and Originality

I hereby declare that this thesis is the result of my own research and that it has been prepared in accordance with the principles of academic integrity of the University of Porto. All ideas, text, data, software, figures, and results originating from other authors or organisations are identified and acknowledged through appropriate references or authorship statements.

Where the thesis incorporates work published or prepared with co-authors, my contribution and the contribution of the other authors are described in the statement on co-authored work and, where appropriate, in the corresponding chapters. No part of this thesis has been submitted for another degree, except if explicitly declared.

I accept full responsibility for the accuracy of the claims, the interpretation of the results, and the final content of the thesis.

Porto, December 2026

Tiago Fonseca

Declaration on the Use of Generative Artificial Intelligence

Generative Artificial Intelligence was used as a supporting tool during the preparation of this thesis, including for research organisation, drafting, language revision, and software-related assistance.

All material included in the thesis was critically reviewed, verified, rewritten where necessary, and integrated by the author. The author remains fully responsible for the research design, source selection, software, experiments, data analysis, interpretation of results, scientific arguments, and final text.

Statement on Co-Authored Work

This thesis includes research developed in collaboration with supervisors, research colleagues, students, open-source communities, and industrial partners. Some methods, software components, datasets, experiments, and research outputs presented in the thesis are therefore associated with co-authored publications or shared research artefacts.

Where collaborative work contributes directly to a method, scientific claim, experiment, software component, dataset, or demonstrator presented in the thesis, the corresponding contributions and provenance are identified in the relevant chapter or section.

During the doctoral programme, I also supervised, co-supervised, and mentored a number of student theses and research projects developed alongside and, in some cases, contributing directly to this research. This work involved defining research problems, planning and reviewing technical work, coordinating parallel activities, and developing structured methods and pipelines for software development, experimentation, validation, documentation, and research follow-up. It provided substantial experience in leading and supervising research teams and in guiding work from an initial problem to a technically and scientifically defensible outcome. Where results or artefacts originating from these activities are used directly in the thesis, the corresponding student contribution and my role in their supervision, development, validation, and integration are stated in the relevant chapter.

Acknowledgements

This work was supported by Fundação para a Ciência e a Tecnologia, I.P. (FCT), under the supervision and superintendence of the Ministry of Education, Science and Innovation, through doctoral grant 2024.00855.BD, financed by the Portuguese State Budget and co-financed by the European Social Fund Plus (FSE+) through the Programa Demografia, Qualificações e Inclusão (PDQI/PESSOAS 2030). Part of the research presented in this thesis was developed within the OPEVA project, which received funding within the Key Digital Technologies Joint Undertaking (KDT JU) from the European Union's Horizon Europe Programme and the National Authorities of France, Belgium, Czechia, Italy, Portugal, Turkey, and Switzerland, under grant agreement 101097267. This work was also supported by the DEMFLEX operation (COMPETE2030-FEDER-01657200 – 19215), supported by the Innovation and Digital Transition Programme (COMPETE 2030), under Portugal 2030, and co-financed by the European Union. The views and opinions expressed in this thesis are those of the author alone and do not necessarily reflect the views of FCT, the European Union, KDT JU, or the respective funding and granting authorities. Neither these institutions nor the granting authorities can be held responsible for them.

I would first like to express my sincere gratitude to my supervisor, Professor Luís Miguel Moreira Lino Ferreira, and to my co-supervisor, Professor Armando Sousa. Their guidance, availability, trust, and willingness to discuss the scientific and practical challenges encountered throughout this work were invaluable. They gave me not only knowledge and advice, but also the freedom to develop my own ideas, question assumptions, make mistakes, and learn from them. Their support throughout these years played an important role in my development as an independent researcher, engineer, and professional.

I would also like to acknowledge the industrial partners who contributed to the research developed during this thesis. I am particularly grateful to CleanWatts, especially Eng. Jorge Landeck, and to i-Charging, especially Eng. Paulo Rodrigues. Our collaboration brought an essential real-world perspective to the work, particularly through the research conducted within OPEVA. Their technical knowledge, operational experience, infrastructure, and understanding of the practical constraints of energy systems and electric-vehicle charging helped ensure that many of the problems investigated in this thesis were confronted with operational reality rather than considered solely as academic abstractions. I am equally grateful to the many other people from these organisations, project partners, and collaborating institutions with whom I had the opportunity to interact and who, directly or indirectly, contributed knowledge, discussions, data, infrastructure, feedback, or assistance to this work.

An especially important part of my doctoral experience was the opportunity to supervise, co-supervise, and work closely with students developing their own theses and research projects. I am grateful to [names to be added], and to all the students I had the opportunity to accompany during these years. Their projects often developed in parallel with my own research and created opportunities to investigate related problems from different perspectives. Supervising this work taught me a great deal about leading research, defining achievable research questions, organising

technical work, establishing development and validation pipelines, reviewing results critically, and helping a team move from an initial idea to a defensible outcome. It also taught me when to provide direction and when to allow someone else to find a different solution. Learning to guide and coordinate the work of others became one of the most valuable parts of my doctoral training.

I also want to thank the friends and colleagues at FEUP who shared their own doctoral journeys with me. A special thanks goes to João Vitorino and Tiago Dias for the conversations, discussions, frustrations, laughs, and countless moments shared throughout these years. Going through a PhD alongside people facing many of the same uncertainties and challenges made the difficult periods easier and the good ones considerably better. I am also grateful to the colleagues, researchers, project partners, and friends with whom I exchanged ideas, code, tools, reviews, experiments, and the many small forms of help that, accumulated over several years, became part of this work.

I have deliberately left my most personal thanks for last. I am deeply grateful to my parents, Vera and Abílio, and to my grandparents. My parents gave me the stability, education, encouragement, and opportunities that made this path possible long before the PhD began. They supported my choices even when those choices demanded time, patience, and sacrifices from them, and they taught me the value of work, responsibility, and perseverance more through their example than through words. I am equally grateful to my grandparents for their affection, presence, and for the values and sense of family that have accompanied me throughout my life. Reaching this point is also the result of the foundation they all helped build for me over many years.

My final and deepest thanks go to my girlfriend, Inês. She experienced this PhD more closely than anyone else, not only through its achievements and good moments, but through the pressure, uncertainty, frustration, exhaustion, and constant presence that such a long process gradually brings into everyday life. There were periods when the work occupied far more of me than it should have, and when I became more impatient, distracted, withdrawn, grumpy, or simply more difficult to be around than I would have liked. Too often, the problems of the PhD followed me home, and she had to live with a version of me shaped by difficulties that were not hers.

Through all of it, she stayed beside me. She listened when I needed to talk, gave me perspective when I had lost it, celebrated progress that sometimes seemed insignificant to me, and supported me through periods when everything appeared to be moving too slowly or not at all. She knew when to encourage me, when to challenge me, and when simply to be there. When I doubted myself or the value of continuing, her belief was often steadier than my own, and giving up was never allowed to become the easy answer. More importantly, she continued to love and support me even when the pressure of this work made that support harder to give than I would like to admit.

A finished thesis inevitably presents research as something ordered: problems lead to methods, methods lead to experiments, and experiments lead to conclusions. Living through it is very different. Much of this journey was uncertain, repetitive, frustrating, and far less elegant than the document that eventually emerged from it. Inês was beside me throughout that real version of the PhD. Her patience, strength, affection, and belief in me made an enormous difference not only to completing this thesis, but to the person I remained while doing it.

Nes, Pai, Mae, Aida, Vo Ek e Vo Fonso.

Abstract

Cyber-physical systems increasingly rely on distributed autonomous control while operating under heterogeneous capabilities, changing configurations, incomplete information, communication limitations, and physical constraints. Multi-Agent Reinforcement Learning (MARL) provides a promising basis for such control, but good performance in simulation does not by itself establish that a learned policy can be used reliably in an operational system. This thesis addresses that transition through four connected requirements: scalability, deployability, safety, and robustness. Renewable Energy Communities and electric-vehicle charging provide the principal empirical domain, combining distributed participants, heterogeneous flexible assets, changing availability, local and collective objectives, and shared physical infrastructure.

The research develops EnergAIze as an integrated architecture for the development and operation of intelligent energy control. It connects data and scenario construction, an extended CityLearn-derived simulation environment, controller development, reproducible experimentation, policy governance, distributed deployment, monitoring, and recovery. Within this architecture, a progression of rule-based, optimisation-based, reinforcement-learning, and multi-agent methods establishes increasingly capable control solutions and provides empirical evidence about the assumptions under which each remains effective.

These findings motivate TI-MARL, the principal algorithmic contribution of the thesis. TI-MARL addresses multi-agent decision problems in which the observation–action interface of an agent can change during operation. A versioned capability schema and runtime health information are compiled into an entity-bound typed interface containing the observation parts, action-group instances, parameterised action ports, dependencies, validity conditions, and bounds currently available to the agent. A decentralised policy operates directly over this variable interface, constructs a shared local representation, and jointly produces a coherent action bundle for the controllable entities that are present. Local hard constraints are incorporated into action construction, while coordination between agents is learned through authorised shared observations, coupled objectives, the joint physical transition, and centralised variable-population critics during training.

The resulting methods and infrastructure are evaluated through simulator verification, common controller comparisons, dynamic reconfiguration, health and disturbance tests, ablations, multi-dimensional scalability experiments, lifecycle and compatibility tests, edge execution, and integrated energy scenarios. Selected components are further assessed using real data, external energy platforms, charging infrastructure, edge services, and physical demonstrators, with the evidence level distinguished explicitly from simulation to direct operational control. Taken together, the thesis shows how algorithmic, representational, experimental, and lifecycle mechanisms can be combined to preserve meaningful and physically valid distributed control as agents, interfaces, operating conditions, and deployed policies change.

Keywords: Multi-Agent Reinforcement Learning; Cyber-Physical Systems; Renewable Energy Communities; TI-MARL; Scalability; Safety and Robustness; Policy Lifecycle.

Resumo

Os sistemas ciberfísicos dependem cada vez mais de controlo autónomo distribuído, apesar de operarem com capacidades heterogéneas, configurações variáveis, informação incompleta, limitações de comunicação e restrições físicas. A Aprendizagem por Reforço Multiagente (MARL) oferece uma abordagem promissora para este contexto, mas um bom desempenho em simulação não demonstra, por si só, que uma política aprendida possa ser utilizada de forma fiável num sistema operacional. Esta tese aborda essa transição através de quatro requisitos relacionados: escalabilidade, capacidade de implantação, segurança operacional e robustez. As Comunidades de Energia Renovável e o carregamento de veículos elétricos constituem o principal domínio empírico, reunindo participantes distribuídos, ativos flexíveis heterogéneos, disponibilidade variável, objetivos locais e coletivos e infraestrutura física partilhada.

A investigação desenvolve o EnergAIze como uma arquitetura integrada para o desenvolvimento e operação de controlo inteligente de energia. Esta arquitetura liga dados e cenários, um simulador alargado derivado do CityLearn, desenvolvimento de controladores, experimentação reprodutível, governação de políticas, implantação distribuída, monitorização e recuperação. Neste contexto, uma progressão de métodos baseados em regras, otimização, aprendizagem por reforço e aprendizagem multiagente estabelece soluções progressivamente mais capazes e produz evidência sobre as condições em que cada uma se mantém eficaz.

Estes resultados motivam o TI-MARL, a principal contribuição algorítmica da tese. O TI-MARL considera problemas multiagente nos quais a interface de observação e ação pode mudar durante a operação. Um schema versionado de capacidades e informação de saúde em runtime são compilados numa interface tipada associada às entidades, contendo as observações, grupos de ação, portas de ação parametrizadas, dependências, condições de validade e limites atualmente disponíveis. Uma política descentralizada atua diretamente sobre esta interface variável, constrói uma representação local comum e produz um bundle coerente de ações para as entidades controláveis presentes. As restrições locais não negociáveis são incorporadas na construção das ações, enquanto a coordenação entre agentes é aprendida através de observações partilhadas autorizadas, objetivos acoplados, da transição física conjunta e de críticos centralizados capazes de representar populações variáveis durante o treino.

Os métodos e a infraestrutura são avaliados por verificação do simulador, comparação de controladores, reconfiguração dinâmica, perturbações e falhas, ablações, escalabilidade, compatibilidade, execução no edge e cenários energéticos integrados. Componentes selecionados são ainda avaliados com dados reais, plataformas externas, infraestrutura de carregamento e demonstradores físicos, distinguindo os níveis de evidência entre simulação e controlo operacional direto. No conjunto, a tese mostra como mecanismos algorítmicos, de representação, experimentação e ciclo de vida podem preservar controlo distribuído e fisicamente válido perante mudanças nos agentes, interfaces, condições e políticas implantadas.

Palavras-chave: Aprendizagem por Reforço Multiagente; Sistemas Ciberfísicos; Comunidades

de Energia Renovável; TI-MARL; Escalabilidade; Segurança e Robustez; Ciclo de Vida de Políticas.

Publications and Research Outputs Arising from the Thesis

The table maps the publications and research outputs associated with the thesis to their principal chapters. Publication status and the relationship between each output and the corresponding thesis contribution follow the available publication and project records.

ID	Publication or output	Status	Chapter
P1	*Flexible Loads Scheduling Algorithms for Renewable Energy Communities*	Published, 2022	2, 5
P2	*EnergAIze: Multi Agent Deep Deterministic Policy Gradient for Vehicle to Grid Energy Management*	Preprint, 2024	3, 5, 8
P3	*EVLearn: Extending the CityLearn Framework with Electric Vehicle Simulation*	Published, 2025	4
P4	*Control of Renewable Energy Communities using AI and Real-World Data*	Preprint, 2025	5, 8
P5	*TI-MARL: Typed-Interface Multi-Agent Reinforcement Learning for Reconfigurable Cyber-Physical Systems*	Manuscript in preparation	6
P6	Progressive control, MATD3, hierarchy, data-efficient learning, and TI-MARL structural and reconfiguration studies	Experiment and manuscript family in preparation	5–6
P7	EnergAIze experiment and MARL policy-set lifecycle orchestration	System contribution in preparation	7
P8	*CityLearn v3 / SoftCPS REC Simulator*: public simulator, scenario, documentation, benchmark, and release record associated with the thesis	Public software/output record	3–8

The publications are outputs of the research rather than substitutes for the thesis contributions. The dissertation harmonises terminology and experimental assumptions across the outputs, incorporates extended and negative results where relevant, and provides the cross-cutting synthesis required to answer the research questions.

Contents

Declaration of Honour and Originality i

Declaration on the Use of Generative Artificial Intelligence iii

Statement on Co-Authored Work v

Acknowledgements vii

Abstract ix

Resumo xi

Publications and Research Outputs Arising from the Thesis xiii

List of Acronyms xxvii

Glossary xxxi

1 Introduction 1
- 1.1 Context and Motivation . . . 1
  - 1.1.1 Distributed and Dynamically Reconfigurable Cyber-Physical Systems . . 1
  - 1.1.2 Renewable Energy Communities as Representative Cyber-Physical Systems 2
  - 1.1.3 MARL for Mixed Cooperative–Competitive Decentralised Control 3
- 1.2 From Simulated MARL to Operational Real-World CPS . . . 5
- 1.3 Thesis Proposition, Research Questions, and Scope . . . 6
  - 1.3.1 Thesis Proposition . . . 6
  - 1.3.2 Research Questions . . . 7
  - 1.3.3 Research Objectives . . . 8
  - 1.3.4 Scope and Assumptions . . . 9
- 1.4 Research Contributions and Outcomes . . . 9
  - 1.4.1 Scientific Contributions . . . 9
  - 1.4.2 System, Software, and Open-Source Contributions . . . 11
  - 1.4.3 Societal and Environmental Relevance . . . 11
- 1.5 Research Path and Thesis Organisation . . . 12

2 Real-World Multi-Agent Control in Cyber-Physical Systems: Foundations, State of the Art, and Research Gaps 13
- 2.1 Literature Scope and Evidence Approach . . . 13
  - 2.1.1 Scope and Sources . . . 14

2.1.2 Evidence Synthesis and Limitations . . . 14
2.2 Distributed, Heterogeneous, and Dynamically Reconfigurable Cyber-Physical Systems . . . 15
2.2.1 Distributed Control and Operational Constraints . . . 15
2.2.2 Heterogeneous Agents, Assets, and Interfaces . . . 16
2.2.3 Dynamic Populations, Capabilities, and Reconfiguration . . . 18
2.3 Renewable Energy Communities as Representative CPSs . . . 20
2.3.1 Participants, Assets, and Heterogeneity . . . 21
2.3.2 Flexibility, Temporal Coupling, and Dynamic Availability . . . 23
2.3.3 Individual and Community Objectives and Physical Constraints . . . 25
2.4 Control Approaches for Distributed Energy Systems and Related CPS . . . 27
2.4.1 Explicit Control: Rules, Optimisation, and Predictive Control . . . 29
2.4.2 Reinforcement Learning and Multi-Agent Control . . . 30
2.4.3 Structured, Federated, Transfer, Hybrid, and Hierarchical Approaches . . . 31
2.4.4 When MARL Adds Value across CPS Domains . . . 32
2.5 Foundations of Reinforcement Learning and MARL Used in This Thesis . . . 34
2.5.1 Sequential Decision-Making and Partial Observability . . . 34
2.5.2 Reinforcement Learning and Actor–Critic Methods . . . 37
2.5.3 Markov Games, MARL, and CTDE . . . 38
2.5.4 MARL Challenges Relevant to This Thesis . . . 40
2.6 From Simulation to Operational Evidence . . . 42
2.7 Critical Analysis of the State of the Art . . . 42
2.7.1 RQ1: Dynamic and Heterogeneous Multi-Agent Control . . . 43
2.7.2 RQ2: Multi-Dimensional Scalability of Learning and Coordination . . . 46
2.7.3 RQ3: Safe and Robust Operation . . . 48
2.7.4 RQ4: Governed and Deployable MARL Lifecycle . . . 49
2.8 Adjacent Challenges and Scope Boundaries . . . 51
2.8.1 Supporting Requirements and Evaluation Conditions . . . 51
2.8.2 Important Challenges Beyond the Central Scope . . . 52
2.9 Synthesis of Residual Gaps and Thesis Requirements . . . 53
2.10 Chapter Summary . . . 56

3 EnergAIze: An Integrated Framework for Intelligent Energy Management 57
3.1 The EnergAIze Framework as a Whole . . . 58
3.1.1 From Heterogeneous Measurements to a Physical Decision . . . 58
3.1.2 A Distributed Framework for Heterogeneous Buildings . . . 59
3.2 Percepta and the Data Provider . . . 60
3.2.1 Historical-Data and Runtime Paths . . . 60
3.3 Prediction, Vehicle State, and User Flexibility . . . 61
3.3.1 Consumption and Photovoltaic-Production Prediction . . . 61
3.3.2 PulseCharge and Flexibility Prediction . . . 62
3.4 Algorithms and the Flexibility Scheduling Decision . . . 62
3.4.1 The Fail-Safe Supervisor and Executed Action . . . 63
3.5 CityLearn, EVLearn, and the Development of Controllers . . . 63
3.6 Deployment Across Buildings and the Community . . . 64
3.7 Component Responsibilities and Thesis Map . . . 64

4 CityLearn v3: A Simulation and Evaluation Environment for Renewable Energy Communities 67
4.1 From Supporting Tool to Research Contribution . . . 68
4.1.1 From an Iterative Research Instrument to Public Research Software . . . 68
4.1.2 CityLearn v3 Across the Controller Lifecycle . . . 69
4.2 Problem Definition, Requirements, and Positioning . . . 71
4.2.1 From the Inherited Environment to the Required Experimental Contract . 71
4.2.2 Lineage and position within existing environments . . . 72
4.3 CityLearn v3 Architecture and REC Modelling Capabilities . . . 74
4.3.1 System Architecture and the Scenario-to-Evidence Path . . . 74
4.3.2 Scenario, Time, Unit, and Data Contracts . . . 76
4.3.3 Stationary storage and electrical-service limits . . . 78
4.3.4 Electric vehicles and chargers . . . 79
4.3.5 Deferrable appliances . . . 79
4.3.6 Local energy sharing and settlement . . . 80
4.3.7 Changing community members and assets . . . 81
4.3.8 Controlled data and equipment failures . . . 82
4.3.9 Demand-response services . . . 83
4.3.10 Multiple communities . . . 84
4.3.11 Controller Interfaces and Applied-Action Traceability . . . 84
4.4 Datasets Developed to Evaluate REC Controllers . . . 85
4.4.1 Common temporal, provenance, and accounting contract . . . 85
4.4.2 Composition and the separation of infrastructure from mobility . . . 87
4.4.3 Experimental ladder and matched scenario variants . . . 89
4.4.4 Validation protocol and annual baseline execution . . . 91
4.5 Evaluation Protocol and Controller Scorecard . . . 91
4.5.1 Reference Controllers and Fair Comparisons . . . 92
4.5.2 Core Scorecard and Result Exports . . . 92
4.6 Chapter Synthesis . . . 93

5 Progressive Control and Learning Solutions for Renewable Energy Communities 95
5.1 Research Progression and Common Evaluation Protocol . . . 96
5.2 Operationally Plausible and Constraint-Aware Control . . . 97
5.3 Adaptive Single-Agent and Community-Level Learning . . . 100
5.4 Decentralised Multi-Agent Coordination . . . 101
5.5 Data-Efficient, Reusable, and Hierarchical Learning . . . 103
5.6 Unified Cross-Method Comparison and Solved-Problem Ledger . . . 106
5.7 Accumulated Capabilities, Remaining Integration Problem, and RQ Contribution 106
5.8 Chapter Summary . . . 108

6 TI-MARL: Typed-Interface Multi-Agent Reinforcement Learning for Reconfigurable Cyber-Physical Systems 111
6.1 Scientific Positioning and Problem Formulation . . . 112
6.1.1 Closest Approaches and the Residual Problem . . . 112
6.1.2 Typed-Interface Decision Processes . . . 113
6.2 TI-MARL Architecture at a Glance . . . 116
6.3 Entity-Bound Runtime Typed Interfaces . . . 117
6.3.1 Schemas, Entities, and Dynamic Module Instances . . . 117

6.3.2 Typed Observation Parts and Community Context . . . 118
6.3.3 Dynamic Action Groups and Typed Parameterised Ports . . . 119
6.3.4 Health-Aware Causal Compilation . . . 120
6.4 Local Typed-Interface Policies . . . 122
6.4.1 Local Information Boundaries and Parameter Sharing . . . 122
6.4.2 Typed Graph Encoding and Local Latent State . . . 122
6.4.3 Dynamic Grouped Action Decoding . . . 123
6.4.4 Constraint-Aware Local Bundle Construction . . . 124
6.5 Multi-Agent Learning and Community Coordination . . . 125
6.5.1 Joint Training and Decentralised Execution . . . 125
6.5.2 Central Set and Graph Critics . . . 125
6.5.3 TI-IPPO, TI-MAPPO, and TI-MATD3 . . . 126
6.5.4 Community Observations, Rewards, and Learned Coordination . . . 127
6.6 Typed Constraints, Safety, and Runtime Robustness . . . 127
6.6.1 The Typed Constraint Graph . . . 127
6.6.2 Hard Shared-Constraint Monitoring . . . 128
6.6.3 Runtime Assurance and Degraded Modes . . . 128
6.7 Compatibility and Typed Explainability . . . 129
6.7.1 Interface Snapshots and Admission . . . 129
6.7.2 Execution-Level Decision Traces . . . 129
6.7.3 Formal and Operational Properties . . . 130
6.8 Experimental Methodology and Baselines . . . 131
6.8.1 Comparison Design and Reference Methods . . . 131
6.8.2 Reconfiguration Scenarios, Ablations, and Metrics . . . 131
6.9 Results and Comparative Analysis . . . 133
6.9.1 Nominal Performance and Comparison with Chapter 5 . . . 133
6.9.2 Multiple Assets, Reconfiguration, and Held-Out Compositions . . . 133
6.9.3 Health-Aware Robustness and Local Safety . . . 133
6.9.4 Multi-Agent Coordination and Shared Constraints . . . 134
6.9.5 Scalability and Runtime Cost . . . 134
6.9.6 Ablations, Explainability, and Boundary Conditions . . . 134
6.10 Discussion, Generality, and Contribution to the Research Questions . . . 135
6.11 Chapter Summary . . . 136

7 Experiment and MARL Policy-Set Lifecycle Orchestration and Distributed Edge Deployment 137
7.1 Lifecycle Problem, Core Entities, and State Model . . . 138
7.2 EnergAIze Lifecycle Architecture and Experiment Orchestration . . . 140
7.3 Policy-Set Governance and Coordinated Edge Deployment . . . 141
7.4 Coordination-Aware Monitoring, Updating, Rollback, and Recovery . . . 142
7.5 Empirical Systems Evaluation . . . 143
7.6 Availability, Limitations, and Contribution to RQ4 . . . 144
7.7 Chapter Summary . . . 145

8 EnergAIze in Operation: Real-World Deployment and Validation 147
8.1 Evidence Levels and Comparative Reading . . . 147
8.2 OPEVA Demonstrator . . . 148
8.2.1 Control Problem and Physical Configuration . . . 148
8.2.2 EnergAIze Integration Path . . . 149
8.2.3 Scenarios and Validation Strategy . . . 151
8.2.4 Results and Demonstrated Boundary . . . 151
8.3 DemFlex Demonstrator . . . 152
8.4 Cross-Demonstrator Lessons and Simulation-to-Operation Differences . . . 153
8.5 Contribution to the Research Questions and Chapter Summary . . . 154

9 General Discussion and Answers to the Research Questions 155
9.1 Reconstructing the Thesis Argument . . . 155
9.2 Answer to RQ1: Dynamic and Heterogeneous Multi-Agent Control . . . 157
9.3 Answer to RQ2: Multi-Dimensional Scalability of Learning and Coordination . . . 158
9.4 Answer to RQ3: Safe and Robust Operation . . . 159
9.5 Answer to RQ4: Governed and Deployable MARL Policy-Set Lifecycle . . . 160
9.6 Interactions among the Four Requirements . . . 161
9.7 Contributions, Generality, and Responsible Operation . . . 162
9.8 Limitations and Threats to Validity . . . 164
9.9 Open Research Challenges and Chapter Summary . . . 165

10 Conclusions and Future Research 167
10.1 Thesis Recap . . . 167
10.2 Main Findings . . . 168
10.3 Principal Scientific and System Contributions . . . 169
10.4 Final Assessment of the Central Thesis . . . 170
10.5 Future Research Directions . . . 171
10.6 Closing Remarks . . . 172

References 175

A Reproducibility and Supplementary Material 185
A.1 Claim and Evidence Inventory . . . 185
A.2 Scenario and Environment Manifests . . . 185
A.3 Canonical Annual Dataset Contract . . . 185
A.3.1 Calendar, price forecasts, and settlement . . . 186
A.3.2 Demand, PV, and stationary storage . . . 186
A.3.3 Electrical connections and phase allocation . . . 187
A.3.4 EVs, sessions, and deferrable services . . . 189
A.3.5 Topology and robustness calendars . . . 190
A.3.6 Matched annual baseline scorecard . . . 192
A.4 Simulator Audits and Demonstrative Results . . . 193
A.4.1 Software and contract verification . . . 194
A.4.2 Selected capability demonstrations . . . 194
A.4.3 Performance and scalability smoke benchmarks . . . 195
A.5 Simulator KPI Catalogue . . . 196
A.6 Controller, TI-RL, and TI-MARL Configurations . . . 198

A.7 Hyperparameters, Seeds, and Statistical Procedures 198
A.8 Supplementary and Negative Results 198
A.9 Lifecycle and Deployment Records 198
A.10 Software, Data, and Archival Record 198

List of Figures

1.1 Renewable Energy Community as the principal real-world cyber-physical-system instantiation used in the thesis. 4

2.1 Taxonomy of dynamicity and reconfiguration used throughout the thesis. 19

3.1 EnergAIze as a complete intelligent energy-management framework. A repeatable building instance connects perception, predictive context, algorithms, safe execution, and traceability to external platforms and physical assets; simulation and training provide candidate controllers, while community information coordinates otherwise local decisions. Adapted from the EnergAIze framework view in the OPEVA demonstrator documentation (Salvan and OPEVA Consortium 2026). . . . 59

4.1 CityLearn v3 representation of one or more Renewable Energy Communities. Each community retains its buildings, assets, controllers, local exchange, events, and KPIs; the enclosing environment synchronises communities and derives combined results. Adapted from the CityLearn v3 simulator manuscript developed alongside this thesis. 74

4.2 Peak visibility when the same physical trace is represented at one-hour, 15-minute, and 15-second resolution. Annual energy is preserved; the observable short-duration peak is not. 77

4.3 A deferrable service with an admissible start window, fixed cycle profile, and completion deadline. Cost or peak reduction is not credited as successful service when the required cycle is omitted or completed late. 80

4.4 Annual local-energy and settlement accounting for one unchanged controller trajectory. The comparison isolates the configured accounting rule and retains heterogeneous member outcomes. 81

4.5 Lifecycle of members and assets in one dynamic-topology scenario. The event log, active entity counts, structured observations, relations, and action rows change at the same simulation steps. 82

4.6 Failure channels in the controller loop. Measurement and forecast events alter controller-visible information; command dropout and equipment unavailability create traceable differences between requested and applied actions. 83

5.1 Cumulative progression from explicit control to adaptive and hierarchical multi-agent coordination. 96

5.2 Common experimental protocol used to compare controller families. 98

5.3 MADDPG and MATD3 training architecture and the evaluation dimensions used in their comparison. 102

5.4 Hierarchical community coordination used in the progressive controller studies. . 105
5.5 Comparative control, learning, and resource results across the progressive methods. 106

6.1 TI-RL as the one-agent reduction of TI-MARL. 115
6.2 TI-MARL architecture. Each agent compiles and encodes its own entity-bound runtime interface, then produces one coherent local action bundle. Community coordination is learned through aggregate community observations, coupled rewards, the joint transition, and a training-only central set/graph critic. The optional shared monitor is reserved for hard exceptional constraints. 116
6.3 Entity-bound interface graph for a building with several chargers and changing EV sessions. 118
6.4 Anatomy of an entity-bound typed observation part and typed action port. 121
6.5 Health-aware recompilation and failure containment. 122
6.6 Local typed graph encoder and dynamic group representations. 123
6.7 Central graph critic for joint training with decentralised local actors. 126
6.8 Typed Constraint Graph spanning local assets and shared community resources. . 128
6.9 Typed execution trace from interface snapshot to executed local bundle. 130
6.10 TI-MARL experimental design from nominal control to integrated reconfiguration. 133
6.11 TI-MARL results across nominal control, reconfiguration, health, coordination, scalability, and trace fidelity. 135

7.1 Lifecycle entity and provenance model. 139
7.2 Policy and policy-set lifecycle states, gates, and rollback relations. 140
7.3 Four-plane EnergAIze lifecycle architecture. 141
7.4 Coordinated policy-set packaging, staged edge deployment, activation, and rollback sequence. 142
7.5 Coordination-aware monitoring and recovery loop. 143

8.1 Validation evidence levels and the claims supported by each level. 148
8.2 OPEVA Demo 9 sites, assets, platforms, and EnergAIze integration path. 150
8.3 Timeline of one audited OPEVA control or integration period. 152
8.4 Observed differences between simulation assumptions and demonstrator conditions. 153
8.5 Verified OPEVA and DemFlex results by evidence level. 154

9.1 Final research-question, contribution, finding, and evidence map. 161
9.2 Interactions and trade-offs among Scalability, Deployability, Safety, and Robustness. 162

10.1 Final contribution stack and cumulative thesis argument. 171

List of Tables

2.1 Examples of recurring properties across distributed CPS domains. 17
2.2 Three layers of heterogeneity in REC control. 22
2.3 Operational classification of dynamics in REC and EV control. 24
2.4 Mapping from renewable-energy-community characteristics to general CPS properties. 28
2.5 Evidence-backed comparison of controller families in REC and EV control. . . . 33
2.6 Recurring MARL challenges and their manifestation in energy communities. . . 41
2.7 Evidence levels used throughout the thesis for real-world-oriented control studies. 43
2.8 Overly broad claims, counterevidence, and defensible replacements. 44
2.9 Representative MARL mechanisms for varying team size, agent loss, or changing interaction graphs. 45
2.10 Lifecycle capabilities that should not be conflated in a MARL deployment. 50
2.11 Residual requirements derived from the state of the art and their connection to the research questions. 54

3.1 Principal EnergAIze components, their role, and the chapters that provide their detailed methods or evidence. 65

4.1 Requirements used to develop and evaluate the extended simulation environment. 72
4.2 Role-based positioning against representative energy-control environments. . . . 73
4.3 Functional layers of CityLearn v3 and their role in an auditable controller experiment. 75
4.4 Installed composition of the canonical 2023 annual dataset suite. Counts are fixed in the full catalogue; the active population and available equipment can change in Dynamic and AllIn scenarios. 87
4.5 Matched Core-30 variants and the experimental question isolated by each schema. All variants reference the same annual physical time series and asset identities. . 90
4.6 Seventeen headline indicators in the core controller scorecard. 92

5.1 Metric families used throughout the controller progression. 98
5.2 Hierarchical coordination matrix. 105
5.3 Solved-problem ledger for the algorithmic progression. 109

6.1 TI-MARL as a cumulative extension of the controller progression in Chapter 5. . 113
6.2 Main TI-MARL components and their interpretation in the running energy example. 117
6.3 Representative typed observation parts for a building with two chargers. Values are illustrative. 119
6.4 Representative action ports for two charger instances and one stationary battery. . 120

6.5 Structural baselines required to isolate the TI-MARL contribution. 132

8.1 OPEVA Demo 9 scenarios and the evidence boundary reported in deliverable D6.3. 151

A.1 Static-asset generation contract by member class. PV and BESS entries apply only to members that own the corresponding asset. 187
A.2 Routine archetypes used to construct heterogeneous demand and flexibility. . . . 187
A.3 Electrical-service and charger contract. Charger powers are compatible with the charging classes described by ERSE (Entidade Reguladora dos Serviços Energéticos 2026b). 188
A.4 Generated EV identities, annual session counts, and topology-observable departure obligations. A censored session is absent from the controller's service denominator because its member or physical charger does not exist at the relevant boundary. 190
A.5 Core-30 dynamic topology calendar. Paired dates denote removal and restoration. 191
A.6 Premium-100 dynamic topology calendar. Member ranges denote simultaneous first-entry cohorts. 191
A.7 Annual matched cost and service scorecard. Paired entries are BAU/RBC Smart. . 192
A.8 Annual safety, demand-response, and robustness scorecard for the stress variants. Paired entries are BAU/RBC Smart. 193
A.9 Verification and validation evidence reported for the simulator manuscript checkout. 194
A.10 Selected application evidence and the bounded claim supported by each example. 195
A.11 Computational-cost evidence from matched simulator configurations. 196
A.12 Expanded CityLearn v3 KPI catalogue. 196

List of Acronyms

AC Alternating Current

AI Artificial Intelligence

API Application Programming Interface

BC Behaviour Cloning

BESS Battery Energy Storage System

BOPTEST Building Optimization Testing Framework

CPS Cyber-Physical System

CPU Central Processing Unit

CSV Comma-Separated Values

CTDE Centralised Training with Decentralised Execution

DC Direct Current

Dec-POMDP Decentralised Partially Observable Markov Decision Process

DER Distributed Energy Resource

DR Demand Response

EMS Energy Management System

EV Electric Vehicle

EVSE Electric Vehicle Supply Equipment

FMI Functional Mock-up Interface

G2V Grid-to-Vehicle

GDPR General Data Protection Regulation

GNN Graph Neural Network

GPU Graphics Processing Unit

HELICS Hierarchical Engine for Large-scale Infrastructure Co-Simulation

HIL Hardware-in-the-Loop

HPC High-Performance Computing

HRL Hierarchical Reinforcement Learning

HVAC Heating, Ventilation, and Air Conditioning

IoT Internet of Things

KPI Key Performance Indicator

LSTM Long Short-Term Memory

MADDPG Multi-Agent Deep Deterministic Policy Gradient

MADRL Multi-Agent Deep Reinforcement Learning

MAPPO Multi-Agent Proximal Policy Optimization

MARL Multi-Agent Reinforcement Learning

MAS Multi-Agent System

MATD3 Multi-Agent Twin Delayed Deep Deterministic Policy Gradient

MDP Markov Decision Process

MILP Mixed-Integer Linear Programming

ML Machine Learning

MLOps Machine Learning Operations

MPC Model Predictive Control

OCPP Open Charge Point Protocol

ONNX Open Neural Network Exchange

OOD Out of Distribution

OPEVA Optimisation of Electric Vehicle Autonomy

PER Prioritized Experience Replay

POMDP Partially Observable Markov Decision Process

PPO Proximal Policy Optimization

PV Photovoltaic

RAL Runtime Assurance Layer

RBC Rule-Based Controller

REC Renewable Energy Community

REST Representational State Transfer

RL Reinforcement Learning

RLOps Reinforcement Learning Operations

RQ Research Question

SAC Soft Actor-Critic

SoC State of Charge

TCG Typed Constraint Graph

TD3 Twin Delayed Deep Deterministic Policy Gradient

TIC Typed Interface Compiler

TI-MARL Typed-Interface Multi-Agent Reinforcement Learning

TIMG Typed-Interface Markov Game

TIP Typed-Interface Policy

TI-RL Typed-Interface Reinforcement Learning

TI-MDP Typed-Interface Markov Decision Process

ToU Time of Use

V2G Vehicle-to-Grid

VPP Virtual Power Plant

Glossary

action-group instance A runtime controllable group bound to a specific module or entity instance, such as the operating mode of one particular EV charger; the same shared decoder can be applied to multiple instances of the same group type

action port A callable typed action exposed by the current agent interface, including its semantic name, parameters, bounds, validity conditions, and any projection or safety rule applied before execution

action group A set of mutually exclusive action ports representing alternative operating modes for one controllable function or entity, while compatible action groups may be controlled simultaneously within the same local action bundle

agent A decision-making entity that receives observations, selects actions, and pursues one or more objectives within an environment. In this thesis, an agent may represent a participant, building, charging installation, or controllable resource, depending on the control setting

behaviour cloning Supervised policy learning from state–action demonstrations produced by an expert, rule-based controller, optimiser, or previously deployed policy

capability schema A versioned description of the potential observations, action groups, action ports, constraints, compatibility requirements, and fallback semantics that an agent or module type can expose

community A set of participants and energy assets coordinated under shared rules or objectives. In the principal case studies, this term refers to a renewable energy community

compatibility signature A versioned summary of interface and capability requirements used to determine whether a policy, compiler, schema, and runtime target can operate together safely

controller The software or physical component responsible for producing control actions. A controller may implement a rule-based strategy, an optimisation method, or a learned policy

coordination-aware monitoring Operational monitoring that detects failures in the relationships among agents and policies, such as incompatible versions, oscillating or conflicting actions, stale messages, unfair shared-resource allocation, or degradation of a community-level objective despite healthy processes

deployability The ability to package, validate, install, configure, execute, monitor, update, and recover a control solution or coordinated policy set within its intended operational environment

dynamic reconfiguration A change in the composition, availability, capabilities, or interaction structure of a system during or between operating periods, such as an electric vehicle arriving or leaving, an asset becoming unavailable, or a new participant joining a community

EnergAIze The integrated research platform and energy-domain reference implementation through which the methods, software components, lifecycle mechanisms, and demonstrators developed in this thesis are connected and evaluated

energy asset A physical or virtual energy resource that can be observed or controlled, such as an electric vehicle, charger, battery, photovoltaic system, heat pump, or flexible electrical load

entity A runtime object that gives observations or actions a concrete identity and operational meaning, such as a connected electric-vehicle session, charger, battery, or other controlled or observed object within an agent

entity-bound typed interface A runtime observation–action interface in which observation parts, action groups, and ports explicitly identify the module and controlled entity that give them meaning

experiment manifest An immutable or content-addressed record that links an experiment to its source revision, environment, scenario and data, reward, observations and actions, constraints, seeds, algorithm configuration, dependencies, resources, and resulting artefacts

fail-safe mode A controlled operating state in which a learned controller is constrained, bypassed, or replaced by a conservative controller because data, communication, policy compatibility, confidence, or safety conditions are not sufficient for normal operation

health closure The deterministic propagation of source or module health through declared dependencies to observation validity, action-port availability, parameter bounds, action groups, constraints, and operational mode

interface snapshot An immutable record of the observation parts, action ports, dependencies, validity states, bounds, health, and resource couplings exposed to an agent at one decision step

local action bundle The structured action produced by one local typed-interface policy, containing one selected port and its parameters for each active action-group instance after local hard-feasibility construction

local feasibility The construction or adjustment of an agent's local action bundle so that simultaneous actions remain within non-negotiable local physical, device, and service constraints while preserving policy intent as far as possible

module instance A concrete runtime instance of a capability-bearing component associated with an agent, such as one charger, battery, or controllable subsystem; module instances may appear, disappear, or change availability without redefining the complete policy architecture

observation part A typed block of information exposed to a policy, such as self state, scanner status, battery state of charge, deadline, entity relation, or measurement-quality indicator

offline reinforcement learning Reinforcement learning from a fixed dataset of previously collected transitions or trajectories without additional environment interaction during the offline training stage

physics-informed reinforcement learning A family of approaches that incorporates physical models, invariants, residuals, constraints, or model-based structure into policy learning, value estimation, action validation, or training objectives

policy A mapping from the information available to an agent to the action selected by that agent. A policy may be rule-based, optimisation-based, or learned

policy registry A lifecycle service that records policy identity, versions, provenance, evaluation evidence, compatibility metadata, promotion state, deployment associations, and rollback lineage, rather than storing only model bytes

policy set A governed collection of interdependent policies intended to operate together, including their agent-role mappings, observation and action schemas, communication assumptions, safety dependencies, compatibility rules, and shared deployment version

real-world operation Execution under physical, computational, communication, organisational, and lifecycle constraints that are not fully represented by a conventional isolated simulation experiment

resilience The ability of a system to recover after a fault, disruption, or period of degraded operation

robustness The ability to maintain acceptable behaviour and performance under uncertainty, disturbances, faults, imperfect data, dynamic reconfiguration, or differences between training and operation

runtime health state The current quality and availability state of observations, modules, communication channels, or controlled entities used by the Typed Interface Compiler to determine validity, bounds, degraded modes, and fallback behaviour

safety The prevention of unacceptable actions or system states through the satisfaction of physical, operational, and user-related constraints

scalability The ability of an algorithm, simulator, infrastructure, or deployed system to accommodate increasing numbers of agents, assets, interactions, experiments, or devices without unacceptable loss of performance or excessive resource use

shared hard-constraint monitor An exceptional runtime assurance mechanism that detects and, when necessary, minimally corrects or rejects joint actions that would violate non-negotiable constraints shared across agents; it is not the normal community optimisation mechanism

transferability The extent to which knowledge, representations, or policies learned in one setting can be reused or adapted in a different agent, configuration, scenario, or environment

typed decision trace An execution record grounded in the actual interface snapshot, including active entities, observations and health, available and invalid ports, selected actions, local feasibility adjustments, shared safety interventions, and final commands

typed constraint graph A runtime graph connecting agents, modules, entities, action ports, resources, topology, and local or shared constraints through typed relations; it supports local bundle feasibility, training-time graph critics, hard shared checks, compatibility, and decision traces

typed interface A runtime contract that describes an agent's available observation parts, action ports, parameter types and bounds, validity conditions, capabilities, and constraints rather than assuming one permanently fixed flat observation and action vector

Chapter 1

Introduction

This dissertation was developed within the Doctoral Programme in Informatics Engineering at the Faculty of Engineering of the University of Porto. It investigates the development and operation of MARL-based control for distributed, heterogeneous, and dynamically reconfigurable CPSs, using smart-energy systems as the principal empirical domain.

This chapter introduces the context that motivates the research, establishes the transition from simulated learning to operational control, states the central thesis proposition and research questions, defines the scope of the work, summarises its main contributions, and presents the research path reflected in the organisation of the dissertation.

1.1 Context and Motivation

The research begins from a class of control problems in which decision-making is distributed across physical assets, software components, and participants whose capabilities and availability may change over time. The following sections move from this broader cyber-physical setting to Renewable Energy Communities and, finally, to the role of multi-agent reinforcement learning in decentralised control.

1.1.1 Distributed and Dynamically Reconfigurable Cyber-Physical Systems

A CPS combines computation with physical processes. Sensors provide information about the physical environment, software interprets that information and decides what to do, and actuators change the physical process, creating a feedback loop between computation and the world. This connection is what distinguishes a CPS from a conventional software system: a software error is no longer confined to incorrect information on a screen; it can also lead to an incorrect physical action. Early work on CPSs already emphasised that physical dynamics, timing, safety, and reliability require abstractions that go beyond those used in general-purpose computing (Lee 2008).

Many contemporary CPSs are distributed. A factory includes machines, local controllers, sensors, and supervisory services. A fleet-management system coordinates vehicles that move in and

out of communication range. A group of robots may share a mission while retaining local sensing and control. An energy community connects buildings, meters, renewable generation, batteries, charging stations, and external platforms. In each case, the information needed for control and the ability to act are spread across several locations rather than concentrated in one perfectly informed controller.

Distribution changes the control problem in several ways. Each part of the system may observe only a local portion of the environment; communication introduces delays, bandwidth limits, and possible failures; available computation may range from cloud servers to embedded devices; and local decisions can have system-wide consequences. A local controller can often react quickly and preserve local autonomy, but it may not have enough information to coordinate effectively. A central controller can use a broader view, but may create communication bottlenecks, scaling limitations, or a single point of failure.

The system may also be dynamically reconfigurable. In this thesis, dynamic reconfiguration refers to changes in participating agents, available assets, capabilities, or interaction structure. Consider an electric vehicle connected to a household charger. Before the vehicle arrives, the charger has no energy request to satisfy. During charging, the vehicle's state, required departure charge, and available power become relevant to control. Once the vehicle leaves, that particular flexibility disappears even though the physical charger remains. Similar changes occur when a robot joins a mission, a machine is taken offline, a device fails, or a communication link becomes unavailable.

Such changes are different from ordinary variation in the value of a fixed observation. A temperature changing from 20 °C to 21 °C changes the state of an otherwise stable interface. A device or participant joining or leaving can instead change which observations exist, which actions are meaningful, which constraints apply, and which objectives can still be achieved. Control methods built around a fixed and homogeneous population may therefore be effective in a stable benchmark while requiring substantial adaptation when the operational configuration changes.

Centralised optimisation remains a strong option when the system is sufficiently well characterised and the corresponding optimisation can be completed within the available decision time. Decentralised or hierarchical decision-making becomes increasingly relevant as configurations change, local information cannot be shared continuously, and communication or computation constrain central control. The resulting question is not whether every responsibility should be centralised or decentralised, but how control can remain coherent when information, capabilities, and decision authority are distributed.

1.1.2 Renewable Energy Communities as Representative Cyber-Physical Systems

The energy transition provides a concrete instance of these challenges. Electricity systems are incorporating increasing numbers of distributed and renewable resources, while electrification introduces flexible loads such as heat pumps and electric vehicles. Flexibility—the ability to shift

production, consumption, storage, or exchange over time—is increasingly important for accommodating uncertainty and aligning demand with available renewable energy (Hussain et al. 2023).

European legislation defines a REC as a legal entity based on open and voluntary participation, effectively controlled by members located near the renewable-energy projects it owns or develops, and primarily intended to provide environmental, economic, or social community benefits rather than financial profit (European Parliament and Council of the European Union 2018). The control setting considered in this thesis is more concrete: a community may contain participants who consume and produce electricity and who expose different forms of flexibility through buildings, batteries, electric vehicles, chargers, heating systems, and other controllable loads.

A single participant may already combine assets with very different information and control interfaces. A battery can charge or discharge subject to power and SoC limits. A charger can exchange energy only while a compatible vehicle is connected and must account for the service expected at departure. A heat pump is coupled to thermal dynamics and comfort. These differences are not implementation details: they determine the observations, actions, constraints, and temporal dependencies of the control problem.

Time couples those decisions. Charging a battery now affects what it can do later. Delaying an electric-vehicle charge creates flexibility only until the vehicle must depart. Reducing heating demand in one interval may create a larger requirement later. Energy management is therefore a sequential decision problem in which the value of an action depends on both its immediate effect and the options it preserves for the future. This is one reason why RL has attracted sustained interest in demand response and building control (Vázquez-Canteli and Nagy 2019).

At community level, objectives are also mixed. A participant may seek lower cost or guaranteed service, while the community may seek lower collective peaks, increased use of local renewable production, or adherence to a shared infrastructure limit. These objectives can align, but they are not identical. Several participants may benefit from coordination while simultaneously competing for a limited resource. The resulting setting is therefore neither purely cooperative nor purely competitive.

The same community can reconfigure during operation. Vehicles connect and disconnect, devices become unavailable, participants can join or leave, and external signals can change the value of different actions. Renewable Energy Communities consequently provide a useful empirical setting in which distributed control, heterogeneous capabilities, changing availability, mixed objectives, and shared physical constraints appear together. These properties also arise in other CPSs, including robotics, transport, and industrial systems, although the empirical claims of this thesis remain grounded in the energy systems that are actually evaluated.

1.1.3 MARL for Mixed Cooperative–Competitive Decentralised Control

Rule-based and optimisation-based controllers remain important in energy management. A rule-based controller can encode transparent operational behaviour, while an optimisation model can schedule several assets from forecasts, explicit constraints, and models of their dynamics. These

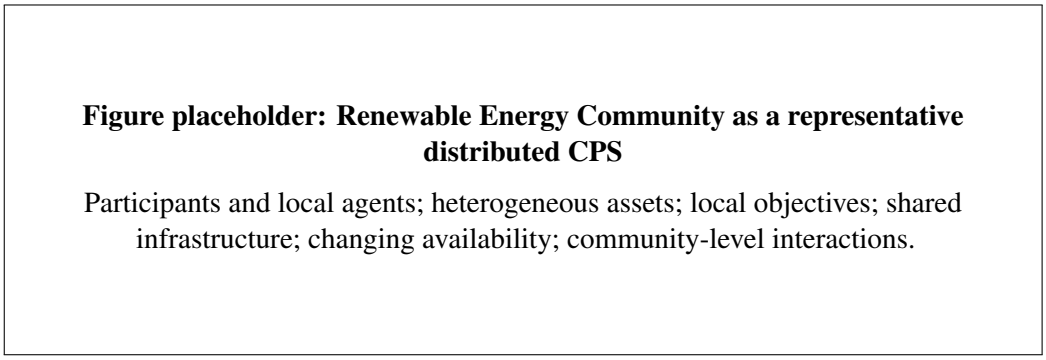


Figure 1.1: Renewable Energy Community as the principal real-world cyber-physical-system instantiation used in the thesis.

approaches provide strong references and, in some cases, appropriate operational fallbacks. Learning-based control becomes attractive when the decision strategy is difficult to specify completely in advance, when repeated interaction provides useful experience, or when decentralised decision makers must adapt to a changing environment.

RL formulates sequential control as a learning problem: an agent observes the environment, selects an action, receives feedback, and improves its policy through experience. The observations, actions, rewards, constraints, and training environment still have to be designed; learning does not remove the need for a sound control formulation. Its value lies in learning useful decision behaviour without encoding the complete strategy as a fixed rule set or solving a full optimisation problem at every decision step.

MARL extends this setting to several agents interacting in a shared environment. It includes cooperative, competitive, and mixed formulations as well as different assumptions about information, communication, and training (Albrecht et al. 2024; Gronauer and Diepold 2022). A common approach is CTDE, in which broader information can be used during training while each agent executes from its authorised local observations (Lowe et al. 2017). This is particularly relevant when operational control is distributed but learning can exploit richer information offline.

The mixed cooperative–competitive character is central to the energy setting. Two households with electric vehicles may share an interest in respecting a community capacity limit while each still prioritises the service required by its own vehicle. The agents therefore need to coordinate around shared outcomes without collapsing all participant objectives into one indistinguishable objective.

MARL introduces additional difficulties of its own. Other learning agents make the environment appear non-stationary from the perspective of an individual learner; joint decision spaces grow with the population; heterogeneous agents may expose different observations and actions; and communication or coordination can create additional computational and operational dependencies (Gronauer and Diepold 2022; Hernandez-Leal et al. 2019). These difficulties become more pronounced when the physical system itself also changes. The thesis therefore treats the learning algorithm as one part of a wider operational control system rather than as an isolated

solution.

1.2 From Simulated MARL to Operational Real-World CPS

Simulation is indispensable for training and evaluating learning-based control. It enables repeated experiments, controlled comparisons, and the deliberate creation of disturbances that would be expensive, unsafe, or impractical to reproduce in a physical installation. At the same time, a simulator can provide cleaner observations, more stable configurations, and more computational resources than will exist during operation. Moving from simulated MARL to operational control therefore requires evidence beyond learning performance alone.

This thesis organises that transition around four cross-cutting requirements: scalability, deployability, safety, and robustness. These four requirements define the dimensions investigated explicitly across the architecture, algorithms, experimentation, lifecycle, and operational evidence developed in the thesis; they are not intended as a complete taxonomy of the challenges associated with real-world intelligent control.

Scalability. Scalability concerns more than increasing the number of agents. It includes the computational cost of learning and inference, growth in heterogeneous observations and actions, simulator throughput, communication and coordination overhead, the reproducible execution of many experiments, and the resources required by deployed components. A method that scales in one dimension can still fail in another; for example, parameter sharing can reduce model count while communication or critic cost continues to grow. The relevant evidence therefore reports what dimension increased, the resources consumed, and the control quality maintained at that scale.

Deployability. A trained policy is not yet an operational controller. It must remain associated with the simulator, scenario, interface, reward, and software versions under which it was produced; it must be validated for the intended target; and it must be packaged, installed, monitored, updated, and recoverable. In a distributed system, these concerns apply to compatible sets of runtime components rather than to one model file in isolation. Deployability in this thesis therefore covers the controlled path from an experimental result to continued operation on heterogeneous edge and server-side infrastructure.

Safety. Physical control imposes limits that cannot be traded away for a higher reward. Batteries, chargers, buildings, and network connections have operating constraints, while users impose service requirements such as departure charge or comfort. Safety is used here in this operational sense: preventing unacceptable actions and states through the control formulation, validation, runtime constraints, and fallback mechanisms appropriate to each stage. Cybersecurity is important to real infrastructure but is not used as a synonym for safety in this thesis.

Robustness. Operational conditions do not remain identical to training conditions. Forecasts are imperfect, observations can be delayed or missing, devices can become unavailable, communication can fail, and the population or control interface can change. Robustness concerns whether useful control behaviour is maintained under these deviations and how performance degrades when assumptions no longer hold. Recovery after a disruption is treated as a related resilience concern and is addressed particularly through lifecycle and runtime mechanisms.

These requirements interact. More communication can improve coordination while increasing dependence on connectivity; a larger policy can improve control while exceeding an edge latency budget; conservative safety mechanisms can reduce violations while limiting performance; and frequent policy updates can support adaptation while increasing validation and operational risk. The thesis therefore evaluates the four requirements jointly where their interactions matter and separately where causal attribution is needed.

The detailed state-of-the-art analysis in Chapter 2 examines how these challenges have been addressed in prior work and identifies the residual gaps that motivate the contributions developed in the remainder of the thesis. Other concerns are also important to real-world CPSs, including cybersecurity and privacy, human factors and interpretability, regulatory and market constraints, maintainability, and long-term organisational adoption. They are considered where they constrain the proposed methods or their operation, but they are not investigated directly as primary dimensions of this thesis.

1.3 Thesis Proposition, Research Questions, and Scope

The research lies at the intersection of artificial intelligence, distributed systems, cyber-physical systems, edge computing, and intelligent energy control. Its central concern is computational: how distributed controllers can be represented, learned, evaluated, and operated when the physical and software interfaces through which they act are heterogeneous and can change. Energy is not treated as an interchangeable benchmark; asset dynamics, user requirements, shared infrastructure, and operational interfaces determine which decisions are meaningful and which evidence is required.

1.3.1 Thesis Proposition

The central proposition evaluated by this dissertation is:

> *Operationalising Multi-Agent Reinforcement Learning in distributed, heterogeneous, and dynamically reconfigurable cyber-physical systems requires the integrated co-design of suitable system representations and simulation environments, scalable learning and coordination mechanisms, safe and robust control processes, and lifecycle-aware distributed deployment infrastructure.*

The proposition places the learning method within the system that makes its claims meaningful. A controller is trained against a representation of the physical system, evaluated under

explicit assumptions, executed through concrete observation and action interfaces, and maintained through an operational lifecycle. The thesis investigates these elements together while evaluating the individual claims in the chapters where the corresponding evidence is produced.

1.3.2 Research Questions

Four research questions structure the investigation.

RQ1 – Dynamic and Heterogeneous Multi-Agent Control.

> *How can MARL-based control be formulated to accommodate heterogeneous and dynamically changing agent populations, capabilities, and availability, while balancing individual and collective objectives in distributed cyber-physical systems?*

RQ1 concerns the representation and learning problem when agents and the entities they control are not assumed to expose one permanently fixed interface. It also addresses the coexistence of local and collective objectives under decentralised control.

RQ2 – Multi-Dimensional Scalability of Learning and Coordination.

> *Under which conditions, and through which algorithmic, computational, and communication mechanisms, can MARL-based learning and coordination maintain acceptable control performance and resource use as agent population size, heterogeneity, environment complexity, and dynamic reconfiguration increase, while computational and communication resources remain constrained?*

RQ2 treats scale as a conditional, multi-dimensional property. It considers not only population size but also heterogeneity, representation size, simulation and training cost, communication, inference, orchestration, and the resources available at deployment time.

RQ3 – Safe and Robust Operation.

> *How can physical and operational safety constraints, uncertainty, faults, dynamic reconfiguration, and shifts between training and operation be incorporated into and systematically assessed for MARL-based control?*

RQ3 addresses both the prevention of unacceptable control actions and the maintenance of useful behaviour when observations, forecasts, devices, communication, or system configuration deviate from nominal conditions.

RQ4 – Governed and Deployable MARL Policy-Set Lifecycle.

What lifecycle abstractions and orchestration mechanisms allow coordinated MARL policy sets to be reproduced, governed, and deployed across heterogeneous and dynamically reconfigurable edge cyber-physical systems, while supporting compatibility validation, monitoring, staged updates, rollback, and failure recovery?

RQ4 follows the controller beyond training. It addresses experiment and environment lineage, compatibility between policies and runtime interfaces, validation and promotion, distributed deployment, monitoring, controlled updates, rollback, and recovery.

1.3.3 Research Objectives

The research questions are pursued through six objectives that connect the scientific argument to the experimental and systems work developed throughout the dissertation.

O1 – Characterise and formalise the control setting.

Establish the requirements of distributed, heterogeneous, and dynamically reconfigurable cyber-physical control, including mixed local and collective objectives and the four cross-cutting dimensions of scalability, deployability, safety, and robustness.

O2 – Establish the simulation and experimental foundation.

Extend and validate a reproducible environment capable of representing heterogeneous energy assets, changing participants and capabilities, physical constraints, disturbances, and the scales required for controlled learning experiments.

O3 – Develop and compare progressive control and learning solutions.

Design, implement, and evaluate rule-based, optimisation-based, reinforcement-learning, and multi-agent formulations under common scenarios and metrics, identifying what each approach solves and the assumptions under which it remains effective.

O4 – Develop and evaluate TI-MARL.

Design a multi-agent method that operates over entity-bound typed interfaces whose observations, action capabilities, bounds, and health can change at runtime, and assess its behaviour under heterogeneity, reconfiguration, local feasibility constraints, faults, and scale.

O5 – Develop the governed policy lifecycle and distributed runtime.

Define and evaluate the lineage, compatibility, validation, packaging, deployment, monitoring, update, rollback, and recovery mechanisms required to operate learned policies and policy sets across heterogeneous edge infrastructure.

O6 – Integrate and validate the resulting approach in the energy domain.

Connect the architectural, simulation, control, learning, and lifecycle contributions within

EnergAIze and evaluate their interactions using Renewable Energy Communities, electric-vehicle charging, real data, external platforms, edge execution, and physical demonstrators at the evidence levels actually available.

1.3.4 Scope and Assumptions

The methods and system abstractions are developed for distributed and reconfigurable CPSs, while the empirical evidence is concentrated in smart-energy systems: Renewable Energy Communities, flexible buildings, electric vehicles, charging installations, and related energy services. Examples from robotics, transport, or industrial systems are used only when they clarify the broader class of problems; they are not presented as equivalent empirical validation. Generality is therefore claimed at the level of the formulated mechanisms and abstractions, while measured performance remains specific to the evaluated energy configurations.

The thesis focuses on sequential operational control rather than long-term infrastructure planning or market design. Rule-based, optimisation-based, single-agent, and multi-agent controllers are all retained when they provide useful baselines, references, fallbacks, or solutions to particular subproblems. Simulation is used for training, controlled comparison, scaling, and disturbance injection, while real data, external platforms, edge devices, charging infrastructure, testbeds, and physical demonstrators provide additional evidence where available. These evidence levels are distinguished explicitly; the use of real data inside a simulator is not treated as equivalent to direct physical control.

The work assumes that observations can be obtained through an operational interface and that admissible actions can be applied to the controlled system. Lower-level device protections remain active. The thesis evaluates selected physical and operational constraints, disturbances, failures, and reconfiguration events, but it does not claim universal formal safety certification. Security, privacy, regulation, and human factors are considered where they affect the architecture or operation, but they are not primary research questions unless supported by dedicated methods and evidence.

1.4 Research Contributions and Outcomes

The thesis contributes scientific methods and findings together with the software and experimental infrastructure required to realise and evaluate them. The publications and research outputs associated with these contributions are listed in the front matter; detailed provenance is recorded in the chapters where collaborative work contributes directly to the corresponding evidence.

1.4.1 Scientific Contributions

The scientific contributions form seven related families. TI-MARL is the principal algorithmic contribution, while the remaining contributions establish the empirical and systems foundations,

solve complementary control problems, and provide the evidence needed to evaluate the complete thesis proposition.

C1 – Evidence-based characterisation of operational MARL requirements. The thesis develops a structured account of the conditions that separate conventional simulated MARL experiments from operational distributed CPSs. The literature analysis identifies existing solutions and counterexamples to simple absence claims, narrowing the residual challenge to the combined treatment of dynamic heterogeneity, multi-dimensional scalability, safe and robust operation, and a governed deployment lifecycle.

C2 – The EnergAIze integrated research architecture. The thesis defines an architecture connecting data and scenario preparation, simulation, controller development, reproducible experimentation, policy governance, edge deployment, runtime monitoring, fallback, and recovery. Its contribution is the explicit organisation of responsibilities and contracts needed to study the path from a control problem to continued operation within one coherent research system.

C3 – A physically audited simulation and experimentation environment. The CityLearn-derived environment is extended with heterogeneous assets, electric vehicles and chargers, electrical and community mechanisms, dynamic topology, entity-structured observations and actions, operational events, data and unit contracts, diagnostics, reproducible scenarios, and computational performance evidence. The same versioned environment, scenario, interface, constraint, and evaluation contracts provide inputs to experiment reconstruction, compatibility validation, policy promotion, packaging, and incident replay in the deployment lifecycle. The contribution therefore lies in the verified combination, in the class of experiments that it makes possible, and in the evidence continuity it establishes between training and deployment, rather than in the isolated presence of any one asset model.

C4 – Progressive control and learning solutions. The thesis develops and evaluates a progression of rule-based, optimisation-based, single-agent, and multi-agent control methods together with mechanisms for more stable, data-efficient, reusable, and hierarchical learning. Each stage addresses a concrete control problem and contributes empirical evidence about the assumptions under which it remains effective. These results provide both independent contributions and the design evidence from which the requirements of TI-MARL are derived.

C5 – TI-MARL as the principal algorithmic contribution. TI-MARL is an optimizer-agnostic multi-agent learning method for systems in which the active agents, controlled entities, and observation–action interfaces can change. Versioned capability information and runtime health are compiled into entity-bound typed interfaces containing the observation parts, action-group instances, parameterised action ports, dependencies, validity conditions, and bounds currently available to each

agent. A decentralised typed policy operates over this variable interface and produces one coherent local action bundle, while multi-agent coordination is learned through authorised shared observations, coupled outcomes, the joint physical transition, and variable-population central critics during training. The method is evaluated through controlled reconfiguration, health and disturbance tests, structural baselines, ablations, and multi-dimensional scaling.

C6 – A governed lifecycle for MARL policies and policy sets. The thesis develops lifecycle abstractions linking experiments, environments, scenarios, interfaces, reward definitions, policies, compatible policy sets, validation states, packages, deployment targets, and runtime observations. The contribution focuses on lineage, compatibility-aware validation and promotion, distributed deployment, monitoring, staged updates, rollback, and recovery in multi-agent operational settings.

C7 – Integrated real-world-oriented evidence in energy systems. The preceding contributions are connected through real data, external platforms, edge devices, charging infrastructure, energy communities, and fail-safe operation. The resulting evidence captures effects that do not appear in isolated learning experiments, including interface mismatches, runtime resource constraints, data-quality failures, deployment interventions, and the limits of the achieved operational evidence.

1.4.2 System, Software, and Open-Source Contributions

The scientific work is supported by a set of interconnected research artefacts whose energy-domain integration is realised through EnergAIze. These include data-collection and validation components; scenario and configuration tools; the extended simulation environment; rule-based, optimisation-based, single-agent, and multi-agent controllers; TI-MARL components; experiment workers and orchestration services; manifests and registries for experiments, policies, policy sets, packages, and deployment targets; validation and deployment tooling; edge-runtime components; monitoring, fallback, rollback, and recovery mechanisms; and reproducible configurations, documentation, and open-source releases where licensing permits.

These artefacts are not treated as substitutes for scientific evidence. They provide the implementation through which the corresponding claims can be tested, reproduced, integrated, and, where appropriate, transferred to operational infrastructure. Repository, release, licence, configuration, and experiment identifiers are retained for the submitted version, with access limitations stated where partner or data restrictions apply.

1.4.3 Societal and Environmental Relevance

Renewable Energy Communities aim to give participants a more active role in the production, use, and sharing of energy. Intelligent control can help use local flexibility without requiring users to coordinate every flexible asset manually, while community-level objectives can support local renewable-energy use and reduce simultaneous peaks. The technical results of this thesis are

therefore relevant to wider questions of flexibility, electrification, and distributed energy participation.

The thesis nevertheless distinguishes measured technical outcomes from broader societal claims. Lower simulated cost does not establish long-term adoption, peak reduction in a testbed does not establish system-wide emissions impact, and edge execution does not by itself establish a complete privacy guarantee. Societal and environmental implications are discussed from the evidence actually produced and within the limits of the evaluated systems.

1.5 Research Path and Thesis Organisation

The research developed cumulatively. Early work established the data, simulation, and experimental foundations and examined progressively more capable control formulations, from rule-based and optimisation approaches to single-agent and multi-agent reinforcement learning. Each stage addressed a concrete problem under explicit assumptions and produced results that remained useful in their own right. As the systems and experiments became more heterogeneous and dynamic, the remaining difficulty increasingly concerned how to preserve meaningful control when agents, assets, observations, actions, constraints, and operating conditions changed.

These findings motivated TI-MARL and, in parallel, the lifecycle and deployment mechanisms required to operate learned controllers beyond isolated experiments. The development was not a linear sequence: simulator limitations became visible during algorithm experiments, real operational data exposed assumptions in scenario design, and deployment targets introduced constraints that fed back into the control and software architecture. EnergAIze provides the common context in which these algorithmic and systems contributions are connected and evaluated.

The organisation of the dissertation follows this progression. Chapter 2 establishes the foundations and residual gaps; Chapter 3 presents the integrated architecture; and Chapter 4 develops the data, scenario, and simulation foundation. Chapter 5 presents the progression of control and learning solutions, leading to TI-MARL in Chapter 6. Chapter 7 addresses the governed lifecycle and distributed deployment of learned control, and Chapter 8 examines the integrated system in operational energy settings. Finally, Chapter 9 combines the evidence to answer the research questions, and Chapter 10 concludes the thesis and identifies the remaining research directions.

Chapter 2

Real-World Multi-Agent Control in Cyber-Physical Systems: Foundations, State of the Art, and Research Gaps

Chapter 1 established the thesis proposition and four research questions around dynamic and heterogeneous control, multi-dimensional scalability, safe and robust operation, and the governed deployment of MARL-based control. This chapter examines the knowledge that supports those questions and determines which parts of the problem are already addressed by prior work.

The argument begins with the class of systems under study rather than with a learning algorithm. Distributed, heterogeneous, and dynamically reconfigurable cyber-physical systems define the operational conditions. Renewable Energy Communities then provide the principal empirical instantiation, making heterogeneity, temporal flexibility, mixed objectives, and physical constraints concrete. The chapter next compares established control families and identifies the conditions under which MARL offers a meaningful alternative. Only after that comparison are the RL and MARL foundations needed by the remainder of the thesis introduced formally.

Simulation is treated as an experimental instrument rather than as evidence of deployment. The chapter therefore distinguishes controlled simulation from higher levels of operational evidence, while the detailed comparison of simulation and benchmarking platforms is developed in Chapter 4, immediately before the simulator contribution itself. The final sections return explicitly to RQ1–RQ4, test broad gap claims against counterexamples, retain adjacent problems that lie outside the thesis scope, and derive the residual requirements used by the later contribution chapters.

2.1 Literature Scope and Evidence Approach

The state of the art is organised around the four research questions defined in Chapter 1. The objective is not to produce an exhaustive census of reinforcement learning, energy management,

simulation, or MLOps. It is to establish the strongest relevant prior capabilities, identify counterexamples to overly broad absence claims, and determine which limitations remain when the requirements of this thesis are considered together.

2.1.1 Scope and Sources

The literature base combines four bodies of evidence that are needed by the thesis: MARL in distributed cyber-physical systems; control and learning in Renewable Energy Communities and EV charging; simulation, benchmarking, and operational evidence; and lifecycle mechanisms for learned controllers and coordinated policy sets. These bodies are not presented as four independent reviews. They form one structured evidence base whose purpose is to support the argument developed across this chapter and the more detailed local state-of-the-art analyses in Chapters 4 and 7.

Peer-reviewed primary studies are preferred when a scientific claim depends on experimental evidence. Surveys and tutorials are used to establish terminology and to identify important lineages. Recent preprints are retained when they provide unusually relevant counterexamples or emerging capabilities, but their evidential status is stated explicitly. Official documentation, standards, and software repositories are used where a platform capability, interface, release history, or deployment mechanism cannot be established reliably from a research paper alone.

Searches combined targeted concept searches with backward and forward citation tracing. The concepts were selected from RQ1–RQ4 and from the operational conditions introduced in Chapter 1: heterogeneity and reconfiguration; scale and resource use; safety, robustness, and uncertainty; and deployment, monitoring, rollback, and recovery. The energy domain remained the principal empirical lens, while selected studies from robotics, transport, and industrial systems were retained when they tested whether a limitation was specific to energy or recurrent across distributed CPSs.

Detailed search expressions, screening records, evidence matrices, and source notes are retained with the research artefacts supporting the thesis. Keeping that operational record outside the chapter allows the dissertation to present the scientific synthesis without turning the state of the art into a procedural report of individual search passes.

2.1.2 Evidence Synthesis and Limitations

Three kinds of statement are kept distinct throughout the chapter. A *foundation* is an established concept used to formulate the work, such as an MDP, a Markov game, or CTDE. A *reported limitation* belongs to a particular study, platform, or line of work. A *research gap* is a broader conclusion that must remain defensible after strong counterexamples have been considered. This distinction is important because the existence of one limitation in one study does not establish that an entire field lacks the corresponding capability.

The synthesis is therefore deliberately counterexample-oriented. Candidate gaps were tested against work demonstrating dynamic teams, heterogeneous agents, large populations, explicit safety mechanisms, physical testbeds, operational deployment, model registries, edge orchestration, rollback, and related mechanisms. Where such evidence exists, the claim is narrowed rather than preserved for convenience. The resulting gaps concern combinations, assumptions, and evidence boundaries, not the blanket absence of individual techniques.

Evidence is also interpreted according to what was actually demonstrated. A documented software feature is not equivalent to an evaluated control mechanism; a physical testbed is not the same as sustained operational service; a policy evaluated on a larger population is not automatically scalable in communication, simulation throughput, or deployment cost. The relevant evidence level is stated when it changes the interpretation of a result.

This review is structured and scoping in nature rather than exhaustive. The retained literature was selected to cover the mechanisms, counterexamples, domains, and evidence levels needed to answer the thesis questions, and it includes recent work whose status may change before submission. Absence from the retained evidence is therefore not treated as proof that a capability has never been studied. Claims are limited to what the cited literature and archived evidence support, and the detailed records are retained so that the synthesis can be audited and updated.

2.2 Distributed, Heterogeneous, and Dynamically Reconfigurable Cyber-Physical Systems

The first step is to characterise the system class independently of any particular learning method. Chapter 1 introduced distribution, heterogeneity, and reconfiguration as recurring properties of the thesis problem. Here they are examined more closely because they determine what information is available, which actions are meaningful, how controllers can be coordinated, and what changes when the operational configuration evolves. These properties establish the setting for RQ1 and introduce communication, computation, and runtime constraints that later shape RQ2–RQ4.

2.2.1 Distributed Control and Operational Constraints

A CPS connects computation, communication, and physical processes through feedback. Sensors measure the physical world, software estimates or predicts its state, controllers select actions, and actuators affect the next physical state. Timing and physical dynamics are part of the computation rather than external details (Lee 2008).

Distribution changes this feedback loop. Information may be generated near an asset, processed at an edge device, aggregated by a platform, and used by a controller running elsewhere. Different actions may also be applied at different locations. A distributed architecture can reduce latency, preserve local data, and avoid a single control bottleneck, but it also creates coordination

and consistency problems. Reviews of distributed CPS control show that these trade-offs appear across domains rather than only in energy (Hamzah et al. 2023).

It is useful to distinguish three broad patterns.

Centralised control. A central controller receives the relevant information and computes the actions for the complete system. This can support globally coordinated decisions, but it depends on communication, a sufficiently accurate global model, and enough computation within the decision interval.

Decentralised control. Local controllers select actions using local information, possibly without a central decision maker. This can reduce latency and improve fault isolation, but global coordination is more difficult.

Hierarchical control. Local controllers retain selected responsibilities, while higher levels coordinate shared objectives, constraints, or resource allocation. Many operational systems use this intermediate pattern.

Decentralisation is a control structure, not an objective in itself. A centralised optimiser may remain the strongest baseline for a small, well-modelled system. The relevant question is whether the structure remains effective as the system grows, changes, or cannot share all data continuously. Some MARL studies describe decentralised agents while retaining a central simulator, global training information, or coordinated deployment outside the learning algorithm; those assumptions are reported explicitly.

Examples across CPS application domains. Table 2.1 shows how recurring system properties appear in energy, robotics or mobility, and industrial CPS. The examples establish generality at the level of the modelling and systems problem; they do not imply direct transfer of one controller between domains.

Evaluation remains domain-aware. Collision avoidance in robotics and battery or grid constraints in energy require different physical models and validation criteria, even when the systems share representations or lifecycle principles. The architectural argument is broader than energy, while the measured empirical claims remain grounded in the energy domain.

2.2.2 Heterogeneous Agents, Assets, and Interfaces

Heterogeneity can refer to several different properties. Agents may have different physical capabilities, sensors, action spaces, objectives, computing resources, communication links, or software interfaces. Treating all of these differences as one category can hide the reason a method fails.

The distinction between agent and energy asset is particularly important. An asset is the physical or virtual resource being observed or controlled. An agent is the decision-making entity. One agent may control several assets, several agents may coordinate around one shared resource, or a

Table 2.1: Examples of recurring properties across distributed CPS domains.

Property	**Energy community**	**Robotics or mobility**	**Industrial CPS**
Distribution	Buildings and chargers act at different sites	Robots or vehicles act at different locations	Machines and controllers are distributed across a plant
Heterogeneity	Batteries, EVs, PV, and heat pumps expose different interfaces	Aerial, ground, or specialised agents have different capabilities	Machines have different processes, sensors, and cycle times
Dynamic population	Vehicles arrive and depart; assets become unavailable	Agents join, leave, or move out of range	Machines enter maintenance or production cells are reconfigured
Mixed objectives	Individual cost and comfort coexist with community peaks and emissions	Local task progress coexists with mission-wide coordination	Local throughput coexists with plant-wide quality and energy targets
Physical constraints	Power, energy, comfort, and network limits	Collision, dynamics, battery, and motion constraints	Safety envelopes, capacity, timing, and maintenance constraints
Communication limits	Local meters and community platforms may be intermittently connected	Wireless links vary with position and interference	Field networks have timing, bandwidth, and reliability limits
Edge operation	Policies run in building or charger controllers	Policies run on robots or roadside units	Control runs on industrial PCs, PLC-connected devices, or embedded nodes

rule-based controller may operate an asset without being a learning agent. These alternatives lead to different observation spaces, action spaces, and deployment units.

Consider three energy assets. A battery accepts a continuous charge or discharge power constrained by its SoC. An EV charger may accept a current limit but only while a vehicle is connected. A heat pump affects a temperature state whose response is delayed by building thermal dynamics. Even if all three actions are normalised to the interval $[-1, 1]$, the meaning of the actions is not the same. A shared neural network cannot safely treat them as interchangeable without an explicit representation of asset type and capability.

The same issue appears in other CPS. A heterogeneous robot team may contain aerial and ground robots. A transport system contains vehicles, traffic signals, and charging infrastructure. A factory contains machines with different cycle times and maintenance states. The general challenge is not merely to train several policies; it is to connect each policy to an interface that correctly represents what its agent can observe and do.

Heterogeneity therefore affects both learning and software architecture. During learning, it influences parameter sharing, representation, coordination, and generalisation. During deployment, it influences compatibility, packaging, hardware requirements, testing, and updates. A method that supports heterogeneous tensors inside a simulator may still require a separate operational process for each physical interface.

The literature also reveals a useful distinction between *environmental heterogeneity* and *architectural heterogeneity*. Buildings can have different demand profiles, capacities, or locations while all agents still share one observation structure, action structure, and policy class. This is environmental heterogeneity. Architectural heterogeneity is stronger: agents differ in what they can observe, what they can control, how often they act, or which role they play. Residential-flexibility research has considered heterogeneous flexible resources under partial observability, and building-control work has explicitly compared homogeneous and heterogeneous MARL formulations (Charbonnier et al. 2022; Wu, Nweye, et al. 2023). These studies are important counterexamples to a claim that heterogeneity is absent. At the same time, most evidence remains tied to fixed agent sets and predeclared interfaces. Supporting a battery, an EV charger, and a heat pump in one experiment is not yet the same as supporting previously unseen interface types that join an operational system without controller redesign.

2.2.3 Dynamic Populations, Capabilities, and Reconfiguration

A fixed-agent problem assumes that the same agents remain present throughout training and operation. A dynamically reconfigurable system relaxes this assumption. Agents or assets may join, leave, fail, recover, or change capabilities. The relevant interaction topology may change with them.

Electric-vehicle charging provides the recurring example. The charger exists continuously, but the vehicle that creates the energy request is temporary. Its arrival time, departure time, initial SoC, desired SoC, battery capacity, and admissible charging power can vary. The control problem at 18:00 may therefore contain a different set of active requests from the problem at 15:00.

There are several levels of change:

- **state variation**: values change while the agent and interface remain fixed;
- **availability variation**: an existing asset is temporarily unavailable;
- **population variation**: agents or controllable requests enter or leave;
- **capability variation**: the observation or action interface changes;
- **topology variation**: communication or coordination relationships change.

These cases should not be collapsed into one generic disturbance. A missing sensor value can be imputed. A departed vehicle should no longer receive a charging action. A new charger type

may require a policy compatible with a different action space. The representation and lifecycle mechanisms must therefore know not only the current data but also which agents and interfaces are valid.

Dynamic populations interact strongly with training. Padding a fixed-size input can handle a known maximum number of agents, but it does not by itself provide generalisation to unseen team sizes or types. Training separate policies for every configuration is often impractical. Type-conditioned policies, permutation-invariant representations, graph-based models, attention, role-based policies, or hierarchical coordination are possible responses, but their suitability depends on the domain and evidence.

Dynamic teams have already received targeted attention, and this must be acknowledged before defining the residual gap. In traffic, MARL has been applied to a varying number of connected vehicles under changing connectivity penetration (Su et al. 2020). In robotics, asynchronous policies have been designed to tolerate different action durations and robot loss (Yu, Yang, et al. 2023); event-triggered communication has been evaluated when the number of transport robots differs from training (Shibata et al. 2023); and physics-informed policies have handled time-varying communication graphs and robot teams larger than those observed during training, including physical Robotarium experiments (Sebastián et al. 2025). The remaining difficulty is therefore not simply "a variable number of agents". It is the joint case in which capabilities, observations, actions, timing, and availability differ while agents join, leave, fail, recover, or change role during continuing operation.

Figure 2.1 fixes the reconfiguration vocabulary used in the remaining chapters and prevents weaker forms of dynamicity from being mistaken for open heterogeneous agent populations.

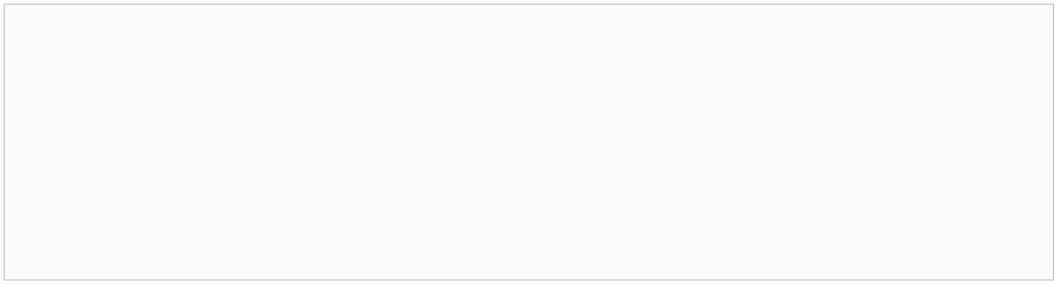


Figure 2.1: Taxonomy of dynamicity and reconfiguration used throughout the thesis.

Physical, computational, and communication constraints. A real CPS is constrained at several levels.

Physical constraints. Actions must respect power, energy, temperature, speed, mechanical, and other domain-specific limits. Physical systems also have inertia and delays: an action may not have an immediate or perfectly predictable effect.

Operational constraints. Users, operators, and organisations impose requirements beyond physical feasibility. A vehicle may need a minimum charge before departure. A building must preserve comfort. Maintenance windows, market rules, and contractual limits can restrict otherwise feasible actions.

Computational constraints. A policy may be trained on a cluster but executed on an embedded device. Memory, inference time, processor capability, and energy consumption constrain the deployed model. Edge computing can reduce latency and keep data local, but it also distributes hardware and maintenance responsibilities (Shi et al. 2016).

Communication constraints. Data can be delayed, lost, compressed, or unavailable. Bandwidth and privacy may limit what is shared. A local controller should know what to do when a community service is temporarily unreachable.

Lifecycle constraints. The controller must be installed, configured, monitored, updated, and, when necessary, rolled back. The operating system, dependencies, interface version, and policy version become part of the control solution. These concerns are not represented in a conventional Markov game, but they can determine whether the policy is usable.

The constraints interact. Sending more data can improve coordination while increasing communication cost and privacy exposure. A larger model can improve simulated performance while exceeding edge memory or latency limits. A conservative safety filter can reduce violations while also reducing nominal reward. Scalability, deployability, safety, and robustness are therefore evaluated as connected properties rather than independent checkboxes.

Taken together, these properties define the operational problem before any controller family is selected. For RQ1, they establish that reconfiguration can change the set and meaning of observations, actions, capabilities, and relationships rather than merely the numerical state. For RQ2, they expose communication, computation, and representation costs. The same changes also alter the conditions under which safety evidence and deployed policy compatibility remain valid, connecting the system model directly to RQ3 and RQ4.

2.3 Renewable Energy Communities as Representative CPSs

Renewable Energy Communities make the preceding CPS properties concrete in the principal domain of this thesis. They combine physically distributed participants, heterogeneous assets, time-coupled flexibility, changing availability, local and collective objectives, and shared electrical constraints. The purpose of this section is not to claim that energy is unique, but to show how the general system properties become measurable control requirements in a domain where the thesis can evaluate them directly.

This domain perspective gives RQ1 a concrete representation problem, gives RQ3 explicit physical and service constraints, and provides the operational scales and shared resources needed to examine RQ2. It also exposes the interfaces and organisational boundaries that later matter to RQ4.

2.3.1 Participants, Assets, and Heterogeneity

European legislation defines a REC through participation, control, geographical proximity, and community benefit (European Parliament and Council of the European Union 2018). This legal framing distinguishes a REC from an arbitrary collection of simulated buildings. It also means that technical control operates within an organisational setting containing members, operators, service providers, network actors, and possibly market participants. Reviews of energy-community business models show that governance, value distribution, and stakeholder roles are central to implementation (Reis et al. 2021).

For the control problem, participants can consume, produce, store, or shift energy. The term *participant* is preferable to inventing a different agent label for every possible role. A participant may be a consumer, a prosumer with local generation, or an entity providing flexibility through controllable assets. Whether that participant corresponds to one learning agent is a design choice addressed later by the EnergAIze architecture and controller formulations.

Typical assets include:

- photovoltaic generation;
- stationary batteries;
- electric vehicles and chargers;
- heat pumps and thermal storage;
- water heaters and other shiftable loads;
- smart meters, sensors, and controllable switches;
- external signals such as prices, emissions, weather, and grid limits.

The asset list is less important than the interfaces it creates. A photovoltaic system may be largely uncontrollable but provides uncertain production. A battery is controllable in both directions but constrained by energy and power limits. A flexible load may only be shifted within a comfort or deadline window. These differences explain why a community cannot be modelled simply as many copies of the same agent.

Stakeholders also define who is allowed to see data and apply actions. A community platform may know aggregate demand without receiving every raw household measurement. A charger operator may control the charging station but not the building's other loads. A network operator

may impose a limit without participating in the optimisation of individual costs. These boundaries later become part of deployment and information architecture.

Three layers of heterogeneity. The energy literature often describes a system as heterogeneous, but that word can refer to different things. Keeping the distinction explicit is important because each layer creates a different control problem.

Asset heterogeneity. Batteries, EVs, photovoltaic systems, heat pumps, and flexible loads have different physical dynamics, actions, limits, and time scales. This is well represented in optimisation, predictive-control, and learning studies (Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Srithapon and Månsson 2023; Zhu, Yang, et al. 2022).

Participant and objective heterogeneity. Buildings and users differ in demand, routines, comfort, mobility requirements, tariffs, available data, and preferred objectives. MERLIN, for example, studies buildings with distinct occupant behaviour, while EnergAIze permits different local management objectives within one community (Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Nweye, Sankaranarayanan, et al. 2023).

Controller and interface heterogeneity. Agents can expose different observation fields, valid actions, capabilities, operating timescales, communication interfaces, or policy architectures. This layer is harder. A system may contain different assets while still mapping all of them into a fixed, homogeneous controller schema. Dynamic participation becomes substantially more demanding when a joining controller introduces a capability or action meaning that was not present during training.

Table 2.2: Three layers of heterogeneity in REC control.

Layer	Example	Control implication
Assets	One building has a battery and EV, another has a heat pump and photovoltaic production.	Models, constraints, and actions differ, but a common abstraction may still preserve one fixed controller interface.
Participants and objectives	One participant prioritises cost, another mobility readiness, while the community limits aggregate import.	Local and community outcomes must be represented and reported separately.
Controllers and interfaces	A new agent exposes different measurements, valid actions, timing, or hardware resources.	The architecture must discover or describe capabilities and bind a compatible controller without assuming an unchanged input–output contract.

The first two layers are already well represented in REC and EV research. The third is the stronger residual problem for this thesis. It explains why a community with different load profiles is not automatically evidence of an architecture that can accept a new charger type, a new action space, or a participant with a different policy role during operation.

2.3.2 Flexibility, Temporal Coupling, and Dynamic Availability

Energy flexibility is the ability to change production, consumption, storage, or exchange relative to a baseline in response to an internal or external need. The literature uses several related definitions and modelling approaches, but a common element is the possibility of moving an energy action in time or changing its magnitude within constraints (Hussain et al. 2023).

Different assets provide flexibility in different forms:

- a battery moves energy across time;
- an EV charge can be delayed or modulated before departure;
- a heat pump uses thermal inertia to move electricity consumption while maintaining comfort;
- a flexible industrial process can be rescheduled within production requirements;
- controllable generation or curtailment changes the balance directly.

The value of flexibility depends on time. One kilowatt-hour of battery capacity is not equally useful in every interval. It may be valuable before a predicted community peak and less valuable after the peak has passed. Similarly, an EV with eight hours before departure provides more scheduling flexibility than an EV with twenty minutes remaining.

This temporal coupling is why one-step optimisation can be insufficient. A controller must consider the future opportunity cost of using flexibility now. Model predictive control addresses this through a receding horizon. RL addresses it through expected return. Rule-based control may encode selected temporal knowledge explicitly. The controller families differ, but the underlying sequential problem is shared.

Uncertainty further complicates the value of flexibility. Photovoltaic production, demand, arrival times, departure times, and prices may be forecast rather than known. A controller can reserve flexibility for a predicted event that does not occur or use it too early when the actual event is larger than expected. Robustness therefore includes not only sensor failures but also ordinary forecast error.

Electric vehicles and dynamic asset availability. Electric vehicles are both mobility assets and energy assets. Their primary purpose is transport, which means that energy control is subordinate to a mobility requirement. A charging policy that reduces cost but fails to provide the energy needed for departure is not acceptable.

An EV charging request is characterised by arrival time, expected or required departure time, initial SoC, target SoC, battery capacity, charger power, efficiency, and possibly vehicle-to-grid capability. These quantities differ between sessions and may be uncertain. The same charger can therefore present a different control problem each time a vehicle connects.

This makes EV charging an intuitive example of a dynamic system, but the word *dynamic* must be used carefully. In most studies, the physical charger or controller identity remains fixed while the session state changes. Puech et al. model uncertain starts, ends, energy requests, and early disconnections over 32 fixed charging slots using MPC and stochastic programming (Puech et al. 2024). Hermans et al. forecast occupancy and control 174 fixed chargers in an operational microgrid (Hermans et al. 2024). In both cases, users and sessions change, but the controller topology is known beforehand.

Table 2.3 defines the vocabulary used throughout the thesis. The categories are ordered by the extent to which the controller architecture itself changes.

Table 2.3: Operational classification of dynamics in REC and EV control.

Category	**What changes**	**Representative evidence and limitation**
Session-level dynamics	An EV plugs in, disconnects, changes requested energy, or changes state while a charger/controller slot remains fixed.	Common in EV scheduling; Puech et al. and Hermans et al. explicitly represent changing sessions over fixed infrastructures (Hermans et al. 2024; Puech et al. 2024).
Asset availability	An asset becomes usable or unavailable inside a pre-existing building or agent.	Useful for masks and local fallbacks, but it does not create a new controller identity or interface.
Variable instance size	The same method is evaluated with a different number of chargers, vehicles, or buildings between runs.	Sadeghianpourhamami et al. use a size-independent aggregate state, while GNN-DT transfers one graph-compatible architecture across charger counts (Orfanoudakis, Panda, et al. 2025; Sadeghianpourhamami et al. 2020).
Dynamic participation	A participant or controller joins or leaves during the modelled process.	Wu et al. explicitly model buildings joining and departing a VPP and adapt a Multi-Agent Transformer within a predefined family of building representations (Wu, Qiu, et al. 2024).
Agent loss	A previously active controller fails or disappears during an episode.	Stronger evidence was found in robotics than in the energy literature; it tests continuity after failure rather than only planned session completion.
True open population	Controllers can be instantiated or removed online, potentially with new capabilities and interfaces, without redesigning or retraining for that exact composition.	No retained REC/EV study demonstrates this complete case. Recent graph methods approach variable scale and heterogeneous node types, but preserve a known feature and action grammar.

The distinction changes how evidence should be interpreted. Sadeghianpourhamami et al. avoid a state representation that grows directly with the number of charging stations and demonstrate generalisation to larger station sets, but the policy still controls an aggregate problem with known semantics (Sadeghianpourhamami et al. 2020). GNN-DT uses graph embeddings so that a model trained at one charger count can be evaluated at other counts; this is size generalisation, not online creation of a new controller interface (Orfanoudakis, Panda, et al. 2025). HetGPS represents three EV types in a parameter-shared graph controller and evaluates 200–3218 EVs, but its formal agent index set remains fixed for each problem instance even though session arrival and

departure are part of the exogenous process (Wang et al. 2026). As a recent preprint, it is a strong design counterexample but not yet peer-reviewed or physically deployed.

Wu et al. provide the clearest energy-domain counterexample to a claim that dynamic participation has not been studied. Their Multi-Agent Transformer with an adaptation module handles buildings that join or depart a virtual power plant and is evaluated across configurations with three to nine buildings of different types and regions (Wu, Qiu, et al. 2024). The result narrows the thesis gap. It does not remove the harder operational question of accepting agents with previously unseen observation fields, actions, roles, timing, or hardware requirements.

The vehicle can also participate at different architectural levels. It may be controlled as an individual agent, as one asset within a building agent, or as part of a central charging-site controller. Each choice changes the number of agents, the information available to the controller, and the meaning of coordination. Chapter 5 compares several such formulations rather than assuming that an EV must always correspond to one MARL agent.

Vehicle-to-grid operation adds another degree of freedom by allowing discharge to the building or grid. It can increase community flexibility, but it also introduces degradation, user acceptance, availability, market, and safety considerations. The presence of a technically possible action does not mean that it is contractually or operationally admissible.

2.3.3 Individual and Community Objectives and Physical Constraints

A community contains objectives at several levels.

At participant level, common objectives include electricity cost, comfort, mobility readiness, battery usage, self-consumption, and preservation of local autonomy. At community level, objectives can include aggregate cost, peak reduction, renewable self-consumption, emissions, energy sharing, and compliance with a shared import or export limit. Network and service operators may introduce additional objectives related to stability, congestion, or quality of service.

These objectives are not always contradictory. Charging a vehicle when local photovoltaic production is high can reduce both individual cost and community import. Conflict appears when several participants prefer the same limited period or when one participant must sacrifice local benefit to improve a community metric.

The formulation therefore needs more than one aggregate score. A community can achieve a low peak while one vehicle repeatedly misses its target. It can minimise total cost while allocating benefits unevenly. It can reduce emissions by shifting demand into a period that increases an individual bill. Reporting only the global objective can hide these effects.

There are several possible responses:

- combine local and community terms in each reward;
- use separate rewards and study the resulting general-sum interaction;
- impose participant requirements as constraints;

- coordinate local controllers through prices, incentives, budgets, or shared signals;
- use hierarchical control in which local and community decisions have different responsibilities.

A single reward cannot resolve the social and economic governance of a REC. The control formulations represent local and community objectives and report their measured effects, while broader questions of fairness and governance require evidence beyond aggregate control performance.

Cooperative and competitive interactions. The interaction between participants changes over time. During a period of abundant local renewable production, several participants can consume without conflict. During an evening peak or under a community import limit, the same participants may compete for available capacity. They still cooperate by accepting a shared coordination mechanism, but their preferred local actions are no longer simultaneously feasible.

A simple example illustrates the mixed motive. Suppose three EVs need energy before morning and the installation can safely supply only two at full power. All drivers benefit from reliable operation of the site, which is a cooperative objective. Each driver also prefers their own vehicle to charge first, which creates competition. A policy that ignores the shared limit is unsafe. A policy that always prioritises the same vehicle may be operationally unfair. A policy that divides power equally may be inefficient when departure times differ.

This example shows why "cooperative" and "competitive" should not be treated as fixed labels attached to the complete system. The relationship depends on the objective, resource, and time. Agents can share one reward component and retain different local reward components. They can cooperate during training through shared information while executing individual policies. They can also use a central coordinator for a constrained resource while retaining local control elsewhere.

The energy domain therefore motivates general-sum formulations and hierarchical architectures. It also provides interpretable metrics for analysing the outcome: missed energy requirements, individual cost, community peak, renewable use, and distribution of benefits. These metrics make the mixed interaction more visible than a single game score.

Physical constraints and mapping to general CPS properties. Energy control is bounded by physical constraints. Batteries and EVs have minimum and maximum SoC, charge and discharge power, efficiency, and possibly degradation limits. Chargers have rated current and connection states. Buildings have thermal dynamics and comfort limits. Electrical installations have import, export, phase, and equipment limits.

Operational constraints add deadlines, preferences, contracts, and service requirements. An EV target may be expressed as a minimum departure energy. A participant may allow a heat pump

to be interrupted only within a comfort band. A charger operator may limit how often a session can be modulated. A community may have an agreed peak or sharing rule.

Constraints can be handled at different layers:

- the environment can clip or reject invalid actions;
- the policy can be constructed to output only admissible actions;
- an optimisation or safety layer can project proposed actions into a feasible set;
- a fallback controller can replace a learned policy under selected conditions;
- validation gates can reject a policy before deployment if violations exceed a threshold;
- lower-level physical protections can remain independent of the learning system.

Safety mechanisms provide different levels of evidence. Action clipping protects the physical model but can hide frequent invalid proposals. A fallback improves operational safety while changing the controller that is effectively executed. Formal guarantees are stronger than empirical tests but may depend on restrictive models. Each later experiment therefore identifies whether its evidence concerns policy behaviour, runtime enforcement, empirical stress testing, or a formal property.

Mapping REC characteristics to general CPS properties. Table 2.4 consolidates the role of energy communities in the thesis. Each REC characteristic is connected to a more general CPS property and to the resulting research concern.

The mapping establishes RECs as a representative real-world CPS while keeping the empirical scope explicit. It also prepares the comparison of control approaches: the relevant issue is how MARL compares with established alternatives and what evidence each family provides.

The domain therefore provides more than an application example. It exposes a compact combination of the properties that motivate the thesis: heterogeneous capabilities, temporal coupling, dynamic availability, mixed objectives, partial information, and non-negotiable physical limits. The later experiments use these properties to test the research questions without claiming that results measured in energy automatically generalise to unrelated CPS domains.

2.4 Control Approaches for Distributed Energy Systems and Related CPS

The preceding sections establish the control problem but do not determine the controller that should solve it. Energy systems already use successful rule-based, optimisation-based, predictive, learning-based, and hybrid approaches. The relevant comparison is therefore between the

Table 2.4: Mapping from renewable-energy-community characteristics to general CPS properties.

REC characteristic	General CPS property	Research implication
Buildings and assets are geographically distributed	Distributed sensing, actuation, and control	Decide which information and responsibilities remain local or are coordinated
Batteries, EVs, PV, heat pumps, and loads expose different interfaces	Heterogeneous agents and assets	Represent capabilities explicitly and support compatible policies
EV sessions begin and end; assets become unavailable; selected participants may join or leave	Session dynamics, availability, and possible population reconfiguration	Distinguish state change from controller membership and test whether interfaces remain compatible
Individual cost and comfort coexist with community objectives	Mixed cooperative–competitive interaction	Evaluate both local and global outcomes
Decisions affect later flexibility	Sequential and temporally coupled control	Use models or learning that account for long-term consequences
Forecasts and measurements are imperfect	Partial observability and uncertainty	Assess robustness to error, delay, and missing data
Power, energy, comfort, and network limits are mandatory	Safety-critical physical constraints	Prevent or quantify invalid and unacceptable actions
Policies may run on local controllers	Edge and lifecycle constraints	Consider inference resources, packaging, updates, monitoring, and recovery

assumptions, capabilities, and operational costs of these families, rather than between "traditional" and "intelligent" control as if one necessarily replaced the other.

The discussion introduces RL and MARL at the level needed to compare them with credible alternatives. Their formal foundations are developed in Section 2.5, after the motivation for using them has been established. This order keeps the state of the art centred on the control problem rather than turning the chapter into a general tutorial on reinforcement learning.

The useful question is therefore not "why replace traditional control with MARL?" It is "which controller fits which assumptions, and what additional value justifies its complexity?" A well-modelled charging site may be controlled effectively through MPC. A geographically distributed community with private observations and repeated local decisions may benefit from decentralised learning. A safety-critical action may still be projected through an optimisation or physics layer even when a learned policy proposes it.

2.4.1 Explicit Control: Rules, Optimisation, and Predictive Control

A RBC selects actions through predefined conditions. Examples include charging when a tariff falls below a threshold, reducing charger power when a site limit is approached, or activating a heat pump when temperature leaves a comfort band. Rules can be local or coordinated through a MAS, in which software agents exchange information and follow designed protocols.

Rule-based control is sometimes treated as a weak baseline because it does not learn. This is misleading. A small set of carefully designed rules can be reliable, inexpensive, easy to inspect, and suitable for constrained hardware. It can also provide a safe fallback when a learned controller is unavailable. Fonseca et al., for example, represent heterogeneous flexible loads through a common FlexOffer abstraction and compare explicit scheduling mechanisms in a REC without requiring RL (Fonseca, Ferreira, Landeck, et al. 2022). The result is a useful reminder that a common interface can solve part of the heterogeneity problem before learning is introduced.

The limitation is that rules express the situations anticipated by their designers. As the number of assets, contexts, and interactions grows, the rule set can become difficult to maintain. Rules that are reasonable independently can conflict when combined, and a threshold selected for one participant may be inappropriate for another. RBCs therefore remain important as transparent baselines, operational fallbacks, and lower-level protections, but they should not be assumed to provide long-horizon adaptation or coordination in every setting.

Mathematical optimisation and multi-agent optimisation. Mathematical optimisation expresses the task through decision variables, an objective, and constraints. In an energy community, decision variables can represent charger power, battery operation, flexible-load schedules, or energy exchanges. Constraints encode energy balances, device limits, deadlines, comfort, and network limits. When models and forecasts are adequate, the result is explicit and interpretable: feasibility can be checked and the effect of a new constraint can be inspected directly.

Optimisation is not inherently centralised. Decomposition and agent-based structures can distribute a problem while retaining mathematical solvers. Faia et al. organise an energy community as local agents and a coordinator, formulate the scheduling problem as mixed-integer linear optimisation, and report a large reduction in solution time relative to a centralised implementation for their high-EV-penetration case (Faia et al. 2023). This is a direct counterexample to the idea that a multi-agent architecture must be learning-based.

The practical burden lies in maintaining models, constraints, forecasts, and solver performance as the system changes. A new asset type may require new variables and equations. Integer decisions, network constraints, uncertainty, and participant-specific models can increase computation. These are problem-dependent limitations rather than universal reasons to prefer RL. If a model can be maintained and solved within the decision interval, a learning-based alternative must justify itself through adaptation, local autonomy, privacy, transfer, or lower online optimisation cost.

Model predictive and stochastic control. MPC repeatedly solves a finite-horizon optimisation problem using the latest measurements and predictions. Only the first action is applied; the horizon then moves forward. This receding-horizon structure is well suited to energy systems because prices, photovoltaic production, occupancy, and charging requests change during the day.

Srithapon and Månsson combine home-level MPC with community coordination and AC optimal power flow for EVs, heat pumps, thermal storage, photovoltaic generation, and network constraints (Srithapon and Månsson 2023). Puech et al. use MPC and two-stage stochastic programming to represent uncertain session starts, ends, requested energy, and early disconnections over 32 charging slots (Puech et al. 2024). These studies demonstrate that heterogeneous assets and dynamic EV sessions can be treated explicitly without MARL.

The strongest operational counterexample in the energy literature is the implementation by Hermans et al. A quadratic-programming MPC controller was integrated into a corporate microgrid with photovoltaic production and 174 EV chargers. Over three weeks, it achieved a reported 59% average daily reduction in grid peak power (Hermans et al. 2024). The system uses charger-occupancy and photovoltaic forecasts while the charger infrastructure remains fixed. This result prevents two overstatements: large EV populations do not intrinsically require MARL, and operational intelligent charging is not absent from the literature.

MPC and stochastic control require models, forecasts, and repeated solver execution. Their value is strongest when constraints must be explicit and the optimisation remains tractable. Learning and MPC are also complementary. A learned model can support prediction; a policy can supply a warm start; an optimiser can project a learned action into a feasible set; and an MPC or rule-based controller can remain as fallback.

2.4.2 Reinforcement Learning and Multi-Agent Control

Single-agent RL can act locally or centrally. A local agent may control one battery or building. A central agent may observe an entire charging site and produce a joint or aggregate action. Both are single-agent formulations even when many physical assets are affected.

Sadeghianpourhamami et al. use fitted Q-iteration to coordinate EV charging through a state representation designed not to grow directly with the number of charging stations. Simulations based on real charging data test generalisation to larger station sets (Sadeghianpourhamami et al. 2020). This is important for the thesis because it shows that variable session availability and size-independent coordination can sometimes be achieved without modelling every EV as a separate MARL agent.

Aggregate representations reduce dimensional growth, but they can remove information that matters to individual participants or to network constraints. A central policy also concentrates computation and may become difficult to deploy when actions must be produced by several local systems. Conversely, a local single-agent policy can be easy to deploy but may ignore shared limits. Single-agent RL is therefore a valid controller family and a necessary comparison, not merely an incomplete version of MARL.

Multi-agent, federated, and CTDE learning. MARL represents decision makers separately and models their interaction during learning. Agents can correspond to buildings, participants, charging sites, EVs, or selected assets. The representation enables local observations and actions while coordination is learned through shared rewards, centralised critics, communication, markets, or community signals.

The retained studies demonstrate several forms of coordination. GridLearn combines building energy models with distribution-network power flow and uses MARL to pursue building and voltage objectives (Pigott et al. 2022). Zhu et al. use centralised training with decentralised execution, soft actor–critic, attention, a counterfactual baseline, and a Lagrangian reward for multi-energy industrial-park management (Zhu, Yang, et al. 2022). Shojaeighadikolaei et al. compare centralised and decentralised critics for cooperative EV charging under CTDE (Shojaeighadikolaei et al. 2024). EnergAIze uses MADDPG for local prosumer objectives and community-level V2G and flexibility management (Fonseca, Ferreira, Cabral, Severino, and Praça 2024). Wilk et al. reduce data exchange by sharing strategic information during training and using local information during execution (Wilk et al. 2024).

Federated RL offers another decentralisation mechanism. Chu et al. coordinate a residential EV fleet while retaining local experiences and aggregating learned parameters (Chu, Wei, et al. 2022). This can reduce raw-data centralisation, but compatible model and interface structures are still required. Federated averaging does not by itself solve the case in which one agent observes temperatures, another controls charger current, and a third has a different action frequency.

These examples show why the label MARL is not enough. A method can be multi-agent while retaining a fixed population, homogeneous policy interfaces, ideal communication, or centralised deployment. The scientific comparison must state what is local, what is shared, how the agents differ, and what happens when composition or communication changes.

2.4.3 Structured, Federated, Transfer, Hybrid, and Hierarchical Approaches

Recent work introduces mechanisms that address selected limitations of fixed-vector and fixed-population controllers.

MERLIN trains independent offline policies in a digital twin of a real 17-building community and transfers policies between buildings with different occupant behaviour, reducing training effort while retaining comparable performance in the reported cases (Nweye, Sankaranarayanan, et al. 2023). Wu et al. use a Multi-Agent Transformer and adaptation modules for buildings that dynamically join or leave a VPP (Wu, Qiu, et al. 2024). GNN-DT combines a graph encoder with a Decision Transformer so that one architecture can process different charger counts and unseen action-space sizes (Orfanoudakis, Panda, et al. 2025).

Graph and set representations remove the need to concatenate every participant into one fixed vector, but they normally preserve a feature and action grammar. A new node can be added only if its attributes and actions can be expressed in the representation expected by the model. This is more flexible than fixed padding, but it is not arbitrary interface discovery.

HetGPS provides a particularly strong recent counterexample to simple scalability and safety claims. The August 2026 preprint combines parameter-shared heterogeneous-graph soft actor–critic with a learned risk model and a physics-anchored action projection. It evaluates 200–3218 EVs across nested distribution networks and keeps the learned parameter count constant across fleet sizes (Wang et al. 2026). It also demonstrates zero-shot topology transfer and runtime voltage protection. At the same time, graph computation and topology data still grow with system size, agent identities are fixed within each problem instance, and the work is not yet peer-reviewed or physically deployed.

The broader lesson is that modern energy control is increasingly hybrid. Learning can provide adaptive decisions, while explicit models, masks, optimisation, and runtime filters protect constraints or make the policy deployable. The complete control stack, rather than only the neural network, should be the unit of evaluation.

Centralised, decentralised, and hierarchical structures. Controller family and control structure are related but different. Optimisation can be centralised or distributed. Rules can be local or coordinated. RL can use one central policy, independent local policies, federated learning, or CTDE. A useful comparison therefore asks both how the action is computed and where information, computation, constraints, and responsibility reside.

Centralised control can exploit global information and coordinate shared constraints directly. It is often appropriate when communication is reliable, privacy constraints are manageable, and the global problem remains tractable. Decentralised control places decisions near assets, potentially reducing communication and preserving local operation, but local actions can collectively violate a shared limit. Hierarchical control combines both: local controllers preserve device and user requirements, while a community level allocates shared capacity or objectives.

Controller families can also be combined. A community optimiser can send budgets to local policies, a learned controller can propose an action checked by a physics layer, and a rule-based fallback can take over when monitoring rejects a policy. MARL is therefore one part of a hybrid control stack, not a synonym for complete decentralisation.

2.4.4 When MARL Adds Value across CPS Domains

The energy literature supports a more demanding standard for claims about the need for MARL. Beating uncontrolled charging or a simple threshold rule is not sufficient when a realistic optimisation or MPC baseline is feasible. The deployed 174-charger MPC study is especially important because it shows that a conventional model-based controller can operate at substantial physical scale (Hermans et al. 2024).

MARL is most strongly motivated when several conditions occur together: decisions are naturally distributed; participants possess local or private information; models are difficult to maintain; interactions repeat often enough to support learning; online solver budgets are restrictive; local

and community objectives must coexist; and the controller must adapt across configurations. Even then, MARL must demonstrate value relative to the engineering and assurance cost it introduces.

A simpler method can remain preferable when the system is small and stable, a trusted model is available, constraints require transparent guarantees, operational data are limited, or the optimiser meets the decision deadline. The later algorithm chapter will therefore compare controller families and will not present MARL as the default winner.

Table 2.5: Evidence-backed comparison of controller families in REC and EV control.

Family	**Representative evidence**	**Where it is strong**	**What must be justified or maintained**
Rules and explicit scheduling	FlexOffer-based scheduling of heterogeneous flexible loads (Fonseca, Ferreira, Landeck, et al. 2022).	Transparency, low runtime cost, fallback, and known operating logic.	Rule coverage, conflicts, maintenance, and long-horizon adaptation.
Optimisation and agent-based optimisation	Multi-agent MILP for high-EV REC scheduling (Faia et al. 2023).	Explicit objectives, feasibility, interpretable constraints, and strong baselines.	Model and formulation maintenance, solver time, and uncertain inputs.
MPC and stochastic control	Heterogeneous REC MPC/OPF, uncertain 32-slot charging, and operational control of 174 chargers (Hermans et al. 2024; Puech et al. 2024; Srithapon and Månsson 2023).	Feedback planning, explicit constraints, and uncertainty modelling.	Forecasts, models, repeated computation, and infeasibility handling.
Aggregate or single-agent RL	Size-independent EV coordination and central/community policies (Sadeghianpourhamami et al. 2020).	Model-free adaptation and avoidance of repeated online optimisation.	Information loss through aggregation, central action growth, and training data.
MARL, CTDE, and federated RL	GridLearn, industrial-park MARL, EV CTDE, federated charging, EnergAIze, and smart-community MARL (Chu, Wei, et al. 2022; Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Pigott et al. 2022; Wilk et al. 2024; Zhu, Yang, et al. 2022).	Local autonomy, coordination, privacy-aware execution, and mixed objectives.	Non-stationarity, compatible interfaces, communication, scale, and policy lifecycle.
Transfer, graph, and hybrid learning	MERLIN, dynamic-participation Transformer, GNN-DT, and HetGPS (Nweye, Sankaranarayanan, et al. 2023; Orfanoudakis, Panda, et al. 2025; Wang et al. 2026; Wu, Qiu, et al. 2024).	Reuse, variable-size representations, topology generalisation, and runtime protection.	Stable feature/action grammar, graph computation, evidence beyond simulation, and safety guarantees.

Evidence from other CPS domains. The same controller families appear in other CPS, although physical models and metrics differ. Robot teams combine trajectory optimisation, local

rules, learned policies, and MARL. Traffic systems use model-based scheduling, independent signal agents, coordinated learning, masks, and operational guards. Industrial systems combine production optimisation with local machine controllers and fault-handling logic.

These comparisons support the generality of the design questions, not direct policy transfer. A policy trained for EV charging cannot control a robot by changing a label. What can transfer are abstractions and questions: how capabilities are represented, how information is shared, how a changing population is processed, how constraints are enforced, and how policies are validated and deployed.

This comparison sets the standard used throughout the thesis. RQ1 cannot be answered simply by showing that MARL can coordinate several agents; the formulation must justify why separate decision makers and changing interfaces matter. RQ2 requires resource and performance evidence rather than agent count alone. RQ3 requires explicit treatment of constraints and disturbances, including mechanisms outside the learned policy when appropriate. RQ4 requires the resulting controller to remain identifiable, compatible, and governable after training. These requirements motivate the formal concepts introduced next.

2.5 Foundations of Reinforcement Learning and MARL Used in This Thesis

The controller comparison above motivates the use of reinforcement learning and MARL in selected parts of the thesis. This section introduces only the formal concepts and notation needed to understand those formulations and the later TI-MARL method. It is not intended as a comprehensive treatment of reinforcement learning, game theory, or multi-agent systems; broader introductions and surveys are available in Sutton and Barto (2018), Oliehoek and Amato (2016), Hernandez-Leal et al. (2019), Gronauer and Diepold (2022), and Albrecht et al. (2024).

The common starting point is sequential decision-making. An action changes not only the present state but also the decisions available later. Charging a battery now can reduce a local electricity bill while leaving less storage capacity for later photovoltaic production; delaying an EV charge can avoid a community peak only while sufficient time remains before departure. The concepts below provide the formal language used to represent those long-term effects, partial information, interacting agents, and centralised-training/decentralised-execution structures.

2.5.1 Sequential Decision-Making and Partial Observability

Markov processes and Markov decision processes. A Markov process describes a system that moves between states according to transition probabilities. Its defining idea is the Markov property: once the current state is known, the probability of the next state does not depend on the complete sequence of earlier states. This does not mean that the past is irrelevant. It means that

the state representation is assumed to contain the information from the past that is needed for future prediction.

A simple battery example helps clarify this point. If the state contains only the current electricity price, it is not sufficient for deciding whether to charge. The decision also depends on the battery's current SoC, its power limits, and possibly the expected future demand. If these relevant quantities are included in the state, the controller no longer needs the entire history of earlier charging actions to understand the immediate consequences of the next action. The quality of the Markov assumption therefore depends on the quality of the state representation.

An MDP adds decisions and rewards to this process. A discounted MDP is commonly represented by the tuple

$$\mathscr{M} = \langle \mathscr{S}, \mathscr{A}, P, R, \gamma \rangle, \tag{2.1}$$

where $\mathscr{S}$ is the state space, $\mathscr{A}$ is the action space, $P(s' \mid s,a)$ is the probability of reaching state s' after action a is selected in state s, $R(s,a,s')$ defines the immediate reward, and $\gamma \in [0,1]$ discounts rewards obtained further in the future (Sutton and Barto 2018).

At each decision step t, the agent observes a state S_t, selects an action A_t, receives a reward R_{t+1}, and the environment moves to S_{t+1}. A policy $\pi(a \mid s)$ specifies the probability of selecting action a in state s. The objective is usually expressed through the expected return

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \tag{2.2}$$

The state-value function $V^\pi(s)$ and action-value function $Q^\pi(s,a)$ describe the expected return obtained by following policy π from a state or state–action pair. These functions connect local decisions to long-term consequences. A charging action with a small immediate cost can still be valuable if it prevents a larger cost later.

The MDP model is useful because it separates the elements of the decision problem. States describe what the controller knows, actions describe what it can change, transitions describe the environment dynamics, and rewards describe the objective. This separation is also useful when a control problem is implemented in software: observation and action interfaces can be checked independently from the learning algorithm.

The same separation exposes several modelling risks. A reward is not the complete physical objective of the system. It is a numerical signal designed to guide behaviour. If a community reward penalises electricity cost but ignores a charger's minimum departure requirement, an agent can obtain a good reward by leaving the vehicle insufficiently charged. The missing requirement should be represented as a constraint or included explicitly in the objective. Similarly, a state that omits a relevant variable can make the problem appear unpredictable even when the underlying physical system is not.

Finally, the standard MDP contains one decision-making agent. Other actors can be hidden inside the transition dynamics, but their behaviour is then treated as part of the environment. This

becomes problematic when several actors are learning and adapting at the same time. Markov games, introduced in Section 2.5.3, extend the model to that setting.

Partial observability and sequential decision-making. The state of a physical system is rarely available to a controller in full. Sensors cover only selected variables, measurements contain noise, data may arrive late, and some information is intentionally kept local. A charger can observe the connected vehicle and the power available at its installation, but it may not know the exact flexibility of every other participant in the community. A local building controller can measure indoor temperature without knowing the complete thermal state of the building envelope.

A POMDP separates the underlying state from the observation available to the agent. It extends the MDP with an observation space $\mathcal{O}$ and an observation model $O(o \mid s)$, or a related formulation in which the observation also depends on the previous action. The agent selects actions from observations or from an internal representation of their history rather than from the true state directly (Kaelbling et al. 1998).

In the classical formulation, a belief state represents a probability distribution over possible underlying states. The belief is updated when a new action and observation are received. In practical deep reinforcement learning, recurrent networks, finite observation windows, learned latent states, or explicit forecasts may serve a similar purpose without maintaining an exact probability distribution.

Partial observability matters for two reasons. First, the same observation can correspond to different physical situations. A household consuming 3 kW may be close to a local limit or may still have considerable spare capacity, depending on other unobserved loads. Second, information sharing changes the decision problem. Providing every agent with the complete community state can improve coordination during an experiment, but it may be unrealistic in deployment because of communication, privacy, latency, or ownership constraints.

When several agents act under partial observability, the formal problem can be represented as a Dec-POMDP. In a Dec-POMDP, agents cooperate under a common objective but receive local observations and select actions using decentralised policies (Oliehoek and Amato 2016). Markov games are more general with respect to individual rewards, while Dec-POMDPs are particularly useful for cooperative planning under uncertainty. The two perspectives overlap in many practical MARL systems: training may use global information, while execution relies on local observations.

Execution-time information is an architectural and algorithmic choice. A policy that requires every global variable may perform well in simulation but be difficult to deploy; a policy based only on local observations is easier to place near the asset but may require stronger coordination mechanisms. This information boundary later affects scalability, deployability, safety, and robustness.

2.5.2 Reinforcement Learning and Actor–Critic Methods

RL addresses sequential decision-making through interaction. Rather than receiving a labelled correct action for every state, an agent acts, observes consequences, and adjusts its policy to increase expected return (Sutton and Barto 2018). This is attractive when a useful control strategy is difficult to write as a complete set of rules, when decisions have delayed consequences, or when the environment contains uncertainty that is difficult to model exactly.

The interaction loop is simple to describe:

1. the agent receives an observation;
2. the policy selects an action;
3. the environment changes;
4. a reward and a new observation are produced;
5. the agent updates its estimates or policy during training.

The difficulty is hidden inside each step. The observation may be incomplete, exploration may be unsafe, useful rewards may be delayed, and the data collected by the current policy may differ from the data that will be seen after deployment.

RL methods are often grouped according to what they learn. Value-based methods estimate the expected return of states or actions and derive a policy from those estimates. Policy-based methods optimise a parameterised policy directly. Actor–critic methods combine a policy, the actor, with a value estimate, the critic. Model-free methods learn without an explicit transition model, while model-based methods learn or use a model to plan. On-policy methods learn primarily from data generated by the current policy; off-policy methods can reuse data generated by other behaviour policies.

These distinctions are useful, but they do not by themselves determine whether a method is appropriate for a CPS. An off-policy method may improve data reuse, but stale experience can be difficult to interpret when other agents are also changing. A policy-gradient method may support continuous actions, but it still needs mechanisms for constraint satisfaction. A model-based controller can use physical knowledge, but the learned or specified model may be inaccurate outside the training distribution.

Energy management illustrates both the attraction and the risk of RL. The controller can learn to shift consumption without requiring an exact closed-form model of every user and device. It can also account for future prices, renewable production, and vehicle departures. Reviews have consequently reported extensive interest in RL for demand response, buildings, and power systems (Cao et al. 2020; Chen, Qu, et al. 2022; Vázquez-Canteli and Nagy 2019). At the same time, the cost of exploration, the need for large amounts of interaction data, and the consequences of invalid actions make direct learning on physical infrastructure difficult. Simulation therefore becomes

central, but it also introduces a gap between the environment used for learning and the system used for operation.

Real-world RL research identifies additional conditions that are often simplified in benchmarks, including delays, constraints, changing objectives, non-stationarity, limited data, explainability requirements, and the need to act safely while learning (Dulac-Arnold et al. 2021). These conditions become more difficult when several learning agents share the environment.

2.5.3 Markov Games, MARL, and CTDE

Stochastic games extend sequential decision-making to several players whose actions jointly influence the state and rewards. The model was introduced by Shapley (1953) and later adopted as a foundation for multi-agent reinforcement learning by Littman (1994). A Markov game can be represented by

$$\mathscr{G} = \langle \mathscr{N}, \mathscr{S}, \{\mathscr{A}_i\}_{i\in\mathscr{N}}, P, \{R_i\}_{i\in\mathscr{N}}, \gamma \rangle, \tag{2.3}$$

where $\mathscr{N} = \{1,\dots,N\}$ is the set of agents, $\mathscr{S}$ is the state space, $\mathscr{A}_i$ is the action space of agent i, P defines transitions from the current state and joint action, and R_i defines the reward of each agent. The joint action is

$$\boldsymbol{a} = (a_1, a_2, \dots, a_N) \in \mathscr{A}_1 \times \mathscr{A}_2 \times \cdots \times \mathscr{A}_N. \tag{2.4}$$

The transition depends on the combined decisions. A household may reduce charging, but the resulting community peak also depends on what every other participant does. Each agent can therefore affect the experience and learning signal of the others.

This creates the central difficulty of MARL: from the perspective of one learning agent, the environment is no longer stationary because the policies of the other agents change during training. In a single-agent MDP, the transition model is assumed to be stable. In MARL, an identical local action can produce different outcomes as other agents learn new behaviours. The agent is effectively trying to learn while part of the environment is learning back.

Early MARL work compared independent learners, which treat other agents as part of the environment, with cooperative agents that exchange information, experience, or policies (Tan 1993). Other foundational work distinguished agents that learn individual action values from agents that model joint actions, showing how exploration and game structure can lead to different equilibria even under a shared reward (Claus and Boutilier 1998). Modern deep MARL extends these ideas with neural policies, centralised critics, value factorisation, learned communication, role representations, and other mechanisms (Albrecht et al. 2024; Gronauer and Diepold 2022; Hernandez-Leal et al. 2019).

A standard Markov game normally assumes a fixed set of agents and defined action spaces. Real systems can violate both assumptions. If an electric vehicle leaves, an action that previously existed may disappear. If a new type of charger is added, its admissible actions and observations

may differ from those of existing agents. These changes are not fully represented by simply changing the numerical state. They motivate explicit attention to dynamic populations, heterogeneous interfaces, and policy compatibility.

Cooperative, competitive, and mixed-motive settings. Multi-agent problems are often described as cooperative or competitive. In a fully cooperative setting, agents share a common reward or optimise the same team objective. In a two-player zero-sum setting, one agent's gain is the other's loss. Many real systems lie between these extremes and are better described as general-sum or mixed-motive games (Lowe et al. 2017; Wang et al. 2022).

An energy community provides a useful example. Participants may cooperate to increase collective self-consumption, reduce a peak, or avoid carbon-intensive electricity. At the same time, each participant may seek to minimise an individual bill, preserve comfort, and keep enough flexibility for personal needs. Two households can therefore cooperate on a community objective while competing indirectly for a limited shared capacity. The word *competitive* does not imply malicious behaviour; it means that individual objectives are not always perfectly aligned.

The distinction affects problem formulation. If every agent receives only the same global reward, individual disadvantages can be hidden. A policy might reduce the community peak by repeatedly curtailing one participant. Conversely, if every agent optimises only its own cost, the community may experience synchronised behaviour, such as many vehicles charging at the beginning of a cheap tariff period. Mixed objectives require an explicit design choice regarding how local and collective terms are represented, weighted, constrained, or negotiated.

A higher total reward does not automatically imply a fair or acceptable outcome for all participants. Community-level and individual-level metrics are therefore reported together. Where relevant, distributional measures and participation constraints expose outcomes hidden by an aggregate reward; a complete fairness analysis remains an adjacent research problem.

Centralised training and decentralised execution. CTDE is a common response to the tension between coordination and deployment. During training, a critic or learning process may access the global state, joint actions, or information from several agents. During execution, each agent selects actions using only its local policy and the observations available at its deployment location. This pattern was used in mixed cooperative–competitive actor–critic learning by Lowe et al. (2017) and in cooperative methods such as COMA and QMIX (Foerster et al. 2018; Rashid et al. 2018).

The appeal of CTDE is easy to see in an energy community. Historical data from all participants may be available in a secure training environment. A centralised critic can use this information to learn how local actions affect the community. The resulting actor policies can then be deployed to local controllers, where they operate without continuous access to every participant's raw data.

CTDE does not remove the deployment problem. The decentralised policies still need compatible observation and action interfaces, sufficiently small inference cost, appropriate packaging, and

a process for coordinated updates. Training may also rely on agent identities, a fixed population, or global variables that are not stable over time. A method can therefore satisfy decentralised execution in an algorithmic sense while remaining difficult to install and maintain across heterogeneous devices.

The distinction between training information and execution information must also be respected experimentally. A decentralised policy should not be evaluated with observations that are only available to the training critic. Otherwise, the experiment measures a more informed controller than the one that can be deployed. This thesis treats the information boundary as part of the system architecture rather than as a minor implementation detail.

2.5.4 MARL Challenges Relevant to This Thesis

Several challenges recur across MARL research (Albrecht et al. 2024; Gronauer and Diepold 2022; Hernandez-Leal et al. 2019). They are introduced here because later sections show how CPS conditions make them more severe.

Non-stationarity. Each agent changes its policy during learning, altering the effective environment experienced by the others. Experience collected earlier can become less representative of the current joint behaviour.

Credit assignment. When a community reward improves, it may be unclear which agent actions caused the improvement. Poor credit assignment can slow learning or encourage agents to rely on the efforts of others. Methods such as counterfactual baselines and value factorisation were developed partly to address this problem (Foerster et al. 2018; Rashid et al. 2018).

Combinatorial growth. The joint action space grows as the product of individual action spaces. Ten agents with five actions each already produce 5^{10} possible joint actions. Continuous actions and heterogeneous interfaces make the problem more complex still.

Partial observability. Local policies act with incomplete information. Communication can reduce uncertainty, but it introduces bandwidth, latency, reliability, and privacy considerations.

Exploration and sample efficiency. The number of possible joint behaviours is large, useful coordination may be rare, and physical systems cannot support arbitrary trial and error. Training often requires extensive simulated interaction.

Heterogeneity. Parameter sharing is effective when agents are sufficiently similar, but agents controlling batteries, chargers, and heat pumps do not necessarily share observations, actions, dynamics, or objectives. A shared policy may need explicit type or role information, while separate policies increase training and lifecycle complexity.

Dynamic populations and generalisation. Policies trained for a fixed group may fail when agents join, leave, or change capabilities. Generalisation across team size, topology, asset type, or operating distribution becomes part of robustness rather than an optional extension.

Evaluation. A high average return can hide unstable training, unequal individual outcomes, constraint violations, or excessive communication. Reliable evaluation requires repeated runs, appropriate baselines, uncertainty reporting, and metrics that reflect the physical task.

Table 2.6 connects these challenges to the recurring energy example and to later thesis concerns.

Table 2.6: Recurring MARL challenges and their manifestation in energy communities.

Challenge	**Example in an energy community**	**Later concern**
Non-stationarity	The value of delaying one vehicle depends on how other charging policies are changing	Learning stability and robustness
Credit assignment	A reduced peak results from several participants shifting load at different times	Individual and community objectives
Combinatorial growth	More buildings and assets increase joint observations and actions	Algorithmic and computational scalability
Partial observability	A local controller does not observe every community asset or user preference	Information architecture and communication
Sample efficiency	Unsafe or inconvenient actions cannot be explored freely on physical assets	Simulation, safety, and sim-to-real transfer
Heterogeneity	Batteries, chargers, and heat pumps expose different interfaces and dynamics	Policy representation and deployability
Dynamic population	Vehicles and participants can appear, disappear, or become unavailable	Reconfiguration and generalisation
Evaluation	Average reward can hide missed departures or local constraint violations	Multi-level evidence and validation

These challenges do not by themselves establish the thesis gaps. They are properties of the learning problem, and many have mature solution families. The critical question for RQ1–RQ4 is which assumptions remain when these mechanisms are placed back into the distributed, heterogeneous, physically constrained, and operational setting already established in Sections 2.2–2.4. The critical synthesis in Section 2.7 makes that distinction explicit.

2.6 From Simulation to Operational Evidence

Learning-based control requires experience, but physical systems cannot safely or economically provide unlimited trial and error. Simulation is therefore a central experimental instrument for this thesis: it supports training, controlled comparison, scaling studies, deliberate disturbance injection, and experiments with configurations that would be unsafe, expensive, or unavailable in a physical demonstrator. The same strength can also hide the assumptions that matter most in operation. Perfect observations, fixed populations, instant communication, deterministic devices, and unconstrained compute can make an algorithm appear more operationally mature than the evidence supports.

Simulation quality must therefore be interpreted relative to the claim being tested. A simplified environment can be entirely appropriate for isolating a learning mechanism, while a deployment claim requires stronger evidence about interfaces, timing, physical limits, hardware, communication, and recovery. Fidelity is not the number of equations in a model; it is the extent to which the mechanisms that could change the conclusion are represented with sufficient accuracy. Reproducibility is equally important: a result should identify the scenario, environment version, data, randomisation, controller configuration, and evaluation procedure needed to reproduce it.

The detailed state of the art in energy, EV-charging, grid-aware, building-physics, and co-simulation environments is presented in Chapter 4, where it directly positions the extended CityLearn-derived environment developed in this thesis. That comparison includes platforms such as CityLearn, SustainGym, EV2Gym, PowerGridworld, CommonPower, BOPTEST, PowerGym, Chargax, and HELICS, but their capabilities are interpreted according to the claims they can support rather than treated as interchangeable benchmarks.

The empirical chapters use the evidence levels in Table 2.7 to keep this distinction explicit. The levels are descriptive rather than a maturity ranking: an experiment does not become invalid because it remains in simulation, but its conclusions must remain commensurate with that evidence.

This evidence distinction supports all four research questions. RQ1 and RQ2 rely heavily on simulation because controlled heterogeneity, population change, and scale are difficult to reproduce physically. RQ3 requires disturbance and failure injection that would often be irresponsible to create on operational assets. RQ4 requires progressively stronger evidence as policies move from reproducible experiments to packaging, edge execution, monitoring, updates, and recovery. Simulation is therefore part of the path to real-world control, but it is never used as a synonym for it.

2.7 Critical Analysis of the State of the Art

The preceding sections establish the system class, the principal energy-domain instantiation, the available controller families, the formal MARL concepts used by the thesis, and the distinction

Table 2.7: Evidence levels used throughout the thesis for real-world-oriented control studies.

Evidence level	Interpretation used in this thesis
Domain simulation	The task represents a physical domain, but sensing, actuation, communication, and timing remain simulated.
High-fidelity simulation or digital replica	The environment is calibrated or physically detailed, but the controller does not close the loop with physical assets.
Hardware-in-the-loop or mixed reality	Some physical controllers, devices, or agents interact with simulated components.
Physical testbed	The controller acts on physical assets in an experimental setting, without a claim of sustained operational service.
Shadow operation	The controller receives operational data and produces decisions, but another controller remains responsible for actuation.
Operational closed-loop deployment	The controller participates in sustained operation under real interfaces and constraints.

between simulated and operational evidence. The remaining task is to determine what the literature leaves unresolved after the strongest counterexamples are taken seriously. This synthesis is organised directly around RQ1–RQ4 so that the gaps retained here correspond to the questions stated in Chapter 1.

The evidence does not support claims that dynamic populations, large-scale control, safety mechanisms, sim-to-real transfer, or operational deployment are absent from MARL and related control research. Strong examples exist in each category. The residual problem is narrower: these capabilities are usually demonstrated under different assumptions, interfaces, scales, and evidence levels. Table 2.8 records several broad claims that are rejected and the scoped conclusions that replace them.

The simulation literature requires the same restraint. Chapter 4 shows that current environments already provide EV session dynamics, V2G, grid constraints, large physical networks, accelerated simulation, safeguards, and dynamic co-simulation. The relevant limitation is therefore not the absence of these individual capabilities, but the difficulty of evaluating dynamic composition, heterogeneous interfaces, physical validity, safety, scale, and reproducibility together under a common control contract.

2.7.1 RQ1: Dynamic and Heterogeneous Multi-Agent Control

Dynamic and heterogeneous control combines several changes that are often studied separately. An agent population can vary while all agents remain identical. Agents can be heterogeneous while the population remains fixed. A communication graph can change while observation and action interfaces remain unchanged. EV sessions can begin and end while every charger remains

Table 2.8: Overly broad claims, counterevidence, and defensible replacements.

Tempting claim	**Assessment**	**Counterevidence**	**Defensible replacement**
MARL has not been deployed in real-world CPS.	False	The I-24 programme used 67 operational VSL controllers (Zhang, Zhang, Ji, et al. 2025; Zhang, Zhang, Quiñones-Grueiro, et al. 2024).	Operational deployment is possible, but remains exceptional and concentrated in very few documented programmes.
Dynamic MARL populations have not been studied.	False	Varying vehicle populations, robot loss, changing team size, and time-varying graphs have been studied (Sebastián et al. 2025; Shibata et al. 2023; Su et al. 2020; Yu, Yang, et al. 2023).	Persistent operation with heterogeneous capabilities and online entry, exit, or role change remains weakly demonstrated.
MARL does not scale.	Too broad	Studies report 2–64 buildings, teams larger than training, and 67 field controllers (Almilaify et al. 2023; Sebastián et al. 2025; Zhang, Zhang, Quiñones-Grueiro, et al. 2024).	Scale is demonstrated along individual dimensions; multi-dimensional scalability remains less established.
Safety is ignored in MARL.	False	Constrained learning, physics shields, action masks, and runtime guards already exist (Chen, Liu, et al. 2023; Gu et al. 2023; Zhang, Dehghanpour, et al. 2021; Zhang, Zhang, Quiñones-Grueiro, et al. 2024).	Safety mechanisms exist, but are rarely assessed jointly with faults, reconfiguration, communication imperfections, and deployment shift.
Sim-to-real MARL has not been demonstrated.	False	Physical autonomous-driving, robot, and heating studies exist (Blad et al. 2023; Candela et al. 2022; Yu, Yang, et al. 2023).	Physical transfer exists, but testbed evidence must not be conflated with sustained operational deployment.

a permanent controller slot. Each relaxation is useful, but none alone represents the complete problem of an operational CPS.

The cross-domain literature contains clear counterexamples to a claim that dynamic teams are unexplored. Su et al. formulate traffic coordination with a varying number of connected vehicles and a random proportion of connected participants (Su et al. 2020). Yu et al. design an asynchronous, team-size-invariant exploration policy partly to accommodate unequal action duration and robot loss (Yu, Yang, et al. 2023). Shibata et al. consider cooperative transport with event-triggered communication and agent counts that differ from training (Shibata et al. 2023). Sebastián et al. use a policy representation for time-varying robot interaction graphs and demonstrate teams substantially larger than those seen during training (Sebastián et al. 2025).

The mechanisms used in these studies are not interchangeable. Table 2.9 separates four representative approaches. The first preserves a fixed neural-network shape by padding absent agents. The second learns a representation that can aggregate a variable set and explicitly tests agent loss during an episode. The third combines local consensus with event-triggered communication so that execution can use a different number of robots from training. The fourth embeds the interaction graph directly in a shared attention-based policy. Each approach solves a real part of the problem, but each also keeps part of the agent–system contract stable.

Table 2.9: Representative MARL mechanisms for varying team size, agent loss, or changing interaction graphs.

Study	**MARL basis**	**Mechanism used**	**What it demonstrates and retains fixed**
Su et al. (2020)	Multi-agent actor–critic with decentralised actors and centralised critics	A fixed local observation uses the closest neighbouring vehicles; artificial entries pad missing observations and a fixed maximum number of actor–critic pairs pads the population.	Supports varying traffic population and connected-vehicle penetration in simulation. The maximum slots, feature meanings, vehicle actions, and controller architecture remain fixed.
Yu, Yang, et al. (2023)	ACE, extending MAPPO to asynchronous execution	Action-delay randomisation; team-size-invariant feature extraction; attention-based fusion of a variable set; low-dimensional feature exchange; explicit robot-loss experiments.	Demonstrates loss of agents during an episode and real-robot asynchronous execution. Robots remain largely homogeneous and expose the same observation and action semantics.
Shibata et al. (2023)	Event-triggered communication and consensus-based MARL	Each robot estimates resultant force and torque through local consensus and jointly decides control and when to communicate.	Demonstrates execution with robot counts different from training and real cooperative transport. The task, payload-control semantics, and minimum communication structure remain predefined.
Sebastián et al. (2025)	Soft actor–critic with a self-attention port-Hamiltonian policy	A shared physics-informed policy represents interactions through a time-varying graph; self-attention aggregates a variable neighbourhood without a fixed joint vector.	Demonstrates larger teams and changing interaction graphs. Robot roles, node features, and action meanings remain compatible with the learned policy.

The energy review sharpens this distinction. Most EV studies represent *session-level dynamics*: users connect and disconnect while charger or agent slots remain fixed (Hermans et al. 2024; Puech et al. 2024). Sadeghianpourhamami et al. and GNN-DT address *variable instance size* through aggregate or graph-compatible representations, but the change occurs between evaluated configurations rather than through arbitrary controller creation during one operational episode (Orfanoudakis, Panda, et al. 2025; Sadeghianpourhamami et al. 2020). HetGPS combines heterogeneous EV types, graph coordination, and session availability at very large simulated scales, but retains a fixed agent set within each instance (Wang et al. 2026).

Wu et al. are the strongest direct energy-domain counterexample. Their Multi-Agent Transformer with adaptation modules explicitly represents buildings that join or leave a VPP and is

evaluated across dynamic configurations of three to nine buildings (Wu, Qiu, et al. 2024). This study means that the thesis cannot claim that dynamic energy participants are absent from MARL research. It also helps define the remaining boundary: the participating buildings still belong to a predefined family of representations, actions, and roles.

The environment literature, developed in detail in Chapter 4, reaches the same conclusion from a different direction. SustainGym contains realistic EV arrivals and departures while retaining a fixed EVSE-agent set (Yeh et al. 2023). EV2Gym and CityLearn/EVLearn can instantiate different system sizes and represent connection states, but this normally occurs over predefined asset and interface schemas (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Nweye, Kaspar, et al. 2025; Orfanoudakis, Diaz-Londono, et al. 2025). HELICS demonstrates online entry and exit at co-simulation-federate level, yet leaves MARL observation, action, policy, and safety compatibility to the application (Hardy et al. 2024). These counterexamples make the policy–environment contract, rather than component creation alone, the central issue.

The residual gap is the joint and persistent case in which agents differ in physical capabilities, observation and action interfaces, objectives, timing, or hardware; their availability changes during operation; and joining, leaving, failing, recovering, or changing role does not require rebuilding the complete controller. Targeted studies solve important pieces, but the cited literature provides little mature operational evidence for this combined setting.

This residual gap is classified as **strong when stated jointly**. Targeted studies solve important pieces, but the cited literature does not provide a mature body of operational evidence for open heterogeneous agent populations. The gap motivates RQ1 and constrains RQ2: population scaling must be tested together with composition and reconfiguration, not only by increasing a fixed set of homogeneous agents.

2.7.2 RQ2: Multi-Dimensional Scalability of Learning and Coordination

Scalability is not a binary property. A method can scale in agent count while requiring global communication. It can reduce communication while increasing training time. It can keep the number of learned parameters fixed while graph processing and environment simulation continue to grow. It can run a large experiment on a cluster but exceed the memory or latency budget of an edge device. A useful assessment must therefore identify what grows, what remains fixed, and which outcome is preserved.

Meaningful counterexamples already exist across CPS domains. SCALEX explicitly compares building-control configurations from 2 to 64 buildings (Almilaify et al. 2023). Charbonnier et al. design distributed residential coordination to avoid the growth and information-sharing requirements of a fully centralised controller (Charbonnier et al. 2022). PowerNet exploits locality and learned neighbour communication for distributed grid control (Chen, Chen, et al. 2022). Large-scale traffic-signal control has been evaluated with local information structures (Chu, Wang, et al.

2020). Sebastián et al. test robot teams larger than those used during training and physical experiments with up to 16 robots (Sebastián et al. 2025). The I-24 deployment provides operational evidence from 67 field controllers (Zhang, Zhang, Quiñones-Grueiro, et al. 2024).

The energy literature adds several stronger population-scale counterexamples. Sadeghianpourhamami et al. design a state representation independent of charger count and evaluate generalisation to larger station sets (Sadeghianpourhamami et al. 2020). GNN-DT is evaluated at changing charger counts, including a 250-charger case, by combining graph embeddings with an offline Decision Transformer (Orfanoudakis, Panda, et al. 2025). The recent HetGPS preprint evaluates 200–3218 EVs, retains a constant learned parameter count through parameter sharing, and transfers a policy to larger distribution-network topologies (Wang et al. 2026). Hermans et al. provide a different form of scale: 174 physical chargers controlled operationally through MPC rather than MARL (Hermans et al. 2024). These studies make population count alone an indefensible research gap.

None of the results proves general scalability, nor should it be expected to. Each demonstrates selected dimensions while holding others sufficiently controlled. In GNN-DT, the feature and action grammar remains graph-compatible. In HetGPS, learned parameter count remains fixed, but node-wise computation, topology data, and simulation burden still grow. In the deployed MPC study, the infrastructure is large but predeclared and centrally modelled. The residual problem is therefore *multi-dimensional scalability*: whether acceptable control performance and resource use persist when agent count, interface heterogeneity, observation and action complexity, communication demand, computational limits, environment complexity, and reconfiguration increase together.

The evidence supports a **moderate residual gap**. RQ2 asks under which conditions algorithmic, computational, and communication mechanisms maintain acceptable performance and resource use as several dimensions grow together. The empirical evaluation therefore separates simulator throughput, training and sample cost, communication, inference latency, orchestration, deployment resources, and reconfiguration behaviour before combining them into a scalability envelope.

The environment literature adds a further warning against collapsing these dimensions. Chargax reports very large simulator-throughput gains in its evaluated JAX configurations (Ponse et al. 2025). PowerGym includes an 8500-node feeder, demonstrating physical network scale (Fan et al. 2022). BOPTEST provides standard high-fidelity building emulators and has evolved toward vectorised experiment execution (Blum et al. 2021). EV2Gym exposes configurable EV, charger, and transformer topology (Orfanoudakis, Diaz-Londono, et al. 2025). These are all scalability results, but they measure different resources and preserve different assumptions. Chapter 4 and Chapter 7 must therefore report environment, learning, communication, orchestration, and deployment scale separately before discussing their interaction.

2.7.3 RQ3: Safe and Robust Operation

Safety and Robustness answer different questions. Safety concerns unacceptable states or actions. Robustness concerns how behaviour changes when assumptions are violated. A controller may satisfy voltage limits under nominal communication and still fail when observations are delayed. Conversely, a controller may tolerate noise while occasionally violating a mobility or physical requirement.

Safety is already an explicit MARL research area. Safe policy learning for networked microgrids incorporates AC power-flow and operational constraints (Zhang, Dehghanpour, et al. 2021). Physics-shielded voltage control constrains learned actions using a model-based safety layer (Chen, Liu, et al. 2023). Constrained multi-agent policy optimisation has been evaluated for safe multi-robot control (Gu et al. 2023). Zhu et al. include storage constraints through a Lagrangian reward in multi-energy MARL (Zhu, Yang, et al. 2022). The I-24 operational stack combines the learned policy with invalid-action masking and runtime guards (Zhang, Zhang, Ji, et al. 2025; Zhang, Zhang, Quiñones-Grueiro, et al. 2024). These examples show that Safety can belong to the complete control stack rather than to the learned policy alone.

The energy literature also contains robustness mechanisms outside MARL. Puech et al. model uncertain charging-session start and end times, requested energy, and early disconnection through MPC and stochastic programming (Puech et al. 2024). Hermans et al. integrate occupancy and photovoltaic forecasting in an operational MPC system (Hermans et al. 2024). HetGPS separates a learned task policy from a graph risk model and a physics-anchored action projection, reducing voltage violations across large simulated networks (Wang et al. 2026). As a recent preprint, this is important counterevidence but not a formal guarantee or deployment result.

Simulation frameworks also encode safety at different layers. CommonPower provides symbolic models, MPC baselines, and automatically synthesised safeguards around RL and MARL controllers (Eichelbeck et al. 2026). PowerGridworld and PowerGym expose grid variables and voltage-related physical constraints through power-flow models (Biagioni et al. 2022; Fan et al. 2022). These environments show that constraint-aware experimentation is already possible. They do not remove the need to test whether a safety mechanism remains valid under missing observations, communication faults, changing interfaces, or online reconfiguration.

Robustness has likewise been studied against selected disturbances in other CPS. Asynchronous robot exploration introduces action-delay randomisation and robot-loss handling (Yu, Yang, et al. 2023). Sim-to-real autonomous driving uses domain randomisation (Candela et al. 2022). Physics-informed multi-robot policies are evaluated under imperfect communication and changing team sizes (Sebastián et al. 2025). Real REC work reports a different set of problems: incomplete and noisy measurements, missing EV state of charge, heterogeneous subsystem integration, and synchronisation difficulties (Fonseca, Sousa, et al. 2025). These observations are operationally important even when they are not yet part of a complete controlled stress-test campaign.

The residual gap is not the absence of safe or robust methods. It is the limited systematic evidence for safe operation under the *combination* of uncertainty, missing or delayed information,

communication failures, agent or device faults, dynamic reconfiguration, unseen configurations, and train-to-operation shift. A paper may demonstrate voltage constraints but not agent loss; robustness to early disconnection but not interface change; or physical transfer without a defined fault campaign.

This residual gap is classified as **moderate**. RQ3 is defensible as an integration and evaluation question: how constraints are incorporated and how acceptable operation is assessed when several real-world disturbances occur, with the evidence level and protection layer stated explicitly.

2.7.4 RQ4: Governed and Deployable MARL Lifecycle

The literature does not support the claim that RL lacks lifecycle tooling or that MARL has never reached an operational CPS. Physical testbeds exist in autonomous driving, multi-robot systems, and building control (Blad et al. 2023; Candela et al. 2022; Shibata et al. 2023; Yu, Yang, et al. 2023). The I-24 programme provides the strongest counterexample: a simulation-trained MARL controller was integrated with real traffic-management infrastructure, deployed over 67 variable-speed-limit controllers, protected by invalid-action masking and independent runtime safety guards, and assessed over millions of decisions and several months of operation (Zhang, Zhang, Ji, et al. 2025; Zhang, Zhang, Quiñones-Grueiro, et al. 2024).

Operational intelligent energy control also exists, although the controller is not always MARL. Hermans et al. deploy MPC over 174 chargers (Hermans et al. 2024). Patrício et al. deploy a non-learning multi-agent community energy-management system with second-life batteries in a Portuguese REC (Patrício et al. 2026). Gonçalves et al. describe an RL-based MLaaS architecture with parallel training and an autoscalable inference service (Gonçalves et al. 2025). Fonseca et al. report preliminary evidence from an operational four-building REC and document practical data and integration problems (Fonseca, Sousa, et al. 2025). These examples reject an absolute claim that intelligent REC, EV, or MARL control has not reached real systems.

The lifecycle review further shows that the missing capability is not generic MLOps. An explicit RLOps lifecycle already covers specification, development, validation, production serving, monitoring, reproducibility, and safety/security for RL (Li, Thomas, et al. 2022). MLflow provides a general precedent for experiment lineage, reproducibility, and deployment-oriented model management (Zaharia et al. 2018). Distributed RL frameworks such as RLlib and Acme support scalable actor–learner execution, while MARLlib and MALib provide flexible agent–policy mappings and policy-population orchestration (Hoffman et al. 2020; Hu et al. 2023; Liang et al. 2018; Zhou et al. 2023). Pearl is explicitly designed as a production-oriented RL library that treats dynamic action spaces and safety as engineering concerns rather than benchmark details (Zhu, Braz, et al. 2024).

These systems are important counterexamples, but they govern different objects. A distributed training framework can execute several policies. A registry can version a model artefact. A container platform can deploy and roll back software. None of those facts, by itself, establishes that

an interdependent policy set is represented, validated, promoted, deployed, observed, and recovered as one governed unit. The difference can be seen through five boundaries summarised in Table 2.10.

Table 2.10: Lifecycle capabilities that should not be conflated in a MARL deployment.

Existing capability	**What it establishes**	**Additional requirement for coordinated MARL operation**
Experiment tracking	Parameters, metrics, artefacts, code revisions, and run history are recorded.	The environment, scenario, reward, constraints, observation/action schemas, seeds, agent–policy mapping, dependencies, and runtime must be recoverable as one experiment contract.
Model storage or registry	A model version has identity, lineage, metadata, and lifecycle state.	A policy-set registry must also represent member policies, roles, interface compatibility, communication and safety dependencies, allowed combinations, and rollback predecessors.
Serialisation or packaging	Weights, code, and dependencies can be moved to a target runtime.	The package must preserve preprocessing, schemas, communication adapters, safety layers, target requirements, configuration, and the identity of the complete policy set.
Multiple-model serving	Several independent models can be loaded and served.	The system must coordinate agent–policy assignment, mixed-version behaviour, atomic or staged policy-set updates, and compatibility checks across dependent policies.
Infrastructure monitoring	Processes, devices, resources, and communications appear operational.	Monitoring must also detect data-quality, control-performance, safety, and coordination failures, including incompatible versions, oscillation, unfair resource use, or loss of community-level performance.

A simple energy example illustrates why this distinction matters. Suppose a community deploys one policy for batteries, another for EV charging, and a third for community-level coordination. Each container may be healthy and each policy may work in isolation. Nevertheless, replacing only the EV policy can alter its action range, message schema, or response timing. The remaining policies may then operate with assumptions that are no longer valid. A generic rolling update has succeeded from a software perspective, while the coordinated controller has entered an unevaluated mixed-version state.

Dynamic composition narrows the gap further without closing it. HELICS supports federates that join after a co-simulation begins and explicitly documents EV federates joining and leaving a running federation (Hardy et al. 2024). This proves that online component composition is feasible at the infrastructure layer. It does not decide which learned policy is compatible with the new component, whether its observation and action schemas match, which safety constraints apply, or how the other active policies should respond. Likewise, MALib's changing policy population is a training abstraction rather than evidence that a previously unknown physical controller can join an operational CPS safely (Zhou et al. 2023).

The residual gap is therefore a *governance and integration gap*. Existing research and production systems provide substantial lifecycle pieces independently: distributed training, experiment tracking, registries, packaging, scalable serving, progressive deployment, rollback, edge orchestration, runtime safety mechanisms, and sustained operational MARL. The available evidence provides much less support for a reusable lifecycle in which coordinated policy sets, agent–policy compatibility, environment–policy lineage, dynamic heterogeneous participation, validation gates, coordination-aware monitoring, staged edge deployment, rollback, and recovery are managed together.

This residual gap is classified as **strong**. RQ4 concerns the reproducibility, governance, validation, deployment, monitoring, updating, and recovery of changing collections of interdependent policies across heterogeneous edge targets. Chapter 7 develops the detailed platform comparison and evaluates the EnergAIze lifecycle. A trained model, a deployable package, and an operational multi-agent control service are treated as distinct research objects.

2.8 Adjacent Challenges and Scope Boundaries

The critical analysis in Section 2.7 isolates the four residual problems that motivate RQ1–RQ4. The same literature also identifies concerns that influence the design, evaluation, or eventual operation of a real-world control system without constituting independent contributions of this dissertation. Distinguishing these roles is necessary because an issue can be operationally important without being a research gap that the thesis claims to close.

The boundary adopted here is evidence-based. A concern is treated as a supporting requirement when it directly shapes the methods or evaluation developed later in the thesis. It remains outside the central scope when answering it would require a separate scientific formulation, dedicated methods, or an evidence base that is not provided by the present work. These boundaries are revisited in Chapter 9 when the final claims and limitations are assessed.

2.8.1 Supporting Requirements and Evaluation Conditions

Several concerns identified in the literature are incorporated into the thesis because they affect how the four research questions can be answered, even though they are not formulated as additional RQs.

Traceability, reproducibility, and interoperability. The lifecycle contribution depends on being able to relate a result to the environment, scenario, reward, observation and action interfaces, source revision, dependencies, policies, and policy-set configuration that produced it. Existing platforms already provide strong experiment-tracking and lineage primitives; the relevant requirement here is to keep those identities coupled when a learned controller is validated, packaged, deployed, updated, and reconstructed. This becomes a supporting element of RQ4 rather than a

claim to solve reproducibility in machine learning as a whole. The same principle applies to interoperability: the thesis defines and tests explicit interfaces between selected components, but it does not propose a universal standard for energy or CPS control software.

The experimental substrate introduces a related requirement. Reproducible comparison depends on explicit scenario semantics, randomisation, physical assumptions, and controller interfaces. Existing environments provide many of these mechanisms individually, as discussed further in Chapter 4; the thesis therefore makes its own environment and scenario contracts explicit instead of claiming a general cross-platform benchmark standard.

Data quality, monitoring, and human oversight. Missing, delayed, noisy, or inconsistent information affects both the behaviour of a learned controller and the meaning of the evidence collected from it. Data-quality disturbances are therefore part of the robustness analysis under RQ3, while runtime monitoring of control performance, safety events, compatibility, and multi-agent coordination supports RQ4. Validation gates, fallback behaviour, and observable runtime state also preserve a meaningful role for human oversight. The thesis does not, however, conduct a human-factors study of trust, workload, or decision support, nor does it develop a general-purpose data-governance platform.

Privacy and data sovereignty. Distributed and edge execution can reduce the movement of raw data and can support local autonomy during communication loss, but these properties are not equivalent to a complete privacy method. Access control, retention, inference attacks, privacy-preserving learning, and data-governance policy require separate treatment. Privacy therefore constrains architectural choices and deployment assumptions without becoming an independent thesis claim.

Mixed objectives and baseline discipline. Personalised and community-level objectives are central characteristics of the control problem rather than evidence of an unaddressed literature gap. Prior work already contains local and collective objectives, hierarchical coordination, and market-based interaction (Faia et al. 2023; Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Srithapon and Månsson 2023). The thesis consequently evaluates both participant and community outcomes without claiming that mixed objectives are absent from prior work. The same evidence leads to an important evaluation condition: MARL must be compared with credible rule-based, optimisation, or MPC alternatives whenever the corresponding model and computational assumptions make those alternatives feasible. Otherwise, apparent gains can result from a weak baseline rather than from the multi-agent learning formulation itself.

2.8.2 Important Challenges Beyond the Central Scope

Other concerns remain outside the central scientific claims of the dissertation. They are acknowledged because they matter to operational CPSs, but resolving them would require methods and

evidence beyond those developed here.

Security, assurance, and regulation. Adversarial security requires explicit threat models and evidence against malicious access, data manipulation, compromised communication, or attacks on software and devices. This is distinct from the operational meaning of safety and robustness adopted in the thesis. Formal verification and certification present a related but separate assurance problem: empirical constraint satisfaction and runtime safeguards do not constitute a formal safety certificate. Regulation and market design likewise shape which energy-community actions are legally and economically admissible, but the thesis treats these as operating conditions rather than proposing new regulation, tariff structures, or market mechanisms.

Explainability, fairness, and long-term adoption. Operators and participants may require understandable reasons for actions, but explaining a multi-agent policy and its interactions is a research problem in its own right. Similarly, reporting individual outcomes can reveal unequal effects without providing a normative definition of fair benefit allocation. Long-term acceptance introduces further questions of trust, incentives, usability, and behaviour that cannot be established from controller performance or short-duration demonstrators alone. These topics remain relevant to the deployment of intelligent energy systems, but they require dedicated human, social, economic, or longitudinal evaluation and are therefore retained as limitations and future research directions rather than central claims.

The resulting scope is deliberately narrower than the full set of challenges associated with real-world intelligent control. RQ1–RQ4 address the residual problems for which the thesis provides a connected methodological and empirical response; the concerns above either constrain that response or define important work that remains beyond it.

2.9 Synthesis of Residual Gaps and Thesis Requirements

The state of the art supports a narrower conclusion than the initial motivation alone might suggest. Dynamic populations, heterogeneous agents, large-scale MARL, constraint-aware control, robustness mechanisms, operational deployment, experiment tracking, registries, rollback, and edge orchestration have all been demonstrated in some form. The residual problem is not the absence of these capabilities. It is the limited evidence that they remain coherent when heterogeneous agents, observation and action interfaces, physical constraints, information quality, computational resources, and deployed policies change within the same distributed CPS.

The four research questions therefore carry forward requirements that are explicitly conditioned by what prior work already establishes. Table 2.11 summarises this transition from state-of-the-art evidence to the evidence required from the thesis.

Table 2.11: Residual requirements derived from the state of the art and their connection to the research questions.

RQ	What prior work already establishes	Residual requirement carried into the thesis	Principal evidence required
RQ1	Session dynamics, variable problem sizes, agent loss, changing graphs, dynamic participation, and reconfigurable co-simulation components have all been demonstrated (Hardy et al. 2024; Orfanoudakis, Panda, et al. 2025; Wu, Qiu, et al. 2024; Yeh et al. 2023; Yu, Yang, et al. 2023).	Control must remain meaningful when runtime membership changes coincide with heterogeneous capabilities, observation and action interfaces, local constraints, and compatible policy operation.	Typed and changing scenarios; controller comparisons across populations and capabilities; reconfiguration tests; integrated operational evidence.
RQ2	Prior work demonstrates scaling in population size, physical-network size, topology, simulator throughput, parallel execution, and operational controller count (Almilaify et al. 2023; Blum et al. 2021; Fan et al. 2022; Orfanoudakis, Diaz-Londono, et al. 2025; Ponse et al. 2025).	Scalability must be evaluated across interacting dimensions rather than inferred from agent count alone, including heterogeneity, representation size, computation, communication, experimentation, reconfiguration, and edge execution.	Simulator and runtime benchmarks; repeated learning runs; compute, memory, communication, inference, orchestration, and control-quality measurements.
RQ3	Constraint-aware learning, stochastic control, shielding, masking, runtime projection, safe-control environments, and selected disturbance tests are established techniques (Chen, Liu, et al. 2023; Eichelbeck et al. 2026; Puech et al. 2024; Yu, Yang, et al. 2023).	Safety and useful control behaviour must be assessed together across physical constraints, uncertain or degraded information, faults, dynamic reconfiguration, communication imperfections, and shifts between training and operation.	Constraint and service metrics; controlled disturbance campaigns; health and reconfiguration tests; fallback and recovery behaviour; evidence separated by simulation, testbed, and operational level.
RQ4	Operational MARL, RLOps proposals, distributed RL/MARL frameworks, flexible policy mapping, registries, packaging, rollback, edge orchestration, and dynamic co-simulation show that the individual lifecycle mechanisms exist (Hardy et al. 2024; Hu et al. 2023; Li, Thomas, et al. 2022; Liang et al. 2018; Zaharia et al. 2018; Zhang, Zhang, Quiñones-Grueiro, et al. 2024; Zhou et al. 2023).	Interdependent policies must be governed as compatible policy sets with coupled environment–policy lineage, validation and promotion, staged distributed deployment, coordination-aware monitoring, rollback, and recovery.	Experiment reconstruction; compatibility tests; policy-set manifests; deployment and mixed-version tests; monitoring, rollback, disconnected-edge, recovery, and sustained-operation evidence.

Two evaluation principles cut across all four questions. First, MARL is treated as one controller family among several rather than as the default solution. Where rule-based control, optimisation, or MPC can operate under credible modelling and computational assumptions, they remain relevant comparisons or complementary mechanisms. Second, evidence levels are kept distinct. A simulator capability, an algorithmic result, successful execution on target hardware, shadow operation, a physical testbed, and sustained closed-loop operation support different claims and are reported accordingly.

The requirements are also interdependent. Dynamic heterogeneity changes representation and coordination cost, complicates safety assessment, and creates new deployment compatibility conditions. Safety mechanisms can alter performance and runtime cost. Monitoring and rollback support Deployability while also contributing to Robustness and recovery. Scalability must therefore be demonstrated through control quality and resource use across the layers at which growth actually occurs, not through a single population-size result.

These requirements define the hand-off from the state of the art to the remainder of the dissertation without prescribing the solutions in advance. Chapter 3 assigns them to architectural responsibilities; Chapter 4 establishes the data, scenario, and simulation evidence needed to test them; Chapters 5 and 6 develop and evaluate the control methods; Chapter 7 addresses governance and distributed operation; and Chapter 8 examines their interaction in integrated energy settings. Chapter 9 then returns to RQ1–RQ4 and answers them against the complete body of evidence.

2.10 Chapter Summary

This chapter established the state-of-the-art basis for the thesis by moving from distributed and dynamically reconfigurable CPSs to their concrete expression in Renewable Energy Communities, the control approaches available for those systems, and the RL and MARL concepts needed by the remainder of the dissertation. It also distinguished the levels of experimental evidence used later, from controlled simulation to operational closed-loop control.

The literature shows that many of the capabilities relevant to real-world MARL already exist individually. The contribution of the synthesis is therefore to identify the residual requirements that remain when heterogeneity, reconfiguration, scale, safety, robustness, and policy lifecycle constraints must be handled together. Section 2.9 carries those requirements forward through RQ1–RQ4, while Section 2.8 keeps adjacent challenges and the limits of the thesis explicit.

Chapter 3 now translates these requirements into the architectural responsibilities of EnergAIze, establishing the common structure for the simulation, control, learning, lifecycle, and operational contributions developed in the following chapters.

Chapter 3

EnergAIze: An Integrated Framework for Intelligent Energy Management

A Renewable Energy Community is not controlled by an algorithm alone. Before a controller can decide when an electric vehicle should charge, it must receive coherent measurements from different platforms, estimate information that is not directly observable, understand the user's flexibility, and translate its decision into a command accepted by the physical device. The same path must continue to operate when measurements are delayed, a vehicle does not expose its state of charge, or an external platform rejects an action. These concerns are usually treated as separate data, forecasting, optimisation, and deployment problems. In a real community, they form one control system.

EnergAIze is the framework developed in this thesis to connect those concerns. Its purpose is to transform heterogeneous energy and mobility data into safe, auditable decisions for flexible assets while retaining a path for simulation, learning, validation, and deployment. Percepta and the Data Provider provide the integration boundary; prediction and vehicle services complete the controller's view of the installation; the Algorithms component computes local and community-aware decisions; the Fail-Safe Supervisor validates those decisions; and CityLearn and EVLearn provide the environment in which alternative controllers can be developed before they receive physical authority (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Fonseca, Sousa, et al. 2025; Salvan and OPEVA Consortium 2026; Sousa et al. 2025).

The framework is distributed by design. Each building can host an EnergAIze instance adapted to its available assets and information, while the instances contribute to Renewable Energy Community objectives such as cost reduction, renewable self-consumption, peak shaving, and smoother demand. A dwelling with photovoltaic production and one charger does not need to expose the same interface as an office with several chargers or a factory with a local power controller. EnergAIze preserves this heterogeneity instead of forcing every participant into one fixed device model.

3.1 The EnergAIze Framework as a Whole

3.1.1 From Heterogeneous Measurements to a Physical Decision

Consider one charging decision in a prosumer building. The meter and photovoltaic inverter may report through an energy-management platform, the charger through a charging-service platform, and the vehicle through a manufacturer-facing service. Their messages use different identifiers, units, sampling periods, and communication mechanisms. Consumption and photovoltaic production for the next decision horizon are not measurements at all, but forecasts. The vehicle's departure time may come from a recurrent schedule, a prediction, or an explicit user request. A useful controller must distinguish these sources and their confidence before deciding how much energy to charge, discharge, or defer.

EnergAIze organises this decision around five cooperating capabilities:

1. **perception and integration**, which turn provider-specific data into one installation-level representation and return validated decisions to the appropriate provider;

2. **prediction and flexibility**, which estimate consumption, photovoltaic production, vehicle state, and the time and energy boundaries within which an asset can be scheduled;

3. **algorithms**, which convert the available observations and objectives into proposed actions for local flexible assets and, where applicable, coordinate those actions with the wider community;

4. **safe execution and traceability**, which check actions against physical and user constraints, apply conservative alternatives when required, and record what was proposed and executed; and

5. **simulation, learning, and lifecycle support**, which use historical or synthetic scenarios to train and compare controllers and move an approved controller towards a cloud or edge runtime.

These are components of one framework rather than independent processing stages. Percepta does not merely clean a dataset: it maintains the boundary between external platforms and the controller during both training and inference. Prediction services do not replace observations silently: their provenance and confidence affect which decision is defensible. The Fail-Safe Supervisor is not part of the learned policy: it retains the authority to restrict or replace an unsafe proposal.

Figure 3.1 presents the framework in the form used throughout the remainder of the thesis. The detailed building instance is repeated according to the composition of each participant. External platforms and physical assets remain outside the EnergAIze boundary because they preserve their own communication, actuation, and electrical-protection responsibilities.

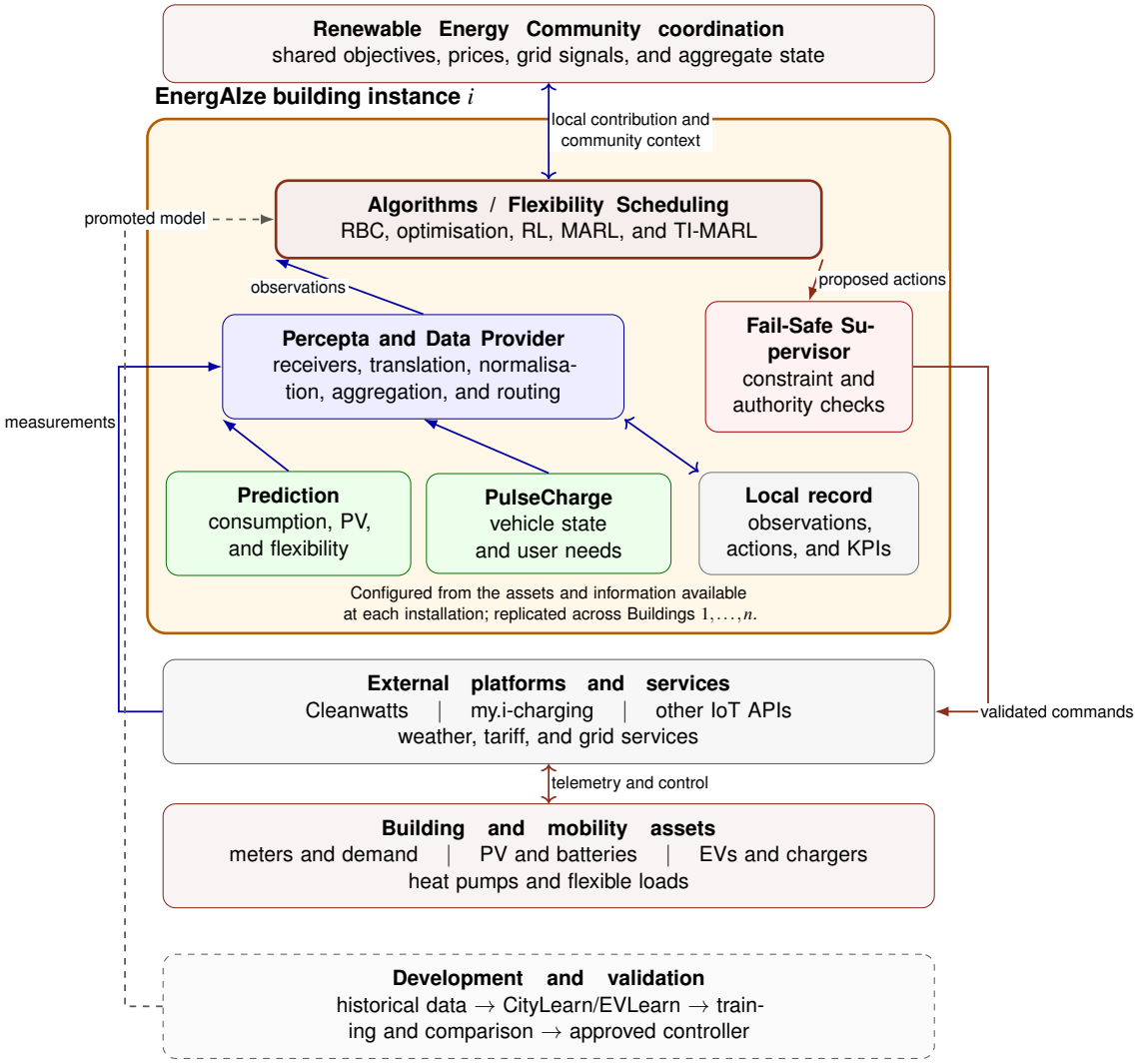


Figure 3.1: EnergAIze as a complete intelligent energy-management framework. A repeatable building instance connects perception, predictive context, algorithms, safe execution, and traceability to external platforms and physical assets; simulation and training provide candidate controllers, while community information coordinates otherwise local decisions. Adapted from the EnergAIze framework view in the OPEVA demonstrator documentation (Salvan and OPEVA Consortium 2026).

3.1.2 A Distributed Framework for Heterogeneous Buildings

The building instance is the basic unit of the framework. It contains the information and decision path required for one participant or installation, but its contents are not identical everywhere. An asset-poor dwelling can contribute demand and photovoltaic measurements without exposing a controllable device. A prosumer can add a stationary battery or domestic EV charger. An office or industrial installation can expose several chargers through one local controller and a shared site-power constraint. The instance is therefore assembled from the available interfaces rather than copied as a fixed observation vector.

Local execution supports privacy and resilience because raw household data and device control need not be centralised solely to compute an action. Community coordination nevertheless requires selected shared information. Depending on the controller, this can include a price or grid signal, aggregate demand, a community peak, or messages exchanged among agents. The algorithms developed in this thesis use these signals to balance participant objectives with community outcomes while keeping the physical action local.

The architecture supports both centralised and edge deployment. In a centralised arrangement, one cloud service hosts multiple installation models and the Data Provider separates their data and decisions. In an edge arrangement, an installation hosts its middleware and controller close to its assets. These are deployment alternatives for the same logical building instance; they do not change the meaning of its observations and actions.

3.2 Percepta and the Data Provider

Percepta addresses the first obstacle to intelligent control: physical installations do not produce model-ready observations. Cleanwatts, my.i-charging, meters, local controllers, vehicle services, and other IoT providers expose different APIs, message formats, identifiers, units, and sampling rates. A controller coupled directly to those details would need to be rewritten whenever a provider or asset changed. Percepta provides the stream-processing capability that separates those details from the decision model (Sousa et al. 2025).

In the OPEVA instantiation, the Data Provider middleware realises this boundary for both historical-data preparation and runtime inference (Salvan and OPEVA Consortium 2026). A receiver contains the communication logic for one source, such as HTTP for a REST service or RabbitMQ for a message-based provider. A translator extracts the relevant fields and converts the provider message into the common EnergAIze representation. Installation queues and an accumulator decouple the remaining processing from the original source. A manager then aligns the decision interval, checks missing values, reconstructs relationships among the installation's assets, and assembles the observation expected by the corresponding controller.

This organisation makes a new provider a local integration task: its receiver and translator are added without changing the controller or the other providers. It also keeps data treatment visible. Resampling a power measurement, interpolating a short gap, repeating a previous state, and substituting a value from a comparable day are not equivalent operations. Percepta records the transformation and source because an apparently complete observation can still contain low-confidence information that should restrict control.

3.2.1 Historical-Data and Runtime Paths

The historical-data path produces installation-level datasets for simulation and training. The user specifies the period and observation interval; receivers retrieve each asset's records; translators align formats and time; and the manager reconstructs the installation view. Energy prices can

follow their own translation path when they are not associated with one installation. The resulting dataset retains the temporal and asset relationships needed by CityLearn, EVLearn, and the controller experiments.

The runtime path performs the same semantic work under a decision deadline. It collects recent messages, aggregates them at the configured interval, identifies unavailable information, and constructs one inference request for the installation. After the model returns a decision, a forwarder applies the communication logic required by the responsible provider. The Data Provider therefore carries information in both directions: observations towards the algorithm and validated device requests towards Cleanwatts, my.i-charging, or another platform.

The OPEVA middleware calls the component that invokes the installation's AI model the *Predictor*. This name refers to the inference gateway inside the Data Provider, not to the consumption, photovoltaic, or flexibility forecasting services discussed next. The distinction matters: the Data Provider Predictor executes a control model, whereas the forecasting components estimate future quantities used by that model.

3.3 Prediction, Vehicle State, and User Flexibility

A controller cannot base every decision on direct measurements. Future consumption and photovoltaic production determine whether charging now is preferable to waiting. An EV charger often exposes delivered energy and connection state but not the vehicle's current state of charge. The time at which the user needs the vehicle, and the required departure energy, are behavioural constraints rather than device telemetry. EnergAIze therefore treats predictive and user-facing services as explicit components around the controller.

3.3.1 Consumption and Photovoltaic-Production Prediction

The consumption and production services estimate the near-term exogenous profiles used by the scheduling algorithm. Their outputs allow the controller to anticipate a demand peak or a period of local photovoltaic surplus instead of reacting only after it occurs. They can also provide a bounded substitute during a temporary sensor interruption. A forecast remains distinguishable from a meter reading, and the controller can be evaluated with perfect, predicted, degraded, or unavailable information without modifying the physical asset model.

These services are updated independently of the scheduling policy. A better photovoltaic model changes one input source; it does not require a new implementation of Percepta, the Fail-Safe Supervisor, or the charging platform. Conversely, a controller can be compared under a common frozen forecast so that performance differences are not confounded with different predictive information.

3.3.2 PulseCharge and Flexibility Prediction

PulseCharge provides the vehicle and user context that charging infrastructure alone frequently cannot supply. It hides manufacturer-specific authentication and telemetry behind a common vehicle-data interface and retrieves information such as state of charge, connection, and charging state when those data are available. Request budgets and periodic-task supervision reflect a real limitation of manufacturer-facing services: the data source can impose rate limits or temporarily fail.

The associated flexibility model ranks information according to its reliability. A non-integrated or uncontrollable vehicle provides no actionable flexibility. A recurrent schedule supplies a low-interaction baseline. Flexibility Prediction uses historical behaviour to estimate the likely connection window and energy requirement. Explicit user input has the highest priority when the current journey differs from the routine. This hierarchy prevents a convenient prediction from silently overriding a declared departure requirement (Salvan and OPEVA Consortium 2026).

The Route Planner extends the same information model to an EV in transit. It combines route, range, charger, price, and community information to propose a charging stop and, where supported, connect that request to the scheduling algorithm of a Renewable Energy Community. It is an optional application component rather than a prerequisite for building-level control, and its physical integration remains bounded as described in Chapter 8.

3.4 Algorithms and the Flexibility Scheduling Decision

The Algorithms component is the decision-making core of EnergAIze. In the OPEVA demonstrator it is referred to as the Flexibility Scheduling Algorithm. It receives the installation state assembled by the Data Provider, including measurements, prices, forecasts, asset availability, vehicle requirements, and the community information permitted by the selected control formulation. It returns actions such as a battery charge or discharge request, an EV charging limit, or the schedule of another flexible load (Fonseca, Ferreira, Cabral, Severino, and Praça 2024; Salvan and OPEVA Consortium 2026).

Each building-level controller represents a local objective and the assets available to that participant. Community coordination adds outcomes that no building can optimise from its local meter alone: aggregate peak, renewable sharing, ramping, or a common grid request. The original EnergAIze algorithm uses multi-agent reinforcement learning so that local agents can learn from their own state and actions while being shaped by individual and community objectives. Only the information required by the selected coordination mechanism needs to cross the building boundary.

EnergAIze is not restricted to one algorithm family. Rule-based control provides an interpretable baseline and fallback. Optimisation exposes an explicit model and constraints. Single-agent reinforcement learning supports adaptive local decisions, while MARL and hierarchical control coordinate several participants. TI-MARL later removes fixed-interface assumptions that

limit these approaches when assets, agents, and capabilities change. The important architectural property is that these controllers receive a declared observation interface and return a declared action interface; Chapters 5 and 6 provide their formulations and evidence.

3.4.1 The Fail-Safe Supervisor and Executed Action

The result produced by an algorithm is a proposed action, not an unconditional physical command. The Fail-Safe Supervisor checks it against rated power, state-of-charge bounds, vehicle departure requirements, connection state, site limits, unavailable observations, and the authority granted to the current controller. It can bound the action, reject it, retain a previously safe value, or invoke a conservative rule-based controller. The external platform and device retain their lower-level protections and can still reject the resulting request.

EnergAIze records the observation context, controller and model version, proposed action, supervised action, provider response, and relevant KPIs in the local or central data store. This distinction between proposed and executed actions is essential. A physically stable trajectory accompanied by frequent supervisor corrections demonstrates an effective protection layer, but does not by itself demonstrate a reliable learned policy.

3.5 CityLearn, EVLearn, and the Development of Controllers

Trial-and-error learning cannot begin on occupied buildings or connected vehicles. An exploratory charging action can violate a site limit, increase cost, or leave a user without the required departure energy. EnergAIze therefore develops controllers against a simulated representation of the same installations before considering real-world inference or actuation.

CityLearn provides the building, demand, photovoltaic, storage, weather, tariff, and community environment. EVLearn extends it with vehicles, charging stations, connection sessions, battery dynamics, and V1G/V2G flexibility (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Nweye, Kaspar, et al. 2025). Historical datasets produced through Percepta can populate this environment, while synthetic or modified scenarios expose the controller to events and conditions that are rare or unsafe to create physically. Rule-based, optimisation, RL, and MARL methods can then be compared under the same assets, information, time period, and KPIs.

The simulator is not a copy of the complete physical system. Provider latency, rejected commands, unavailable vehicle telemetry, human overrides, and electrical protections require separate representation or real-world evidence. The purpose of the simulation path is to make controller development controlled and comparable, not to convert simulation performance into an automatic deployment claim. Chapter 4 establishes the simulator and scenario contracts, and Chapter 8 identifies which assumptions survived contact with the demonstrators.

Training and evaluation are supported by experiment workers, artefact storage, and policy records. A training request binds the algorithm configuration to a scenario, data version, seed, and compute target. The resulting model is associated with its observation and action interfaces,

preprocessing, validation results, and intended installation. Chapter 7 develops this supporting lifecycle in detail. Here, its role is deliberately secondary: it provides a controlled route by which a model created in CityLearn or EVLearn becomes the version invoked by the Data Provider Predictor.

3.6 Deployment Across Buildings and the Community

An EnergAIze instance can run centrally for several installations or locally beside one installation. The central arrangement suits a community platform that already aggregates household data. The edge arrangement suits a local controller that requires lower latency, reduced external data movement, or continued local operation during a cloud interruption. The Algorithms component and Data Provider can be packaged separately so that a model update does not require the integration middleware to be replaced, and a new provider adapter does not require the policy to be retrained solely because the transport protocol changed (Salvan and OPEVA Consortium 2026).

At the external boundary, the framework uses the interface exposed by each platform. In OPEVA, OCPP links chargers and the charging platform; Modbus TCP/IP links selected chargers, meters, and local controllers; OCPI carries charging information and profiles between charging and community platforms; and REST or RabbitMQ connects data and algorithm services. These protocols belong behind Percepta receivers and forwarders. They should not appear as assumptions inside the policy.

The distributed arrangement permits Buildings $1, \ldots, n$ to retain different assets, providers, and local constraints while participating in a shared control problem. It does not imply that every component was deployed with the same authority at every site. A building can contribute historical data only, execute a controller in simulation, run inference without actuation, or accept bounded physical commands. The OPEVA and DemFlex demonstrators are therefore reported by evidence level in Chapter 8 rather than being represented as one uniform deployment.

3.7 Component Responsibilities and Thesis Map

Table 3.1 summarises the complete framework without replacing the component descriptions above. The table follows the direction of one decision: external information is made coherent, missing context is estimated, an algorithm proposes an action, the action is supervised and executed, and the outcome becomes evidence for evaluation or a later update.

The framework boundary is intentional. EnergAIze produces and governs energy-management decisions; it does not replace device firmware, electrical protection, charging-service operation, or every external data source. Likewise, the presence of a component in Figure 3.1 records its architectural role, not an equal level of real-world validation. Simulation, external-platform integration, target-runtime execution, bounded actuation, and sustained closed-loop use support different claims.

Table 3.1: Principal EnergAIze components, their role, and the chapters that provide their detailed methods or evidence.

Component	**Role in EnergAIze**	**Detailed treatment**
Percepta and Data Provider	Acquire, align, normalise, quality-check, aggregate, and route provider-specific observations and device requests	this chapter and Chapter 8
Consumption and PV Prediction	Estimate the near-term exogenous profiles required for anticipatory scheduling and controlled data-gap substitution	this chapter and demonstrator evidence in Chapter 8
PulseCharge and Flexibility Prediction	Provide vehicle state, user requirements, behavioural flexibility, and confidence-ranked availability information	this chapter and Chapter 8
Algorithms / FSA	Convert local and community observations into proposed actions using RBC, optimisation, RL, MARL, hierarchical, or typed-interface control	Chapters 5 and 6
Fail-Safe Supervisor and data store	Validate proposed actions, apply conservative alternatives, and preserve the distinction between proposed, requested, and executed control	Chapters 7 and 8
CityLearn and EVLearn	Represent buildings, flexible assets, EV sessions, scenarios, events, constraints, and KPIs for training and controlled comparison	Chapter 4
Experiment and policy lifecycle	Bind data, scenarios, controller versions, validation, packaging, targets, monitoring, updates, and recovery	Chapter 7
External platforms and local controllers	Expose physical data and actuation interfaces while retaining provider, device, and electrical authority	Chapter 8

Chapter synthesis. EnergAIze connects the complete path from heterogeneous energy and mobility data to a supervised physical decision. Percepta and the Data Provider create a stable information and actuation boundary. Prediction, PulseCharge, and flexibility services complete the installation state. The Algorithms component coordinates local assets and community objectives, while the Fail-Safe Supervisor preserves physical and user constraints. CityLearn and EVLearn provide the development environment, and the experiment and policy lifecycle governs the movement of selected controllers towards cloud or edge execution.

This component-centred view provides the map for the remaining thesis. Chapter 4 develops the simulated world in which controllers are trained and compared. Chapter 5 presents the cumulative progression of energy-management algorithms. Chapter 6 develops the principal algorithmic contribution for changing typed interfaces. Chapter 7 addresses the supporting experiment and deployment lifecycle. Chapter 8 then returns to the complete EnergAIze path and examines which components, interfaces, and decisions were exercised in the OPEVA and DemFlex demonstrators.

Chapter 4

CityLearn v3: A Simulation and Evaluation Environment for Renewable Energy Communities

A controller can appear successful for the wrong reasons. It can reduce electricity cost by leaving an electric vehicle below its required departure state of charge, postpone a flexible load beyond its deadline, request more charging power than the building connection can supply, or exploit measurements that would be incomplete in operation. These failures are difficult to recognise when an experiment represents a Renewable Energy Community (REC) as a fixed set of buildings, assumes one uniform and perfectly observed clock, and reports only aggregate energy or cost.

The simulator developed in this thesis addresses that methodological problem. Starting from the CityLearn lineage, it extends the environment from a predominantly building-centred benchmark into CityLearn v3: an auditable, dataset-driven simulation and evaluation environment for realistic REC control studies. The extension combines heterogeneous buildings and flexibility assets, explicit temporal and unit semantics, local energy sharing, demand-response services, changing participants and equipment, controlled data and equipment failures, structured controller interfaces, multiple communities, service-aware reference controllers, and traceable evaluation outputs.

The purpose is not to reproduce every physical process at maximum fidelity. It is to represent, with sufficient fidelity and explicitness, the mechanisms that can change the conclusion of a controller experiment. The resulting environment is the experimental substrate for the controller progression in Chapter 5 and the typed-interface methods in Chapter 6. It also provides the controlled counterpart to the OPEVA and DemFlex evidence examined in Chapter 8.

The central argument of this chapter is therefore methodological: a REC controller should be evaluated against a declared scenario, under the same service and physical rules as its reference controller, with a record that connects what the controller observed, what it requested, what the simulator applied, and what outcome followed.

4.1 From Supporting Tool to Research Contribution

At the beginning of this research, the simulator could reasonably have been treated as supporting infrastructure: a means of training a controller and obtaining the results needed elsewhere in the thesis. That interpretation became inadequate as the research moved from conventional building-level demand response towards heterogeneous RECs, EV charging, flexible services, changing system composition, and field-informed operating conditions. Under these conditions, the environment determines which physical and service obligations exist, what information reaches the controller, which actions are feasible, and how success is measured. It is therefore part of the experimental method rather than a neutral implementation detail.

This distinction matters because a missing simulator capability can change the conclusion of an experiment. If mobility requirements are omitted, a controller can reduce cost by withholding energy from an EV. If a short connection peak is averaged into an hourly value, an electrically infeasible action can appear acceptable. If a changing asset population is converted into a fixed vector before it reaches the policy, an experiment no longer evaluates adaptation to online composition change. If requested and applied actions are not separated, a failed command can be misinterpreted as a poor control decision. In each case, the issue is not visual or architectural completeness; it is the construct validity of the reported comparison.

The substantial effort devoted to CityLearn v3 consequently followed from a dependency in the research question. The controllers developed in this thesis could only be assessed under realistic heterogeneity and change if the environment could express those conditions, apply them consistently, and retain enough evidence to explain their effects. Simulator development, controller development, and demonstrator interpretation therefore progressed together. New operating requirements exposed missing environmental semantics; implementing those semantics changed the controller contract; and the revised contract required new baselines, indicators, tests, and documentation.

4.1.1 From an Iterative Research Instrument to Public Research Software

CityLearn v3 emerged through repeated movement between three contexts. Algorithm development identified what controllers needed to observe and control; OPEVA and DemFlex exposed the timing, integration, asset, and data conditions that occur outside a benchmark; and simulator experiments made those conditions repeatable before a controller was taken to a physical site. This produced a feedback loop rather than a one-way software implementation. A field constraint was formalised as a scenario or service contract, introduced under controlled conditions, evaluated with a reference controller, and then reflected in the interfaces used by the learning and deployment pipeline.

The importance of this loop is visible in several design decisions developed later in the chapter. Absolute measured PV data prevent a normalisation convention from altering the energy balance.

Requested and applied actions distinguish policy behaviour from physical enforcement or command failure. Dynamic topology events retain the identity and active period of a changing entity instead of hiding the change through padding alone. Mobility, cycle, and demand-response indicators state whether the requested service was delivered, while participant-level settlement prevents a favourable community total from concealing an unfavourable allocation. These are simulator features, but each was introduced to protect an experimental inference.

The resulting scope is intentionally broader than the minimum needed to run any one controller reported in this thesis. That breadth allowed the same environment to support conventional RBC and optimisation baselines, single-agent and multi-agent learning, entity-structured TI-MARL experiments, robustness studies, and multi-community execution. It also reduced the need to create a separate, weakly documented simulation script for every study. Reuse alone is not a scientific result, but a common and inspectable environment makes differences between controller experiments easier to attribute to the method under study.

Public release and external reuse. The simulator subsequently became a research output in its own right. Throughout this thesis, *CityLearn v3* denotes the generation developed in this work and distributed by the SoftCPS Research Group as `softcpsrecsimulator`, while retaining the `citylearn` Python import path for migration. The source identifies it as a research fork that extends, but does not replace, the separate upstream CityLearn codebase (Intelligent Environments Laboratory 2026; SoftCPS Research Group 2026a). At the thesis snapshot of 20 August 2026, version 1.7.0 was publicly available through the Python Package Index under the MIT licence, with source and wheel distributions, scenarios, examples, tests, a user interface, migration records, and bilingual documentation (SoftCPS Research Group 2026b).

Public release made the environment reusable beyond the experiments reported here and led to technical exchanges with researchers from several universities. These contacts indicate external relevance and initiated reuse and collaboration, but are not treated as a measured adoption rate or independent validation. The contribution is therefore fourfold: CityLearn v3 enabled the thesis experiments, preserved the contracts required by the controller-to-deployment pipeline, provided a methodological and software contribution to credible REC-controller evaluation, and created a public basis for subsequent research.

4.1.2 CityLearn v3 Across the Controller Lifecycle

CityLearn v3 connects controller development to deployment by preserving the assumptions under which a policy was produced and challenged. It is not an edge-runtime component and it does not certify a controller for field operation. Its role is to provide the controlled environment, compatibility information, and replay evidence needed before and after a deployment decision. This distinction places the simulator inside the EnergAIze lifecycle without confusing simulation with operation.

During *controller development*, the simulator binds the policy to a declared scenario: data and event versions, timestep and units, active entities, observations, actions, rewards, constraints, reference controllers, seeds, and evaluation outputs. RBC, optimisation, RL, MARL, and TI-MARL methods can therefore be compared under the same physical and service obligations. The result is not merely a model checkpoint; it is a checkpoint accompanied by the experimental conditions that give its reported performance meaning.

During *pre-deployment qualification*, the same contract becomes a source of challenge cases and compatibility checks. Topology changes, missing measurements, forecast deviations, command loss, equipment unavailability, phase limits, mobility deadlines, and demand-response requests are introduced reproducibly. The resulting evidence shows whether a candidate depends on unavailable inputs, assumes a different timestep or asset role, repeatedly requests infeasible actions, or loses required service under the declared disturbances. Chapter 7 uses these artefacts as inputs to promotion, packaging, and target-compatibility decisions.

During *operation and recovery*, selected field traces and incidents can be translated back into regression or replay scenarios. A problematic action sequence can then be reconstructed against the relevant software, interface, event, and constraint versions before a policy is updated, rolled back, or returned to service. This feedback path is deliberately asymmetric: field evidence can strengthen the simulator's challenge set, whereas a successful simulation remains only one part of the evidence required for safe operation.

The handoff across these stages is a versioned evidence package rather than an informal claim that the same algorithm ran in two places. It links scenario declaration, controller-visible information, requested and applied actions, service and physical outcomes, software identity, and evaluation records. In this way, the simulator supports continuity between experimentation and deployment while the runtime adapters, monitoring, governance, and physical demonstrators retain their distinct responsibilities.

This lifecycle role connects the simulator to all four research questions without assigning one requirement to one chapter. For RQ1, it represents stable identities, heterogeneous capabilities, and online changes to entity, observation, and action sets. For RQ2, it varies population, community count, temporal resolution, interface payload, and computational workload. For RQ3, it makes physical and service constraints explicit, separates requested from applied actions, and introduces repeatable data, communication, equipment, and topology disturbances. For RQ4, it preserves the scenario, interface, constraint, baseline, software, and result lineage needed for reconstruction, compatibility checks, incident replay, and revalidation.

These mechanisms make scalability measurable, deployability traceable, safety constraints observable, and robustness deviations reproducible. The simulator supplies the controlled conditions and evidence contract; the controller chapters determine algorithmic behaviour, Chapter 7 governs transfer and continued operation, and Chapter 8 establishes the available real-world evidence.

4.2 Problem Definition, Requirements, and Positioning

Consider one plausible operating day. A household joins the community and its photovoltaic (PV) system enters the local balance. An EV arrives at a charger and must receive a specified amount of energy before departure. A dishwasher or industrial process can be shifted, but its full cycle must still finish before a deadline. During the evening peak, a system operator asks the community to reduce import. At the same time, a meter stops reporting and one battery command is lost. None of these conditions is exceptional in isolation; the scientific difficulty lies in representing them together without making the controller interface or the reported result ambiguous.

Three roles follow from this problem. First, the simulator is a *comparative benchmark*: rule-based control (RBC), mathematical optimisation, single-agent reinforcement learning (RL), multi-agent reinforcement learning (MARL), hierarchical control, and TI-MARL can be evaluated under common data, assets, constraints, services, and indicators. Second, it is a platform for *controlled intervention*: topology changes, missing observations, forecast errors, action failures, and demand-response requests can be introduced reproducibly without exposing a physical pilot to experimental risk. Third, it is an *integration rehearsal*: the scenario, entity, observation, and action contracts used during simulation can also inform training services, inference packages, and operational adapters. This third role narrows the simulation-to-operation gap, but does not turn the environment into a fully synchronised digital twin.

4.2.1 From the Inherited Environment to the Required Experimental Contract

CityLearn v2 and EVLearn provided a strong starting point: a common simulation loop, coordinated multi-building control, established energy and cost indicators, storage and thermal assets, and an initial representation of EVs, chargers, connection schedules, and bidirectional charging (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Nweye, Kaspar, et al. 2025). The research developed here, however, required these mechanisms to remain coherent when participants and assets changed during an episode, temporal resolution exposed short power peaks, several devices competed for a building or phase connection, service deadlines became part of the objective, and data, commands, or equipment could become unavailable.

This gap cannot be closed by adding isolated asset classes. Introducing a charger during an episode, for example, also requires persistent identity, an event model, updated observations and actions, electrical headroom enforcement, mobility accounting, stable exports, and compatibility tests. The same dependency extends across scenario definition, physical progression, controller visibility, action feasibility, evaluation, and reproducibility. A plausible trajectory is not sufficient if a baseline receives different information, an infeasible request is silently clipped, or the export omits the event that explains a KPI.

Table 4.1 is therefore the single formal summary of the required transition. The first eight requirements define what the simulated REC must represent; the remaining five define the evidence needed to interpret and reproduce a controller comparison. Together they turn the inherited

simulation basis and its identified research pressures into an experimental contract.

Table 4.1: Requirements used to develop and evaluate the extended simulation environment.

ID	Requirement	Experimental consequence
D1	Multiple communities	Preserve local physics, controllers, and results while also reporting portfolio outcomes.
D2	Demand-response requests	Declare the issuer, direction, target, window, baseline, delivered response, shortfall, and settlement.
D3	Local sharing and settlement	Retain participant-level local import/export, residual grid exchange, and economic allocation.
D4	Changing members and assets	Apply time-stamped additions and removals consistently to state, control interfaces, settlement, and evaluation.
D5	Flexible-load services	Represent cycle profiles, start windows, completion deadlines, and missed or completed service.
D6	Data and equipment failures	Introduce repeatable changes to observations, forecasts, actions, and logical equipment availability.
D7	Time and unit semantics	Declare the timestep and distinguish power, energy per step, price, emissions, and normalised versus measured data.
D8	Electrical-service limits	Enforce total and per-phase import/export limits while preserving requested and applied power.
S1	Controller traceability	Record controller-visible inputs, availability, feasible actions, requested actions, and applied actions.
S2	Multi-criteria evaluation	Report energy, cost, emissions, service, settlement, robustness, and constraint indicators with inspectable trajectories.
S3	Fair reference operation	Compare controllers against references that face the same services, limits, disturbances, and accounting rules.
S4	Software and physical checks	Protect temporal, physical, interface, accounting, and export behaviour through repeatable tests and audits.
S5	Computational practicality	Measure loading, step, memory, export, interface, and multi-community costs under declared workloads.

These requirements originated in the attempt to translate real REC and charging problems, including those encountered in OPEVA and DemFlex, into repeatable controller studies. They do not imply field validation of every simulator capability. Instead, they identify the operational assumptions that must be made explicit before a simulation result can support a stronger claim.

4.2.2 Lineage and position within existing environments

CityLearn introduced a common Gym-compatible environment for demand response and coordinated control across multiple buildings (Vázquez-Canteli, Kämpf, et al. 2019). The CityLearn Challenge added shared tasks, data, held-out evaluation, and benchmark governance (Nagy et al. 2021). CityLearn v2 broadened the represented assets, occupant and resilience mechanisms, and reference control methods, while EVLearn extended the lineage with EVs, chargers, connection schedules, charging requirements, and grid-to-vehicle/vehicle-to-grid operation (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Nweye, Kaspar, et al. 2025). This lineage provides familiar centralised and decentralised interfaces and established building- and district-level indicators.

It also exposes the limits of a static, building-centred formulation. Fixed flat observation vectors become difficult to interpret when participants or assets change. Hourly data can conceal power–energy mistakes and short peaks. A requested action may differ from physical operation because a vehicle has disconnected, a phase limit is binding, or a command has failed. Finally, aggregate cost and import do not show whether mobility, flexible-load, demand-response, or participant services were delivered.

CityLearn v3 retains compatibility where it remains useful and adds explicit services around the simulation kernel rather than replacing the lineage. The novelty claim is consequently not that EVs, high-fidelity buildings, power-flow simulation, accelerated execution, or dynamic co-simulation were absent from prior work. Representative environments already provide each of those capabilities. The contribution lies in the implemented combination required for auditable REC controller studies and in the evidence retained across scenario declaration, control, and evaluation.

Table 4.2: Role-based positioning against representative energy-control environments.

Environment family	**Strongest relevant capability**	**Boundary relevant to this thesis**
CityLearn, its Challenge, and EVLearn	Shared community scenarios, multi-building control, established KPIs, EV charging, and V2G (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Nagy et al. 2021; Nweye, Kaspar, et al. 2025)	Direct software lineage; online heterogeneous interface change and its audit trail are not established by asset presence alone.
ACN-Sim, SustainGym, and EV2Gym	Real charging sessions, charging-network constraints, configurable EV/V2G scenarios, and single- or multi-agent interfaces (Lee et al. 2021; Orfanoudakis, Diaz-Londono, et al. 2025; Yeh et al. 2023)	EV arrival and departure can occur while the charger-agent roster remains fixed; variable scenario size is not necessarily online agent entry.
Sinergym and BOPTEST	Reproducible, high-fidelity EnergyPlus- or Modelica-based building emulation (Blum et al. 2021; Campoy-Nieves et al. 2025)	Stronger references for detailed building physics; their purpose differs from high-throughput heterogeneous REC/MARL studies.
PowerGridworld, CommonPower, and PowerGym	Power-flow-aware objectives, heterogeneous grid devices, safeguards, and large feeder cases (Biagioni et al. 2022; Eichelbeck et al. 2026; Fan et al. 2022)	Stronger references for network physics and formal safety; site and phase limits in CityLearn v3 do not replace a feeder solver.
HELICS and mosaik	Distributed co-simulation, communication-aware studies, hardware-in-the-loop, and runtime federation changes (Gryga and Rossi 2021; Hardy et al. 2024)	Dynamic simulator composition does not itself define policy compatibility, typed observations/actions, or safe MARL adaptation.

This comparison also clarifies the meaning of *dynamic*. An EV session can begin or end while a known charger remains present; different runs can instantiate different numbers of assets; a variable-set policy can consume a changing collection; and an agent or an entirely new interface can appear online. These are distinct levels of change, as established in Chapter 2. The simulator

exposes several of them explicitly so that the controller's response, rather than the environment's terminology, becomes the object of evaluation.

4.3 CityLearn v3 Architecture and REC Modelling Capabilities

4.3.1 System Architecture and the Scenario-to-Evidence Path

CityLearn v3 is organised around one responsibility: converting a declared REC scenario and a controller request into an auditable physical and service outcome. Figure 4.1 should therefore be read from the scenario boundary towards the evidence boundary. Within each REC, buildings retain their demand, generation, assets, electrical-service limits, services, settlement, and controller-facing identities. An enclosing orchestration layer can advance several independent RECs on a common clock without erasing their local state, policies, or results. Grid exchange, demand-response requests, topology changes, and data or equipment disturbances enter through explicit contracts rather than controller-specific modifications.

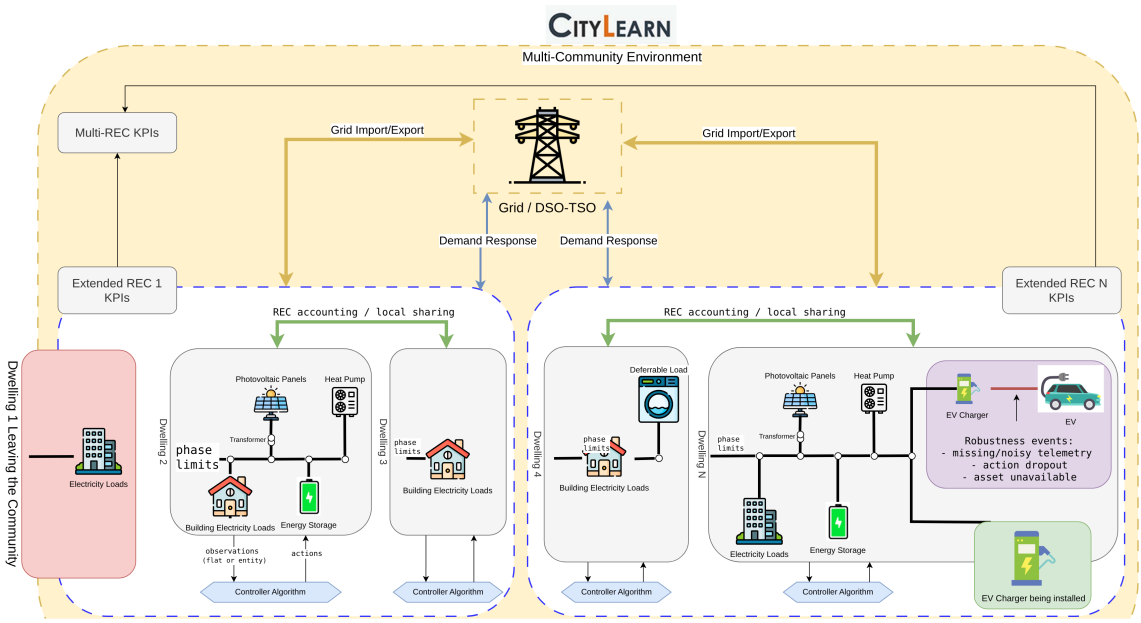


Figure 4.1: CityLearn v3 representation of one or more Renewable Energy Communities. Each community retains its buildings, assets, controllers, local exchange, events, and KPIs; the enclosing environment synchronises communities and derives combined results. Adapted from the CityLearn v3 simulator manuscript developed alongside this thesis.

The architecture separates four functional layers whose outputs form one continuous execution path. Table 4.3 states the responsibility and observable evidence of each layer. The separation is not only a software-engineering choice. It allows a disturbance to be varied without rewriting a policy, an interface to change without altering physical equations, and a KPI to be added without

changing the state transition. Consequently, a matched experiment can identify which assumption changed and where its effect entered the result.

Table 4.3: Functional layers of CityLearn v3 and their role in an auditable controller experiment.

Layer	Responsibility	Evidence retained
Scenario, data, and events	Resolves the clock, units, measured and forecast data, entities, services, topology changes, demand-response requests, and controlled disturbances.	Schema and software versions, source provenance, checksums, seeds, event calendars, active periods, and declared interfaces.
Simulation kernel	Advances building and asset states, energy balances, service progress, electrical limits, local sharing, and settlement.	Physical trajectories, state bounds, balances, feasible headroom, service state, and participant/community exchanges.
Control and evaluation	Constructs flat or entity-structured inputs, receives requested actions, applies feasibility and availability rules, calculates rewards and KPIs, and exports results.	Controller-visible values, masks, requested/applied/rejected actions, reasons, KPI rows, and time-series outputs.
Assurance	Checks schema integrity, physical and accounting invariants, interface stability, regression behaviour, and computational cost.	Test reports, golden cases, audit manifests, performance measurements, and release-linked validation records.

The assurance layer distinguishes four levels of evidence. Software verification covers unit, integration, regression, and export behaviour; physical and accounting checks cover time conversion, energy balance, state bounds, action feasibility, settlement, and KPI identities; behavioural demonstrations confirm that each declared mechanism produces an observable trace; and external validity asks whether the selected abstraction is adequate for the real process behind a claim. The first three levels are reproducible within the simulator and their detailed audits are retained in Appendix A.4. The fourth requires field or specialised-simulator evidence and is therefore addressed through the demonstrators and explicit scope statements. In particular, inherited thermal and HVAC models are preserved for compatibility but are not presented as a new CityLearn v3 physics contribution.

At each timestep, events scheduled for that instant are first resolved; the controller-visible state and available actions are then constructed; the policy returns a requested action; feasibility, availability, command, building, and phase rules determine the applied action; and the simulation kernel advances the physical and service state. In compact form,

$$\begin{aligned} \tilde{o}_t &= \mathcal{O}(x_t, w_t, e_t), & a_t^{\mathrm{req}} &= \pi(\tilde{o}_t), \\ a_t^{\mathrm{app}} &= \Gamma(a_t^{\mathrm{req}}, x_t, e_t), & (x_{t+1}, y_t) &= F(x_t, a_t^{\mathrm{app}}, w_t, e_t), \end{aligned} \tag{4.1}$$

where x_t is the physical state, w_t contains exogenous data, e_t contains active topology, service, or failure events, $\tilde{o}_t$ is the information actually exposed to the controller, and y_t contains the step-level physical, service, and accounting outcomes. The feasibility operator Γ deliberately separates controller intent from execution: a_t^{req} remains visible even when availability, equipment, building, phase, or command constraints reduce or reject it.

The complete trace associated with one decision is represented by the following evidence path:

$$\boxed{\mathscr{S}_t \longrightarrow \tilde{o}_t \xrightarrow{\pi} a_t^{\text{req}} \xrightarrow{\Gamma} a_t^{\text{app}} \xrightarrow{F} (x_{t+1}, y_t) \longrightarrow \mathscr{R}_t} \tag{4.2}$$

Here, $\mathscr{S}_t$ is the resolved scenario state at timestep t and $\mathscr{R}_t$ is the archived evidence record from which rewards, KPIs, service summaries, event logs, and exported trajectories are derived. This formulation makes the causal chain explicit: the result can be traced back to what existed, what was known, what was requested, what was physically applied, and which outcome followed. The remaining subsections explain the capabilities that implement each part of this path.

4.3.2 Scenario, Time, Unit, and Data Contracts

A dataset is not, by itself, a reproducible scenario. A time series does not state whether PV is an absolute measurement or a normalised profile, whether a price applies per kWh, which charger is connected to which building and phase, which services are mandatory, or which disturbances are active. CityLearn v3 therefore defines a scenario through a schema, referenced data and event files, and versioned assumptions. The schema declares the simulation and episode windows, temporal resolution, units, buildings, assets, control mode, observations, actions, services, events, KPI scope, and export settings.

This separation allows the same base data to support controlled interventions without editing simulator code. A nominal scenario can disable disturbances; a robustness scenario can add measurement and command failures; a topology scenario can activate time-stamped additions and removals; and a demand-response scenario can add externally issued service requests. These are different experimental scenarios even when their load and PV files are identical.

The physical contract uses kW for power and instantaneous limits, kWh per simulation step for energy-series columns, currency/kWh for prices, and kgCO_2/kWh for carbon intensity. If P_t is an average power measurement in kW and Δt is the step duration in seconds, the corresponding energy is

$$E_t = P_t \frac{\Delta t}{3600}. \tag{4.3}$$

The conversion is elementary but consequential. A 7.2 kW charger delivers 7.2 kWh in a one-hour step, 1.8 kWh in a 15-minute step, and 0.03 kWh in a 15-second step. Interpreting each value as 7.2 kWh would overstate the 15-second energy by a factor of 240.

The schema also distinguishes a normalised PV profile from measured production. The former is multiplied by installed capacity; the latter already represents the physical output and is not scaled again. Timestamp spacing, the declared simulation clock, and episode bounds are checked during loading so that a 15-second trace is not silently treated as hourly data. Fixtures exercise 60, 300, 900, and 3600-second operation, while dedicated datasets extend the checks to 15-second power traces.

Temporal resolution changes more than storage requirements. Figure 4.2 replays the same synthetic annual physical trace at three resolutions. Annual load, PV, and EV energy remain

unchanged, but the observed peak changes from 96.8 kW at one hour to 188.2 kW at 15 minutes and 281.5 kW at 15 seconds. On the selected day, the hourly view reduces a 231.0 kW short event to 63.9 kW. The example is a deterministic trace-replay test rather than field validation, but it demonstrates why the timestep belongs to the scientific claim.

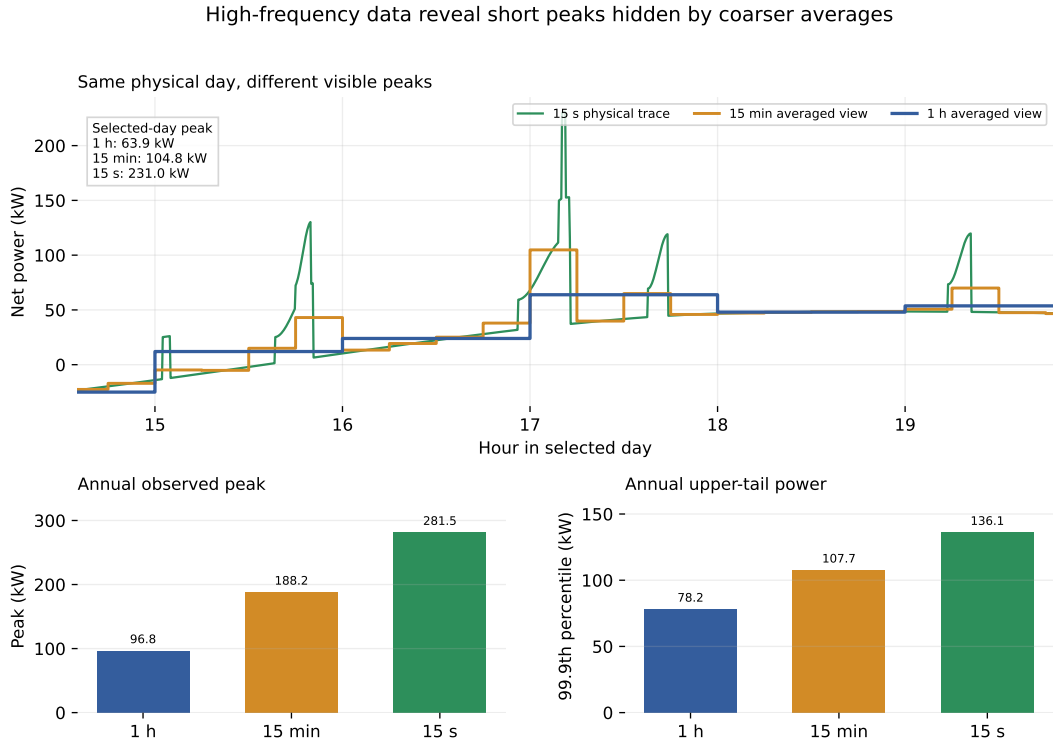


Figure 4.2: Peak visibility when the same physical trace is represented at one-hour, 15-minute, and 15-second resolution. Annual energy is preserved; the observable short-duration peak is not.

High-frequency access is implemented through windowed loading and columnar formats. In the evaluated evidence bundle, two 15-second CSV datasets of 2.446 and 2.619 GB were represented as 12.405 and 12.740 MB Parquet files. Initialisation time decreased from 25.37 to 15.61 s and from 28.59 to 18.92 s, respectively, while post-load step time remained similar. These figures characterise those datasets and encodings; they are not a universal compression ratio.

The simulator treats controllable demand as a collection of services embedded in buildings, rather than as one unconstrained aggregate signal. Buildings retain their non-controllable demand, weather-dependent or inherited thermal components, PV generation, and grid exchange. Stationary batteries, EVs, chargers, and deferrable appliances introduce different state, availability, and service semantics. This separation matters because the same energy shift can have different consequences for mobility, comfort, appliance completion, and electrical feasibility.

4.3.3 Stationary storage and electrical-service limits

Stationary storage is represented through energy capacity, charge/discharge power, efficiency, and state-of-charge bounds. The simulator calculates the feasible action from the current state and retains feedback whenever the requested action is clipped or cannot be applied. Storage can therefore support self-consumption, peak reduction, price response, and demand-response headroom without allowing an infeasible request to disappear from the record.

Each building can additionally declare total import/export limits and, for three-phase connections, a separate limit for each phase. In the canonical suite, household and small-service import ceilings use Portuguese normalised contracted-power levels. Single-phase residential connections do not exceed 10.35 kVA; three-phase BTN levels extend to 41.4 kVA; and the larger-consumption scenario class uses representative BTE-level connections of 69, 103.5, or 138 kVA (E-REDES 2018; Entidade Reguladora dos Serviços Energéticos 2026a). Residential chargers use 3.7, 7.4, or 11 kW classes and service chargers use 11 or 22 kW, within the normal and fast charging powers described by ERSE (Entidade Reguladora dos Serviços Energéticos 2026b). Single-phase connections and chargers are rotated across the community's L1, L2, and L3 labels; balanced three-phase uncontrollable demand and three-phase assets are divided across all three phases. Equipment therefore carries explicit phase-allocation metadata: a charger may be connected only to L1, while a three-phase battery can distribute its power across L1–L3.

CityLearn currently advances active energy and does not calculate reactive power. The suite therefore maps the numerical kVA connection level to a kW ceiling under an explicit unity-power-factor assumption; a balanced three-phase ceiling is one third of the total on each phase, whereas a single-phase member receives the complete ceiling on its assigned phase. The export ceiling is the smaller of the import ceiling and the installed PV-plus-battery power. These rules produce Portuguese-comparable connection-headroom cases, not an engineering design or certification of a particular installation. They do not model voltage drop, feeder impedance, protection coordination, or phase-specific measured household baselines. With positive power denoting import, an applied state is feasible when

$$-X_b^{\max} \leq P_{b,t} \leq I_b^{\max}, \qquad -X_{b,\phi}^{\max} \leq P_{b,\phi,t} \leq I_{b,\phi}^{\max} \quad \forall \phi \in \Phi_b, \tag{4.4}$$

where I and X are import and export limits. A building can be below its total contracted limit and still exceed the phase to which a charger is attached. For example, if a house imports 4 kW on L1 and L1 is limited to 7 kW, a 7.2 kW single-phase charging request has only 3 kW of feasible headroom even if the total building limit would allow more. CityLearn v3 applies the feasible portion and records both the original request and the rejected power. Two KPI families preserve the causal distinction. *Requested pressure* measures the excess implied by the controller request before projection and therefore shows how strongly a policy depends on the safety layer. *Residual violation* is calculated from the applied total and phase trajectories after projection and is the quantity used for the electrical hard gate. If non-controllable demand already violates a limit,

that residual remains visible rather than being erased by the constraint layer. The controller-visible constraint signal retains requested-pressure semantics so that an unsafe intention can be penalised even when the applied trajectory is safe.

4.3.4 Electric vehicles and chargers

EV control is both time-coupled and service-constrained. The relevant state includes connection, vehicle and charger identities, arrival state of charge, target or minimum departure state, charging/discharging bounds, expected departure, and vehicle-to-grid capability. From these quantities the environment derives remaining energy, time to departure, average required power, slack, and urgency.

The implementation distinguishes the vehicle session from the charging asset. A charger may remain installed while successive vehicles connect and disconnect; this is session dynamicity, not necessarily a change in agent identity. A topology event can separately install, remove, activate, or deactivate the charger or its controller-facing entity. This distinction allows the experiments in Chapters 5 and 6 to state precisely which form of reconfiguration they exercise.

Several edge cases are explicitly tested: reproducible initial state of charge, arrival and departure transitions, state progression when the charger action is zero, disconnected behaviour, charging and V2G bounds, terminal-step handling, and required departure service. The KPI layer reports departure counts, target/minimum/tolerance success, energy deficit, charged energy, and V2G energy. It also applies an independent energy-accounting invariant: the positive connected-session increase in stored EV energy cannot exceed metered charger input, apart from numerical tolerance. A positive unexplained shortfall is a hard audit failure. A controller cannot therefore improve a grid metric by silently failing the mobility service, nor can a boundary-state reference introduce unmetered energy.

Episode and topology boundaries receive explicit state semantics. If an experiment starts, a member enters, or a charger is installed while its declared session is already occupied, the runtime initializes the EV from the session's current-SoC boundary reference rather than from an unrelated annual-catalogue initial value. The reference is used once at runtime association; subsequent SoC is determined by the applied actions and battery model. Likewise, a deferrable request whose admissible start window closed before a member or appliance became active is excluded rather than recorded as a controller failure. These rules make partial-window training and dynamic full-year evaluation comparable without inventing decisions in history that the controller never observed.

4.3.5 Deferrable appliances

The earlier appliance-specific representation was generalised into a reusable deferrable-service model. A cycle catalogue defines the energy profile $q_{c,\tau}$ and duration d_c; a schedule defines the earliest and latest admissible start, completion deadline, priority, and whether the service is mandatory. The controller selects a start time but does not reshape or truncate the cycle.

For requested cycle c, start s_c is feasible when

$$t_c^{\text{earliest}} \leq s_c \leq t_c^{\text{latest}}, \qquad s_c + d_c - 1 \leq t_c^{\text{deadline}}. \tag{4.5}$$

Once started, the stored profile is applied until completion and remains subject to the building's electrical limits. Figure 4.3 shows the modelling boundary. The simulator does not reproduce the internal physics of every appliance or industrial process; it represents the complete time-shiftable service so that omitted or late demand is visible through completed cycles, missed cycles, start delay, served energy, and unserved energy.

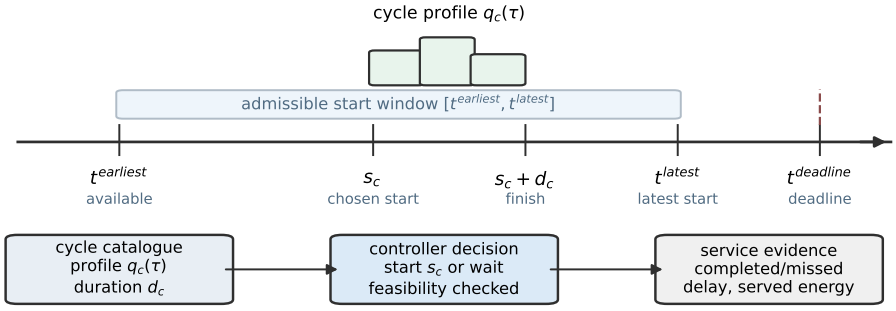


Figure 4.3: A deferrable service with an admissible start window, fixed cycle profile, and completion deadline. Cost or peak reduction is not credited as successful service when the required cycle is omitted or completed late.

4.3.6 Local energy sharing and settlement

REC operation cannot be inferred from district net load alone. If one member exports PV while another imports, aggregation may show a near-zero balance without revealing whether local exchange was permitted, which participant supplied or consumed it, or how value was allocated. CityLearn v3 therefore separates member net demand, local import/export, residual grid exchange, and settlement.

For active members $\mathscr{C}_t$, let $E_{b,t}^{\text{net}}$ be positive for demand and negative for surplus. Community demand, surplus, and locally shared energy are

$$D_{\mathscr{C},t} = \sum_{b \in \mathscr{C}_t} \max(E_{b,t}^{\text{net}}, 0), \tag{4.6}$$

$$S_{\mathscr{C},t} = \sum_{b \in \mathscr{C}_t} \max(-E_{b,t}^{\text{net}}, 0), \tag{4.7}$$

$$L_{\mathscr{C},t} = \min(D_{\mathscr{C},t}, S_{\mathscr{C},t}). \tag{4.8}$$

Participant weights allocate scarce surplus without exceeding demand, and residual demand or surplus is exchanged with the grid. The current settlement configuration prices local energy as a declared fraction of the retail import price and retains counterfactual cost, savings, and member-level outcomes. It is an experimental accounting model, not a universal legal or market rule.

The historical annual diagnostic in Figure 4.4 uses the same community RBC trajectory in both accounting cases. Under the configured rule, 17.0 MWh is shared locally, annual cost changes from 24.8 to 22.0 kEUR, and all 17 members reduce cost, although savings range from 68.8 to 525.7 EUR. This result predates the canonical suite and is retained for exact reproduction. It demonstrates that participant-level consequences are measurable; it does not establish that the selected rule is equitable or applicable to every REC.

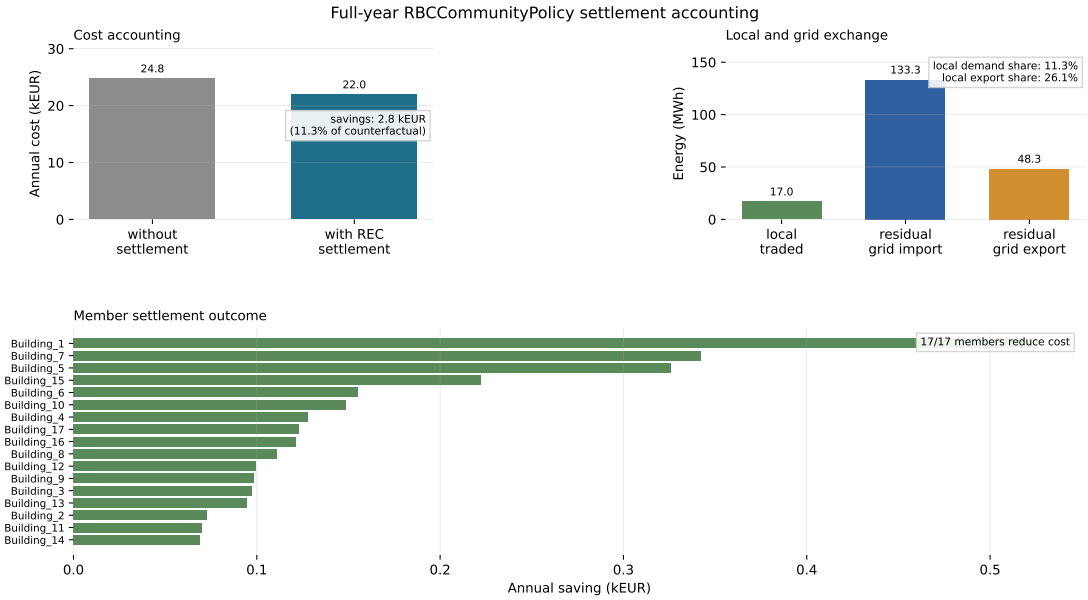


Figure 4.4: Annual local-energy and settlement accounting for one unchanged controller trajectory. The comparison isolates the configured accounting rule and retains heterogeneous member outcomes.

Asset models describe what can operate; event layers describe how the conditions of operation change. CityLearn v3 represents these changes as scenario data rather than controller-specific code. The same policy can therefore be run against a nominal scenario and against a matched scenario containing topology, demand-response, or robustness events.

4.3.7 Changing community members and assets

A member may join or leave, a charger may be installed, or PV and storage may become part of the community only during a defined active period. Each topology event contains a timestep, operation, affected identity, and, where required, replacement data or parameters. Events scheduled at t are applied before the controller acts at t, so the physical state, available observations and actions, settlement, and KPI denominators use the same active topology.

Removal does not erase history. A departed member retains its earlier rows but no longer contributes demand, generation, settlement, or actions. Similarly, an installed charger has no action or service denominator before its activation. A reinstalled stateful asset is represented by a new runtime instance initialized at its declared boundary state; it does not silently inherit SoC or pending service from the removed instance, whose earlier trajectory remains available to KPI aggregation. Energy carried at re-entry is therefore an exogenous boundary condition rather than grid energy created inside a control step. The exported topology log identifies which entity changed and why the shape of the controller interface or result set changed.

Figure 4.5 shows one executable lifecycle scenario. Building 18 is added at step 1000, a charger is added at 1300, a charger is removed at 1500, and PV is removed at 1600. At the same points, identified controller-input rows change from 67 to 74 and finish at 71; relation rows change from 74 to 84 and finish at 80; and action rows change from 26 to 30 and finish at 29. These figures are interface-propagation evidence, not a claim about controller performance under reconfiguration.

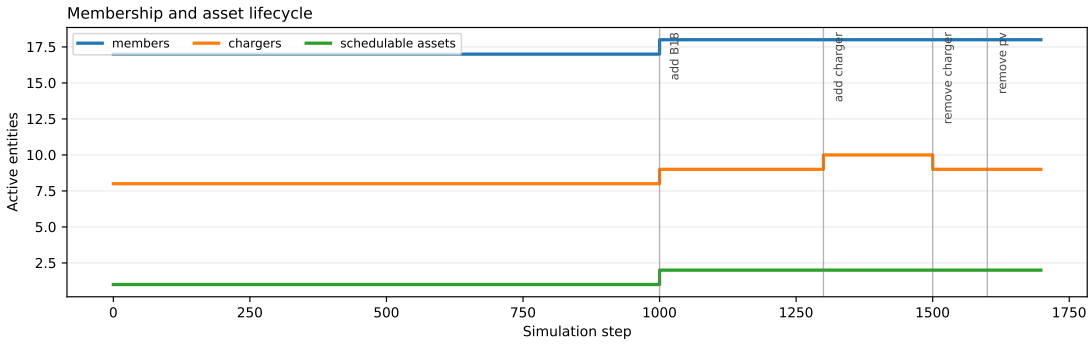


Figure 4.5: Lifecycle of members and assets in one dynamic-topology scenario. The event log, active entity counts, structured observations, relations, and action rows change at the same simulation steps.

4.3.8 Controlled data and equipment failures

Operational failures enter different parts of the control loop and must not be conflated. A missing measurement or biased forecast changes what the controller knows; an action dropout changes what is actuated; equipment unavailability changes which action is physically possible. CityLearn v3 stores these conditions as time-stamped event records with channel, target, feature, interval, failure mode, and parameters.

Let $\mathscr{E}_t^o$, $\mathscr{E}_t^f$, $\mathscr{E}_t^a$, and $\mathscr{E}_t^u$ denote active observation, forecast, action, and availability failures. Their location in the control loop is

$$\tilde{o}_t = \rho_o(o_t, \mathscr{E}_t^o, \mathscr{E}_t^u), \qquad \tilde{f}_t = \rho_f(f_t, \mathscr{E}_t^f), \tag{4.9}$$

$$a_t^{\text{app}} = \rho_a(a_t^{\text{req}}, \mathscr{E}_t^a, \mathscr{E}_t^u), \qquad x_{t+1} = F(x_t, a_t^{\text{app}}, w_t). \tag{4.10}$$

Corrupting observations or forecasts affects controller-facing data without changing the underlying physical state. Action and availability failures preserve the request but alter the applied action. Supported modes include missing, noise, bias, stuck, and clipping for data; dropout, delay, noise, bias, stuck, and clipping for actions; and temporary unavailability for assets. A seed makes stochastic events repeatable, and a missing-value marker remains distinct from physical zero.

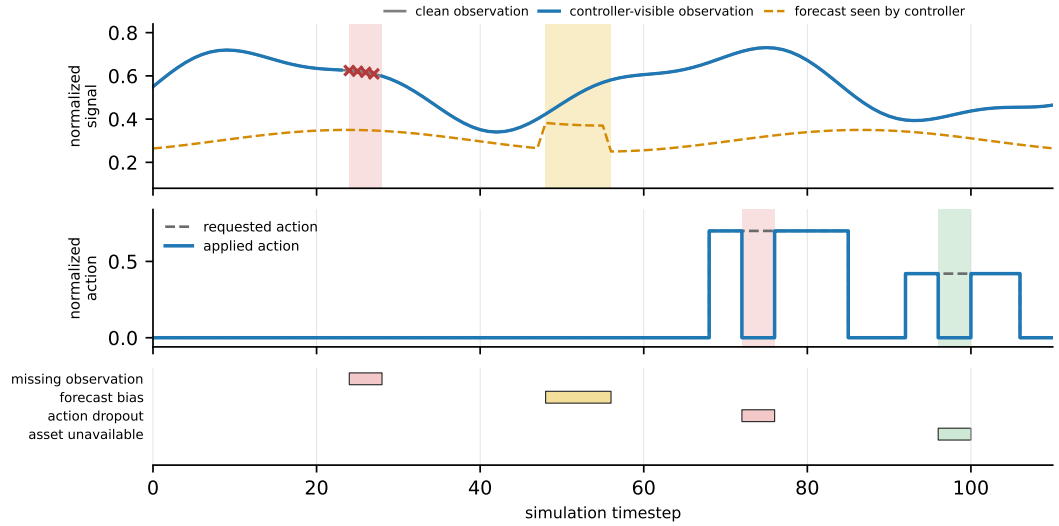


Figure 4.6: Failure channels in the controller loop. Measurement and forecast events alter controller-visible information; command dropout and equipment unavailability create traceable differences between requested and applied actions.

In the diagnostic scenario underlying Figure 4.6, a missing load measurement, biased price forecast, storage-command dropout, and temporary storage unavailability are activated in separate windows. The resulting export records changed measurements and forecasts, lost commands, unavailable-equipment timesteps, and requested/applied-action differences. The supported claim is that the failure is reproducible and measurable, not that the diagnostic policy is robust to every failure class.

4.3.9 Demand-response services

A demand-response request is represented as an explicit service rather than hidden inside a price or reward. Each request declares an issuer, direction, start and end, target power, tolerance, activation price, shortfall penalty, and baseline method. With positive P_t denoting community import and $B_{r,t}$ the baseline for request r, delivered power is

$$d_{r,t} = \begin{cases} P_t - B_{r,t}, & \text{up request,} \\ B_{r,t} - P_t, & \text{down request.} \end{cases} \tag{4.11}$$

Positive delivery therefore always means movement in the requested direction. Credited response is non-negative and capped by the target, whereas full delivery remains in the record. Shortfall is calculated after tolerance, and both quantities are converted from power to per-step energy using Equation (4.3). An invalid or unavailable baseline is reported rather than scored silently.

The evaluated 180-step scenario contains three non-overlapping requests totalling 190.0 kWh. Relative to reference operation, a diagnostic event-aware dispatch reduced shortfall from 101.1 to 51.6 kWh, increased compliance from 0.43 to 0.70, and reduced cost after settlement from 863.6 to 790.6 EUR. Net demand-response revenue remained negative after penalties. The conclusion is consequently not that every request is profitable or optimally served; it is that request, delivery, credit, shortfall, payment, penalty, and final cost can be interpreted together.

4.3.10 Multiple communities

`MultiCommunityEnv` advances several independent CityLearn environments in lockstep. Each community retains its scenario, members, assets, events, controller, reward, KPIs, and identifier. Actions and observations are organised by community, enabling local, portfolio, or hierarchical controllers without merging all buildings into one artificial district.

Energy, cost, emissions, payments, and event counts are additive. Ratios are aggregated through declared reporting weights that affect only the combined result, not the simulated physics or local control. Communities in one synchronised run must share timestep, episode length, controller-interface mode, and control setting, whether centralised or decentralised.

In the four-community evidence case, the environments were evaluated over the same 600-step window. Individual import/export trajectories remained visible alongside the portfolio result, and 2.45 MWh of simultaneous export and demand was identified as a potential inter-community match. This was an accounting diagnostic under independent control, not physical power transfer, centralised market clearing, or optimised inter-community exchange.

4.3.11 Controller Interfaces and Applied-Action Traceability

The simulator preserves the original CityLearn/Gymnasium interaction pattern: reset the environment, receive an observation, submit an action through `step`, and receive reward and diagnostic information. Two complementary controller views are available.

The *flat interface* preserves compatibility with existing RBC, optimisation, and RL implementations that expect fixed arrays. The *entity interface* provides tables organised by entity type, stable identifiers, relations, masks, feasible ranges, action ports, and metadata. A newly installed charger is therefore an identified row linked to its building and current EV, rather than an unexplained

change in vector position. Entity bundles can expose basic or more detailed feature sets for graph-, set-, or transformer-based policies and for offline datasets.

The structured interface does not itself make a policy capable of handling a new entity or interface. It makes the change explicit and testable. A fixed-head controller can use a compatible fixed view or padding and masks; a variable-set controller can consume active rows; and TI-MARL can be tested against the stronger typed-interface conditions developed in Chapter 6. The environment remains neutral about whether the controller succeeds.

Action tables carry identity, bounds, masks, and feasible capacity. At every step, requested, clipped, applied, or rejected values can be exported with a reason. This prevents a recurrent interpretive error: a policy that repeatedly proposes infeasible power may produce an apparently safe physical trajectory only because the environment intervenes. Controller intent and executed operation must therefore remain separate in learning diagnostics and evaluation.

4.4 Datasets Developed to Evaluate REC Controllers

The simulator capabilities described above require an experimental progression that makes their effects separable. A collection of unrelated scenarios would permit each controller to be tested, but would make it difficult to determine whether a result changed because of the algorithm, the participant set, the data resolution, the service obligations, or an unnoticed change in the physical inputs. CityLearn v3 therefore includes a canonical annual suite constructed as a ladder: a small diagnostic case, a reduced development case, a 30-member reference family with isolated interventions, and a 100-member integrated stress case. The purpose of the ladder is not to make one dataset representative of every REC. It is to preserve a common contract while increasing the dimensions that matter to RQ1–RQ4.

The earlier hourly case with 17 members is retained for historical reproduction of experiments developed before this suite. It remains useful when an exact earlier result must be reconstructed, including the annual settlement diagnostic reported later in this chapter. It is not treated as a member of the new canonical family because its timestep and asset contract differ.

4.4.1 Common temporal, provenance, and accounting contract

Every canonical dataset spans the civil year 2023 with 35,040 steps of 15 min and `seconds_per_time_step=900`. The progression uses one continuous UTC timeline from 1 January at 00:00 to 31 December at 23:45. A separate calendar table preserves the corresponding `Europe/Lisbon` local time, UTC offset, weekday, hour, minute, and daylight-saving status. This choice avoids an artificial duplicate or missing simulator index at the daylight-saving transitions while retaining the local attributes needed by prediction and control.

The canonical files contain exactly 35,040 physical interval rows. Since the environment needs the state at both ends of an action interval, a complete annual rollout uses 35,041 state points to execute 35,040 control transitions. The extra point is an observation-only boundary at 1 January

2024 00:00; it carries no additional demand, generation, price, settlement, or service-energy interval. This distinction is part of the runtime contract and prevents the final 15-minute action, including a departure at midnight, from silently disappearing from evaluation.

The price series is the historical component held common across the suite. It uses the official 2023 Portuguese day-ahead prices distributed by OMIE (OMIE 2023). The daily source files contain the physical market periods, including the 23- and 25-period daylight-saving days. Concatenating those periods produces 8,760 physical hours. Since the day-ahead market only moved to a 15-minute market time unit in October 2025 (OMIE 2025), each official 2023 hourly price is repeated in the four corresponding simulation steps, without interpolation, and converted from EUR/MWh to EUR/kWh. The generated provenance table retains the daily OMIE filename, published revision, URL, number of periods, and SHA-256 digest. The transformation is consequently inspectable down to the source file used for each day.

The remaining exogenous series are reproducible scenario profiles, but they are not copies of one daily template with independent noise. Each complete member class is stratified over five explicit routine archetypes. Residential members include early, standard, and late commuters, home workers, and predominantly home-day routines. Small services include office, six-day retail, split-day clinic, extended hospitality, and early-workshop schedules; larger consumers include daytime, extended, continuous, morning-intensive, and evening-intensive processes. The archetype changes the operating window, the relative morning, daytime, and evening demand, occupancy, eligible EV visits, and deferrable-service windows. Member-specific peak locations, correlated short-term variation, day-level temporal shifts, reduced-activity days, weekends, holidays, and seasonality then prevent members within one archetype from becoming exact replicas.

Each annual demand trace is scaled to a target drawn from a bounded log-normal scenario distribution, without accumulating observations at clipping limits. Its class centre is anchored to 2023 Portuguese consumption per consumer: approximately 2.49 MWh for residential, 16.58 MWh for non-domestic, and 175.89 MWh for industrial consumption (Direção-Geral de Energia e Geologia 2023; Pordata 2023). These national classes provide an order-of-magnitude anchor; neither the ranges nor the frequency of each routine are presented as a fitted distribution of REC participants. PV capacity is coupled to the host's annual demand and separately varies east/south/west orientation, derating, and reproducible shading. Stationary batteries use several product-like energy and power classes selected from a stratified daily-load-autonomy rule with a host-PV floor; efficiency, initial state of charge, usable depth, standing loss, and degradation coefficient vary by member. Weather and carbon remain constructed scenario series, while weather forecasts include temporally correlated errors that increase with forecast horizon. Load and PV forecast bundles use causal daily persistence: a value at horizon h is based on the observation one day before $t+h$, with the current step used only for cold start. They therefore contain no realised future load or generation. The OMIE price bundles are publication-aware: an official realised price is visible only after the declared 13:00 local day-ahead cut-off; an as-yet unpublished horizon uses causal daily persistence instead. Stored price forecasts cover 1, 6, and 24 hours. The entity interface obtains

its additional 15-minute and 3-hour features from an earlier issue of the shortest stored horizon that targets the same delivery step, rather than reading the realised future-price series. The cut-off and alignment rule are conservative benchmark assumptions recorded in the schema and interface metadata.

The resulting datasets are therefore deliberately described as *hybrid*: the price is historical; annual demand has a Portuguese statistical anchor; and PV, weather, carbon, mobility, and deferrable behaviour create controlled heterogeneous operating conditions rather than measurements of one physical community. This boundary prevents the controlled evidence in Chapters 5 and 6 from being confused with the field-derived data and demonstrator evidence in Chapter 8. The exact generation ranges, asset parameters, topology calendars, and claim boundaries are fixed in Appendix A.3 and in the machine-readable family manifests.

All families use the same local settlement rule. Surplus and demand are matched within the same 15-minute step; importers receive equal allocation weights; local energy is priced at 80% of the OMIE value; and any residual export receives no remuneration. A grid-only calculation, with no local matching, is retained as the counterfactual for participant and community results. Retail margins, network tariffs, taxes, and regulated compensation are intentionally excluded. The settlement is therefore an internally consistent experimental accounting rule, not a reconstruction of a Portuguese electricity bill.

4.4.2 Composition and the separation of infrastructure from mobility

Table 4.4 fixes the installed catalogue for each level. The counts refer to distinct members and physical assets, not to observation rows or EV visits. Core-15-Stripped is a byte-identical physical subset of the first 15 members of Core-30 for the files and assets it retains. This makes it possible to move from debugging to the principal comparison without changing the behaviour of the shared members.

Table 4.4: Installed composition of the canonical 2023 annual dataset suite. Counts are fixed in the full catalogue; the active population and available equipment can change in Dynamic and AllIn scenarios.

Dataset	Members	PV	BESS	Charging members	Chargers	Deferrables
Micro-4-Q	4	2	1	2	2	1
Core-15-Stripped	15	7	3	6	7	4
Core-30	30	15	6	12	15	8
Premium-100	100	55	22	45	80	30

Four identities are kept distinct throughout the suite. A *member* is the participant or building that owns demand, production, and assets. A *charger* is a physical controllable connection attached to that member and to an electrical phase. An *EV* is the mobile energy-storage object whose state of charge and mobility requirement persist beyond one connection interval. A *charging session* associates one EV with one charger over a declared arrival–departure window. The

separation is consequential: counting visits as chargers would inflate the control interface, while assigning one permanent EV to every service charger would suppress the shared-infrastructure problem that those buildings are meant to represent. Contract v1.9 propagates all four identities into the executable schedule. A new session is therefore observable even when the same EV reconnects at the exact boundary at which its previous session ends and charger occupancy never changes. Runtime arrival SoC and departure KPIs use the session boundary, rather than inferring it only from charger or vehicle identity.

The smaller datasets already include more than one charger at one household. Core-15-Stripped has five homes with one charger and one with two. Core-30 has nine homes with one and three homes with two. The 80-charger Premium catalogue expands both multiplicity and building type. Thirty equipped homes contain 36 chargers: 24 have one and six have two. Ten small service or commercial members contain 20 chargers, two per member. Five larger office, public-service, or light-process members contain the remaining 24: three have four chargers and two have six. Chargers at one member remain separate action entities and phase-connected loads, so simultaneous requests must satisfy both individual limits and the shared building and phase headroom.

Residential charger–vehicle relations are normally stable. Shared chargers in service and office members instead receive successive EVs during the day. The Premium catalogue consequently contains 212 EV identities and 41,173 annual session identities for 80 physical chargers. Arrival and departure times vary reproducibly by member, charger, day, weekday, building class, and routine archetype; missed residential visits and several daily service-charging windows prevent an artificial all-at-once connection pattern. At the 15-minute clock, Premium uses all 96 possible arrival times and has no more than 10 simultaneous arrivals in the generated year.

The energy requested in each session is sampled by building class and translated into arrival and target state of charge. The generator reserves a 15% charger-window margin for residential sessions and a 28% margin for shared service sessions, and caps every request at 72% of EV capacity. Hence every ordinary session is individually deliverable at the charger's declared power and 95% efficiency before community constraints are applied. The audit found zero individually infeasible sessions across all four families. This does not make every deadline jointly attainable: simultaneous chargers, non-controllable demand, phase limits, demand-response obligations, outages, or command failures can still create deliberate competition for scarce headroom. The separation is methodologically useful because failure under Safety or AllIn can be attributed to the tested interaction rather than to an impossible mobility request embedded in the clean data. The session distributions are scenario assumptions, not a national mobility survey.

Deferrable services follow the same principle. Each asset has a catalogue of complete cycle profiles with duration and step energy, and a sparse annual request schedule with earliest and latest start, completion deadline, priority, and `must_run`. Residential archetypes produce distinct washing-machine, dishwasher, washer–dryer, and domestic-hot-water services. Service and larger-consumption routines introduce cleaning, cold storage, sterilisation, hospitality hot water,

compressors, pumping, and batch-process loads. Profile energy is scaled per asset; request occurrence, eligible days, clock window, completion margin, and mandatory-service probability vary by routine and day. Requests for one physical appliance are explicitly non-overlapping. Completion, delay, missed deadline, and unserved-energy indicators allow the later algorithm comparisons to distinguish legitimate load shifting from omission of the requested service.

4.4.3 Experimental ladder and matched scenario variants

Micro-4-Q is the smallest end-to-end contract. It is used for loader, interface, accounting, and controller smoke tests where a four-member trajectory is easier to inspect manually. Core-15-Stripped retains enough heterogeneity for initial baselines and algorithm development but reduces training and debugging cost. Its clean data and normally non-binding connection limits make an error easier to attribute before the same method is moved to the full reference family.

Core-30 is the principal controlled family. One annual physical base produces the five variants in Table 4.5. Safety, Health, and Dynamic first introduce one experimental dimension at a time. Combined then asks whether a method that appeared adequate under isolated changes remains adequate when tight limits, faults, topology changes, and demand-response commitments interact. Its dynamic trajectory begins with 26 active members: four previously inactive members join and Members 019 and 008 later leave permanently, producing 26–30 active participants and 28 at year end. A charger, stationary battery, deferrable appliance, and PV system are each removed and later restored independently of their member. A member event separately activates or deactivates all of that participant's assets. Equipment unavailability remains a health event rather than a topology event, preserving the distinction between an entity that no longer exists and an entity that exists but cannot currently be observed or controlled.

The corresponding state semantics are explicit. Re-adding an asset from its catalogue template represents replacement or recommissioning and therefore starts a new runtime instance from the declared initial state; it does not claim continuity of an unobserved SoC, degradation state, or unfinished service across removal. A temporary outage of the same physical instance is instead represented by a robustness event, which retains its identity and accumulated state while blocking the affected observation or action channel. The word "restored" in the scenario calendar consequently denotes restoration of the declared capability, not an assumption that hidden physical history has been reconstructed.

Premium-100 is the integrated scale and heterogeneity case. Its complete catalogue contains 70 residential members, 20 small service or commercial members, and 10 larger-consumption members. Each class contains all five of its routine archetypes; the frequencies are stratified scenario choices rather than inferred population proportions. The scenario begins with 70 active participants; 30 first-entry events activate the remaining catalogue and six active members later leave permanently. The active population therefore spans 70–100 and ends at 94. Two chargers and one PV system, stationary battery, and deferrable appliance each undergo independent removal and restoration. PV, stationary batteries, chargers, deferrables, building types, and service patterns

Table 4.5: Matched Core-30 variants and the experimental question isolated by each schema. All variants reference the same annual physical time series and asset identities.

Variant	Added condition	Purpose
Core-30-Nominal	Static membership, available equipment, and no injected faults	Reference comparison under the complete but undisturbed annual contract.
Core-30-Safety	Total and per-phase import/export limits	Tests whether requested actions remain feasible when several assets at one member act simultaneously.
Core-30-Health	Missing observations, forecast bias, command delay/dropout, and asset unavailability	Isolates degradation caused by data, communication, and equipment health without changing membership.
Core-30-Dynamic	Member entry/exit and independent charger, PV, storage, and deferrable removal/reinstallation	Isolates changes to the active entity, relation, observation, and action sets.
Core-30-Combined	Safety, Health, Dynamic, and demand-response requests	Evaluates interacting obligations and disturbances after each mechanism has been examined separately.

are deliberately unevenly distributed, so a policy must handle different capability sets rather than one residential template repeated 100 times.

Two Premium schemas share the same physical files, topology timeline, electrical limits, settlement, and demand-response services. Premium-100-Clean contains that already demanding operational structure without injected health faults. Premium-100-AllIn adds missing and degraded telemetry, forecast error, delayed or dropped actions, charger and storage unavailability, and selected grid-outage intervals, with some events coinciding with safety pressure, mobility deadlines, or demand-response requests. AllIn is reserved for final confirmation rather than routine method selection: repeated tuning on the integrated stress case would turn it into another development set and weaken its value as a held-back challenge.

Service denominators follow the active topology rather than the complete catalogue. An EV departure is an observable controller obligation only when its member and physical charger exist during the final control interval preceding departure. The static Micro, Core-15, and Core-30 variants therefore expose 554, 1,757, and 3,774 departures, respectively. Core-30-Dynamic and Combined expose 3,690; 56 catalogue sessions occur while their member is inactive and 28 while the charger is absent. Each Premium variant exposes 28,108; 13,048 sessions belong to an inactive member at the relevant boundary and 17 to an absent charger. The same audit distinguishes deferrable requests that are fully observable, never acquire a valid start opportunity, or are interrupted by a topology change before their deadline. The dynamic Core case contains 1,657 fully observable, 234 censored, and one potentially interrupted cycle; Premium contains 5,950, 1,409, and one, respectively. These excluded or interrupted services are reported separately, not silently assigned as controller failures. The annual audit independently reconstructs the expected denominators from the schema and requires agreement with the runtime KPI counts.

This ladder assigns a clear role to each evidence level. Core-15 supports fast implementation

checks and early controller progression. Matched Core-30 variants support the principal attribution of algorithm effects. Premium-100 tests whether the resulting method and interface remain usable under scale, mixed building roles, several action groups per member, dynamic composition, safety constraints, and health degradation. In this way the datasets provide a common bridge from the RBC and learning progression in Chapter 5 to the typed, variable-set TI-MARL evaluation in Chapter 6. They do not guarantee that an algorithm is representative merely because it performs well on all levels; they make the conditions under which that claim is tested explicit.

The suite connects directly to the thesis-wide research questions without repeating their full mapping. Dynamic members, installed assets, EVs, and sessions expose the representation problem in RQ1; population and asset multiplicity provide controlled RQ2 scale; matched Safety and Health variants provide repeatable RQ3 conditions; and versioned schemas, source provenance, deterministic generation, checksums, and validation reports provide the RQ4 lineage that accompanies a controller into the deployment workflow. The validation protocol and its numerical results are reported in Appendix A.3. Passing those checks establishes consistency within the declared dataset and simulator contract; it does not establish population-level representativeness, feeder-level electrical validity, or controller effectiveness.

4.4.4 Validation protocol and annual baseline execution

Structural validation cannot by itself show that a controller can traverse every session boundary, topology event, fault, service window, and settlement step without corrupting the experiment contract. The annual execution audit therefore pairs BAU and RBC Smart on every schema, binds both runs to the same dataset and simulator attestations, and independently re-reads the exported KPIs. It checks the terminal-state convention, topology-derived service denominators, causal price interface, finite outputs, event counts, EV energy conservation, and total and per-phase residuals.

BAU uses no stationary-battery action, charges EVs immediately until they enter the declared departure tolerance, and starts each deferrable cycle at its earliest feasible time. RBC Smart uses current and causal forecast information to shift storage and flexible services. The campaign is an execution and interpretability audit, not an algorithm-ranking result: it verifies that the datasets are traversable, the references are matched, required services remain visible, and competing cost, peak, ramping, demand-response, and feasibility outcomes are retained. The complete structural audit, annual scorecards, service diagnostics, and event evidence are reported in Appendix A.3.

4.5 Evaluation Protocol and Controller Scorecard

A controller comparison is only meaningful when all policies face the same scenario contract and when performance is interpreted together with service and feasibility. CityLearn v3 therefore separates the optimisation reward, the core thesis scorecard, the extended KPI catalogue, and the exported trajectories. Reward guides a controller during training; it is not treated as sufficient evidence of controller quality.

4.5.1 Reference Controllers and Fair Comparisons

The principal reference is business-as-usual (BAU): normal EV charging, earliest feasible execution of mandatory deferrable services, and no deliberate stationary-battery optimisation. A no-battery variant isolates native service demand, while progressively informed RBCs add stationary storage, current price, PV, peak, or community context. These policies are transparent reference behaviours rather than claims of optimality.

Every comparison binds the evaluated controller and its reference to the same data, topology, services, settlement, limits, disturbances, timestep, information boundary, and KPI definitions. Raw indicators are always retained. Ratios or differences to BAU are reported only when the reference value is valid and the direction of improvement is unambiguous. Missing service, invalid demand-response baselines, and constraint violations remain counts, rates, or gates rather than being hidden inside a normalised aggregate.

4.5.2 Core Scorecard and Result Exports

The thesis uses a compact scorecard rather than one composite reward. Run integrity and scenario-specific obligations, including mandatory deferrable completion and valid demand-response baselines, are checked before the headline indicators are interpreted. The reference is the exported BAU when available and otherwise the simulator baseline declared in the result. Cost saving is $C_{\text{ref}} - C_{\pi}$, so a positive value means that policy π is cheaper. Member improvement is reported as both x/N and a percentage, where x is the number of eligible members whose cost is lower than under the matched reference.

Table 4.6: Seventeen headline indicators in the core controller scorecard.

Dimension	Headline indicators	Preferred reading
Economic	**Cost saving vs reference** [EUR and %]	Positive; larger is better.
Service and safety	**EV minimum feasible service** [%]; **EV within tolerance** [%, declared ±5% SoC band]; **Electrical violations** [kWh]	Service ↑; violations = 0.
Flexibility and solar	**Battery throughput** [kWh]; **V2G export** [kWh]; **Solar self-consumption** [%]	Throughput ↓ at equal service; V2G contextual; self-consumption ↑.
Community exchange	**Community import** [kWh]; **Community export** [kWh]; **Community net exchange** [kWh]	Import ↓; export ↓ under the declared zero-remuneration rule; net exchange towards zero.
Grid shape	**Daily peak ratio**; **All-time peak ratio**; **Load factor penalty**; **Ramping** [raw value or reference ratio]	Ratios < 1; ramping ↓.
Participant outcomes	**Members saving vs reference** [x/N and %]	Larger coverage is better; retain the savings range.
Computation	**Inference latency** [ms/action]; **Training time** [h]	Lower under matched hardware, software, and training budget.

Electrical violations use the residual applied total and per-phase trajectory and therefore remain a hard feasibility gate. EV service uses topology-feasible departures as its denominator; the minimum threshold and tolerance band remain separately visible. Battery throughput and V2G export are diagnostic rather than universally monotonic: additional cycling or export is only beneficial when its economic, grid, service, and degradation consequences justify it. The same principle applies to community export whenever the settlement rule changes.

CityLearn v3 retains a wider KPI catalogue covering emissions, local settlement, deferrable and demand-response service, requested electrical pressure, rejected actions, topology, controlled failures, inherited comfort quantities, and multiple communities. Definitions, units, and interpretation boundaries are provided in Appendix A.5. Exports include these KPI rows together with trajectories, service summaries, event logs, entity relations, masks, and requested, applied, reduced, or rejected actions. The user interface supports comparative inspection, while the archived machine-readable result remains the authoritative record.

Computational evidence is part of the comparison rather than a separate simulator claim. Training time and inference latency are reported with hardware, software, controller, population, timestep, interface, executed steps, training budget, and aggregation convention. Matched smoke benchmarks additionally record initialisation, reset, tail step time, memory, export time, data format, and community count. Their detailed current-checkout results are retained in Appendix A.4; they bound the evaluated workloads rather than establishing linear scaling to arbitrary systems.

4.6 Chapter Synthesis

CityLearn v3 contributes a coherent simulation and evaluation substrate rather than a collection of isolated features. Its scenario contract binds data, units, services, topology, disturbances, controller interfaces, and evaluation; the annual dataset ladder turns that contract into matched development, attribution, scale, safety, and robustness cases; and the evidence path connects entity identity, controller-visible information, requested action, applied action, and outcome. Together, these mechanisms make heterogeneous and changing REC-controller experiments comparable, traceable, and reusable across the algorithm and deployment lifecycle developed in the following chapters.

The supported claim remains deliberately scoped. The chapter establishes internal consistency, executable mechanisms, and an auditable comparison contract; it does not establish population-level representativeness, detailed feeder physics, controller superiority, universal settlement validity, or field certification. The corresponding technical limits are stated beside the electrical, settlement, robustness, multi-community, and interface mechanisms to which they apply. Real-world evidence remains the role of the demonstrators in Chapter 8.

> *CityLearn v3 provides an auditable and physically checked experimental substrate for REC controller studies in which heterogeneous assets, service obligations, high-frequency data, dynamic topology, external requests, controlled*

operational failures, and multiple communities remain traceable from scenario declaration to controller-visible input, requested action, applied action, reported outcome, and the versioned validation contract used by the deployment lifecycle.

The next chapter uses this substrate to compare the progression from transparent rules and optimisation to single- and multi-agent learning. The question is no longer only whether a method improves an aggregate metric, but whether it does so under common information, service, physical, and evaluation conditions.

Chapter 5

Progressive Control and Learning Solutions for Renewable Energy Communities

The common simulation and evaluation substrate supports a cumulative sequence of control studies. Passive operation and progressive RBCs establish feasibility and transparent operating logic; optimisation makes objectives and constraints auditable; single-agent learning introduces adaptation; MADDPG introduces decentralised coordination; MATD3 improves the deterministic multi-agent actor–critic baseline; and data-efficient and hierarchical mechanisms address reuse, sample efficiency, and local–community coordination.

The sequence begins with passive operation and rule-based control, where feasibility and transparent operating logic can be checked directly. It then introduces explicit optimisation, which makes objectives and constraints auditable. Single-agent reinforcement learning adds adaptation without requiring a complete model at every decision step. MADDPG adds decentralised multi-agent coordination, and MATD3 addresses part of the training instability and value overestimation observed with deterministic multi-agent actor–critic learning. Behaviour cloning, guided exploration, replay selection, offline learning, distillation, and hierarchical coordination then address sample efficiency, reuse, and the separation between local and community decisions.

Each stage produces a distinct empirical contribution by establishing what can be achieved, under which information and operational assumptions, and at what computational cost. The accumulated evidence also exposes the remaining integration problem: stable, data-efficient, or hierarchical controllers can still depend on fixed observation and action interfaces and on manually maintained physical contracts. Those residual assumptions become the design requirements for TI-MARL.

5.1 Research Progression and Common Evaluation Protocol

A controller can appear better simply because it receives more information, uses a more favourable forecast, or is evaluated on an easier scenario. To prevent this, the methods in this chapter are compared through common physical scenarios and explicit information assumptions. Loads, photovoltaic production, assets, tariffs, deadlines, constraints, and evaluation periods are shared whenever the comparison permits it. Controller-specific models or preprocessing are treated as part of the method and are reported rather than hidden.

The progression is evaluated through four recurring questions:

1. **What problem was the method introduced to solve?**
2. **What did the experiments show that it actually solved?**
3. **Under which information, scale, and operational assumptions was that result obtained?**
4. **What limitation remained and motivated the next step?**

The methods are classified by their role as strong baselines, reusable mechanisms, empirical contributions, or components of TI-MARL. The recurring analysis records the problem addressed, measured result, scope, and remaining assumption for each stage.

Figure 5.1 summarises the cumulative research progression: each stage resolves a concrete problem, retains a useful capability, and reveals the assumption addressed by the next stage.

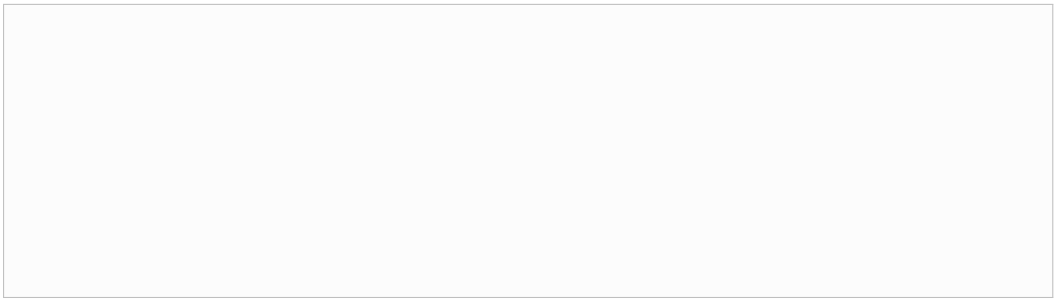


Figure 5.1: Cumulative progression from explicit control to adaptive and hierarchical multi-agent coordination.

Problem instances. The experiments cover related levels of control rather than one universal decision problem.

Asset level. A controller schedules charging, discharging, thermal operation, or a deferrable request for one asset.

Building level. A controller coordinates the assets behind one meter while respecting local service and electrical limits.

Community level. A controller coordinates several buildings or modifies a shared signal to reduce cost, peak demand, ramping, emissions, or grid stress.

Reconfigurable level. Agents, assets, capabilities, observations, actions, or topology change. This level motivates and is evaluated in Chapter 6.

A real deployment can combine all four. For example, a community-level coordinator can issue one price multiplier, while each building continues to schedule its own EV and battery. The high-level problem and the local problem should therefore not be collapsed into one large vector merely because that is convenient for an algorithm.

Comparison principles. The evaluation follows six principles.

Information fairness. Every result states what the method knew at decision time. A perfect-information MILP is an oracle reference, not an operational competitor to a policy that sees only current observations and forecasts.

Strong baselines. The RBCs become progressively more informed. The optimisation formulation includes the relevant constraints. Learning methods are compared with tuned and credible alternatives, not with deliberately weak references.

Repeated learning runs. RL and MARL results are reported over independent seeds. Central tendency, dispersion, learning stability, and checkpoint selection are part of the result.

Multiple outcomes. Aggregate reward is not sufficient. Energy, service, safety, fairness, learning, computation, and communication metrics are reported separately before any combined score is considered.

Operational interpretation. Training cost, inference latency, memory, communication, model requirements, and integration effort are considered together with energy performance.

Negative and boundary results. A method that does not improve one dimension, or improves it only under a narrow assumption, remains informative. Such results define where the method should and should not be used.

Figure 5.2 defines the common protocol required for a fair comparison across controller families with different information and training requirements.

5.2 Operationally Plausible and Constraint-Aware Control

Passive operation. The passive or business-as-usual case follows the native asset or user schedule without active flexibility management. An EV can begin charging when it is connected, a

Table 5.1: Metric families used throughout the controller progression.

Metric family	**Representative measures**
Participant outcomes	electricity cost, grid import/export, self-consumption, comfort or departure service, asset cycling
Community outcomes	aggregate cost, peak demand, load factor, ramping, local renewable use, carbon indicators, demand-response delivery
Safety and service	infeasible proposals, clipping or projection, power-limit violations, missed EV departure targets, incomplete services
Learning behaviour	return, convergence, variance across seeds, sample efficiency, critic disagreement, overestimation diagnostics
Computational cost	training wall time, environment interactions, CPU/GPU time, memory, checkpoint size, inference latency
Distribution and fairness	outcome by participant and asset group, worst-participant result, dispersion of savings
Operational complexity	model and forecast requirements, configuration effort, communication, package size, target compatibility

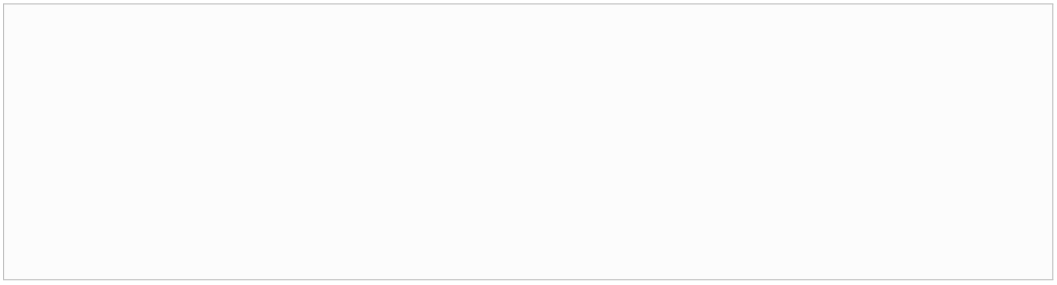


Figure 5.2: Common experimental protocol used to compare controller families.

battery can remain idle, and a deferrable request can follow its default start. This case is not a weak controller. It is the denominator that shows how much value is created by any intervention at all.

Progressive rule-based controllers. The five RBC levels form a ladder of increasing knowledge and coordination. Their repository class names are mapped to the following conceptual levels so that the comparison remains stable across implementation revisions.

Local feasibility. The controller blocks impossible actions, remains inside asset limits, and prioritises urgent service requirements.

Time- and price-aware control. The controller uses tariff periods, time to deadline, and threshold logic.

Forecast- and flexibility-aware control. The controller uses short-horizon forecasts and explicit flexibility quantities, such as remaining EV energy divided by remaining connection time.

Community-aware control. Local rules respond to community demand, headroom, or a shared coordination signal.

Expert and fail-safe control. The strongest rule set combines local feasibility, forecasts, community context, and conservative fallback behaviour. It can also generate expert demonstrations.

The RBC ladder solves several problems early in the thesis. It establishes physically plausible behaviour, gives the simulator and action adapters a deterministic reference, provides a low-cost runtime fallback, and separates the value of richer information from the value of learning. If a learned controller only outperforms a local rule but not a community-aware rule, the observed gain may come from coordination information rather than from the learning mechanism itself.

The remaining limitation is adaptation. Rules can be effective when the expected conditions are known, but they need manual redesign when objectives, tariffs, asset combinations, or patterns of uncertainty change. The more conditions the rule set covers, the more difficult it becomes to reason about conflicts and maintenance.

Evidence from the RBC progression. The RBC comparison follows the complete ladder instead of reporting only the strongest rule set. Performance changes are attributed to local feasibility, temporal or forecast context, community information, and expert fallback behaviour. This creates an interpretable path from passive operation to the rule-based teacher retained by TI-MARL.

Explicit constraint-aware coordination. MILP provides a different form of intelligence. Instead of learning from repeated interaction, it represents decisions, objectives, and constraints explicitly. Binary variables can represent service activation or mutually exclusive modes; continuous variables can represent power and energy; and constraints can encode deadlines, capacity, balance, and electrical limits.

The optimisation is used in two forms.

Operational formulation. It uses forecasts and information that could realistically be available at decision time. Its performance includes solver time, infeasibility handling, and sensitivity to forecast error.

Perfect-information reference. It uses realised future information and provides an oracle-like bound for the simplified formulation. It is not labelled as a deployable controller.

MILP solves important problems. It makes the meaning of the objective explicit, provides a strong constraint-aware benchmark, generates reference schedules, and can supply expert trajectories for behaviour cloning. It also reveals whether a learned method is recovering a useful scheduling pattern or only exploiting a weak baseline.

Its limitations are equally informative. Real communities contain uncertainty, mixed discrete and continuous decisions, heterogeneous devices, long horizons, and changing participation. Solve time can grow, a simplified model can omit relevant behaviour, and repeated model maintenance can be expensive. These limitations motivate adaptive control, but they do not make optimisation obsolete. MILP remains a reference, a teacher, and a possible verifier or fallback within the integrated stack.

Local results and solved problems. Passive, RBC, and MILP evidence is presented beside the corresponding methods. The comparison covers feasibility and service outcomes, the incremental value of each RBC information level, operational MILP performance and solve time, and the gap to any perfect-information oracle. Appendix A retains detailed schedules, sensitivity runs, and per-building tables.

5.3 Adaptive Single-Agent and Community-Level Learning

The next stage asks what can be learned without explicit multi-agent coordination. Depending on the task, one agent can control one asset, one building, or an aggregated community. This level provides a clean comparison between adaptation and coordination.

Proximal Policy Optimisation. PPO is an on-policy actor–critic method that limits the size of policy updates through a clipped objective (Schulman et al. 2017). It is useful as a robust baseline and as the high-level learner in the hierarchical coordinator. Its main cost is data: samples are used for a limited number of updates and are then discarded.

Soft Actor–Critic. SAC is an off-policy method that optimises reward together with policy entropy (Haarnoja et al. 2018). It provides a strong continuous-control reference when exploration and sample reuse matter. In community-level control, SAC can learn an aggregated policy without the non-stationarity introduced by several learning agents, but its action space grows when all assets are controlled centrally.

Twin Delayed Deep Deterministic Policy Gradient. TD3 reduces overestimation in deterministic actor–critic learning through twin critics, target policy smoothing, and delayed actor updates (Fujimoto et al. 2018). It is relevant for bounded continuous energy actions and provides the single-agent basis for MATD3.

Together, PPO, SAC, and TD3 solve part of the adaptation problem. They can improve beyond fixed rules, learn from repeated operation, and avoid solving a full optimisation problem at every step. The comparison also reveals where aggregation becomes restrictive: one community policy can hide participant objectives, create a large joint action space, or require central access to data that should remain local.

Comparative learning results. Repeated-run learning curves, held-out evaluation outcomes, sample use, wall-clock cost, and inference requirements compare PPO, SAC, and TD3 under matched scenarios and information assumptions. Representative curves and the principal statistical comparison remain in the chapter; seed-level tables, hyperparameter sweeps, secondary reward variants, and unsuccessful configurations are archived in the appendix and experiment record.

5.4 Decentralised Multi-Agent Coordination

MADDPG. MADDPG extends deterministic policy gradients to multi-agent settings through centralised training with decentralised execution (Lowe et al. 2017). Each actor uses local observations at execution, while its critic can condition on joint observations and actions during training. This matches the EnergAIze design in which dwelling or asset controllers operate locally but learn from community interactions.

The inherited EnergAIze implementation uses one agent per dwelling or controllable node, continuous actions, individual and community reward components, and RBC-guided early exploration (Fonseca, Ferreira, Cabral, Severino, and Praça 2024). It established that decentralised policies can pursue local service and economic outcomes while learning from community effects. This solved a problem that the centralised single-agent formulations did not: local policy execution with an explicitly multi-agent training objective.

The remaining issues include non-stationarity, sensitivity to reward and exploration settings, critic growth with joint information, and deterministic value overestimation. These observations motivate MATD3 and later structural changes.

MATD3. MATD3 extends the twin-critic and delayed-update ideas of TD3 to centralised multi-agent critics (Ackermann et al. 2019). The standard algorithm is prior work, but the thesis can make substantial contributions through its implementation, adaptation, evaluation, and integration.

MATD3 has three roles in the thesis:

1. **empirically**, by quantifying stability, critic disagreement, performance, and resource cost in heterogeneous REC tasks;

2. **methodologically**, when material modifications such as expert initialisation, event-aware replay, hierarchical coordination, dynamic interfaces, or physics-informed constraints are isolated through ablations; and

3. **operationally**, by integrating the selected MATD3-based policy into the EnergAIze experiment, policy-set, and edge lifecycle.

MATD3 is therefore an important contribution stage and a likely learning backbone for TI-MARL. It is not, however, the unifying algorithmic contribution of the thesis. The distinctive

contribution emerges from the way the later method combines learning, changing interfaces, physical feasibility, and safe execution.

MADDPG–MATD3 comparison. The comparison holds the actor structure, observations, actions, scenarios, replay budget, and evaluation procedure constant where possible. The main independent variables are the critic structure, target construction, policy smoothing, and actor-update schedule. The analysis includes:

- value overestimation and critic disagreement;
- stability and variance across seeds;
- convergence and sample efficiency;
- participant and community KPIs;
- sensitivity to reward scale and exploration noise; and
- training, memory, and inference overhead.

A modest KPI improvement with substantially lower variance can be more useful than a large improvement from one favourable seed. Conversely, a higher return without better service or energy outcomes is not sufficient evidence of progress.

Figure 5.3 separates the standard MADDPG-to-MATD3 algorithmic change from the thesis-specific adaptation and evaluation criteria.

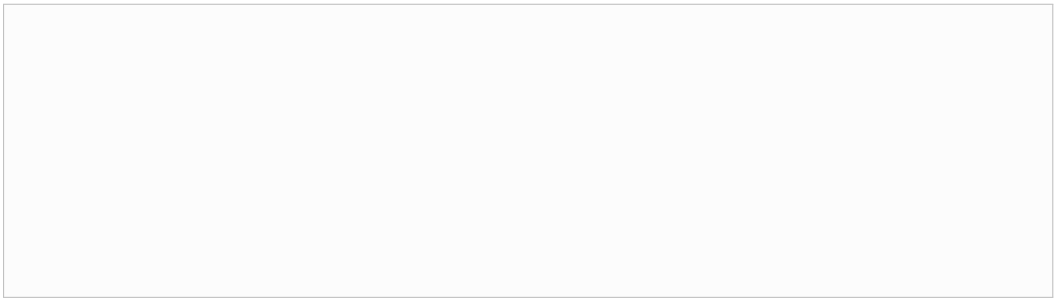


Figure 5.3: MADDPG and MATD3 training architecture and the evaluation dimensions used in their comparison.

Coordination and stability findings. The MADDPG–MATD3 comparison places energy and service outcomes beside seed-to-seed variability, critic diagnostics, training cost, and inference resource use. MATD3 constitutes a substantive stage only where the evidence shows that it resolves a measurable limitation of MADDPG; the thesis does not claim the standard MATD3 algorithm as a new invention. Named modifications and their ablations define any additional method contribution.

5.5 Data-Efficient, Reusable, and Hierarchical Learning

The next stage reuses knowledge that already exists in rules, optimisation trajectories, previous policies, and historical data. These mechanisms solve distinct problems and are evaluated separately before any combined configuration is retained.

RBC-Guided Exploration. Purely random exploration spends many interactions on actions that are infeasible or operationally meaningless. The inherited EnergAIze approach centres early exploration on a plausible RBC action and gradually gives the learned policy more influence. A generic blended action is:

$$a_t = (1-\alpha_t)a_t^{\mathrm{RBC}} + \alpha_t \pi_\theta(o_t) + \varepsilon_t, \tag{5.1}$$

where α_t changes over training and ε_t is bounded exploration noise. This mechanism solves part of the early-training problem by keeping trajectories near feasible operation. The remaining risk is expert anchoring: the learner may reproduce the rule instead of improving beyond it.

Behaviour Cloning. Behaviour cloning trains a policy from expert state–action pairs before RL fine-tuning. Experts can be produced by an RBC, operational MILP, hierarchical teacher, or validated historical controller. For continuous actions, the actor can minimise:

$$\mathscr{L}_{\mathrm{BC}}(\theta) = \frac{1}{|\mathscr{D}_E|} \sum_{(o,a_E)\in\mathscr{D}_E} \|\pi_\theta(o) - a_E\|^2. \tag{5.2}$$

Behaviour cloning is evaluated for its effect on the control problem: whether expert initialisation reduces required interaction, preserves feasible behaviour, and still permits improvement beyond the teacher.

Prioritised and Event-Aware Replay. Uniform replay treats ordinary intervals and rare critical events equally. Prioritised experience replay addresses this imbalance by sampling transitions according to a priority and correcting the resulting bias (Schaul et al. 2016). Energy trajectories contain many nominal steps and relatively few imminent departures, demand-response activations, near-violations, communication failures, or topology changes. The thesis evaluates standard TD-error priority and event-aware priority of the form:

$$p_k = \lambda_{\mathrm{TD}}|\delta_k| + \lambda_{\mathrm{event}} e_k + \lambda_{\mathrm{safety}} c_k + \varepsilon, \tag{5.3}$$

where e_k marks an operational event and c_k reflects proximity to a constraint or intervention. This stage seeks to solve inefficient learning from rare but important experiences. Bias correction, capped priorities, and ablation against uniform replay are necessary to show that the improvement does not come from an uncontrolled change in the data distribution.

Offline Reinforcement Learning. Offline RL learns from a fixed dataset and reduces the need for online exploration (Levine et al. 2020). This is useful when historical trajectories exist but physical trial-and-error is unacceptable. Its limitation is support: a policy that proposes actions not represented in the dataset can be evaluated unreliably. Behaviour cloning remains the conservative baseline, while methods such as CQL or IQL can be used where the action structure permits it (Kostrikov et al. 2022; Kumar et al. 2020).

The thesis uses offline learning as pretraining, as a test of what can be learned from existing operations, and as a pre-deployment stage before simulator fine-tuning. The contribution lies in the dataset and coordination conditions under which it succeeds or fails, especially when the active policy set and joint behaviour matter.

Policy Distillation. Distillation transfers behaviour from a larger or centralised teacher to a smaller or local student (Rusu et al. 2015). It can reduce runtime cost and support edge deployment. It can also remove information or coordination structure that the student needs. The experiments therefore report the performance–resource trade-off and whether the student preserves participant service, safe execution, and compatibility with the target interface.

These mechanisms address sample efficiency, reuse, and runtime simplification while retaining a fixed interface assumption. A cloned MATD3 policy still expects the observation and action contract on which it was trained, and an offline policy can remain data-efficient while failing when a new action appears. Structural reconfiguration therefore remains unresolved.

Hierarchical community coordination. A fully central controller can become difficult to scale and deploy, while independent local controllers can create community peaks. Hierarchical coordination introduces a smaller decision between the two. A community coordinator does not directly command every charger or battery; it modifies a signal that changes how local controllers interpret the current community condition.

The implemented hierarchy uses a high-level PPO coordinator and a low-level signal-aware controller. The coordinator observes community aggregates and emits a price multiplier. In the current implementation the multiplier is bounded in $[0.5, 1.3]$. A value of 1 leaves the parent low-level controller unchanged, which provides a clean comparison between standalone and coordinated operation.

The high-level controller acts every four 15-minute steps, while the local controllers act at every environment step. This temporal abstraction reduces the coordinator's horizon and keeps local feasibility close to the assets. Two variants are evaluated:

Global coordination. One multiplier is broadcast to the community. This is compact and easy to operate, but it cannot distinguish participants with different flexibility.

Per-building coordination. One multiplier is produced for each building. This allows targeted coordination at the cost of a larger observation, action, and monitoring interface.

The current high-level reward contains cost, peak, ramping, export, and violation terms; an additional variant includes EV service. The behaviour-cloning teacher also contains a proactive headroom heuristic. This mismatch is informative: behaviour cloning may teach the coordinator to act before a violation, while PPO can later weaken that behaviour if the reward only penalises the consequence. The analysis therefore compares the teacher, the fine-tuned policy, and the evolution between them, rather than only the terminal KPI.

Table 5.2: Hierarchical coordination matrix.

Low-level controller	**Standalone**	**Global CC**	**Per-building CC**	**Purpose**
BAU	yes	—	—	Passive denominator
RBC Basic	yes	yes	yes	Generalisation across rule sophistication
RBC Smart	yes	yes	yes	Headline controlled comparison
RBC Community	yes	yes	yes	Learned coordination versus community heuristic
MADDPG/MATD3	yes	yes	yes	Hierarchy over an already learned local controller

Hierarchical coordination solves part of the local–community decomposition. It can reuse local controllers, preserve a clear fallback path, and reduce the dimensionality of the community decision. Its remaining assumption is that the lower-level interfaces are already known and compatible. A new asset or capability still needs a representation that the coordinator and local policy can understand.

Figure 5.4 shows how the community coordinator modifies local control without replacing the local controllers or their fallback paths.

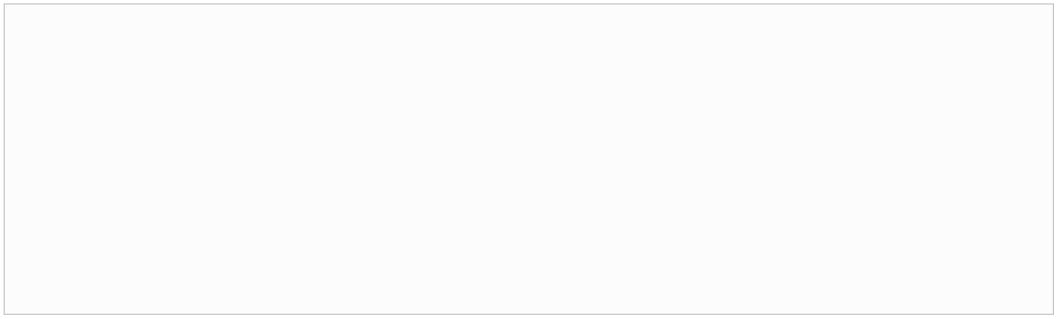


Figure 5.4: Hierarchical community coordination used in the progressive controller studies.

Mechanism-level ablations and results. Each enhancement is evaluated against the problem it was introduced to solve. Behaviour cloning and guided exploration are judged by initial feasibility

and interactions required to reach a performance threshold. Replay variants are judged by learning efficiency and rare-event coverage. Offline learning is judged by policy quality, dataset coverage, and extrapolation failures. Distillation is judged by model size, latency, and retained control quality. Hierarchy is judged by local–community outcomes and the cost of the additional coordination layer. Only the decisive ablations remain in the chapter; exhaustive sweeps and secondary combinations move to the appendix.

5.6 Unified Cross-Method Comparison and Solved-Problem Ledger

The broad experiment campaign is a funnel rather than a full Cartesian product. Deterministic controllers are first verified on small scenarios. Learning algorithms are screened under common fixed-population conditions. Strong candidates are then compared with the highest RBC levels and operational optimisation. Only a small number of backbones and enhancements are carried into TI-MARL and deployment studies.

Figure 5.5 brings the principal control, learning, service, and resource results into one comparison without reducing the chapter to a single winner.

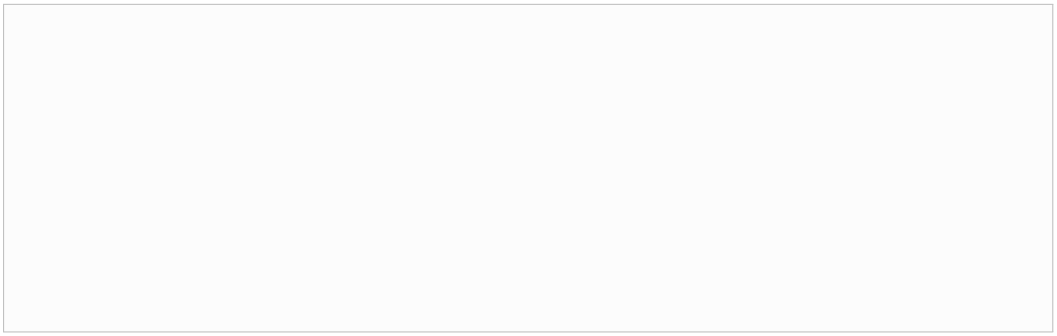


Figure 5.5: Comparative control, learning, and resource results across the progressive methods.

5.7 Accumulated Capabilities, Remaining Integration Problem, and RQ Contribution

The progression solves a substantial part of the original problem. It provides:

- feasible and transparent local control;
- explicit constrained scheduling and reference trajectories;
- adaptive continuous control;
- decentralised multi-agent coordination;
- a more stable deterministic MARL backbone;

- expert and historical data reuse;
- rare-event and safety-relevant replay; and
- hierarchical local–community coordination.

The unresolved problem is not that these capabilities are missing. It is that their assumptions do not automatically compose. A stable MATD3 actor can still expect a fixed observation vector. A cloned policy can inherit a teacher's fixed action grammar. A hierarchical coordinator can assume that every local controller interprets its signal in the same way. A safety mask can become stale if an asset gains a new mode or loses a supporting measurement.

The findings therefore lead to the following design requirements for TI-MARL:

1. represent each agent through a versioned capability schema, explicit module and entity instances, and a separate runtime health state;
2. compile entity-bound typed observations, dynamic action-group instances, parameterised action ports, dependencies, validity, bounds, and local constraint relations from those objects;
3. preserve the local information boundary while allowing parameter sharing by role and authorised aggregate community observations;
4. encode all active local entities and groups into one shared local representation, then jointly decode a coherent action bundle rather than independent asset actions or one fixed flat vector;
5. reuse shared type-conditioned decoders when known assets enter or leave, without adding permanent neural heads or rebuilding the global layout;
6. incorporate non-negotiable local constraints into bundle construction while leaving normal community optimisation to multi-agent learning;
7. learn community coordination through coupled local and community outcomes and variable-population set or graph critics during centralised training;
8. propagate sensor and module health through dependencies to action availability, bounds, degraded modes, and recovery;
9. evaluate population, entity, interface, computation, communication, and graph scale separately and jointly; and
10. expose the same typed contract to lifecycle compatibility, deployment admission, monitoring, and execution-level traces.

Contribution to the research questions. The controller decompositions and comparative evidence support RQ1 by establishing how local and community objectives are represented. Computational cost, data use, coordination, and scale assumptions contribute to RQ2. Explicit constraints, feasible references, expert behaviour, and fail-safe baselines contribute to RQ3. Together, the solved-problem ledger and remaining assumptions provide the causal basis for the TI-MARL design.

5.8 Chapter Summary

The algorithmic research forms a cumulative progression. Rules establish feasibility and fallback behaviour; MILP makes objectives, constraints, and reference schedules explicit; single-agent methods provide adaptive control; MADDPG introduces decentralised coordination; and MATD3 addresses part of the stability and value-estimation problem. Guided exploration, behaviour cloning, replay selection, offline learning, and distillation improve the use of knowledge and data, while hierarchy separates community and local decisions.

Each method retains a role as a reference, reusable mechanism, empirical finding, or component of the integrated solution. The remaining limitation is the fixed or manually maintained control contract, which motivates TI-MARL in the next chapter.

Table 5.3: Solved-problem ledger for the algorithmic progression.

Problem	**Solution stage**	**Capability established**	**Remaining assumption**
Plausible local behaviour	RBC ladder	feasibility, transparency, low-cost fallback, expert demonstrations	anticipated rules and fixed control grammar
Explicit constraints and reference scheduling	MILP	auditable objectives, feasibility, operational and oracle references	model/forecast maintenance and solve-time growth
Adaptive model-free control	PPO, SAC, TD3	adaptation and continuous control without repeated full optimisation	aggregation or fixed single-agent interface
Decentralised coordination	MADDPG	CTDE, local execution, individual/community reward interaction	training instability and fixed joint interface
More stable deterministic MARL	MATD3	twin-critic evidence, delayed updates, stronger backbone	fixed agents, observations, and actions
Efficient use of prior data	guided exploration, BC, replay, offline RL, distillation	faster initial learning, reuse, rare-event sampling, smaller policies	teacher/dataset support and interface compatibility
Local–community decomposition	hierarchical control	slower community signals combined with local feasibility	known lower-level controllers and interfaces

Chapter 6

TI-MARL: Typed-Interface Multi-Agent Reinforcement Learning for Reconfigurable Cyber-Physical Systems

The controller studies in Chapter 5 progressively removed several barriers to intelligent energy control. Rule-based controllers established feasible and transparent behaviour. Optimisation made objectives and constraints explicit. Single-agent reinforcement learning introduced adaptation, while MADDPG, MATD3, and hierarchical control enabled decentralised coordination, improved learning stability, and separated local decisions from community objectives. Behaviour cloning, offline learning, replay selection, and distillation improved the use and reuse of data. These results remain independent contributions and provide the methods, teachers, baselines, and fallback controllers used later in the thesis.

A common assumption nevertheless remained. The controller was designed for a known collection of agents, assets, observations, and action variables. Adding a second charger to a building changed the input and output layout. Replacing an unidirectional charger with a V2G-capable device introduced new observations and actions. A stale state-of-charge measurement altered whether discharging remained safe. A building joining or leaving the community changed the joint learning problem. These changes concern more than the numerical state of the system: they alter the interface through which an agent observes and acts.

TI-MARL addresses this interface shift directly. Each decentralised agent acts through a runtime typed interface compiled from its current modules, controlled entities, observations, health state, constraints, and authorised community signals. The policy encodes a variable entity-bound interface, creates one local latent representation, and jointly decodes a coherent action bundle across the controllable groups that currently exist. The local bundles form the multi-agent joint action. Community-level behaviour is learned through aggregate community observations, cou-

pled rewards, and centralised or structured critics during training; execution remains decentralised. Hard local constraints are incorporated into bundle construction, while a separate shared-constraint monitor is reserved for non-negotiable exceptional violations rather than normal community optimisation.

A building with several EV chargers, a stationary battery, and a heat pump provides the running example. Each charger is a distinct module instance, each connected EV session is an entity, and every observation and action is bound to the entity and module that give it meaning. A new charger can therefore create a new action-group instance without adding a permanent neural head. The same example is used to explain health propagation, local bundle construction, multi-agent learning, shared limits, deployment compatibility, and typed execution traces. The formal model is domain-independent; the empirical evaluation remains centred on Renewable Energy Communities and EV charging.

6.1 Scientific Positioning and Problem Formulation

6.1.1 Closest Approaches and the Residual Problem

Several neighbouring research lines already solve parts of the problem. Parameterised-action methods combine a discrete action type with continuous parameters (Li, Tang, et al. 2022; Xiong et al. 2018). Variable-action policies can process action candidates whose cardinality, order, or semantic content changes (Jain et al. 2020; Sinii et al. 2024). Modular and cross-embodiment policies reuse one controller across different sensors, actuators, and morphologies (Doshi et al. 2025; Huang et al. 2020; Luketina et al. 2025). Heterogeneous and task-open MARL address different action semantics, changing tasks, and dynamic populations (Anil et al. 2025; Yu, Lin, et al. 2024). Sensor-dropout training improves robustness to missing modalities (Liu et al. 2017), while joint-action shields prevent unsafe combinations of actions (Elsayed-Aly et al. 2021).

These results exclude several broad novelty claims. TI-MARL is not the first method to process variable action sets, parameterised actions, heterogeneous agents, or changing populations. Its encoder, graph representation, actor–critic optimiser, and safety monitor are established classes of mechanism. The residual problem is the relationship among those mechanisms. The same runtime contract should identify the active entities, bind observations to their sources, instantiate the action groups and ports that currently exist, propagate health and dependency changes into port validity and bounds, preserve local information boundaries, and remain meaningful to the policy, critic, safety logic, deployment validator, and explanation trace.

This position is especially important for assets that enter and leave. A padded universal vector can reserve ten charger slots and mask the unused positions. Such a representation is often sufficient when the maximum topology and all action semantics are known in advance. It becomes less natural when a newly admitted module brings a coherent observation–action contract, when several instances of the same type must be distinguished, or when a failed observation changes

the safe domain of only the dependent actions. TI-MARL tests whether an explicit entity-bound typed interface provides a useful inductive and operational structure in those cases.

The method also extends, rather than replaces, the progression established in Chapter 5. Table 6.1 identifies the problem solved by each family and the assumption that TI-MARL relaxes.

Table 6.1: TI-MARL as a cumulative extension of the controller progression in Chapter 5.

Stage	**Problem addressed**	**Assumption that remained**	**TI-MARL extension**
RBC and fallback control	Feasible, transparent, low-cost operation	Rules and device interfaces are manually fixed	Health-aware interface compilation and typed fallback scope
MILP and optimisation	Explicit objectives and constraints	Variables and system dimensions are defined before solving	Runtime entity/action instances and typed local constraints
PPO, SAC, and TD3	Adaptive control without a complete online optimiser	One agent and one fixed observation–action signature	TI-RL as the one-agent typed-interface reduction
MADDPG and MATD3	Decentralised multi-agent control and improved actor–critic stability	Fixed agent/action layout and concatenated joint critic input	Local typed actors and set/graph critics over variable agents
Behaviour cloning and offline RL	Reuse of expert and historical data	Demonstrations are tied to a particular interface layout	Schema- and port-referenced trajectories and replay
Hierarchical coordination	Separation of local and community objectives	Participants, roles, and signals are predefined	Community signals as authorised typed observations under dynamic membership

The scientific claim is therefore narrower than "MARL for heterogeneous systems". TI-MARL studies decision problems in which the interface itself is a time-varying structured object. A deterministic compiler gives that object operational meaning; the policy learns over the compiled interface; and the same semantics support reconfiguration, robustness, safety, deployment compatibility, and typed explanations. The comparative experiments in Sections 6.8 and 6.9 test whether that coupling provides measurable value beyond simpler masked, modular, parameterised-action, and dynamic-population baselines.

6.1.2 Typed-Interface Decision Processes

The one-agent reduction isolates interface reconfiguration from cross-agent learning. A TI-MDP is defined by

$$\mathscr{M}^{\mathrm{TI}} = \left\langle \mathscr{S}, \Sigma, \mathscr{H}, \mathscr{C}, \Gamma^{\mathrm{loc}}, P, P_{\Sigma}, R, \gamma \right\rangle, \tag{6.1}$$

where $\mathscr{S}$ is the physical and operational state space, Σ_t is the current versioned capability schema, $\mathscr{H}_t$ is the runtime health state, $\mathscr{C}$ is the Typed Interface Compiler, and Γ_t^{loc} is the local typed relation and constraint graph. P_{Σ} represents interface events such as module addition, module removal, sensor degradation, or recovery. Learning in this one-agent model is denoted TI-RL. It supports controlled evaluation of multiple chargers, variable action groups, health-aware contraction, and local bundle feasibility without introducing multi-agent credit assignment.

A TIMG adds an active agent population, local information boundaries, coupled outcomes, and shared resources:

$$\mathscr{G}^{\mathrm{TI}} = \langle \mathscr{S}, \mathscr{N}, \Sigma, \mathscr{H}, \mathscr{C}, \Gamma, P, P_{\Sigma}, \{R_i\}_{i \in \mathscr{N}}, \gamma \rangle. \tag{6.2}$$

At time t, $\mathscr{N}_t \subseteq \mathscr{N}$ denotes the active population. Each agent i has a schema $\Sigma_{i,t}$ and health state $\mathscr{H}_{i,t}$. The physical state is augmented with the active population and interface state:

$$\tilde{s}_t = (s_t, \mathscr{N}_t, \{\Sigma_{i,t}, \mathscr{H}_{i,t}\}_{i \in \mathscr{N}_t}, \Gamma_t). \tag{6.3}$$

The compiler produces the policy-facing interface of agent i:

$$\mathscr{I}_{i,t} = \mathscr{C}\left(\Sigma_{i,t}, \mathscr{H}_{i,t}, \mathrm{view}_i(\tilde{s}_t)\right) = \left(\mathscr{V}_{i,t}, \mathscr{E}_{i,t}, \mathscr{G}^a_{i,t}, \mathscr{P}_{i,t}, \mathscr{B}_{i,t}\right), \tag{6.4}$$

where $\mathscr{V}_{i,t}$ and $\mathscr{E}_{i,t}$ are the typed interface nodes and relations, $\mathscr{G}^a_{i,t}$ is the variable set of active action-group instances, $\mathscr{P}_{i,t}$ is the set of valid action-port instances, and $\mathscr{B}_{i,t}$ contains the current parameter domains and local hard constraints. The view function enforces the information boundary: local actors receive their local interface and explicitly authorised aggregate context, not the private observations of other agents.

The policy produces one decision for each active action-group instance:

$$a_{i,t} = \{a_{i,g,t}\}_{g \in \mathscr{G}^a_{i,t}}, \qquad a_{i,g,t} = (p_{i,g,t}, \eta_{i,g,t}), \tag{6.5}$$

where $p_{i,g,t}$ is a selected valid port and $\eta_{i,g,t}$ is its parameter vector. The local action is therefore a variable structured bundle rather than one permanent vector. A constraint-aware construction Φ_i maps raw decoded group decisions into a locally feasible bundle:

$$\bar{a}_{i,t} = \Phi_i\left(a_{i,t}, \Gamma^{\mathrm{loc}}_{i,t}\right), \qquad \bar{a}_{i,t} \in \mathscr{A}^{\mathrm{loc}}_{i,t}. \tag{6.6}$$

The multi-agent joint action is

$$\bar{\mathbf{a}}_t = (\bar{a}_{i,t})_{i \in \mathscr{N}_t}. \tag{6.7}$$

An optional hard shared-constraint monitor $\mathscr{S}_\Gamma$ may intervene when a non-negotiable constraint is violated:

$$\mathbf{a}_t = \mathscr{S}_\Gamma(\bar{\mathbf{a}}_t, \Gamma_t). \tag{6.8}$$

Normal community optimisation remains the responsibility of the learned agents. The monitor is not a community controller and does not allocate resources according to utility or fairness; it provides exceptional minimum correction or fallback for hard shared limits. The physical transition is

$$s_{t+1} \sim P(\cdot \mid s_t, \mathbf{a}_t). \tag{6.9}$$

Local and community outcomes are coupled through this joint transition. A representative reward decomposition is

$$r_{i,t} = r_{i,t}^{\text{loc}} + \alpha_i r_t^{\text{com}} - \beta_i c_{i,t}^{\text{loc}} - \eta_i c_t^{\text{shared}}, \tag{6.10}$$

where local terms capture cost, service, comfort, or degradation; the community term captures peak, ramping, renewable utilisation, or a shared demand-response objective; and the constraint terms distinguish local and shared violations or interventions. When $|\mathscr{N}_t| = 1$ and Γ_t contains only local relations, the model reduces to TI-RL.

The formal objects are domain-independent. In energy, a module can be a charger or battery, an entity can be an EV session, a community observation can be transformer loading, and a port can be CHARGE-EV(p). In robotics, the same structure can represent a sensor module, a manipulated object, workspace occupancy, and a parameterised grasp action. The thesis evaluates the method in energy systems while preserving this broader interpretation.

Figure 6.1: TI-RL as the one-agent reduction of TI-MARL.

6.2 TI-MARL Architecture at a Glance

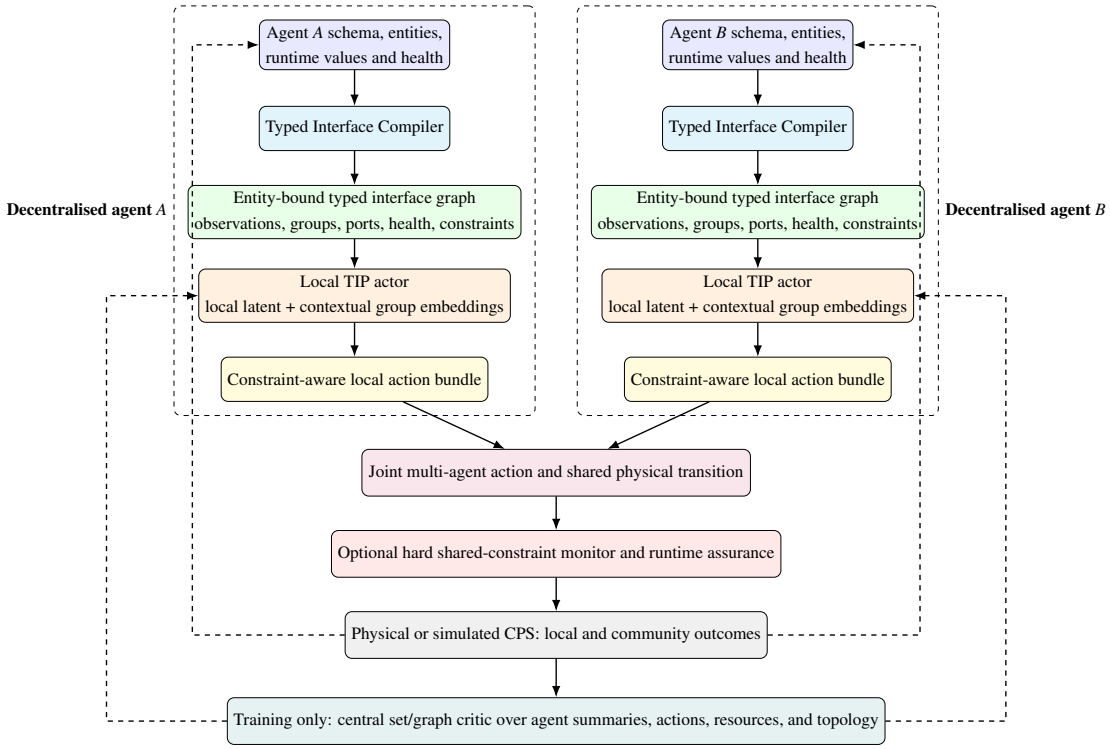


Figure 6.2: TI-MARL architecture. Each agent compiles and encodes its own entity-bound runtime interface, then produces one coherent local action bundle. Community coordination is learned through aggregate community observations, coupled rewards, the joint transition, and a training-only central set/graph critic. The optional shared monitor is reserved for hard exceptional constraints.

Figure 6.2 separates the runtime path from the training path. Each local actor receives its own compiled interface and authorised aggregate signals. It does not receive the raw private state of other agents. During training, a central critic evaluates the joint consequence of all local bundles using a variable-cardinality representation. At deployment, the central critic is removed and the local actors execute independently.

The method contains five principal components.

The architecture is deliberately asymmetric between training and operation. The actors remain local in both phases. Centralisation is introduced only in the critic and experiment infrastructure, where it can improve credit assignment without creating a runtime dependency on unrestricted global state. Community observations remain ordinary typed observation parts with explicit provenance, scope, timestamp, and health.

Table 6.2: Main TI-MARL components and their interpretation in the running energy example.

Component	Generic role	Energy example
Typed Interface Compiler	Converts active modules, entities, observations, health, and constraints into the current policy-facing interface	Adds one charger group when a charger is installed; removes it on failure; contracts V2G bounds when SoC becomes stale
Entity-bound interface graph	Preserves which observation and action belongs to which entity and source module	Binds EV SoC from `charger_2` to the EV session and actions controlled through `charger_2`
Local Typed-Interface Policy	Encodes the variable local graph and jointly decodes the active controllable groups	Produces actions for two chargers, one battery, and one heat pump from one local latent representation
Constraint-aware bundle construction	Enforces local non-negotiable coupled constraints without replacing the policy objective	Keeps all charger, battery, and heat-pump actions within the building and phase limits
Central set/graph critic	Evaluates joint behaviour during training while accepting variable agents and resource relations	Learns the value of the buildings' joint bundles relative to transformer loading, community peak, and service outcomes

6.3 Entity-Bound Runtime Typed Interfaces

6.3.1 Schemas, Entities, and Dynamic Module Instances

A capability schema describes the potential interface of an agent. It identifies the agent role, module types, observation templates, action-group templates, constraints, compatibility requirements, and fallback semantics. Runtime state identifies which module instances and controlled entities are active now. This distinction prevents fast-changing values such as SoC or connection state from being treated as schema revisions.

A building can contain several instances of the same module type. In the running example, agent A contains two charger instances, one battery, one heat pump, and one main meter:

$$\mathscr{M}_{A,t} = \{m_1^{\text{meter}}, m_1^{\text{charger}}, m_2^{\text{charger}}, m_1^{\text{battery}}, m_1^{\text{hp}}\}. \tag{6.11}$$

Each charger can be related to a currently connected EV session. The observation

```
semantic type:  EV_SOC
entity:  ev_session_17
source module:  charger_1
```

cannot be confused with the SoC of the vehicle connected to `charger_2`. The corresponding action group is similarly bound:

```
group type:  EV_CHARGER_MODE
owner module:  charger_1
target entity:  ev_session_17
```

The compiler represents these relationships as typed edges such as `owns`, `connected_to`, `observes`, `controls`, and `requires`. Entity binding is therefore part of the interface semantics rather than a positional convention in a flat vector.

When `charger_3` is installed, a module-addition event creates a new module node, its observation parts, one EV-charger action-group instance, valid port instances, and the relevant constraint edges. The neural network is not extended with a new permanent head. The same group and port decoder used for the existing chargers is applied to the new instance. Removing the charger removes that subgraph. A semantically unknown module is admitted only when a compatible parent type and decoder are available; otherwise its learned control path remains inactive and fallback semantics apply.

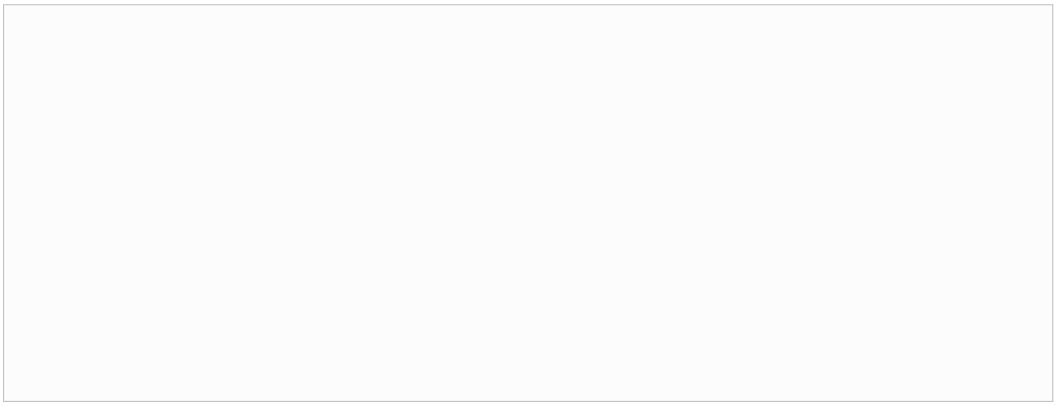


Figure 6.3: Entity-bound interface graph for a building with several chargers and changing EV sessions.

6.3.2 Typed Observation Parts and Community Context

An observation part is a typed object rather than an anonymous vector position. A representative definition is

$$o_k = (\tau_k, \mathrm{id}_k, \mathrm{entity}_k, \mathrm{source}_k, x_k, u_k, q_k, h_k, t_k, s_k), \tag{6.12}$$

where τ_k is the semantic type, x_k the value, u_k the unit, q_k quality or uncertainty metadata, h_k the health state, t_k the timestamp, and s_k the scope. The scope can be self, local, neighbourhood, community, or global. The source can be a physical sensor, derived signal, forecast, external platform, or coordinator message.

Community information is represented through the same object model. A building may receive numerical aggregates such as community net demand, transformer loading, remaining headroom,

photovoltaic surplus, a price multiplier, or a demand-response target. These values do not reveal the raw private state of another building. Their provenance and health remain explicit:

```
semantic type: COMMUNITY_TRANSFORMER_LOADING
scope: community; source: platform_service
unit: percent; health: HEALTHY
```

Flexibility is not a mandatory negotiation object in TI-MARL. In the energy instantiation, an EV connected from 08:00 until 18:00 with a current SoC, target SoC, maximum charging power, and departure requirement already exposes the information from which a policy can learn temporal flexibility. Useful derived quantities such as time to departure, remaining required energy, or required average charging power can be supplied as typed observation parts when they improve learning or interpretability. Their use is an experimental design choice rather than a separate offer phase or community allocator.

Table 6.3 lists representative observation parts for the running example.

Table 6.3: Representative typed observation parts for a building with two chargers. Values are illustrative.

Semantic type	Entity/source	Value	Scope	Decision meaning
House net power	main meter	2.1 kW	self	Current aggregate import used to evaluate local electrical limits
EV SoC	session 17 / charger 1	0.35	local	Energy state of the EV controlled through charger 1
EV target SoC	session 17	0.80	local	Minimum service target at departure
Departure time	session 17	10 h	local	Temporal slack available to the charging policy
EV SoC	session 28 / charger 2	0.62	local	Distinct SoC bound to charger 2 and its connected session
Battery SoC	battery 1	0.70	local	Available storage state
Community headroom	platform service	5.5 kW	community	Aggregate remaining shared capacity made visible to all authorised agents
Community peak state	platform service	high	community	Community context used by local actors and the reward

6.3.3 Dynamic Action Groups and Typed Parameterised Ports

The output interface is organised around action-group instances. Each controllable module creates one or more groups whose ports are mutually exclusive within the group but can be combined with ports from other groups. For example,

$$\mathscr{P}_{\text{charger}_1,t} = \{\text{IDLE-EV}, \text{CHARGE-EV}(p), \text{DISCHARGE-EV}(p)\}, \tag{6.13}$$

$$\mathscr{P}_{\text{battery}_1,t} = \{\text{IDLE-BATTERY}, \text{CHARGE-BATTERY}(p), \text{DISCHARGE-BATTERY}(p)\}, \tag{6.14}$$

$$\mathscr{P}_{\text{hp}_1,t} = \{\text{OFF}, \text{HEAT}(p), \text{COOL}(p)\}. \tag{6.15}$$

An action port is a typed callable capability:

$$p = \left(\tau_p, \text{id}_p, \text{group}_p, \text{owner}_p, \Theta_p, U_p, B_{p,t}, V_{p,t}, D_p, K_p\right), \tag{6.16}$$

where Θ_p is the parameter schema, U_p the units, $B_{p,t}$ the current parameter domain, $V_{p,t}$ the validity state, D_p the observation and capability dependencies, and K_p the resource effects used by local constraints, critics, and optional safety checks.

A new charger creates another instance of the charger action group. It does not create another set of network parameters. The decoder receives the group embedding, port embedding, local context, and current bounds. The same function can therefore act on `charger_1`, `charger_2`, and `charger_3` while preserving their distinct entity bindings.

Table 6.4: Representative action ports for two charger instances and one stationary battery.

Port	**Owner/target**	**Parameter**	**Current validity and bounds**	**Dependencies and local effects**
Charge EV	charger 1 / session 17	power [kW]	valid, $[0, 7.2]$	connected session; increases house import
Discharge EV	charger 1 / session 17	power [kW]	valid, $[0, 4.0]$	V2G, trustworthy SoC, reserve; reduces house import
Charge EV	charger 2 / session 28	power [kW]	valid, $[0, 11.0]$	distinct connection and EV state; increases house import
Discharge EV	charger 2 / session 28	power [kW]	absent	charger 2 has no V2G capability
Discharge battery	battery 1	power [kW]	valid, $[0, 3.0]$	battery SoC and reserve; reduces house import

6.3.4 Health-Aware Causal Compilation

Health is part of interface semantics rather than only a feature supplied to the neural network. Each source or module declares criticality, dependency type, failure scope, degradation policy, and recovery conditions. Representative health states include HEALTHY, DEGRADED, STALE, ESTIMATED, MISSING, FAILED, and UNTRUSTED.

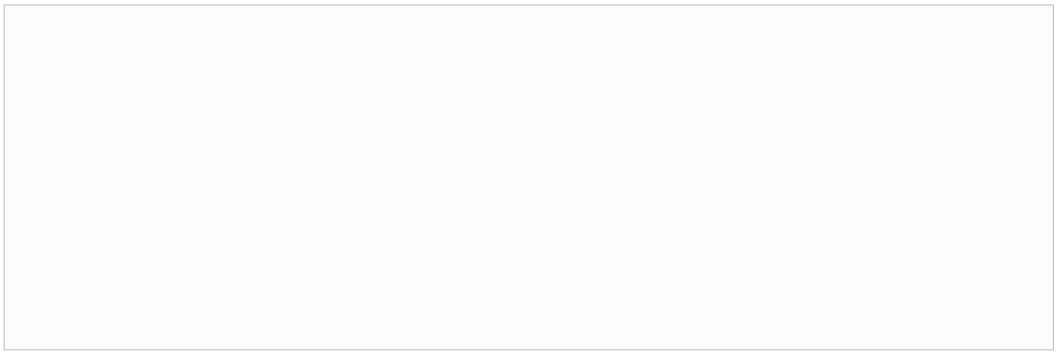


Figure 6.4: Anatomy of an entity-bound typed observation part and typed action port.

A charger failure removes only the associated charger group and its ports. Other modules controlled by the building agent remain active. A stale EV SoC can preserve charging while contracting or disabling discharging. Failure of the main house meter has a wider consequence: if the meter is required to verify the aggregate import or phase limit, learned flexible control for the complete building enters an isolated safe mode and a certified local fallback assumes control. Loss of a community signal does not necessarily suspend local operation; it removes or degrades the corresponding community observation and activates the approved local behaviour for coordination loss.

The compiler computes the transitive effect of a health event through a dependency closure:

$$\mathscr{H}^{\star}_{i,t} = \text{Closure}\left(\mathscr{H}_{i,t}, D_{i,t}\right), \tag{6.17}$$

which identifies affected observations, ports, groups, bounds, local constraints, and operational modes. For the EV SoC example,

$$\text{HEALTHY} \Rightarrow \text{DISCHARGE-EV} \in [0, 4.0]\ \text{kW}, \tag{6.18}$$

$$\text{STALE} \Rightarrow \text{DISCHARGE-EV} \in [0, 1.0]\ \text{kW}, \tag{6.19}$$

$$\text{FAILED} \Rightarrow \text{DISCHARGE-EV} \notin \mathscr{P}^{\text{valid}}_{i,t}. \tag{6.20}$$

This behaviour differs from replacing a missing value with zero and asking the policy to infer the safety consequence. The dependency rule changes the decision interface itself. Recovery is also explicit: a port can be reintroduced only after the declared number of fresh or consistent samples has been observed.

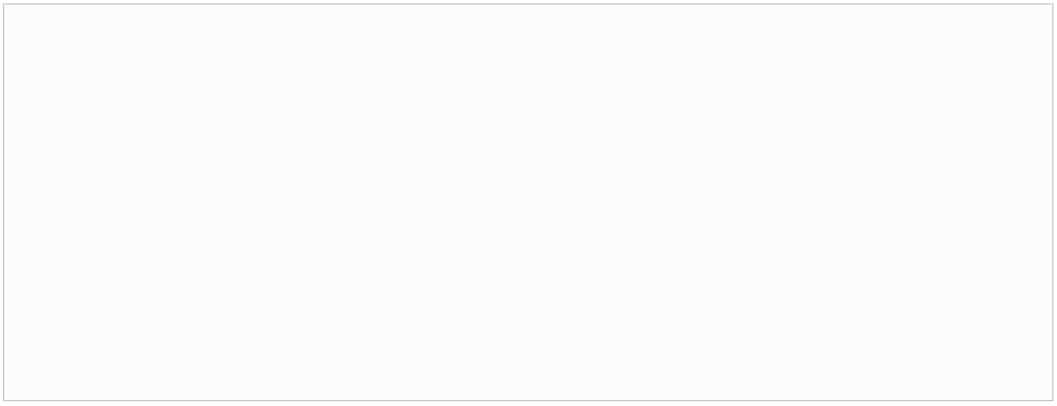


Figure 6.5: Health-aware recompilation and failure containment.

6.4 Local Typed-Interface Policies

6.4.1 Local Information Boundaries and Parameter Sharing

Each agent executes a logically local TIP. A building actor receives its compiled interface, optional memory, and authorised aggregate community observations. It does not receive the raw loads, EV states, or private service requirements of other buildings. This boundary is identical during evaluation and deployment; privileged information used by a central critic during training is not silently passed to the actor.

Agents with compatible roles may share policy parameters:

$$\pi_i = \pi_{\theta_{\rho(i)}}\left(\mathscr{I}_{i,t}, m_{i,t}\right), \tag{6.21}$$

where $\rho(i)$ denotes a role or compatible interface family. Parameter sharing does not imply shared observations. House A and House B can execute the same weights on different local graphs. Role-specific adapters can be introduced when the underlying entity or port families differ materially.

This structure supports population changes without a permanent actor slot per agent. A new building instantiates the appropriate local policy family after schema compatibility has been checked. Removing a building removes its actor instance from the active population; the shared parameter set and the actors of the remaining agents are unchanged.

6.4.2 Typed Graph Encoding and Local Latent State

The compiled local interface is represented as a typed graph or equivalent structured set. Nodes can represent the agent, modules, controlled entities, observation parts, action groups, ports, and local constraint or resource objects. Edges preserve relationships such as ownership, source, target, dependency, conflict, and bounded-by.

Type-specific encoders first map raw objects into a common latent dimension:

$$z_k^o = E_{\tau_k}^o\left(x_k, m_k\right), \tag{6.22}$$

$$z_g^g = E_{\tau_g}^g\left(\text{schema}_g\right), \tag{6.23}$$

$$z_p^p = E_{\tau_p}^p\left(\text{schema}_{p,t}\right). \tag{6.24}$$

A set, graph, or Transformer encoder then produces a local agent representation and contextual group embeddings:

$$\left(z_{i,t}, \{z_{i,g,t}\}_{g \in \mathcal{G}_{i,t}^a}\right) = F_\theta\left(\mathcal{I}_{i,t}\right). \tag{6.25}$$

The common latent $z_{i,t}$ summarises the complete local decision context. The group embeddings are not independent. Message passing or cross-group attention allows the representation of `charger_1` to account for `charger_2`, the battery, the heat pump, the main meter, and the local electrical limits before any port is selected.

Variable cardinality is handled structurally. Adding a charger adds nodes and one group embedding; it does not change the size of the network parameters. The computational cost grows with the number of local entities and relations rather than with a predeclared maximum vector containing inactive slots.

Figure 6.6: Local typed graph encoder and dynamic group representations.

6.4.3 Dynamic Grouped Action Decoding

The actor does not contain a permanent neural head for each possible asset instance. A shared decoder is applied to every active group instance. For group g and candidate port p,

$$\ell_{i,g,p,t} = f_\theta^{\text{select}}\left(z_{i,t}, z_{i,g,t}, z_{i,g,p,t}^p\right), \tag{6.26}$$

and the port is selected from the current valid set:

$$\Pr\left(p_{i,g,t} = p\right) = \text{Softmax}_{p \in \mathcal{P}_{i,g,t}^{\text{valid}}}\left(\ell_{i,g,p,t}\right). \tag{6.27}$$

A type-conditioned decoder generates the parameters:

$$\eta_{i,g,t} = d_{\tau(p)}\left(z_{i,t}, z_{i,g,t}, z^{p}_{i,g,p,t}, B_{i,g,p,t}\right). \tag{6.28}$$

The same selector and decoder can be invoked for several charger instances. Instance identity and current observations are carried by the group and entity embeddings. Ports with different semantic types can use specialised parameter decoders while sharing the higher-level encoder and selection mechanism.

All groups are decoded as one bundle from the same local context. A representative raw bundle is

$$a^{\text{raw}}_{A,t} = \left\{ \begin{array}{l} \texttt{charger_1} : \text{CHARGE-EV}(4.0\ \text{kW}), \\ \texttt{charger_2} : \text{CHARGE-EV}(3.5\ \text{kW}), \\ \texttt{battery_1} : \text{DISCHARGE-BATTERY}(1.0\ \text{kW}), \\ \texttt{heat_pump_1} : \text{HEAT}(1.2\ \text{kW}) \end{array} \right\}. \tag{6.29}$$

The bundle-level representation allows the actor to learn priorities among the two EV sessions, battery use, thermal demand, and community conditions. It does not by itself provide a mathematical guarantee that the combined power remains within the local electrical limits. That guarantee is addressed by the final local construction.

6.4.4 Constraint-Aware Local Bundle Construction

Local hard constraints should not depend exclusively on a negative reward. An actor can learn to reduce violations, but safety-critical electrical and service limits require explicit semantics. TI-MARL therefore separates the learned preference represented by the raw bundle from a constraint-aware parameterisation that returns the final local bundle.

Let $a^{\text{raw}}_{i,t}$ denote the decoded bundle and $\mathscr{F}^{\text{loc}}_{i,t}$ the feasible set induced by the local typed graph. The construction is

$$\bar{a}_{i,t} = \Phi_i\left(a^{\text{raw}}_{i,t}, \mathscr{F}^{\text{loc}}_{i,t}\right), \qquad \bar{a}_{i,t} \in \mathscr{F}^{\text{loc}}_{i,t}. \tag{6.30}$$

The mapping can be implemented through an analytic bounded parameterisation, an autoregressive resource-aware decoder, a differentiable projection, or a small local optimisation layer. The method requires the mapping to preserve the actor's priorities as far as possible and to expose any adjustment. It must not introduce a separate community objective or choose the high-level control strategy on behalf of the policy.

For the running example, one local constraint is

$$P^{\text{ch1}}_{t} + P^{\text{ch2}}_{t} + P^{\text{hp}}_{t} - P^{\text{bess,dis}}_{t} \leq P^{\max}_{A,t}. \tag{6.31}$$

The actor decides which EV is urgent, whether the battery should discharge, and how thermal demand should be served. The construction prevents the resulting combination from exceeding $P_{A,t}^{\max}$. Two diagnostics distinguish a useful feasibility mechanism from one that hides poor policy learning:

$$I_t^{\text{loc}} = \mathbb{1}\left[a_{i,t}^{\text{raw}} \notin \mathscr{F}_{i,t}^{\text{loc}}\right], \tag{6.32}$$

$$D_t^{\text{loc}} = \left\| \bar{a}_{i,t} - a_{i,t}^{\text{raw}} \right\|. \tag{6.33}$$

Frequent or large adjustments indicate that the actor relies on the construction layer rather than learning coherent bundles. These measures are reported alongside reward and constraint outcomes.

6.5 Multi-Agent Learning and Community Coordination

6.5.1 Joint Training and Decentralised Execution

TI-MARL does not require a community controller agent. Each building acts independently from its own compiled interface and authorised community observations. The method becomes multi-agent because the local bundles form one joint action, the physical transition depends on that joint action, local actions compete for shared resources, and rewards include local and community consequences.

At one training step,

$$\bar{\mathbf{a}}_t = (\pi_1(\mathscr{I}_{1,t}), \ldots, \pi_n(\mathscr{I}_{n,t})) \tag{6.34}$$

is applied to the shared environment. The transition produces the next local interfaces, local rewards, and common community outcomes. A low transformer headroom followed by simultaneous charging from two buildings can reduce the community reward or activate the hard shared monitor. Over repeated interaction, the agents learn how their local state and aggregate community signals relate to the likely behaviour of the other agents.

Joint training means that actor updates are derived from the same joint transition. It does not mean that the actors share their private runtime inputs or that one global actor replaces them. Centralised Training with Decentralised Execution is used to improve credit assignment: the critic can observe more of the joint training context, while the actor remains restricted to the information available at deployment.

6.5.2 Central Set and Graph Critics

A conventional central critic often concatenates all agent observations and actions. That representation is tied to a fixed ordering and fixed maximum number of agents. TI-MARL instead

represents the training context as a variable set or graph.

An agent node can contain a summary of its local typed interface, local latent representation, selected action bundle, role, and current availability. Shared-resource nodes represent objects such as a transformer, feeder, lane, CPU pool, or workspace. Typed edges indicate which agents affect which resource and which constraints couple their actions. A graph critic therefore evaluates the same joint transition while accepting a changing number of agents and resource relations:

$$Q_\phi\left(\tilde{s}_t, \bar{\mathbf{a}}_t\right) = G_\phi\left(\{z_{i,t}, \bar{a}_{i,t}\}_{i \in \mathcal{N}_t}, \Gamma_t\right). \tag{6.35}$$

A set critic is sufficient when all agents interact through one global aggregate and explicit topology adds little information. A graph critic is preferable when buildings share different phases, feeders, or local resources. The critic is a training instrument, not an operational coordinator. Its output shapes actor updates but is not executed as a control command.

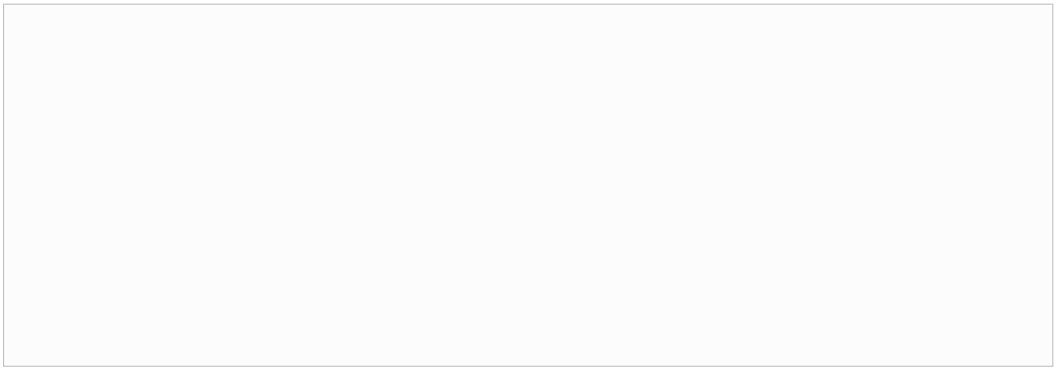


Figure 6.7: Central graph critic for joint training with decentralised local actors.

6.5.3 TI-IPPO, TI-MAPPO, and TI-MATD3

TI-MARL defines the interface, actor structure, local bundle semantics, and variable-agent training representation. The optimiser remains replaceable.

TI-IPPO. Each local TIP is updated with PPO using local trajectories and shared or role-specific parameters. This provides a useful decentralised baseline but gives the actor limited assistance with multi-agent credit assignment.

TI-MAPPO. Local stochastic TIP actors are paired with a central set or graph value critic. The categorical port selections and continuous port parameters can be represented through mixed distributions. The critic uses joint training context; the actors retain local information boundaries. This is the most direct reference implementation for hybrid discrete–continuous action groups.

TI-MATD3. Local deterministic TIP actors are paired with two central graph critics, target networks, and delayed actor updates. Port selection requires a differentiable relaxation or equivalent

training mechanism, while continuous parameters retain the deterministic actor structure. The critics evaluate the joint bundle and can preserve MATD3's twin-value and delayed-update advantages identified in Chapter 5. TI-MATD3 is especially relevant when the dominant actions are continuous power set-points.

The thesis does not define TI-MARL by one optimiser. The reference implementation is selected according to implementation maturity and the action structure of the final experiments; the remaining backbone is retained as an ablation or comparative instantiation.

6.5.4 Community Observations, Rewards, and Learned Coordination

Community coordination is learned from numerical aggregate observations and coupled outcomes. Typical community-scoped parts include transformer loading, remaining headroom, net community demand, photovoltaic surplus, current peak state, price multipliers, and demand-response targets. They are ordinary typed observations with explicit source, timestamp, health, and scope.

The policy is not given a symbolic flexibility offer. An EV's current SoC, target SoC, departure time, charging limits, connection state, and local price or community context are sufficient for the actor to learn whether charging can be delayed or reduced. Derived features such as remaining required energy and time to departure can be included when supported by the experimental protocol, but they remain observation parts rather than a separate negotiation mechanism.

The local and community reward terms expose the trade-off to the learning algorithm. A building can reduce its own cost while contributing to a community peak, or preserve community headroom while missing an EV service target. Reporting therefore separates participant outcomes, community outcomes, service, constraint interventions, and distributional effects. A single scalar return is used for optimisation where required, but the underlying metrics remain visible.

6.6 Typed Constraints, Safety, and Runtime Robustness

6.6.1 The Typed Constraint Graph

The TCG is the shared semantic representation of local and cross-agent physical relations. Its nodes represent agents, modules, controlled entities, action groups, resources, and constraints. An edge from a charger port to house import records the effect of charging; an edge from house import to transformer loading records the shared coupling. Health events can remove nodes or change edge domains.

Within a building, the graph supports local bundle construction. During training, it supplies topology and resource relations to the graph critic. During deployment, it supports compatibility checks and optional hard shared-constraint validation. The graph does not impose one optimisation algorithm. Its value lies in preserving the same meaning across learning, safety, and runtime artefacts.

For two buildings sharing one transformer, a hard constraint can be written as

$$\sum_{i \in \mathcal{N}_t} K_i^{\text{import}}(\bar{a}_{i,t}) \leq P^{\max}_{\text{transformer},t}. \tag{6.36}$$

The actors receive aggregate indications such as transformer loading or headroom and learn coordinated behaviour through the critic and reward. The graph gives the safety and evidence layers an unambiguous definition of the resource being shared.

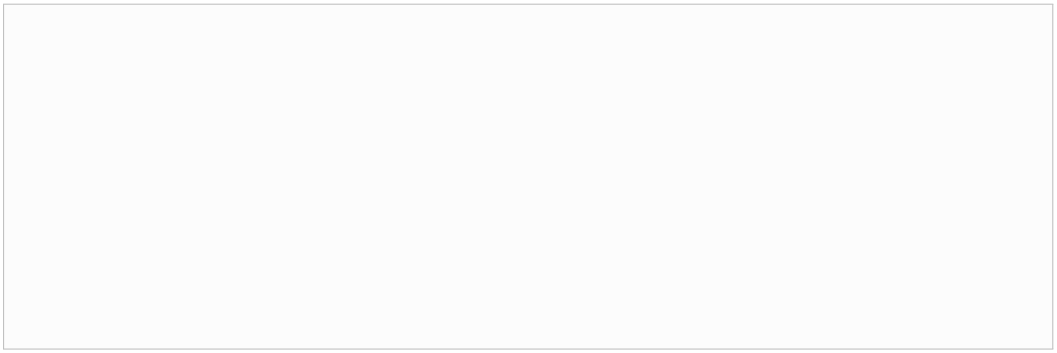


Figure 6.8: Typed Constraint Graph spanning local assets and shared community resources.

6.6.2 Hard Shared-Constraint Monitoring

Decentralised actors can learn to avoid shared violations, but a learned policy does not provide an unconditional mathematical guarantee. TI-MARL therefore permits a hard shared-constraint monitor for operational deployments. The monitor has a deliberately narrow role:

$$\mathbf{a}_t = \begin{cases} \bar{\mathbf{a}}_t, & \bar{\mathbf{a}}_t \in \mathscr{F}_t^{\text{shared}}, \\ \text{MinCorrect}\left(\bar{\mathbf{a}}_t, \mathscr{F}_t^{\text{shared}}\right), & \text{otherwise.} \end{cases} \tag{6.37}$$

It does not implement community optimisation, fairness, or service prioritisation. It applies an approved minimum correction or safe fallback to non-negotiable violations. The monitor is present during training and evaluation so that the actors experience the consequences of relying on it. Its intervention rate and correction magnitude are reported. Frequent intervention indicates insufficient learned coordination and weakens the algorithmic claim.

This separation assigns the normal control intelligence to the local policies and the hard guarantee to the safety mechanism. It also enables a clean experimental comparison among reward-only coordination, centralised critics, explicit aggregate observations, and the exceptional monitor.

6.6.3 Runtime Assurance and Degraded Modes

Runtime assurance handles conditions that cannot be delegated to policy quality. These include incompatible schema versions, unknown action types, compiler failure, missing safety-critical sensors, inference timeout, actuator failure, or an infeasible hard-constraint correction. Responses are scoped according to the dependency graph.

Representative operational modes are NORMAL, DEGRADED, ISOLATED-SAFE, SUSPENDED, and EMERGENCY. A failed charger removes one group and leaves the other groups active. A stale SoC contracts the dependent discharge port. A failed main meter can place the complete building actor in ISOLATED-SAFE because the aggregate local limit can no longer be verified. A missing community signal can retain local operation under an approved conservative behaviour. Recovery requires the declared health and consistency conditions rather than the arrival of one unverified sample.

The runtime-assurance decision and the learned action remain separate records. This distinction supports both scientific evaluation and operational audit: the policy's behaviour can be assessed without attributing fallback actions to the policy itself.

6.7 Compatibility and Typed Explainability

6.7.1 Interface Snapshots and Admission

Every decision is associated with an immutable interface snapshot containing active entities, observation parts, action groups, ports, validity, bounds, dependencies, health, and schema versions. The snapshot allows an experiment transition to be reconstructed even when the runtime topology subsequently changes.

Before a policy is activated, the lifecycle layer checks whether the policy package supports the current schema families, observation types, action-port types, units, parameter signatures, and health semantics. A new instance of a known charger type can be admitted without changing the network architecture. A held-out combination of known module types tests compositional generalisation. A semantically unknown action type is rejected unless an explicitly compatible parent contract and decoder are available.

This contract links the algorithmic method to Deployability. It prevents a model from being moved to a device merely because the tensor dimensions happen to match. The deployment decision uses the same semantics that the actor and compiler use at runtime.

6.7.2 Execution-Level Decision Traces

Typed interfaces support explanations grounded in the actual decision path. A trace can identify

- the active entities, modules, and schema versions;
- the local and community observations available to the actor;
- observations that were stale, estimated, or failed;
- the action groups and ports that existed at that step;
- invalid ports and the dependency or health rule that invalidated them;

- the port and parameter selected for each active group;
- any local feasibility adjustment and the active local constraint;
- any hard shared-monitor intervention; and
- the final action executed by each asset.

For example, a trace can state that `charger_1` selected CHARGE-EV at 2.8 kW, while DISCHARGE-EV was unavailable because the bound EV SoC source was stale. It can separately state that the local bundle construction reduced `charger_2` by 0.4 kW to preserve the house phase limit. These statements follow compiler and constraint semantics; they are not inferred from attention weights alone.

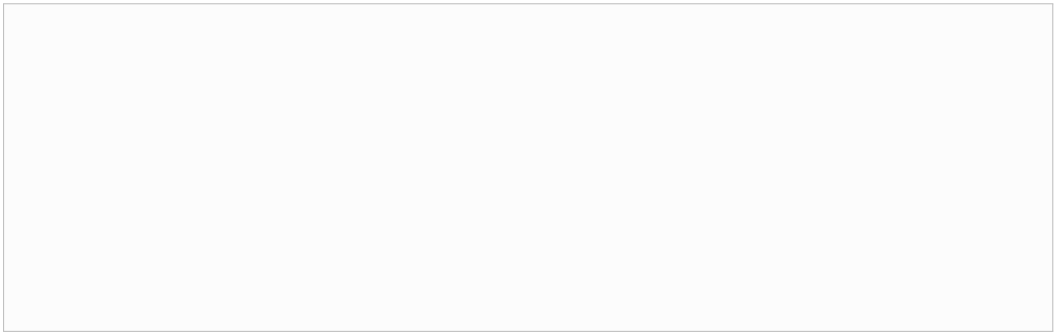


Figure 6.9: Typed execution trace from interface snapshot to executed local bundle.

6.7.3 Formal and Operational Properties

The method supports several properties whose validity can be checked independently of control performance.

Entity-binding integrity. Every observation and action instance identifies its source, owner, and target entity. The compiler rejects ambiguous or orphaned bindings.

Interface soundness. A compiled valid port has an active owner module, satisfied hard dependencies, and a recognised parameter schema. Health-dependent contraction is deterministic under a fixed snapshot.

Local bundle feasibility. The final local bundle belongs to the local feasible set when the construction layer terminates normally and the encoded constraints are correct.

Information-boundary preservation. The deployed actor consumes only local interface objects and authorised aggregate observations. Privileged critic information is excluded from the actor package.

Safe admission. Unknown or incompatible schema elements do not become executable action ports merely because a numerical embedding can be produced.

Trace consistency. The explanation record references the same interface snapshot, selected ports, constraint rules, and final action used by execution.

These properties do not prove global optimality, universal robustness, or the correctness of every physical model. They define testable invariants that separate interface and runtime correctness from empirical policy quality.

6.8 Experimental Methodology and Baselines

6.8.1 Comparison Design and Reference Methods

TI-MARL is evaluated under the common scenarios, information assumptions, repeated-run protocol, and physical metrics established in Chapter 5. The nominal comparison determines whether typed structure imposes an unacceptable cost when the interface remains stable. Reconfiguration experiments then test the capability for which the method is designed.

The Chapter 5 reference set includes the strongest progressive RBC, operational and oracle optimisation where appropriate, the best single-agent or community-level RL configuration, MADDPG, MATD3, and the selected hierarchical controller. Structural baselines are additionally required because nominal MARL comparisons cannot isolate the typed-interface mechanism.

6.8.2 Reconfiguration Scenarios, Ablations, and Metrics

The scenarios are organised from controlled changes to integrated operation.

Entity and module reconfiguration. A second or third charger is added to a building; an EV session connects to a different charger; one charger is removed; a battery or V2G capability is introduced; and a complete building joins or leaves the community.

Health reconfiguration. Charger failure, stale or failed SoC, main-meter failure, degraded community observations, simultaneous local and shared communication loss, and recovery events test failure containment and interface recompilation.

Held-out composition. Each module type is observed during training, while selected combinations or instance counts are held out. Stronger tests introduce a held-out port family only when a compatible semantic parent is defined.

Table 6.5: Structural baselines required to isolate the TI-MARL contribution.

Baseline	**Question tested**
Fixed universal vector with masking	Are typed entity and port semantics more useful than a sufficiently large predeclared layout?
Capability-conditioned fixed head	Does schema conditioning add value beyond telling a conventional policy which capabilities exist?
Transformer or graph encoder with fixed output head	Does the benefit come only from variable-cardinality input encoding?
Modular observation/action controller	Does TI-MARL outperform reusable adapters around a shared controller?
Variable candidate-action or UAS-style policy	Does runtime typed port structure add value beyond candidate descriptions or a fixed semantic union with masks?
Dynamic-population graph MARL	Does the method add value beyond handling a changing number of agents?
Reward-only local feasibility	Are explicit local constraints necessary beyond penalties?
No central critic / set critic / graph critic	Which part of coordination and scaling is explained by the critic representation?
No shared monitor / hard shared monitor	How often does learned coordination require exceptional correction?

Local and shared constraints. Several chargers compete for one house phase or import limit, while several houses share a transformer or demand-response target. The shared monitor is evaluated as exceptional protection rather than as an optimiser.

Ablations remove semantic type metadata, entity binding, health dependencies, dynamic bounds, cross-group contextualisation, local bundle construction, community observations, community reward terms, graph-critic topology, parameter sharing, compatibility checks, or typed traces. The evaluation reports five evidence groups:

1. control and service performance;

2. reconfiguration and held-out-composition behaviour;

3. Safety and Robustness, including local adjustments and assurance interventions;

4. Scalability and runtime cost across agents, modules, groups, ports, and graph edges; and

5. Deployability and explainability, including admission, compatibility, and trace completeness.

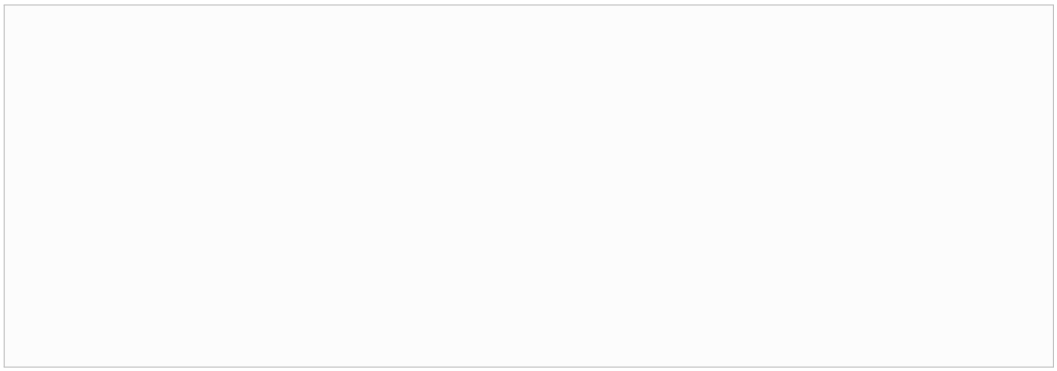


Figure 6.10: TI-MARL experimental design from nominal control to integrated reconfiguration.

6.9 Results and Comparative Analysis

The results are organised by scientific claim rather than by implementation component. This structure keeps nominal control quality, interface reconfiguration, health behaviour, multi-agent coordination, scale, and typed trace evidence separate.

6.9.1 Nominal Performance and Comparison with Chapter 5

The nominal comparison places TI-MARL beside the strongest relevant configurations from Chapter 5 under a stable interface. It reports energy and service KPIs, community outcomes, training stability, sample efficiency, inference cost, and local feasibility adjustments. The comparison establishes whether the typed representation preserves competitive control quality before any reconfiguration advantage is claimed.

6.9.2 Multiple Assets, Reconfiguration, and Held-Out Compositions

This analysis measures whether a policy trained over typed entity and group instances remains competent when the number of chargers changes, EV sessions move between chargers, modules are added or removed, buildings join or leave, and known module types appear in held-out combinations. Results report both control quality and interface correctness: entity-binding errors, invalid actions, admission decisions, retraining requirements, and recovery after topology changes.

The key comparison is against fixed-superset masking, capability-conditioned fixed heads, modular controllers, and dynamic-population graph MARL. A positive result requires more than accepting a different tensor length. It must preserve the semantic relation between the new entity, its observations, its action group, and the constraints affected by its ports.

6.9.3 Health-Aware Robustness and Local Safety

Health experiments compare typed dependency-based recompilation with default imputation, missingness masks, recurrent estimation, and sensor-dropout training. They measure service preserva-

tion, affected scope, time to degrade, bounds contraction, unnecessary shutdowns, fallback use, violations, and recovery time.

Local safety results separate policy quality from the constraint-aware bundle construction. The analysis reports raw-bundle feasibility, final-bundle feasibility, adjustment frequency and magnitude, and the effect of removing local constraint semantics. Main-meter failure, charger failure, and stale SoC provide distinct failure scopes and should not collapse into one binary robustness score.

6.9.4 Multi-Agent Coordination and Shared Constraints

Community coordination is evaluated through aggregate observations, local and community reward components, and critic structure. The comparison includes independent critics, a set critic, a graph critic, and the Chapter 5 multi-agent baselines. It reports participant outcomes, community peak and ramping, transformer margins, distribution of benefits, and the frequency with which the exceptional shared monitor intervenes.

A credible TI-MARL result should show that a capable local actor and central critic learn most coordination behaviour before the hard monitor is applied. Feasibility provided almost entirely by repeated shared corrections would demonstrate a useful safety wrapper but weak multi-agent learning.

6.9.5 Scalability and Runtime Cost

Scale experiments vary the number of agents, module instances per agent, charger and EV entities, action-group instances, port instances, observation parts, graph nodes and edges, and shared-resource relations. Actor latency, critic latency, memory, message volume, training throughput, local adjustment cost, and optional monitor cost are measured alongside control performance.

The results must identify which dimension increased and which remained fixed. Shared decoders can keep model parameters constant while graph processing and inference cost still grow. A variable-population critic can avoid rebuilding a fixed joint vector while communication or attention cost becomes the next bottleneck.

6.9.6 Ablations, Explainability, and Boundary Conditions

The ablations determine whether typed semantics, entity binding, health closure, contextual group decoding, local feasibility, graph critics, and compatibility checks materially affect the claimed capability. Typed explanation evidence is audited for completeness and fidelity: every explanation must match the actual interface snapshot, selected ports, active constraints, adjustments, and executed commands.

Boundary results are reported explicitly. If a masked fixed-space baseline matches TI-MARL under the tested reconfiguration, the typed interface remains useful for software contracts and

traces but the algorithmic claim narrows. If the method requires retraining for every new combination of known modules, the compositional claim is weakened. If the shared monitor intervenes frequently, community coordination remains incomplete.

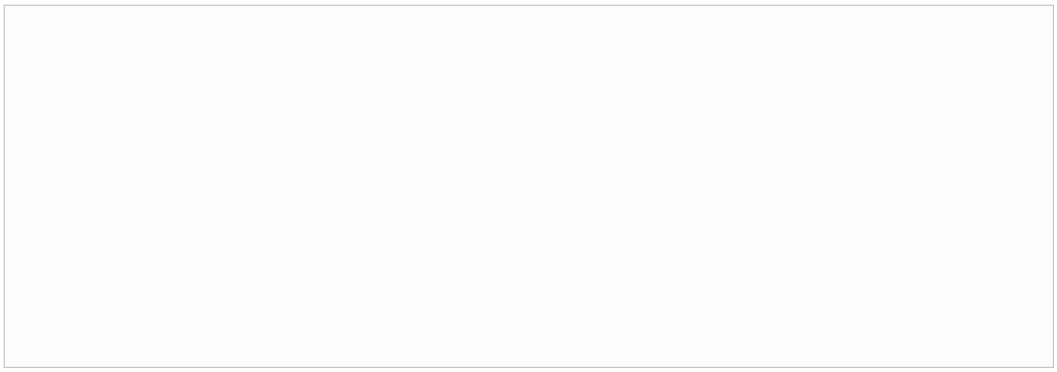


Figure 6.11: TI-MARL results across nominal control, reconfiguration, health, coordination, scalability, and trace fidelity.

6.10 Discussion, Generality, and Contribution to the Research Questions

TI-MARL extends the methods of Chapter 5 by changing the object over which local policies and central critics operate. The actor no longer assumes one fixed list of assets and action coordinates; it acts over an entity-bound interface compiled from the modules and health conditions that exist now. The critic no longer requires a permanent concatenated population; it evaluates a set or graph of active agents and shared resources. The same semantics support local hard constraints, admission, deployment compatibility, and decision traces.

The contribution to RQ1 lies in the representation of heterogeneous and changing agents, module instances, observations, and action groups while preserving local information boundaries. The contribution to RQ2 lies in shared type-conditioned decoders, variable-cardinality local encoders, set/graph critics, and the explicit measurement of scale across agents, entities, ports, and relations. The contribution to RQ3 lies in health-aware interface recompilation, constraint-aware local bundles, scoped degraded modes, and exceptional hard shared protection. The method also contributes to RQ4 through versioned schemas, interface snapshots, policy compatibility, safe admission, and execution traces, although lifecycle orchestration is evaluated in Chapter 7.

Typed explainability is a cross-cutting outcome. Port identities, entity bindings, health rules, bounds, local adjustments, and shared interventions are already part of the executable contract. They can therefore be reported without translating anonymous output indices after the decision. This structure does not explain the complete internal causal reasoning of a neural network, but it provides a faithful account of what the policy could observe and invoke, what the interface excluded, and how the final command was obtained.

The formal model can represent modular robots, industrial systems, edge services, and other CPSs with changing capabilities and local/shared resources. The evidence in this thesis remains energy-specific. Generality is therefore claimed for the abstraction and implementation pattern; performance transfer to another domain requires new experiments, constraints, and safety evidence.

The principal limitations are also explicit. Known type families and compatible decoders remain necessary. Aggregate community observations can be delayed or insufficient for some coordination tasks. Local constraint construction is only as correct as the encoded physical model. A central critic improves training but may increase computational cost. An exceptional shared monitor provides hard protection without proving that decentralised policies have learned an optimal community strategy. These boundaries determine the future extensions and the strength of the final claims.

6.11 Chapter Summary

TI-MARL treats the current observation–action interface of each agent as a versioned, entity-bound runtime object. The Typed Interface Compiler derives observation parts, dynamic action-group instances, parameterised ports, health-dependent validity, bounds, dependencies, and local constraint relations from the active modules and entities. A local policy encodes the variable interface into one shared local representation and jointly decodes an action bundle across the controllable groups that currently exist. New instances reuse the same type-conditioned decoders instead of requiring permanent asset-specific heads.

The method is multi-agent through the joint physical transition, coupled local and community outcomes, shared resources, dynamic population, and centralised set or graph critics used during training. Execution remains decentralised: each actor receives its local interface and authorised aggregate community numbers. Local hard feasibility is incorporated into bundle construction, while the optional shared monitor is reserved for exceptional non-negotiable violations. Health events recompile the interface at the appropriate scope, from one port to the complete local agent.

This structure links the accumulated controller results to the four real-world requirements of the thesis. It provides a concrete mechanism for dynamic heterogeneity, variable-cardinality scaling, explicit Safety and Robustness semantics, deployable interface contracts, and typed execution traces. The following chapter moves from the policy and learning method to the lifecycle required to reproduce, validate, package, deploy, monitor, update, and recover the resulting policies and policy sets.

Chapter 7

Experiment and MARL Policy-Set Lifecycle Orchestration and Distributed Edge Deployment

The preceding chapters established a progression of controllers and then brought the selected capabilities together in TI-MARL. That still does not produce an operational control service. A method becomes usable only when the experiment that created it can be reconstructed, the resulting policy can be validated against its intended interfaces and physical conditions, and the approved runtime can be installed, observed, updated, and recovered on the target system.

CityLearn v3 remains part of this transition rather than disappearing after a policy has been trained. Its environment, scenario, data, event, interface, reward, constraint, baseline, and evaluation versions form part of the experiment and validation record. The simulator provides controlled regression and disturbance cases before promotion and a route for replaying selected field observations or incidents after deployment. It does not perform edge deployment itself; it supplies the reproducible evidence and compatibility contracts consumed by the lifecycle mechanisms developed in this chapter.

For MARL, this transition is more demanding than copying one checkpoint. The deployed object can be a set of behaviourally interdependent policies. A partial update may succeed from a software perspective while placing the control system in a combination that was never evaluated. A new agent may be technically reachable while lacking a compatible policy or safety adapter. A rollback may restore model files without restoring the policy-set version and interface contract that were known to work together.

The EnergAIze lifecycle integrates established MLOps, RLOps, distributed RL, registry, container, edge-orchestration, and deployment mechanisms around RL- and MARL-specific operational objects (Hardy et al. 2024; Hoffman et al. 2020; Hu et al. 2023; Li, Thomas, et al. 2022; Liang et al. 2018; Zaharia et al. 2018; Zhang, Quiñones-Grueiro, et al. 2024; Zhang, Zhang, Quiñones-Grueiro, et al. 2024; Zhou et al. 2023). Environment and scenario versions, rewards,

observation and action schemas, policy sets, agent-role mappings, safety dependencies, validation evidence, edge targets, and coordination-aware telemetry remain linked throughout the lifecycle. The same machinery can govern TI-MARL, MATD3, MADDPG, hierarchical controllers, and deterministic fallbacks.

7.1 Lifecycle Problem, Core Entities, and State Model

Generic MLOps already provides experiment tracking, artefact lineage, registries, packaging, serving, progressive rollout, and monitoring (Kreuzberger et al. 2023; Sculley et al. 2015; Zaharia et al. 2018). RLOps adds the coupling between a learned policy and an environment, interaction data, evaluation, and safety (Li, Thomas, et al. 2022). Distributed RL frameworks provide workers, actors, learners, replay, and multi-policy mappings (Hoffman et al. 2020; Hu et al. 2023; Liang et al. 2018; Zhou et al. 2023).

Semantic integration is the remaining problem. Reproducibility requires the source revision, environment, scenario, data, reward, constraints, observation and action schemas, seeds, algorithm configuration, dependencies, and runtime to be recoverable as one experiment contract.

A model store answers where the bytes are. A registry adds identity, version, lineage, status, and metadata. A MARL policy-set registry must additionally answer which policies may operate together, which agents and roles they serve, which interfaces they expect, which communication and safety dependencies apply, and which predecessor should be restored during rollback.

Serialisation stores weights. Packaging preserves the executable semantics around them: preprocessing, adapters, dependencies, schemas, safety layers, target requirements, and the identity of the complete policy set. Multiple-model serving loads several independent models. Coordinated MARL deployment governs the relationships between them.

Experiment Manifest. The experiment manifest is the reproducibility anchor and contains:

- experiment and parent identifiers;
- source repository and commit;
- environment package and version;
- scenario, data, and event versions;
- observation, action, and entity schemas;
- reward and constraint configuration;
- algorithm and network configuration;
- seeds and training/evaluation windows;

- dependencies and container image;
- requested compute resources;
- produced checkpoints, metrics, logs, and reports; and
- completion, cancellation, and failure status.

The manifest is immutable after run completion. Corrections create a new experiment or an explicit derived record rather than silently changing the provenance of an existing policy.

Policy and Policy-Set Manifest. A policy record links model artefacts to role, schemas, preprocessing, evaluation, and target constraints. A policy-set manifest lists member policies, agent-role mappings, allowed versions, compatibility relations, required communication schemas, safety layers, and a shared set version.

For homogeneous parameter sharing, one policy can map to many agents. For heterogeneous control, several policies can coexist. The manifest permits both while preserving the distinction between a shared policy and a set of independent files.

Deployment and Operational Record. A deployment record identifies package, policy set, target group, rollout strategy, activation time, previous version, and observed state. An operational record links runtime telemetry and significant events to that deployment. This makes it possible to answer which policy version produced an action during an incident.

Figure 7.1 identifies the lifecycle entities and immutable provenance relations required to reconstruct an experiment and its deployed policy set.

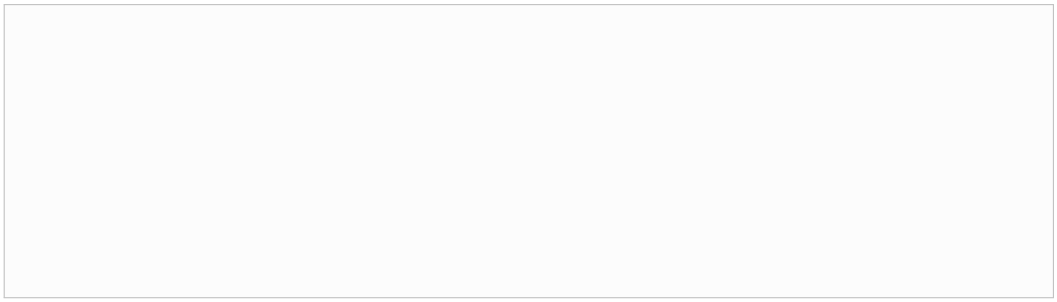


Figure 7.1: Lifecycle entity and provenance model.

Lifecycle States and Invariants. Candidate artefacts move through states such as:

$$\begin{aligned} \texttt{trained} \rightarrow \texttt{evaluated} \rightarrow \texttt{validated} \rightarrow \texttt{approved} \\ \rightarrow \texttt{packaged} \rightarrow \texttt{deployed} \rightarrow \texttt{active}. \end{aligned} \tag{7.1}$$

Alternative states include rejected, failed, superseded, rolled back, and archived. Invariants include complete provenance, schema compatibility, policy-set completeness, passing validation gates, and target compatibility. State transitions are recorded with actor, reason, and evidence.

Figure 7.2 defines the states and evidence gates that separate a stored checkpoint from a validated and deployable policy set.

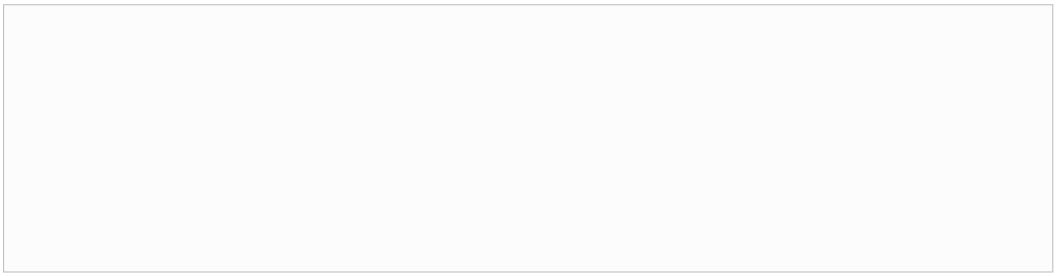


Figure 7.2: Policy and policy-set lifecycle states, gates, and rollback relations.

7.2 EnergAIze Lifecycle Architecture and Experiment Orchestration

The implementation can be understood through four planes.

The *experiment plane* receives configurations, creates jobs, assigns workers, collects metrics, and stores artefacts. The *governance plane* maintains experiments, policies, policy sets, validation, aliases, and promotion states. The *deployment plane* builds packages, resolves target profiles, distributes versions, and tracks desired versus actual state. The *observation and assurance plane* receives logs and telemetry, evaluates runtime health and control behaviour, and initiates alerts, fallback, rollback, or retraining.

The repository ecosystem comprises a CityLearn-derived simulator, algorithm-training package, backend API, remote workers, an inference service, a web interface, infrastructure definitions, and simulated-asset services. Workers execute containerised jobs on local and remote/HPC resources, while message brokers and shared storage decouple execution from the control plane.

Figure 7.3 presents the lifecycle through experiment, governance, deployment, and observation/assurance responsibilities.

Experiment and training orchestration. A job begins with validation of its manifest and resource request. The backend creates a persistent record, places or assigns the job, and exposes status to the UI. Workers advertise capability and heartbeat state, retrieve the job, prepare the environment, execute training or analysis, publish progress, and return artefacts.

The orchestration layer supports training, evaluation, model comparison, missing-data robustness, segment analysis, and hyperparameter search. Cancellation is cooperative: a cancellation flag is checked at defined iteration boundaries, and incomplete training is not promoted automatically. Atomic state writes and explicit terminal statuses reduce ambiguity after crashes.

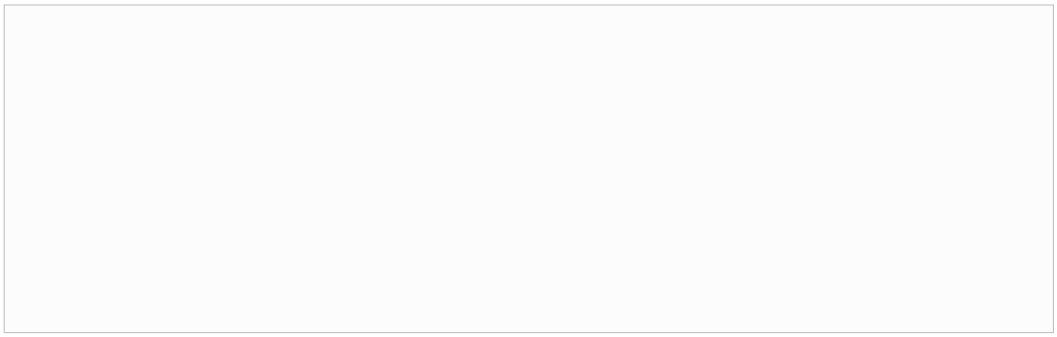


Figure 7.3: Four-plane EnergAIze lifecycle architecture.

The worker model supports scaling the number of concurrent jobs. Evaluation must distinguish throughput from scientific scalability. Relevant metrics include queue wait, startup, execution overhead, worker utilisation, failure and retry rates, artefact-transfer time, and reproducibility across workers.

7.3 Policy-Set Governance and Coordinated Edge Deployment

Candidate policies are registered with their experiment lineage. Validation combines controller KPIs, seed statistics, constraint and robustness evidence, runtime-resource measurements, and schema compatibility.

A reusable MARL promotion gate can be expressed as a set of target-scoped predicates:

$$G_{\text{target}}(P) = G_J(P) \wedge G_S(P) \wedge G_R(P) \wedge G_C(P) \wedge G_H(P), \tag{7.2}$$

where G_J represents control performance, G_S safety, G_R robustness, G_C compatibility and policy-set completeness, and G_H hardware/resource suitability.

The gate does not require one universal threshold. A shadow deployment can use a different threshold from closed-loop actuation. Every approved state records the target and evidence.

Policy-set validation detects missing members, incompatible observation or action schemas, invalid role mappings, communication-version mismatch, and safety-layer mismatch. A policy that is individually valid can still be rejected as part of an unevaluated combination.

Packaging and coordinated distributed edge deployment. A package contains or references weights, code, dependencies, manifests, preprocessing, adapters, schemas, safety logic, configuration, and checksums. ONNX or another portable model format can reduce runtime dependence, while rule-based controllers can be packaged through the same interface.

Deployment associates the package with target profiles describing architecture, CPU/GPU, memory, operating system, container support, connectivity, sensors, actuators, and permitted control roles. Pre-activation checks verify package integrity and interface availability.

A coordinated policy-set rollout can be atomic where infrastructure permits, staged through a canary target, or explicitly mixed-version compatible. The architecture does not assume atomic fleet updates over intermittent networks. It records stale targets and blocks activation when required partners are not ready.

Edge inference remains available during temporary central disconnection according to local policy and cache rules. The runtime retains the last approved version and a fallback. Updates are downloaded and verified before replacing the active controller. A failed activation restores the previous version.

Figure 7.4 shows the coordinated rollout semantics required when several interdependent policies and edge targets must become ready together.

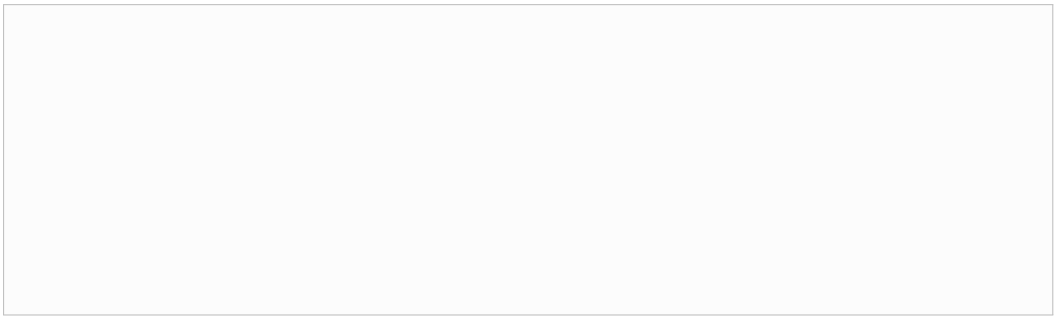


Figure 7.4: Coordinated policy-set packaging, staged edge deployment, activation, and rollback sequence.

7.4 Coordination-Aware Monitoring, Updating, Rollback, and Recovery

Infrastructure health answers whether processes, devices, resources, and communication are available. Data monitoring answers whether observations are complete, current, and plausible. Control monitoring answers whether the system meets performance objectives. Safety monitoring answers whether constraints or guards are being approached or activated. Coordination monitoring answers whether the active policy set behaves coherently.

Examples of coordination failure include incompatible versions, oscillating charge requests, repeated competition for the same site headroom, unfair allocation, stale community messages, and loss of aggregate performance despite healthy individual processes.

Monitoring events can trigger:

- a warning without control change;
- local data substitution or action masking;
- activation of guarded or fallback mode;

- policy or policy-set rollback;
- target isolation;
- a new validation run; or
- a retraining request linked to the observed operating period.

Rollback is evaluated as an operational sequence, not only a Kubernetes feature. The test verifies that the previous policy set, adapters, schemas, and safety dependencies are restored and that control performance recovers.

Figure 7.5 connects infrastructure, data, control, safety, and coordination monitoring to the corresponding operational response.

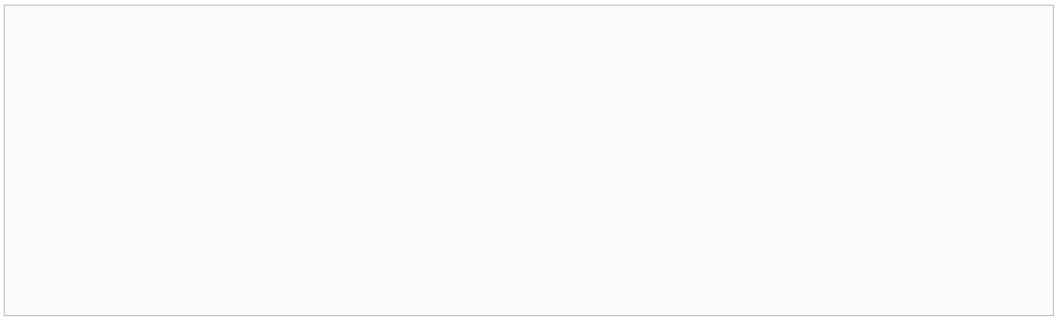


Figure 7.5: Coordination-aware monitoring and recovery loop.

7.5 Empirical Systems Evaluation

Experiment Reconstruction. Historical experiments are reconstructed from their identifiers in a clean environment. The evaluation checks source, dependencies, scenario, reward, schemas, seeds, and resulting metrics. Reproduction tolerance is stated for deterministic and stochastic components.

Compatibility and Policy-Set Completeness. Negative tests deliberately submit missing members, incompatible schemas, invalid roles, and mismatched safety dependencies. The expected result is rejection before activation. The false-acceptance and false-rejection rates of compatibility rules are reported where automatic inference is used.

Orchestration Throughput and Worker Scaling. The benchmark varies job count, worker count, job duration, artefact size, and failure rate. It reports queue time, throughput, overhead, resource use, retry success, and scaling efficiency. Separate experiments use local containers and remote/HPC execution where supported.

Packaging, Deployment, and Runtime Resources. Measurements include package size, build time, transfer time, install time, activation time, inference latency, memory, CPU, and network use. Results are reported by target class rather than averaged across dissimilar devices.

Staged Rollout and Mixed Versions. A policy-set update is deployed to a subset of targets. The experiment measures convergence to the desired fleet state, duration of mixed-version operation, compatibility behaviour, KPI change, and rollback latency. A network interruption during rollout tests disconnected-edge semantics.

Join, Leave, and Failure. A controller joins with a known or unknown schema, an active agent leaves, communication is lost, and a device restarts. The system must identify the role and capabilities, select or reject a compatible policy, restore state, activate a fallback where required, and monitor the effect on the remaining policy set.

Soak Testing. Long-duration operation records decisions, outages, restarts, manual interventions, safety interventions, promotions, rollbacks, active agents, resource use, and data-quality incidents. A stable 24-hour or multi-day run provides stronger operational evidence than a brief inference demonstration.

7.6 Availability, Limitations, and Contribution to RQ4

Implementation and open-source availability. The implementation is distributed across repositories with distinct responsibilities. An architecture manifest records each repository, release, role, licence, contribution ownership, and archived DOI or commit. Internal project services that cannot be released are documented through their interfaces and reproducible test artefacts.

Software size and commit counts can contextualise the engineering effort but are not scientific results. The contribution is supported by lifecycle abstractions, failure semantics, compatibility rules, and empirical evaluation.

Discussion, limitations, and RQ4 contribution. The chapter is the primary response to RQ4. It moves the unit of deployment from an isolated checkpoint to a governed policy set linked to the world in which it was trained and the interfaces through which it acts.

The architecture inherits mature mechanisms rather than recreating them. Distributed training, generic registries, containers, and edge module management already exist. The residual contribution is the MARL-specific semantic layer and its evaluation in a changing CPS.

The demonstrated lifecycle includes manual steps, incomplete atomicity across disconnected targets, target-specific adapters, and limited long-duration operational evidence where those conditions apply. The state model and evaluation expose these boundaries instead of presenting them as automated guarantees.

7.7 Chapter Summary

The EnergAIze lifecycle turns experiments into governed distributed control. It defines experiment, policy, policy-set, package, deployment, and operational records; separates experiment, governance, deployment, and assurance planes; and links orchestration, validation, packaging, monitoring, rollback, and recovery through explicit state transitions and evidence.

Coordinated MARL policy sets require additional lineage, compatibility, update, and monitoring semantics when agents and edge targets are heterogeneous and dynamically reconfigurable. This requirement extends established MLOps and RLOps mechanisms with policy-set and control-interface semantics. The next chapter examines what happened when selected pieces of this architecture interacted with real data, partner platforms, physical controllers, and operational REC and charging contexts.

Chapter 8

EnergAIze in Operation: Real-World Deployment and Validation

The preceding chapters developed the representation, simulation, control, and lifecycle mechanisms required to move learned energy control beyond an isolated experiment. This chapter examines that movement through two distinct demonstrators. The OPEVA demonstrator connects Renewable Energy Community operation, residential and corporate charging installations, external platforms, vehicle information, simulation, and distributed control. The DemFlex demonstrator provides a second and independently attributable setting. Their sites, assets, control authority, evaluation periods, and results are kept separate even where both instantiate the EnergAIze architecture.

Real-world evidence spans several levels. Historical measurements can drive a simulation; a target device can execute a policy without actuating an asset; a controller can operate in shadow beside the active system; and a bounded field test can apply physical commands. None of these results establishes sustained closed-loop operation by itself. Each demonstrator is therefore reported through its physical configuration, integration path, controller authority, duration, interventions, and evidence level.

8.1 Evidence Levels and Comparative Reading

The validation uses the following hierarchy:

1. **real-data simulation**: historical operational data drive the simulator;
2. **target-hardware inference**: the controller runs on the intended class of device;
3. **hardware- or software-in-the-loop**: the policy interacts with physical or emulated interfaces in a closed loop;
4. **shadow operation**: decisions are produced and compared but not applied;

5. **limited physical or field control**: actions affect assets during a bounded pilot; and
6. **sustained operational control**: repeated closed-loop use forms part of normal site operation.

The evidence level is part of each claim. Real-data simulation can compare controllers over long and identical periods, but cannot establish that a device accepted the requested action. Integration tests can establish protocol and interface compatibility, but not long-term energy benefit. Physical control can expose latency, rejection, fallback, and user constraints, but a short pilot cannot establish seasonal performance. The two demonstrators are consequently assessed against the same questions: which assets were represented, which information was available, which decisions were produced, which commands were applied, which protection remained external, and what outcome was measured.

Figure 8.1 maps each thesis demonstrator to the exact evidence level and action authority it supports.

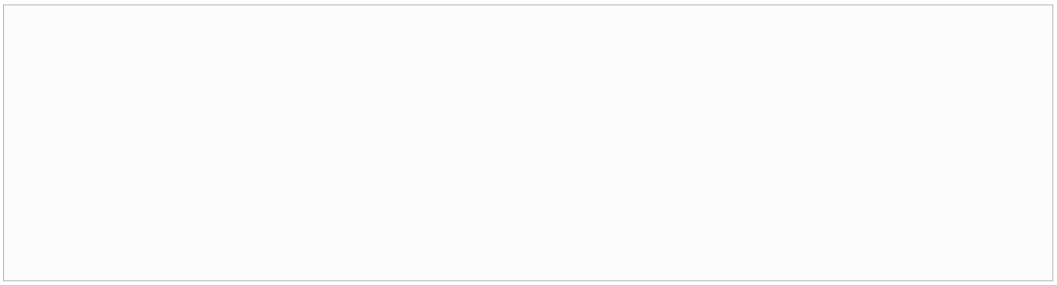


Figure 8.1: Validation evidence levels and the claims supported by each level.

Controller selection follows the promotion logic in Chapter 7. A method is included in a demonstrator only when its control performance, stability, interface compatibility, runtime resources, and protection mechanisms are adequate for the intended evidence level. A transparent rule-based controller remains available as a reference or fallback. The EnergAIze MADDPG controller preserves continuity with the OPEVA work; later MATD3, hierarchical, or TI-MARL policies require their own target-scoped validation before receiving physical authority.

8.2 OPEVA Demonstrator

8.2.1 Control Problem and Physical Configuration

OPEVA Demo 9 addresses the coordination problem created when EV charging is added to buildings that already have non-controllable demand, photovoltaic generation, storage, and site-power limits. Charging immediately at connection can satisfy the driver while increasing a building or community peak. Delaying or modulating the session can reduce cost and align demand with local photovoltaic production, but only within the vehicle's connection window and required departure

energy. Corporate installations add a further allocation problem because several simultaneous charging requests can exceed the power available to the charging switchboard or site (Salvan and OPEVA Consortium 2026).

The demonstrator connects three complementary contexts. The Cleanwatts Living Lab represents a heterogeneous community of real residential participants, including consumers without controllable assets, prosumers with photovoltaic production, and flexsumers with batteries or domestic EV charging. The i-charging S. Mamede factory represents an industrial site with one 50 kW DC charger, two AC charging units with two outlets each, photovoltaic generation, meters, and a local controller. The i-charging headquarters represents a larger office installation with one 50 kW DC charger, eighteen 22 kW AC chargers, photovoltaic generation, building loads, meters, and a common charging switchboard. Simultaneous charging can request more than 400 kW, making site-level power allocation a physical requirement rather than only an optimisation preference (Salvan and OPEVA Consortium 2026).

Across the three sites, the reported installation comprises four domestic 7.4 kV A AC chargers, two corporate 7.4 kV A AC units with two outputs, eighteen corporate 22 kV A AC chargers, two 50 kW DC chargers, two local charging controllers, and three AC meters. These devices are distributed across installations with different demand, generation, storage, communication, and control capabilities. The resulting heterogeneity is central to the demonstrator: the controller cannot assume that every participant owns the same assets or exposes the same telemetry.

The later real-data study associated with the demonstrator represents 17 installations. The first 15 form a heterogeneous residential and commercial REC; the remaining two represent an office and an industrial facility with photovoltaic generation and multiple EV chargers. The dataset contains 18 months of operational measurements aligned to 15-minute intervals. This simulation configuration and the physical Demo 9 installation are related research objects, but they are not interchangeable: the dataset supports long-horizon comparison, while the physical sites provide interface, device, and bounded-actuation evidence.

8.2.2 EnergAIze Integration Path

The demonstrator instantiates EnergAIze as a path from heterogeneous physical interfaces to a controlled action. Cleanwatts provides the community-facing platform for residential measurements, photovoltaic and consumption forecasts, batteries, energy prices, and selected actuation. My.i-charging monitors charging sessions and connects the residential and corporate chargers to the wider system. At the corporate sites, omni-e local controllers acquire charger and meter data and enforce local power-management functions. OCPP connects chargers with the charging platform, Modbus TCP/IP connects selected meters and chargers to local controllers, OCPI exchanges charging information and profiles between platform boundaries, and REST or RabbitMQ interfaces connect the data and algorithm services (Salvan and OPEVA Consortium 2026).

Percepta and the associated Data Provider middleware isolate the controller from those provider-specific details. Receivers acquire measurements from each platform; translators normalise identifiers, units, timestamps, and message formats; per-installation managers align the required decision interval, identify asset relationships, and handle data gaps; and predictors invoke the policy associated with the installation. The reverse path sends the validated result to the provider responsible for the corresponding battery or charger. Training and inference use the same physical meanings while following different data paths: historical records are assembled into reproducible datasets for simulation and learning, whereas runtime messages are collected and aggregated periodically for the active controller (Fonseca, Sousa, et al. 2025; Salvan and OPEVA Consortium 2026).

Vehicle SoC illustrates why the integration layer is part of the control problem. Many chargers report session status and energy but do not expose the vehicle's current SoC. PulseCharge obtains available vehicle telemetry through manufacturer-facing services and combines it with user-declared or predicted flexibility. Recurrent user schedules provide a low-interaction baseline, predictions use historical routines when available, and explicit user input has priority when a journey departs from the routine. Consumption, photovoltaic-production, and flexibility prediction services provide additional derived observations and can temporarily compensate for unavailable direct measurements. Missing or low-confidence information remains distinguishable from a healthy observation because it changes which charging or V2G decisions can be justified.

The Flexibility Scheduling Algorithm receives the resulting observations and produces building-level actions for controllable assets. CityLearn and EVLearn reproduce the relevant building, EV, charger, battery, and community conditions so that rule-based, optimisation, single-agent, and multi-agent controllers can be trained or compared without trial and error on the physical installations (Fonseca, Ferreira, Cabral, Severino, Nweye, et al. 2025; Fonseca, Ferreira, Cabral, Severino, and Praça 2024). At runtime, the policy can execute centrally as one model instance per installation in a cloud environment or locally beside an installation. In both arrangements, provider and device protections retain the final authority to accept, limit, reject, or override a command.

Figure 8.2 presents the three OPEVA sites, the assets available at each, and the data and command path through the partner platforms and EnergAIze components.

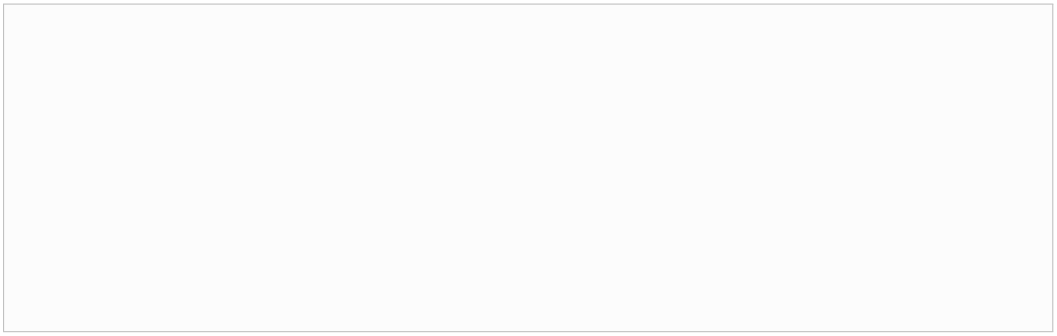


Figure 8.2: OPEVA Demo 9 sites, assets, platforms, and EnergAIze integration path.

8.2.3 Scenarios and Validation Strategy

Four use cases organise the demonstrator. Their value lies in the progression from a common residential charging problem to V2G, in-transit charging, and constrained fleet charging. Their implementation levels must remain explicit.

Table 8.1: OPEVA Demo 9 scenarios and the evidence boundary reported in deliverable D6.3.

Scenario	Control question	Reported implementation	Boundary
1	Schedule residential charging within the connection window and departure requirement	Simulation and real setting	Physical action authority and duration require final log confirmation
2	Use a connected EV for local storage and V2G	Simulation and partial real integration	Bidirectional charger operation remained simulated
3	Select and reserve an in-route charging option using trip, tariff, range, and community information	Simulation and partial integration	Public charging and community/charger communication were not available for complete physical execution
4	Prioritise several corporate charging requests under building loads, photovoltaic production, user needs, and site-power limits	Simulation and partial real integration	Testing was bounded to avoid disturbing office and industrial operation

The real-data simulation study compares unmanaged operation, passive REC energy sharing, rule-based scheduling, and EnergAIze coordination under common data and pricing assumptions. The 18-month records are aligned to 15-minute intervals, with accumulated and instantaneous quantities treated differently during resampling. Short gaps, longer missing periods, timestamp inconsistencies, physically invalid values, and post-gap accumulation spikes are handled according to declared data-type rules. This processing enables a common experiment but does not erase the fact that some controller inputs were interpolated, repeated, estimated, or unavailable in the original system.

The final evidence must keep this campaign separate from the earlier nine-building benchmark reported in D6.3. That benchmark combined demonstrator measurements with CityLearn data and simulated EV flexibility. It supported the initial controller and simulator evaluation, whereas the later 17-installation campaign addresses a different population, time resolution, data horizon, and performance-and-equity question. Numerical results from the two campaigns are not pooled.

8.2.4 Results and Demonstrated Boundary

The four-building operational study provides preliminary evidence that the complete data and coordination path can work with real participants and imperfect measurements. It reports approximately 9% lower daily peak demand and 5% lower energy cost for the evaluated coordinated operation (Fonseca, Sousa, et al. 2025). These values are interpreted with the reported site, baseline, duration, weather, participant behaviour, and degree of direct control; they are not extrapolated to all 17 installations or to sustained autonomous operation.

The later real-data simulation campaign provides the long-horizon comparison. Relative to the unmanaged reference, EnergAIze reports a 10.2% reduction in grid energy demand, a 13.6% reduction in energy cost, a 14.2% improvement in renewable self-consumption, a 20.7% reduction in peak demand, and a 30.1% reduction in ramping under the stated scenario and pricing assumptions. Participant-level results remain heterogeneous, which is scientifically important: aggregate improvement does not establish that participants with different assets and flexibility obtain equal benefits.

The systems evidence is similarly layered. The Cleanwatts and my.i-charging integration exchanged charging-session data; OCPI charging profiles provided a path for control; local controllers operated at the corporate sites; the middleware produced periodic real-data messages for inference; and selected Cleanwatts-device actuation tests had begun at the reporting point (Salvan and OPEVA Consortium 2026). The same report described containerised model and middleware deployment as an early-stage activity. The defensible conclusion is therefore that OPEVA established substantial real-platform, real-data, local-controller, and bounded-actuation evidence, while V2G, in-route charging, fleet-wide policy deployment, and sustained closed-loop operation retained scenario-specific limitations.

Figure 8.3 aligns one audited OPEVA period with observation quality, policy state, requested and applied charging actions, site demand, limits, and interventions.

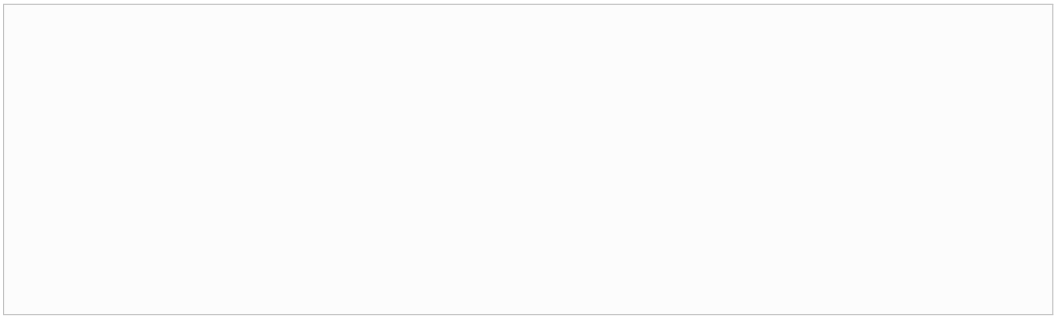


Figure 8.3: Timeline of one audited OPEVA control or integration period.

8.3 DemFlex Demonstrator

DemFlex constitutes the second demonstrator context of this chapter. It is kept separate from OPEVA because its physical setting, available assets, data lineage, control authority, experimental periods, and deployment history define a different body of evidence. The same EnergAIze abstractions can be reused, but compatibility of an interface or component does not permit a result, asset count, or evidence level to be transferred between demonstrators.

8.4 Cross-Demonstrator Lessons and Simulation-to-Operation Differences

The demonstrators expose differences that are easily hidden by a nominal simulator. Real timestamps and APIs are not perfectly aligned. SoC can be unavailable or estimated. User-declared departure times can be absent or incorrect. Physical platforms can reject, quantise, delay, or override commands. Assets can be unavailable for reasons not represented in the training scenario. Edge devices can restart and communication can be interrupted.

These differences are classified as data, interface, dynamics, execution, constraint, or lifecycle shifts. The appropriate response depends on the source of the mismatch. A unit or timestamp error requires a data-contract correction; a provider-specific action requires an adapter; a known observation failure requires an explicit degraded mode; a persistent distribution shift can justify retraining; and an incompatible capability can require rejection rather than numerical imputation. The learning algorithm is changed only when the evidence identifies it as the relevant failure source.

Fail-safe behaviour is evaluated through proposed and executed actions separately. Missing or stale critical data can contract the current interface, activate conservative bounds, invoke a rule-based fallback, hold a previously safe value, or suspend learned control. The number, duration, cause, and effect of these interventions are part of the result. A stable physical trajectory produced almost entirely by external correction is evidence for a useful assurance layer, but weak evidence for the policy itself.

Figure 8.4 turns the observed differences between simulation assumptions and demonstrator conditions into explicit findings, mitigations, and remaining limits.

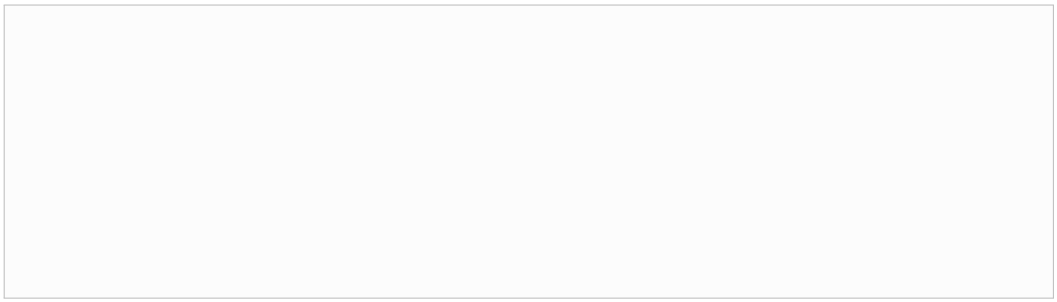


Figure 8.4: Observed differences between simulation assumptions and demonstrator conditions.

The evidence remains bounded by the number of sites, deployment duration, seasonal coverage, partner access, manual steps, and degree of direct actuation. Weather, occupancy, prices, and participant behaviour can confound energy outcomes. Where randomised field periods are unavailable, matched baselines, replay, and counterfactual simulation can support interpretation without extending the causal claim beyond the study design.

Figure 8.5 reports the final demonstrator evidence without collapsing simulated, integration, and physical outcomes into one score.

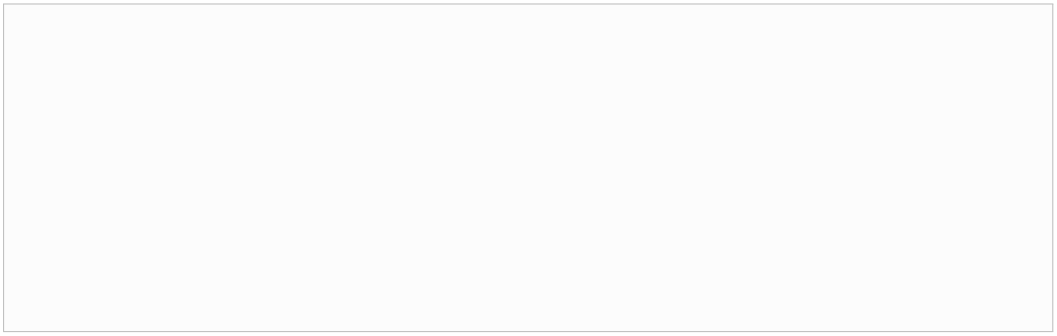


Figure 8.5: Verified OPEVA and DemFlex results by evidence level.

8.5 Contribution to the Research Questions and Chapter Summary

The two demonstrators contribute to all four research questions by testing interactions that the preceding chapters isolate experimentally. For RQ1, they identify the heterogeneous interfaces, participant and asset configurations, and availability changes that occurred outside a fixed benchmark. For RQ2, they establish the installation, device, data, and runtime scales actually reached, while larger controlled population claims remain in simulation. For RQ3, they expose data-quality failures, missing vehicle state, physical limits, platform interventions, and degraded operation. For RQ4, they test the path from historical data and training to packaging, external-platform integration, edge or cloud inference, monitoring, updates, and recovery.

Chapter synthesis. OPEVA demonstrates how EnergAIze joins heterogeneous community and charging assets, external platforms, data normalisation, vehicle and flexibility information, simulation, learned control, and bounded actuation. Its strongest current evidence combines long-horizon real-data simulation, real-platform integration, local charging control, and preliminary field outcomes, while several scenarios remain partially simulated or bounded by platform access and site-operation constraints. DemFlex is treated as a separate demonstrator so that its contribution can be stated from its own configuration and evidence.

Together, the demonstrators show what changes when the architectural and algorithmic contributions meet physical assets, provider interfaces, imperfect data, runtime resources, and human authority. The next chapter synthesises that evidence with the controlled results from Chapters 4–7 and answers the research questions explicitly.

Chapter 9

General Discussion and Answers to the Research Questions

The thesis began with a practical observation: a controller that performs well in an isolated simulation is not automatically ready to operate in a distributed physical system. The state-of-the-art review showed that this difference cannot be reduced to one missing algorithm. Scalable MARL methods, safe RL mechanisms, dynamic-team methods, physical testbeds, operational deployments, RLOps, and dynamic co-simulation all have strong counterexamples (Almilaify et al. 2023; Chen, Liu, et al. 2023; Hardy et al. 2024; Li, Thomas, et al. 2022; Sebastián et al. 2025; Zhang, Zhang, Quiñones-Grueiro, et al. 2024). The remaining problem is that real systems require several of these capabilities at the same time, while the assumptions that support one capability may change the meaning of another.

The contribution chapters approached this problem cumulatively. EnergAIze made the complete path visible. The simulator made the relevant system conditions reproducible. The controller progression solved a sequence of increasingly difficult problems and established the conditions under which each solution was useful. TI-MARL brought the selected capabilities together around versioned entity-bound interfaces, variable local action groups, health-aware compilation, coherent local bundles, and variable-population joint training. The lifecycle contribution governed the relationship between experiments, policies, policy sets, packages, targets, and runtime evidence. The real-world chapter then tested which assumptions survived contact with operational data, software, devices, and external stakeholders.

The discussion combines the findings, answers the research questions, analyses interactions among the four requirements, and states the limits of the conclusions.

9.1 Reconstructing the Thesis Argument

The complete argument rests on five connected findings.

First, real-world readiness is an integration problem. The literature did not support the original temptation to claim that dynamic populations, Safety, scalability, or deployment were absent. It supported a narrower conclusion: the evidence is distributed across methods, domains, and layers, and often depends on stable assumptions about agents, interfaces, communication, constraints, or runtime infrastructure. The research problem is therefore not to invent every missing piece, but to preserve meaning and evidence as those pieces are combined.

Second, the architecture must expose the control contract. EnergAIze treats data sources, scenarios, agents, assets, observations, actions, constraints, controllers, policies, packages, and deployment targets as related objects. This prevents a common failure in experimental systems: a checkpoint is moved into another runtime while the assumptions that gave its inputs and outputs meaning remain undocumented. The architecture does not prove control quality, but it establishes where that meaning is represented and where each claim is evaluated.

Third, the algorithmic progression produced solutions, not only motivation. The five RBC levels established feasible and increasingly informed operation. MILP made objectives and constraints explicit and provided an operational or oracle reference depending on its information. PPO, SAC, and TD3 established what model-free adaptation could achieve without multi-agent coordination. MADDPG introduced decentralised continuous control with centralised training. MATD3 addressed part of the stability and overestimation problem and may serve as a principal learning backbone. Behaviour cloning, guided exploration, replay selection, offline learning, distillation, and hierarchy addressed data use, training efficiency, reuse, and local–community coordination. These findings remain contributions even where the corresponding mechanism is not retained in the final deployed configuration.

Fourth, the remaining assumptions did not automatically compose. A MATD3 policy can be more stable than MADDPG and still assume a fixed observation and action layout. Behaviour cloning can reduce the interactions needed to reach a useful policy and still inherit the teacher's restricted action grammar. A hierarchical coordinator can improve community outcomes while assuming that the set of local controllers and their capabilities are known. A safety adapter can enforce current bounds while becoming stale when the interface changes. TI-MARL was designed in response to this integration problem.

Fifth, operation requires governance after learning. Even an effective TI-MARL policy is not an operational service by itself. The environment, scenario, reward, schemas, dependencies, policy members, target requirements, safety logic, and compatible rollback state must remain identifiable. Runtime monitoring must include data, control, safety, and coordination, not only process health. These requirements motivate the governed policy-set lifecycle and the distinction between Chapter 3 as the blueprint, Chapter 7 as the lifecycle machinery, and Chapter 8 as the deployed instance.

9.2 Answer to RQ1: Dynamic and Heterogeneous Multi-Agent Control

Research question. How can MARL-based control be formulated to accommodate heterogeneous and dynamically changing agent populations, capabilities, and availability, while balancing individual and collective objectives in distributed cyber-physical systems?

Answer. The evidence supports a formulation in which the system does not begin with one permanently fixed joint observation and action vector. Agents, module instances, and controlled entities are distinguished, and each agent's current control interface is represented explicitly. In TI-MARL, a versioned capability schema and runtime health state are compiled into entity-bound typed observations, dynamic action-group instances, parameterised action ports, dependencies, validity, bounds, and local constraint relations. Each local policy encodes the complete current interface and jointly decodes one coherent action bundle across the assets that exist at that step.

The local actors preserve asset-level decisions and local data use, while aggregate community observations and coupled reward terms expose shared objectives such as peak reduction, ramping, renewable use, and service delivery. Centralised set or graph critics use the joint training transition to improve credit assignment without becoming runtime controllers. TI-MARL extends the progression in Chapter 5 to cases in which the local control contract itself changes, for example when a second charger is installed, a charger gains V2G capability, an EV session changes, an observation becomes unavailable, or a building joins the active population.

Individual and collective objectives require explicit reward, constraint, or coordination structures. Participant-level service and cost outcomes are evaluated alongside community KPIs, and the hierarchical studies compare global and per-building signals as mechanisms for coordinating local controllers. Acceptable trade-offs are characterised by both aggregate performance and the distribution of benefits and service outcomes across participants.

What was solved along the way. RBCs established feasible local behaviour and explicit service priorities. MILP provided constraint-aware references. MADDPG and MATD3 demonstrated decentralised execution under continuous multi-agent control. Hierarchy separated community and local decisions. TI-MARL extends these capabilities through causal interface compilation, entity-bound local policies, dynamic group decoding, constraint-aware local bundles, set or graph critics, health-aware degraded modes, and runtime compatibility.

Boundary of the answer. Graph, Transformer, and fixed-head policies provide strong counterexamples for variable team size and configurable systems. The TI-MARL claim concerns changes that alter the semantics, parameters, validity, or physical consequences of what can be observed

and invoked, not cardinality alone. Its scope is defined by the tested asset types, schemas, population changes, adapters, and retraining requirements; arbitrary previously unknown agents in sustained operation remain outside the demonstrated evidence.

9.3 Answer to RQ2: Multi-Dimensional Scalability of Learning and Coordination

Research question. Under which conditions, and through which algorithmic, computational, and communication mechanisms, can MARL-based learning and coordination maintain acceptable control performance and resource use as agent population size, heterogeneity, environment complexity, and dynamic reconfiguration increase, while computational and communication resources remain constrained?

Answer. The evidence supports treating scalability as an envelope rather than a binary attribute. Different mechanisms address different bottlenecks:

- local observations, decentralised execution, and hierarchy reduce dependence on one growing central action;
- parameter sharing, structured entity representations, graph or set encoders, and TI-MARL interface tokens reduce dependence on permanent concatenation;
- MATD3 and selected training mechanisms address stability and data use, but do not remove the cost of interaction or centralised critics;
- efficient scenario formats, memory corrections, vectorisation, and distributed workers reduce simulator and experiment wall time;
- policy packaging, target profiles, and edge inference make runtime resource limits measurable; and
- explicit communication and coordination metrics reveal costs that may be hidden by reward alone.

The answer is conditional because a mechanism can scale one dimension while worsening another. A shared policy can hold parameter count constant while message and graph computation increase. A hierarchy can reduce the local action problem while creating a new dependency on the high-level signal. A simulator can run more agents while the selected network no longer fits the edge target. A policy can infer quickly while requiring an expensive training campaign.

The scalability result is expressed as an envelope over population size, heterogeneity, reconfiguration frequency, representation size, communication, training wall time, memory, inference

latency, and control quality. Operational promotion is conditioned on compatibility with the target resource budget and control requirements.

What was solved along the way. The simulator work solved major data and runtime bottlenecks, including high-frequency storage, memory growth, and structured scenario representation. The controller progression identified stable and unstable learning configurations and established the effect of stronger baselines. Hierarchical coordination decomposed local and community decisions. TI-MARL addresses population, interface-object, action-group, and resource-coupling scale without requiring one manually fixed universal joint layout. The lifecycle infrastructure separated scientific algorithm scale from campaign throughput and deployment-fleet scale.

Boundary of the answer. RQ2 is answered by a measured scalability envelope rather than one maximum agent count. Each result identifies the dimensions that increased, the assumptions held fixed, the resources consumed, and the next bottleneck. Evidence from one REC size, edge device, or communication topology remains specific to that configuration.

9.4 Answer to RQ3: Safe and Robust Operation

Research question. How can physical and operational safety constraints, uncertainty, faults, dynamic reconfiguration, and shifts between training and operation be incorporated into and systematically assessed for MARL-based control?

Answer. The evidence supports a layered answer. Safety is not inferred from reward and robustness is not inferred from one noisy test. Physical and user constraints are represented in the scenario, capability contracts, and typed constraint graph. The interface compiler exposes current validity and feasible bounds; TI-MARL jointly decodes local action bundles; local hard constraints are incorporated into bundle construction; and a narrow shared monitor protects exceptional non-negotiable limits. Runtime assurance handles incompatibility, critical health failure, and fallback. Monitoring records interface snapshots, raw and feasible local bundles, shared interventions, and service outcomes separately.

Robustness is assessed through explicit disturbance models. The analysis reports disturbance type and severity, baseline, degradation, violation or service failure, intervention, and recovery for missing or stale observations, forecast errors, communication impairment, agent or asset loss, changed capabilities, topology changes, and unseen scenarios.

The progression in Chapter 5 contributes several mechanisms. RBCs provide deterministic feasible behaviour and fallback. Behaviour cloning and guided exploration reduce time spent in implausible regions. Offline learning uses historical data without direct physical exploration. Prioritised or event-aware replay gives rare violations or events greater learning attention. Distillation

can reduce runtime cost. TI-MARL integrates the selected mechanisms with health-aware interface compilation, entity-bound dynamic action groups, constraint-aware local bundles, variable-population critics, and explicit runtime assurance.

What was solved along the way. The research moved from implicit penalty-based protection towards explicit layers. It separated invalid action prevention from learning quality, distinguished teacher guidance from the final reward, represented data quality instead of silently imputing every value, and made fallback activation observable. These are substantive results even before the complete TI-MARL configuration is considered.

Boundary of the answer. Formal safety is claimed only where a proof or verified monitor supports it. Projection is bounded by the correctness of the feasible set; fallback reliability depends on its observations and adapter; and physics-informed learning depends on the physical relation being represented. The answer therefore separates guarantees by construction, runtime enforcement, empirical observation, and unverified assumptions. It also separates control-level robustness in TI-MARL from infrastructure resilience and rollback in Chapter 7.

9.5 Answer to RQ4: Governed and Deployable MARL Policy-Set Lifecycle

Research question. What lifecycle abstractions and orchestration mechanisms allow coordinated MARL policy sets to be reproduced, governed, and deployed across heterogeneous and dynamically reconfigurable edge cyber-physical systems, while supporting compatibility validation, monitoring, staged updates, rollback, and failure recovery?

Answer. The lifecycle must treat the experiment, environment, scenario, reward, constraints, observation and action schemas, agent–policy mapping, source revision, dependencies, policies, policy set, package, target, deployment, and operational record as linked versioned entities. A policy-set manifest identifies the policies intended to operate together, their roles and interfaces, communication and safety dependencies, compatible combinations, and rollback predecessor.

Candidate promotion is a target-scoped decision. Performance, safety, robustness, compatibility, and resource gates are evaluated together. Packaging preserves preprocessing, schemas, adapters, safety logic, target requirements, and the identity of the complete policy set rather than storing only weights. Deployment tracks desired and observed state and permits atomic, staged, or explicitly compatible mixed-version operation. Monitoring includes infrastructure, data quality, control performance, safety interventions, and coordination failures. Rollback restores the previous compatible operational contract, not only a previous container.

Distributed training, registries, containers, edge orchestration, progressive rollout, rollback, and sustained operational MARL already exist in different systems. The RQ4 contribution is

their MARL-specific integration around governed policy sets, interface compatibility, deployment evidence, and recovery in a reconfigurable CPS context.

What was solved along the way. Experiment jobs, workers, progress reporting, cancellation, analysis tasks, forecasting model selection, atomic replacement, policy metadata, target packaging, runtime telemetry, and fail-safe modes solve real operational problems independently. The policy-set lifecycle brings them under one state and provenance model. A deployment can then be rejected because a policy is incompatible, not only because a file is missing.

Boundary of the answer. The demonstrated lifecycle distinguishes implemented automation from design intent and manual procedure. Incomplete atomicity across intermittent edge networks, site-specific adapters, and operational evidence restricted to selected sites or policy versions define its current maturity and scope.

Figure 9.1 traces each final answer to numbered findings, contribution chapters, evidence levels, and stated boundaries.

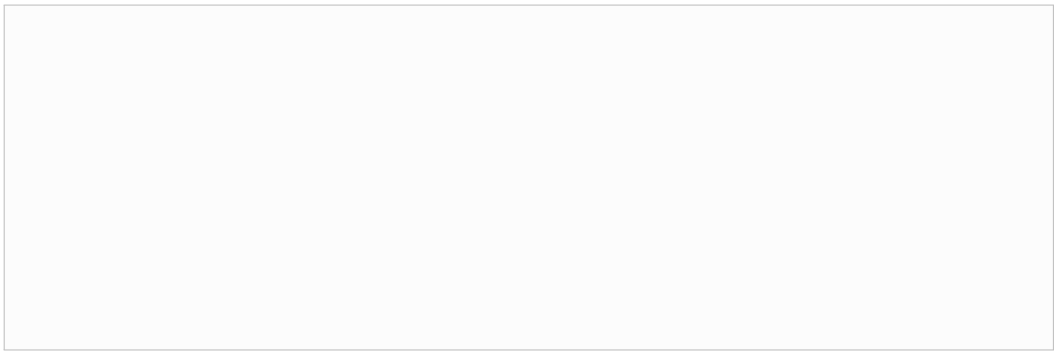


Figure 9.1: Final research-question, contribution, finding, and evidence map.

9.6 Interactions among the Four Requirements

The four requirements interact throughout the thesis. Treating them as independent scorecards would hide several important effects.

Scalability and Control Quality. Reducing information or sharing parameters can improve scale but remove distinctions needed for local objectives or constraints. More expressive structured interfaces can support heterogeneity while increasing token, graph, and communication cost. The relevant trade-off is not maximum size against reward alone; it is the control quality retained per unit of training, communication, memory, and runtime cost.

Safety and Optimality. A safety layer can reject an action that improves the nominal objective because that objective does not fully represent feasibility. Frequent repair, however, can hide a

poor policy and alter the behaviour executed at runtime. Pre- and post-intervention outcomes are therefore reported together, with intervention rate treated as part of policy quality.

Robustness and Specialisation. A policy specialised to one community or season can outperform a transferable policy in the nominal case. Offline pretraining, domain randomisation, structured representations, or physical priors can improve generalisation but can also reduce peak nominal performance. The suitable balance depends on the operational envelope and the cost of retraining and validation.

Deployability and Model Complexity. A large policy can perform well in simulation while exceeding the target's memory or latency budget. Multiple policies can fit individually while their combined package, communication, and update semantics are impractical. Distillation, hierarchy, local inference, and target-specific profiles address this problem, but the final choice remains a multi-criteria promotion decision.

Reconfiguration and Assurance Drift. A change in an interface can invalidate evidence obtained under the previous contract. A safety mask can become stale. A fallback can require a missing observation. A policy-set combination can be software-valid but behaviourally untested. This is the central reason why the thesis connects TI-MARL to lifecycle governance: interface and compatibility changes must be visible to both the controller and the operational system.

Figure 9.2 summarises the cross-effects that prevent Scalability, Deployability, Safety, and Robustness from being interpreted as independent scores.

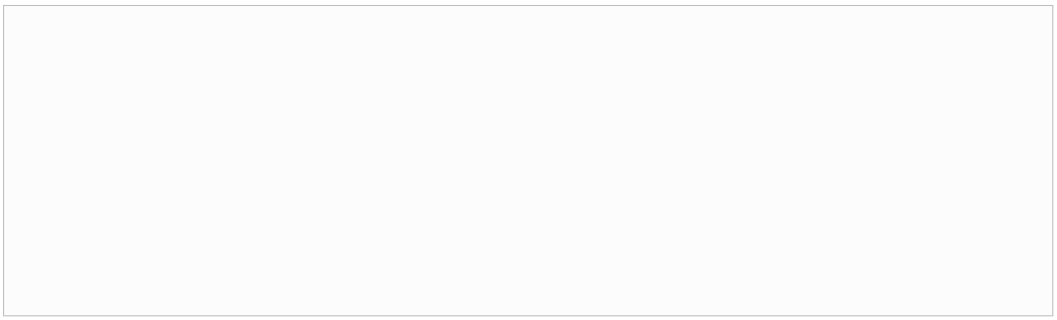


Figure 9.2: Interactions and trade-offs among Scalability, Deployability, Safety, and Robustness.

9.7 Contributions, Generality, and Responsible Operation

Scientific contributions and advances over the state of the art. The thesis advances the state of the art through a connected set of contributions rather than one claim that replaces all existing methods.

1. It replaces broad absence claims with an evidence-based account of the residual integrated problem in real-world MARL.
2. It provides the EnergAIze architecture, which keeps scenario, control, safety, lifecycle, and runtime contracts visible across the research process.
3. It extends and audits the simulation substrate required to study high-frequency, heterogeneous, event-driven, and reconfigurable energy communities.
4. It provides a progressive empirical account of what RBC, MILP, single-agent RL, MADDPG, MATD3, data-efficient learning, and hierarchy solve, and of the assumptions under which the solutions remain valid.
5. It introduces TI-MARL as the principal algorithmic contribution: a method that treats the entity-bound runtime typed interface as part of the multi-agent decision problem, jointly decodes variable local action bundles, and learns community coordination through local information boundaries and centralised set or graph critics.
6. It provides a governed lifecycle for coordinated MARL policy sets, with explicit lineage, compatibility, validation, packaging, deployment, monitoring, rollback, and recovery semantics.
7. It supplies integrated real-world-oriented evidence and documents where simulator and architectural assumptions meet operational data, devices, services, and stakeholder constraints.

The contribution taxonomy distinguishes new methods, material adaptations, verified combinations, new empirical findings, and operational abstractions. MATD3, behaviour cloning, offline RL, typed software interfaces, containers, and rollback provide established building blocks; novelty is isolated through the method definition, ablations, comparisons, and operational evidence associated with each thesis contribution.

Generality beyond renewable energy communities. The empirical evidence remains energy-domain evidence. A controller trained for EV charging does not control a robot or traffic signal by changing a label. Generality is argued at the level of problem structure and system contracts.

Other distributed CPS also contain agents that acquire or lose capabilities, observation fields that become unavailable, actions whose validity depends on physical state, local and shared objectives, heterogeneous edge targets, and policy sets that must be updated safely. TI-MARL may therefore be relevant to modular robots, industrial equipment, traffic control, or distributed edge services. EnergAIze's lifecycle abstractions may also transfer conceptually. New evidence is still required in each domain because the meaning of a safety violation, feasible action, fallback, and acceptable degradation is domain-specific.

The thesis consequently distinguishes three levels of generality:

Direct evidence. Results measured in the evaluated REC, EV, simulator, testbed, or operational setting.

Mechanism-level transfer. A formulation or architecture that addresses a shared structural problem and can be instantiated elsewhere.

Plausible implication. A direction suggested by the work but not yet validated outside the energy domain.

Ethics, privacy, security, and responsible operation. A controller can reduce aggregate cost while distributing benefits unfairly. It can preserve service on average while failing one participant at departure. An edge architecture can reduce raw data transfer while still exposing sensitive telemetry through logs. A technically successful deployment can also reduce human visibility if decisions and interventions are not recorded.

Responsible operation therefore requires more than aggregate performance. The thesis reports participant-level outcomes where possible, distinguishes human requirements from optimisation preferences, records fallback and manual intervention, and keeps operator authority over promotion and actuation. Data minimisation, access control, retention, and partner agreements remain necessary even where local inference is used. Security is not treated as a synonym for Safety; a full adversarial security contribution would require a dedicated threat model and evidence beyond the central scope of the thesis.

The use of generative AI and automated tools in preparing the dissertation is declared in the front matter. Scientific claims, sources, software, experiments, analysis, and interpretation remain the responsibility of the author.

9.8 Limitations and Threats to Validity

Construct Validity. Terms such as dynamic population, real-world, safety, robustness, and scalability can hide different meanings. Explicit taxonomies and evidence levels distinguish EV session dynamics from agent entry, real-data simulation from direct control, runtime clipping from policy safety, and population count from multi-dimensional scalability.

Internal Validity. RL results can be affected by implementation errors, reward choices, checkpoint selection, hyperparameter tuning, and stochastic variance. The simulator audits, common experiment contracts, independent seeds, deterministic tests, strong baselines, and ablations reduce this risk. They do not remove it. Any method that received a larger tuning budget or different information must be identified.

External Validity. The evaluated communities, participants, seasons, tariffs, devices, and operational periods bound the conclusions. Simulated scaling provides controlled coverage, while field studies reveal integration and operational conditions; neither substitutes for the other.

Operational and Causal Validity. Energy outcomes in physical settings can be confounded by weather, occupancy, prices, user behaviour, maintenance, and concurrent platform changes. Where randomised or controlled field periods are unavailable, the analysis should use matched baselines, replay, or counterfactual simulation and avoid causal claims beyond the design. Operational screenshots or a successful demonstration are integration evidence, not automatically evidence of sustained closed-loop benefit.

Implementation and Attribution. The work builds on CityLearn, algorithms from the literature, partner platforms, open-source software, student work, and collaborative European projects. The statement of author contributions must distinguish original method design, adaptation, integration, data provision, implementation, experiment execution, and writing. Repository history and paper contribution statements should support that account.

Review Completeness. The Chapter 2 evidence base comes from structured scoping reviews rather than an exhaustive census. Search protocols, screening records, and counterexamples make the synthesis auditable, while unindexed or unpublished work remains a possible source of additional evidence. Publisher metadata and institutional-database searches are retained with the review archive.

9.9 Open Research Challenges and Chapter Summary

Several challenges remain after this thesis.

First, truly open heterogeneous populations require admission, policy selection, interface validation, safety configuration, and coordination adaptation without assuming all future agent types. Second, formal assurance must account for interface and monitor changes rather than only fixed dynamics. Third, continual and offline policy improvement must avoid bypassing promotion and forgetting previously validated behaviour. Fourth, policy-set updates over intermittent networks require stronger consistency and local rollback semantics. Fifth, long-duration replicated deployments are needed to understand drift, maintenance, operator intervention, and benefit distribution over seasons. Finally, fairness, explainability, privacy, security, certification, and governance deserve dedicated research beyond their role as design constraints here.

Chapter synthesis. The thesis evidence supports an integrated answer. Progressive control and learning methods solve important problems under explicit assumptions. TI-MARL is motivated by the remaining difficulty of composing those solutions when agents, interfaces, physical constraints,

and information change. The governed lifecycle preserves the identity, compatibility, and operational evidence of the resulting policy sets. EnergAIze connects these contributions and exposes their behaviour in real-world-oriented energy systems.

The answers identify demonstrated mechanisms, operating envelopes, failure modes, and operational requirements. Chapter 10 condenses these conclusions, assesses the central thesis statement, and derives the main future directions.

Chapter 10

Conclusions and Future Research

This thesis investigated how multi-agent reinforcement learning can move from controlled experiments towards the operation of distributed, heterogeneous, and dynamically reconfigurable cyber-physical systems. Renewable Energy Communities and EV charging provided the principal empirical setting because they combine local and community objectives, heterogeneous physical assets, temporal flexibility, changing availability, incomplete data, operational constraints, and edge deployment.

The research followed a cumulative path in which each stage solved a concrete problem and exposed the assumptions that remained. TI-MARL provides the principal algorithmic contribution, while EnergAIze provides the architecture, simulation, lifecycle, and operational context needed to develop and evaluate it.

10.1 Thesis Recap

The state-of-the-art review first established that the research gap is not the absence of scalable MARL, safe RL, dynamic teams, physical testbeds, or deployment tooling. Strong counterexamples exist for each. The residual problem is their composition: real systems require heterogeneous and changing agents, multi-dimensional scale, physical and user constraints, robustness to imperfect information and faults, and a governed lifecycle at the same time.

EnergAIze made this complete path explicit. It connected data collection and validation, scenario construction, simulation, controller development, experiment orchestration, policy-set governance, edge deployment, monitoring, fail-safe operation, and recovery. The architecture did not replace the contributions of the later chapters; it showed where their responsibilities and evidence belonged.

The CityLearn-derived simulator provided the experimental foundation. It introduced or consolidated explicit time and unit contracts, high-frequency data support, heterogeneous flexibility assets, EV and charger behaviour, electrical-service and community mechanisms, dynamic topology and operational events, structured observation and action interfaces, multiple communities,

diagnostics, automated tests, physical audits, and performance benchmarks. This made the real-world requirements observable and reproducible before policies were placed near physical assets.

The algorithmic progression then established what different controller families could achieve. Passive operation provided a denominator. Five RBC levels solved feasibility, transparency, progressively richer context, and fallback behaviour. MILP made objectives and constraints explicit and provided operational or oracle references. PPO, SAC, and TD3 established model-free adaptation at asset or community level. MADDPG introduced decentralised continuous multi-agent control, and MATD3 addressed part of the observed stability and overestimation problem. Guided exploration, behaviour cloning, prioritised replay, offline learning, distillation, and hierarchical coordination solved further problems of data use, reuse, training, and local–community decomposition.

The significance of this progression is not only the final ranking. It provides an empirical account of what was solved, under which conditions, and what remained. More stable learning did not automatically support changing action semantics. Better data efficiency did not automatically provide physical feasibility. Hierarchy did not automatically admit a new type of local controller. These non-composable assumptions led to TI-MARL.

TI-MARL is the principal algorithmic contribution. It represents each active agent through a versioned capability schema, entity-bound runtime interface, and explicit health state. The compiler creates local typed observations and dynamic action-group instances for the modules and controlled entities that currently exist. Each local policy uses one shared local representation and type-conditioned decoders to produce a coherent action bundle across those groups. Local hard constraints are incorporated into bundle construction, while community coordination is learned through aggregate observations, coupled outcomes, and centralised set or graph critics during training. Runtime assurance handles exceptional incompatibility, critical failure, and fallback. The learning optimiser remains replaceable, allowing TI-MAPPO, TI-MATD3, and future backbones to share the same interface semantics.

The lifecycle contribution then addressed the path after training. Experiments, environments, scenarios, rewards, schemas, policies, policy sets, packages, targets, and operational records were treated as related governed entities. Compatibility, validation, promotion, staged deployment, monitoring, rollback, and recovery were made explicit. The real-world chapter finally examined which assumptions survived integration with operational data, external APIs, edge devices, charging infrastructure, and REC demonstrators.

10.2 Main Findings

The combined evidence supports the following high-level conclusions.

Real-world MARL readiness is an integration problem. Individual mechanisms already exist, but their evidence does not automatically compose when the system contract changes. Interface,

safety, runtime, and lifecycle assumptions must be visible and tested together.

Progressive methods solve real subproblems and remain part of the contribution. RBC, optimisation, single-agent RL, MADDPG, MATD3, data-efficient learning, and hierarchy are not discarded preliminaries. They provide strong controllers, reusable mechanisms, baselines, teachers, fallbacks, and empirical knowledge. Their conditions and limitations motivate the integrated method.

Controller complexity must be justified. MARL is not the default winner. A strong RBC, MILP, MPC, or single-agent method can be preferable when the system is stable, well modelled, small enough, or safety-critical. MARL adds value when local autonomy, distributed information, mixed objectives, adaptation, or repeated coordination justify its learning and lifecycle cost.

Scalability is conditional and multi-dimensional. Population size, heterogeneity, reconfiguration, representation, communication, training, simulator throughput, orchestration, and edge resources must be measured separately. A successful result defines an envelope and a bottleneck rather than a universal label.

Interfaces and physical constraints are part of the decision problem. When capabilities, observations, valid actions, or bounds change, the policy must know more than the current physical state. TI-MARL treats the current entity-bound interface as a policy-facing object and separates deterministic interface compilation, learned local bundle construction, training-time joint value estimation, and exceptional runtime assurance.

Safety and robustness require layers and explicit evidence. Reward penalties alone are insufficient. Physical constraints, action construction, validation, interface health, fail-safe modes, fallback, stress tests, and runtime monitoring provide different forms of evidence and must not be conflated.

Deployment is a governed lifecycle rather than a model-copy operation. Reproducibility, policy-set compatibility, packaging, target profiles, staged updates, monitoring, rollback, and recovery are part of operational control. Process health alone cannot show that a coordinated policy set remains valid.

10.3 Principal Scientific and System Contributions

Scientific contributions. The thesis makes the following contribution families:

1. an evidence-based formulation of the residual real-world MARL problem;

2. the EnergAIze integrated research architecture;
3. a physically audited and extended simulation environment for REC and EV-control research;
4. a progressive empirical body of control and learning solutions, including the conditions and limitations of RBC, optimisation, RL, MADDPG, MATD3, data-efficient learning, and hierarchy;
5. TI-MARL, the principal algorithmic contribution for learning over changing entity-bound typed interfaces, jointly decoding variable local action bundles, and coordinating decentralised agents through aggregate context and centralised set or graph critics;
6. a governed lifecycle and reference implementation for coordinated MARL policy sets and distributed edge deployment; and
7. integrated real-world-oriented evidence and lessons from energy systems.

Each contribution is linked to its publications, software releases, experiments, quantitative findings, limitations, and author contribution. This traceability is particularly important for collaborative software and project demonstrators.

System, software, and open-source contributions. The research produced or extended scenario and data tools, the CityLearn-derived REC simulator, controller implementations, TI-MARL components, experiment workers and backend services, policy and policy-set metadata, forecasting and analysis services, packaging and edge runtimes, monitoring and fail-safe mechanisms, demonstrator integrations, and reproducible configurations. Each releasable artefact is associated with a repository release or archived commit, licence, documentation, and experiment manifest.

These artefacts support reuse and auditability, but their existence is not used as a substitute for scientific evidence. The simulator is supported by tests and audits. Algorithms are supported by comparisons and ablations. The lifecycle is supported by systems and failure experiments. The operational system is supported by explicitly classified evidence levels.

Figure 10.1 closes the thesis by showing how the simulator, progressive solutions, TI-MARL, lifecycle, EnergAIze integration, and real-world evidence combine into the answers to the research questions.

10.4 Final Assessment of the Central Thesis

The central thesis statement proposed that operationalising MARL in distributed, heterogeneous, and dynamically reconfigurable CPS requires the integrated co-design of representations and simulation environments, scalable learning and coordination, safe and robust control, and lifecycle-aware deployment infrastructure.

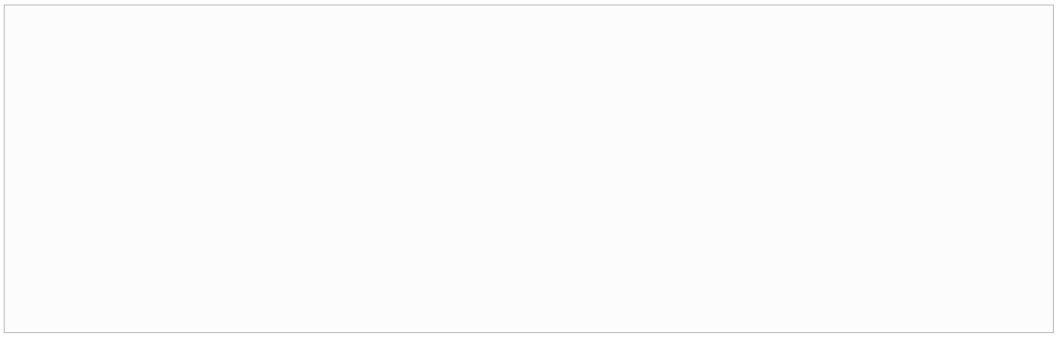


Figure 10.1: Final contribution stack and cumulative thesis argument.

The complete body of work supports this statement within the evaluated energy-domain scope. The simulator alone cannot establish control quality. A strong policy alone cannot preserve its meaning under changing interfaces or guarantee that it belongs on a target. A runtime safety layer alone can hide a poor policy. An edge platform alone can deploy an incompatible policy combination. The contributions become useful together because they preserve the relationship between the physical problem, the experiment, the policy, the operational interface, and the observed result.

The central thesis applies to CPS settings in which distributed information, heterogeneous and changing interfaces, physical constraints, and coordinated deployment make an integrated approach necessary. The evaluated scale, asset types, devices, durations, disturbances, and evidence levels define the reach of the conclusion in RECs and EV charging.

10.5 Future Research Directions

Operational Plug-and-Control Energy Assets. The next step is to admit a new energy asset from its declared capabilities and interfaces: identify the asset, compile observations and action ports, select a compatible policy or fallback, validate safety and policy-set compatibility, activate it, and monitor the effect on coordination. This requires broader asset schemas, admission tests, and longer operational evidence.

TI-MARL across Backbones, Domains, and Larger Systems. TI-MARL should be evaluated with alternative learning backbones, larger agent and interface budgets, different communication structures, and additional CPS domains. These studies should preserve the narrow claim discipline used in the thesis: distinguish the value of the interface formulation from the value of a stronger optimiser or encoder.

Physics under Model Error. Physics-informed features, residuals, action sets, and safety adapters can fail when the physical model or measurements are wrong. Future work should represent model confidence, detect inconsistent physical assumptions, and study when physical priors help or harm learning and transfer.

Formal and Adaptive Runtime Assurance. The current layered fail-safe approach can be extended with formal barrier, reachability, or shield methods where suitable models exist. Assurance must also adapt when observations, ports, bounds, and monitors change, and it must report when its own assumptions are no longer valid.

Continual, Offline, and Federated Policy Improvement. Historical and distributed data can reduce online exploration and support adaptation across communities. Future work should combine offline evaluation, conservative policy improvement, federated or privacy-aware learning, and continual updates without bypassing promotion gates or forgetting previously validated behaviour.

Policy-Set Consistency over Intermittent Edge Networks. Coordinated updates require stronger semantics for mixed versions, local activation, disconnected operation, atomicity, and rollback. Future work should investigate which policy combinations remain safe during partial rollout and how edge nodes can recover locally while preserving community coordination.

Long-Duration and Replicated Operational Studies. The field needs more replicated, seasonal, and long-duration evidence. Future studies should report decisions, outages, data-quality incidents, interventions, policy promotions, rollbacks, active agents, resource use, and participant outcomes, not only average energy savings.

Fairness, Explainability, Human Oversight, and Governance. Community control affects participants differently. Future research should examine benefit distribution, explanation of interface and policy decisions, operator and participant override, data governance, certification, cybersecurity, and regulatory accountability as dedicated research questions rather than secondary metrics.

10.6 Closing Remarks

The work began with multi-agent learning for energy coordination and expanded because real operation made the hidden assumptions visible. The resulting thesis is not a catalogue of algorithms and not a software-platform report. It is the account of a cumulative research process: feasible control led to explicit optimisation; adaptation led to decentralised coordination; stability and data-efficiency mechanisms improved learning; hierarchy separated local and community decisions; the limits of composing those solutions led to TI-MARL; and operational use led to governed policy sets, fail-safe execution, and lifecycle evidence.

The central lesson is straightforward. Intelligent control becomes credible not when one method achieves the highest simulated return, but when the complete system can explain what it controls, what the policy can currently observe and do, why an action is admissible, how the

experiment can be reconstructed, which policies may operate together, what happens when assumptions fail, and what evidence supports continued operation. EnergAIze, TI-MARL, and the accompanying experimental work provide one concrete step towards that goal.

References

Ackermann, J., V. Gabler, T. Osa, and M. Sugiyama (2019). *Reducing Overestimation Bias in Multi-Agent Domains Using Double Centralized Critics*. DOI: 10.48550/arXiv.1910.01465. arXiv: 1910.01465 [cs.LG].

Albrecht, S. V., F. Christianos, and L. Schäfer (2024). *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. Cambridge, MA: MIT Press. URL: https://www.marl-book.com.

Almilaify, Y., K. Nweye, and Z. Nagy (2023). "SCALEX: Scalability Exploration of Multi-Agent Reinforcement Learning Agents in Grid-Interactive Efficient Buildings". In: *Proceedings of the 10th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*, pp. 261–264. DOI: 10.1145/3600100.3623749.

Anil, G., P. Doshi, D. A. Redder, A. Eck, and L.-K. Soh (2025). "MOHITO: Multi-Agent Reinforcement Learning using Hypergraphs for Task-Open Systems". In: *Proceedings of the Forty-first Conference on Uncertainty in Artificial Intelligence*. Vol. 286. Proceedings of Machine Learning Research. PMLR, pp. 149–171. URL: https://proceedings.mlr.press/v286/anil25a.html.

Biagioni, D., X. Zhang, D. Wald, D. Vaidhynathan, R. Chintala, J. King, and A. S. Zamzam (2022). "PowerGridworld: A Framework for Multi-Agent Reinforcement Learning in Power Systems". In: *Proceedings of the Thirteenth ACM International Conference on Future Energy Systems*. Association for Computing Machinery, pp. 565–570. DOI: 10.1145/3538637.3539616.

Blad, C., S. Bøgh, C. Kallesøe, and P. Raftery (2023). "A Laboratory Test of an Offline-Trained Multi-Agent Reinforcement Learning Algorithm for Heating Systems". In: *Applied Energy* 337, p. 120807. DOI: 10.1016/j.apenergy.2023.120807.

Blum, D., J. Arroyo, S. Huang, J. Drgoa, F. Jorissen, H. T. Walnum, Y. Chen, K. Benne, D. Vrabie, M. Wetter, and L. Helsen (2021). "Building Optimization Testing Framework (BOPTEST) for Simulation-Based Benchmarking of Control Strategies in Buildings". In: *Journal of Building Performance Simulation* 14.5, pp. 586–610. DOI: 10.1080/19401493.2021.1986574.

Campoy-Nieves, A., A. Manjavacas, J. Jiménez-Raboso, M. Molina-Solana, and J. Gómez-Romero (2025). "Sinergym: A Virtual Testbed for Building Energy Optimization with Reinforcement Learning". In: *Energy and Buildings* 327, p. 115075. DOI: 10.1016/j.enbuild.2024.115075.

Candela, E., L. Parada, L. Marques, T.-A. Georgescu, Y. Demiris, and P. Angeloudis (2022). "Transferring Multi-Agent Reinforcement Learning Policies for Autonomous Driving Using Sim-to-Real". In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 8814–8820. DOI: 10.1109/IROS47612.2022.9981319.

Cao, D. et al. (2020). "Reinforcement Learning and Its Applications in Modern Power and Energy Systems: A Review". In: *Journal of Modern Power Systems and Clean Energy* 8.6, pp. 1029–1042. DOI: 10.35833/MPCE.2020.000552.

Charbonnier, F., T. Morstyn, and M. D. McCulloch (2022). "Scalable Multi-Agent Reinforcement Learning for Distributed Control of Residential Energy Flexibility". In: *Applied Energy* 314, p. 118825. DOI: 10.1016/j.apenergy.2022.118825.

Chen, D., K. Chen, Z. Li, T. Chu, R. Yao, F. Qiu, and K. Lin (2022). "PowerNet: Multi-Agent Deep Reinforcement Learning for Scalable Powergrid Control". In: *IEEE Transactions on Power Systems* 37.2. DOI: 10.1109/TPWRS.2021.3100898.

Chen, P., S. Liu, X. Wang, and I. Kamwa (2023). "Physics-Shielded Multi-Agent Deep Reinforcement Learning for Safe Active Voltage Control with Photovoltaic/Battery Energy Storage Systems". In: *IEEE Transactions on Smart Grid* 14.4, pp. 2656–2667. DOI: 10.1109/TSG.2022.3228636.

Chen, X., G. Qu, Y. Tang, S. H. Low, and N. Li (2022). "Reinforcement Learning for Selective Key Applications in Power Systems: Recent Advances and Future Challenges". In: *IEEE Transactions on Smart Grid* 13.4, pp. 2935–2958. DOI: 10.1109/TSG.2022.3154718.

Chu, T., J. Wang, L. Codecà, and Z. Li (2020). "Multi-Agent Deep Reinforcement Learning for Large-Scale Traffic Signal Control". In: *IEEE Transactions on Intelligent Transportation Systems* 21.3. DOI: 10.1109/TITS.2019.2901791.

Chu, Y., Z. Wei, X. Fang, S. Chen, and Y. Zhou (2022). "A Multiagent Federated Reinforcement Learning Approach for Plug-In Electric Vehicle Fleet Charging Coordination in a Residential Community". In: *IEEE Access* 10, pp. 98535–98548. DOI: 10.1109/ACCESS.2022.3206020.

Claus, C. and C. Boutilier (1998). "The Dynamics of Reinforcement Learning in Cooperative Multiagent Systems". In: *Proceedings of the Fifteenth National Conference on Artificial Intelligence*. AAAI Press, pp. 746–752.

Direção-Geral de Energia e Geologia (2023). *Consumo por Município e Tipo de Consumidor*. Official Portuguese electricity-consumption statistics. URL: https://www.dgeg.gov.pt/pt/estatistica/energia/eletricidade/consumo-por-municipio-e-tipo-de-consumidor/ (visited on 08/21/2026).

Doshi, R., H. R. Walke, O. Mees, S. Dasari, and S. Levine (2025). "Scaling Cross-Embodied Learning: One Policy for Manipulation, Navigation, Locomotion and Aviation". In: *Proceedings of the 8th Conference on Robot Learning*. Vol. 270. Proceedings of Machine Learning Research. PMLR, pp. 496–512. URL: https://proceedings.mlr.press/v270/doshi25a.html.

Dulac-Arnold, G., N. Levine, D. J. Mankowitz, J. Li, C. Paduraru, S. Gowal, and T. Hester (2021). "Challenges of Real-World Reinforcement Learning: Definitions, Benchmarks and Analysis". In: *Machine Learning* 110.9, pp. 2419–2468. DOI: 10.1007/s10994-021-05961-4.

E-REDES (2018). *Manual de Ligações à Rede Elétrica de Serviço Público*. Official connection manual. URL: https://www.e-redes.pt/sites/eredes/files/2019-02/EDPDistribui%C3%A7%C3%A3o_Manual_Ligacoes_Rede_2018_9%C2%AA_edi%C3%A7%C3%A3o.pdf (visited on 08/21/2026).

Eichelbeck, M., H. Markgraf, and M. Althoff (2026). "CommonPower: A Framework for Safe Data-Driven Smart Grid Control". In: *IEEE Transactions on Smart Grid* 17.1, pp. 71–82. DOI: 10.1109/TSG.2025.3616402.

Elsayed-Aly, I., S. Bharadwaj, C. Amato, R. Ehlers, U. Topcu, and L. Feng (2021). "Safe Multi-Agent Reinforcement Learning via Shielding". In: *Proceedings of the 20th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, pp. 483–491. arXiv: 2101.11196. URL: https://www.ifaamas.org/Proceedings/aamas2021/pdfs/p483.pdf.

Entidade Reguladora dos Serviços Energéticos (2026a). *Manual do Utilizador: Simulador de Potência Contratada*. Official guidance on Portuguese low-voltage contracted-power levels. URL: https://www.erse.pt/media/02ibhrwu/manualutilizador_pt.pdf (visited on 08/21/2026).

— (2026b). *Mobilidade Elétrica: Como Funciona*. URL: https://www.erse.pt/consumidores-de-energia/mobilidade-electrica/como-funciona/ (visited on 08/21/2026).

European Parliament and Council of the European Union (2018). *Directive (EU) 2018/2001 of the European Parliament and of the Council of 11 December 2018 on the Promotion of the Use of Energy from Renewable Sources*. L 328, pp. 82–209. Official Journal of the European Union. URL: https://eur-lex.europa.eu/eli/dir/2018/2001/oj.

Faia, R., B. Ribeiro, C. Goncalves, L. Gomes, and Z. Vale (2023). "Multi-Agent Based Energy Community Cost Optimization Considering High Electric Vehicles Penetration". In: *Sustainable Energy Technologies and Assessments* 59, p. 103402. DOI: 10.1016/j.seta.2023.103402.

Fan, T.-H., X. Y. Lee, and Y. Wang (2022). "PowerGym: A Reinforcement Learning Environment for Volt-Var Control in Power Distribution Systems". In: *Proceedings of the 4th Annual Learning for Dynamics and Control Conference*. Vol. 168. Proceedings of Machine Learning Research. PMLR, pp. 21–33.

Foerster, J. N., G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson (2018). "Counterfactual Multi-Agent Policy Gradients". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1, pp. 2974–2982. DOI: 10.1609/aaai.v32i1.11794.

Fonseca, T., L. L. Ferreira, B. Cabral, R. Severino, K. Nweye, D. Ghose, and Z. Nagy (2025). "EVLearn: Extending the CityLearn Framework with Electric Vehicle Simulation". In: *Energy Informatics* 8.1, p. 16. DOI: 10.1186/s42162-024-00445-w.

Fonseca, T., L. L. Ferreira, B. Cabral, R. Severino, and I. Praça (2024). "EnergAIze: Multi Agent Deep Deterministic Policy Gradient for Vehicle to Grid Energy Management". In: *arXiv preprint arXiv:2404.02361*. DOI: 10.48550/arXiv.2404.02361. arXiv: 2404.02361.

Fonseca, T., L. L. Ferreira, J. Landeck, L. Klein, P. Sousa, and F. Ahmed (2022). "Flexible Loads Scheduling Algorithms for Renewable Energy Communities". In: *Energies* 15.23, p. 8875. DOI: 10.3390/en15238875.

Fonseca, T., C. Sousa, R. Venâncio, P. Pires, R. Severino, P. Rodrigues, P. Paiva, and L. L. Ferreira (2025). "Control of Renewable Energy Communities using AI and Real-World Data". In: *arXiv preprint arXiv:2505.17321*. DOI: 10.48550/arXiv.2505.17321. arXiv: 2505.17321.

Fujimoto, S., H. van Hoof, and D. Meger (2018). "Addressing Function Approximation Error in Actor-Critic Methods". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, pp. 1587–1596. URL: https://proceedings.mlr.press/v80/fujimoto18a.html.

Gonçalves, R., D. Gomes, and M. Antunes (2025). "Intelligent Management of Renewable Energy Communities: An MLaaS Framework with RL-Based Decision Making". In: *Energies* 18.13, p. 3477. DOI: 10.3390/en18133477.

Gronauer, S. and K. Diepold (2022). "Multi-Agent Deep Reinforcement Learning: A Survey". In: *Artificial Intelligence Review* 55.2, pp. 895–943. DOI: 10.1007/s10462-021-09996-w.

Gryga, L. and B. Rossi (2021). "Co-Simulation of Smart Grids: Dynamically Changing Topologies in Failure Scenarios". In: *Proceedings of the 6th International Conference on Complexity, Future Information Systems and Risk*. SciTePress, pp. 63–69. DOI: 10.5220/0010448700630069.

Gu, S., J. G. Kuba, Y. Chen, Y. Du, L. Yang, A. Knoll, and Y. Yang (2023). "Safe Multi-Agent Reinforcement Learning for Multi-Robot Control". In: *Artificial Intelligence* 319, p. 103905. DOI: 10.1016/j.artint.2023.103905.

Haarnoja, T., A. Zhou, P. Abbeel, and S. Levine (2018). "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, pp. 1861–1870. URL: https://proceedings.mlr.press/v80/haarnoja18b.html.

Hamzah, M. et al. (2023). "Distributed Control of Cyber Physical System on Various Domains: A Critical Review". In: *Systems* 11.4, p. 208. DOI: 10.3390/systems11040208.

Hardy, T. D., B. Palmintier, P. L. Top, D. Krishnamurthy, and J. C. Fuller (2024). "HELICS: A Co-Simulation Framework for Scalable Multi-Domain Modeling and Analysis". In: *IEEE Access* 12, pp. 24325–24347. DOI: 10.1109/ACCESS.2024.3363615.

Hermans, B., S. S. W. Walker, J. H. A. Ludlage, and L. Özkan (2024). "Model Predictive Control of Vehicle Charging Stations in Grid-Connected Microgrids: An Implementation Study". In: *Applied Energy* 368, p. 123210. DOI: 10.1016/j.apenergy.2024.123210.

Hernandez-Leal, P., B. Kartal, and M. E. Taylor (2019). "A Survey and Critique of Multiagent Deep Reinforcement Learning". In: *Autonomous Agents and Multi-Agent Systems* 33.6, pp. 750–797. DOI: 10.1007/s10458-019-09421-1.

Hoffman, M. W., B. Shahriari, J. Aslanides, G. Barth-Maron, F. Behbahani, T. Norman, A. Abdolmaleki, A. Cassirer, F. Yang, K. Baumli, S. Henderson, A. Novikov, S. Gómez Colmenarejo, S. Cabi, Ç. Gülçehre, T. Le Paine, A. Cowie, Z. Wang, B. Piot, and N. de Freitas (2020). "Acme: A Research Framework for Distributed Reinforcement Learning". In: *arXiv preprint arXiv:2006.00979*. arXiv: 2006.00979 [cs.LG].

Hu, S., Y. Zhong, M. Gao, W. Wang, H. Dong, X. Liang, Z. Li, X. Chang, and Y. Yang (2023). "MARLlib: A Scalable and Efficient Multi-Agent Reinforcement Learning Library". In: *Journal of Machine Learning Research* 24.315, pp. 1–23. URL: https://www.jmlr.org/papers/v24/23-0378.html.

Huang, W., I. Mordatch, and D. Pathak (2020). "One Policy to Control Them All: Shared Modular Policies for Agent-Agnostic Control". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, pp. 4455–4464. URL: https://proceedings.mlr.press/v119/huang20d.html.

Hussain, S., C. Lai, and U. Eicker (2023). "Flexibility: Literature Review on Concepts, Modeling, and Provision Method in Smart Grid". In: *Sustainable Energy, Grids and Networks* 35, p. 101113. DOI: 10.1016/j.segan.2023.101113.

Intelligent Environments Laboratory (2026). *CityLearn: A Gymnasium Environment for Demand Response*. URL: https://github.com/intelligent-environments-lab/CityLearn (visited on 08/20/2026).

Jain, A., A. Szot, and J. J. Lim (2020). "Generalization to New Actions in Reinforcement Learning". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, pp. 4661–4672. URL: https://proceedings.mlr.press/v119/jain20b.html.

Kaelbling, L. P., M. L. Littman, and A. R. Cassandra (1998). "Planning and Acting in Partially Observable Stochastic Domains". In: *Artificial Intelligence* 101.1–2, pp. 99–134. DOI: 10.1016/S0004-3702(98)00023-X.

Kostrikov, I., A. Nair, and S. Levine (2022). "Offline Reinforcement Learning with Implicit Q-Learning". In: *International Conference on Learning Representations*. arXiv: 2110.06169. URL: https://openreview.net/forum?id=68n2s9ZJWF8.

Kreuzberger, D., N. Kühl, and S. Hirschl (2023). "Machine Learning Operations (MLOps): Overview, Definition, and Architecture". In: *IEEE Access* 11, pp. 31866–31879. DOI: 10.1109/ACCESS.2023.3262138.

Kumar, A., A. Zhou, G. Tucker, and S. Levine (2020). "Conservative Q-Learning for Offline Reinforcement Learning". In: *Advances in Neural Information Processing Systems*. Vol. 33. DOI: 10.48550/arXiv.2006.04779. arXiv: 2006.04779.

Lee, E. A. (2008). "Cyber Physical Systems: Design Challenges". In: *2008 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing*, pp. 363–369. DOI: 10.1109/ISORC.2008.25.

Lee, Z. J., S. Sharma, D. Johansson, and S. H. Low (2021). "ACN-Sim: An Open-Source Simulator for Data-Driven Electric Vehicle Charging Research". In: *IEEE Transactions on Smart Grid* 12.6, pp. 5113–5123. DOI: 10.1109/TSG.2021.3103156.

Levine, S., A. Kumar, G. Tucker, and J. Fu (2020). *Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems*. DOI: 10.48550/arXiv.2005.01643. arXiv: 2005.01643 [cs.LG].

Li, B., H. Tang, Y. Zheng, J. Hao, P. Li, Z. Wang, Z. Meng, and L. Wang (2022). "HyAR: Addressing Discrete-Continuous Action Reinforcement Learning via Hybrid Action Representation". In: *International Conference on Learning Representations*. arXiv: 2109.05490. URL: https://openreview.net/forum?id=wQkaGq7Vz6q.

Li, P., J. D. Thomas, X. Wang, A. Khalil, A. K. Ahmad, R. Inacio, S. Kapoor, A. Parekh, A. Doufexi, A. Shojaeifard, and R. J. Piechocki (2022). "RLOps: Development Life-Cycle of Reinforcement Learning Aided Open RAN". In: *IEEE Access* 10, pp. 113808–113826. DOI: 10.1109/ACCESS.2022.3217511.

Liang, E., R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. E. Gonzalez, M. I. Jordan, and I. Stoica (2018). "RLlib: Abstractions for Distributed Reinforcement Learning". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, pp. 3053–3062. URL: https://proceedings.mlr.press/v80/liang18b.html.

Littman, M. L. (1994). "Markov Games as a Framework for Multi-Agent Reinforcement Learning". In: *Proceedings of the Eleventh International Conference on Machine Learning*. Morgan Kaufmann, pp. 157–163. DOI: 10.1016/B978-1-55860-335-6.50027-1.

Liu, G. H., A. Siravuru, S. Prabhakar, M. Veloso, and G. Kantor (2017). "Learning End-to-End Multimodal Sensor Policies for Autonomous Navigation". In: *Proceedings of the 1st Conference on Robot Learning*. Vol. 78. Proceedings of Machine Learning Research. PMLR, pp. 249–261. URL: https://proceedings.mlr.press/v78/liu17a.html.

Lowe, R., Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch (2017). "Multi-Agent Actor-Critic for Mixed Cooperative–Competitive Environments". In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 6379–6390.

Luketina, J., J. Lanchantin, S. Sukhbaatar, and A. Szlam (2025). "Compositional Interfaces for Compositional Generalization". In: *Proceedings of the 3rd Conference on Lifelong Learning Agents*. Vol. 274. Proceedings of Machine Learning Research. PMLR, pp. 692–709. URL: https://proceedings.mlr.press/v274/luketina25a.html.

Nagy, Z., J. R. Vázquez-Canteli, S. Dey, and G. Henze (2021). "The CityLearn Challenge 2021". In: *Proceedings of the 8th ACM International Conference on Systems for Energy-Efficient*

Buildings, Cities, and Transportation. Association for Computing Machinery, pp. 218–219. DOI: 10.1145/3486611.3492226.

Nweye, K., K. Kaspar, G. Buscemi, T. Fonseca, G. Pinto, D. Ghose, S. Duddukuru, P. Pratapa, H. Li, J. Mohammadi, L. L. Ferreira, T. Hong, M. Ouf, A. Capozzoli, and Z. Nagy (2025). "CityLearn v2: Energy-Flexible, Resilient, Occupant-Centric, and Carbon-Aware Management of Grid-Interactive Communities". In: *Journal of Building Performance Simulation* 18.1, pp. 17–38. DOI: 10.1080/19401493.2024.2418813.

Nweye, K., S. Sankaranarayanan, and Z. Nagy (2023). "MERLIN: Multi-Agent Offline and Transfer Learning for Occupant-Centric Operation of Grid-Interactive Communities". In: *Applied Energy* 346, p. 121323. DOI: 10.1016/j.apenergy.2023.121323.

Oliehoek, F. A. and C. Amato (2016). *A Concise Introduction to Decentralized POMDPs*. Cham: Springer. DOI: 10.1007/978-3-319-28929-8.

OMIE (2023). *Day-Ahead Market Hourly Price in Portugal: Official File Archive*. URL: https://www.omie.es/en/file-access-list?dir=+Day-ahead+market+hourly+price+in+Portugal&parents%5B0%5D=%2F&parents%5B1%5D=Day-ahead+Market&parents%5B2%5D=1.+Prices&realdir=marginalpdbcpt (visited on 08/21/2026).

— (Oct. 1, 2025). *The Day-Ahead Market Begins Trading in 15-Minute Periods*. Official market notice. URL: https://www.omie.es/sites/default/files/2025-10/2501001_go_live_mtu15_md_en_vf.pdf (visited on 08/21/2026).

Orfanoudakis, S., C. Diaz-Londono, Y. E. Ylmaz, P. Palensky, and P. P. Vergara (2025). "EV2Gym: A Flexible V2G Simulator for EV Smart Charging Research and Benchmarking". In: *IEEE Transactions on Intelligent Transportation Systems* 26.2, pp. 2410–2421. DOI: 10.1109/TITS.2024.3510945.

Orfanoudakis, S., N. K. Panda, P. Palensky, and P. P. Vergara (2025). "GNN-DT: Graph Neural Network Enhanced Decision Transformer for Efficient Optimization in Dynamic Environments". In: *arXiv preprint arXiv:2502.01778*. DOI: 10.48550/arXiv.2502.01778. arXiv: 2502.01778.

Patrício, J. T., T. Lopes, A. Rocha, J. Martins, and R. A. Lopes (2026). "Development and Real-World Deployment of a Multi-Agent System to Support the Operation of Renewable Energy Communities". In: *Energy Reports* 15, p. 108967. DOI: 10.1016/j.egyr.2025.108967.

Pigott, A., C. Crozier, K. Baker, and Z. Nagy (2022). "GridLearn: Multiagent Reinforcement Learning for Grid-Aware Building Energy Management". In: *Electric Power Systems Research* 213, p. 108521. DOI: 10.1016/j.epsr.2022.108521.

Ponse, K., J. F. Kleuker, T. M. Moerland, and A. Plaat (2025). "Chargax: A JAX-Accelerated EV Charging Simulator". In: *Reinforcement Learning Journal* 6, pp. 364–383.

Pordata (2023). *Consumo de Energia Elétrica por Consumidor: Total e por Tipo de Consumo*. Series based on DGEG data; 2023 values provisional. URL: https://www.pordata.pt/portugal/consumo+de+energia+eletrica+por+consumidor+total+e+por+tipo+de+consumo-1231 (visited on 08/21/2026).

Puech, A., T. Rigaut, W. Templier, and M. Tournoud (2024). "Controlling Large Electric Vehicle Charging Stations via User Behavior Modeling and Stochastic Programming". In: *arXiv preprint arXiv:2402.13224*. DOI: 10.48550/arXiv.2402.13224. arXiv: 2402.13224.

Rashid, T., M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, and S. Whiteson (2018). "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research, pp. 4295–4304. URL: https://proceedings.mlr.press/v80/rashid18a.html.

Reis, I. F. G., I. Gonçalves, M. A. R. Lopes, and C. H. Antunes (2021). "Business Models for Energy Communities: A Review of Key Issues and Trends". In: *Renewable and Sustainable Energy Reviews* 144, p. 111013. DOI: 10.1016/j.rser.2021.111013.

Rusu, A. A., S. G. Colmenarejo, C. Gulcehre, G. Desjardins, J. Kirkpatrick, R. Pascanu, V. Mnih, K. Kavukcuoglu, and R. Hadsell (2015). *Policy Distillation*. DOI: 10.48550/arXiv.1511.06295. arXiv: 1511.06295 [cs.LG].

Sadeghianpourhamami, N., J. Deleu, and C. Develder (2020). "Definition and Evaluation of Model-Free Coordination of Electrical Vehicle Charging With Reinforcement Learning". In: *IEEE Transactions on Smart Grid* 11.1, pp. 203–214. DOI: 10.1109/TSG.2019.2920320.

Salvan, L. and OPEVA Consortium (Apr. 2026). *OPEVA D6.3: Presentation of All Final Demonstrators*. Tech. rep. D6.3. Version 1.0, approved 22 April 2026. Chips Joint Undertaking Project 101097267 — OPEVA.

Schaul, T., J. Quan, I. Antonoglou, and D. Silver (2016). "Prioritized Experience Replay". In: *International Conference on Learning Representations*. arXiv: 1511.05952. URL: https://arxiv.org/abs/1511.05952.

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov (2017). *Proximal Policy Optimization Algorithms*. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347 [cs.LG].

Sculley, D., G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison (2015). "Hidden Technical Debt in Machine Learning Systems". In: *Advances in Neural Information Processing Systems*. Vol. 28, pp. 2503–2511.

Sebastián, E., T. Duong, N. Atanasov, E. Montijano, and C. Sagüés (2025). "Physics-Informed Multiagent Reinforcement Learning for Distributed Multirobot Problems". In: *IEEE Transactions on Robotics* 41, pp. 4499–4517. DOI: 10.1109/TRO.2025.3582836.

Shapley, L. S. (1953). "Stochastic Games". In: *Proceedings of the National Academy of Sciences* 39.10, pp. 1095–1100. DOI: 10.1073/pnas.39.10.1095.

Shi, W., J. Cao, Q. Zhang, Y. Li, and L. Xu (2016). "Edge Computing: Vision and Challenges". In: *IEEE Internet of Things Journal* 3.5, pp. 637–646. DOI: 10.1109/JIOT.2016.2579198.

Shibata, K., T. Jimbo, and T. Matsubara (2023). "Deep Reinforcement Learning of Event-Triggered Communication and Consensus-Based Control for Distributed Cooperative Transport". In: *Robotics and Autonomous Systems* 159, p. 104307. DOI: 10.1016/j.robot.2022.104307.

Shojaeighadikolaei, A., Z. Talata, and M. Hashemi (2024). "Centralized vs. Decentralized Multi-Agent Reinforcement Learning for Enhanced Control of Electric Vehicle Charging Networks". In: *arXiv preprint arXiv:2404.12520*. DOI: 10.48550/arXiv.2404.12520. arXiv: 2404.12520.

Sinii, V., A. Nikulin, V. Kurenkov, I. Zisman, and S. Kolesnikov (2024). "In-Context Reinforcement Learning for Variable Action Spaces". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, pp. 45773–45793. URL: https://proceedings.mlr.press/v235/sinii24a.html.

SoftCPS Research Group (2026a). *Simulator: Research Fork of CityLearn for Renewable Energy Community Studies*. URL: https://github.com/Soft-CPS-Research-Group/Simulator (visited on 08/20/2026).

— (Aug. 16, 2026b). *SoftCPS REC Simulator*. Version 1.7.0. Python package. URL: https://pypi.org/project/softcpsrecsimulator/ (visited on 08/20/2026).

Sousa, C., T. Fonseca, L. L. Ferreira, R. Venâncio, and R. Severino (2025). "Percepta: High Performance Stream Processing at the Edge". In: *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, pp. 1–4. DOI: 10.1109/ETFA65518.2025.11205806.

Srithapon, C. and D. Månsson (2023). "Predictive Control and Coordination for Energy Community Flexibility with Electric Vehicles, Heat Pumps and Thermal Energy Storage". In: *Applied Energy* 347, p. 121500. DOI: 10.1016/j.apenergy.2023.121500.

Su, H., K. Shi, J. Y. J. Chow, and L. Jin (2020). *Dynamic Queue-Jump Lane for Emergency Vehicles under Partially Connected Settings: A Multi-Agent Deep Reinforcement Learning Approach*. DOI: 10.48550/arXiv.2003.01025. arXiv: 2003.01025.

Sutton, R. S. and A. G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press. URL: https://mitpress.mit.edu/9780262039246/reinforcement-learning/.

Tan, M. (1993). "Multi-Agent Reinforcement Learning: Independent versus Cooperative Agents". In: *Proceedings of the Tenth International Conference on Machine Learning*. Morgan Kaufmann, pp. 330–337. DOI: 10.1016/B978-1-55860-307-3.50049-6.

Vázquez-Canteli, J., J. Kämpf, G. Henze, and Z. Nagy (2019). "CityLearn v1.0: An OpenAI Gym Environment for Demand Response with Deep Reinforcement Learning". In: *Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*, pp. 356–357. DOI: 10.1145/3360322.3360998.

Vázquez-Canteli, J. R. and Z. Nagy (2019). "Reinforcement Learning for Demand Response: A Review of Algorithms and Modeling Techniques". In: *Applied Energy* 235, pp. 1072–1089. DOI: 10.1016/j.apenergy.2018.11.002.

Wang, J. et al. (2022). "Cooperative and Competitive Multi-Agent Systems: From Optimization to Games". In: *IEEE/CAA Journal of Automatica Sinica* 9.5, pp. 763–783. DOI: 10.1109/JAS.2022.105506.

Wang, X., N. Nimalsiri, Y. Xia, P. Wang, and S. Halgamuge (2026). "HetGPS: Scalable Graph Multi-Agent Reinforcement Learning with Physics-Anchored Adaptive Safety for EV Charging". In: *arXiv preprint arXiv:2608.00679*. Preprint, released August 2026. DOI: 10.48550/arXiv.2608.00679. arXiv: 2608.00679.

Wilk, P., N. Wang, and J. Li (2024). "Multi-Agent Reinforcement Learning for Smart Community Energy Management". In: *Energies* 17.20, p. 5211. DOI: 10.3390/en17205211.

Wu, A., K. Nweye, and Z. Nagy (2023). "Heterogeneous Multi-Agent Reinforcement Learning for Grid-Interactive Communities". In: *Proceedings of the 10th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*, pp. 314–315. DOI: 10.1145/3600100.3626276.

Wu, H., D. Qiu, L. Zhang, and M. Sun (2024). "Adaptive Multi-Agent Reinforcement Learning for Flexible Resource Management in a Virtual Power Plant with Dynamic Participating Multi-Energy Buildings". In: *Applied Energy* 374, p. 123998. DOI: 10.1016/j.apenergy.2024.123998.

Xiong, J., Q. Wang, Z. Yang, P. Sun, L. Han, Y. Zheng, H. Fu, T. Zhang, J. Liu, and H. Liu (2018). *Parametrized Deep Q-Networks Learning: Reinforcement Learning with Discrete-Continuous Hybrid Action Space*. DOI: 10.48550/arXiv.1810.06394. arXiv: 1810.06394 [cs.LG].

Yeh, C., V. Li, R. Datta, J. Arroyo, N. Christianson, C. Zhang, Y. Chen, M. M. Hosseini, A. Golmohammadi, Y. Shi, Y. Yue, and A. Wierman (2023). "SustainGym: Reinforcement Learning Environments for Sustainable Energy Systems". In: *Advances in Neural Information Processing Systems*. Vol. 36, pp. 59464–59476.

Yu, C., X. Yang, J. Gao, J. Chen, Y. Li, J. Liu, Y. Xiang, R. Huang, H. Yang, Y. Wu, and Y. Wang (2023). "Asynchronous Multi-Agent Reinforcement Learning for Efficient Real-Time Multi-Robot Cooperative Exploration". In: *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, pp. 1107–1115. arXiv: 2301.03398. URL: https://arxiv.org/abs/2301.03398.

Yu, X., Y. Lin, S. Wang, K. Lv, and S. Han (2024). *Improving Global Parameter-sharing in Physically Heterogeneous Multi-agent Reinforcement Learning with Unified Action Space*. DOI: 10.48550/arXiv.2408.07395. arXiv: 2408.07395 [cs.MA].

Zaharia, M., A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar (2018). "Accelerating the Machine Learning Lifecycle with MLflow". In: *IEEE Data Engineering Bulletin* 41.4, pp. 39–45.

Zhang, Q., K. Dehghanpour, Z. Wang, F. Qiu, and D. Zhao (2021). "Multi-Agent Safe Policy Learning for Power Management of Networked Microgrids". In: *IEEE Transactions on Smart Grid* 12.2, pp. 1048–1062. DOI: 10.1109/TSG.2020.3034827.

Zhang, Y., M. Quiñones-Grueiro, Z. Zhang, Y. Wang, W. Barbour, G. Biswas, and D. B. Work (2024). "MARVEL: Bringing Multi-Agent Reinforcement-Learning Based Variable Speed Limit Controllers Closer to Deployment". In: *IEEE Access* 12, pp. 161995–162014. DOI: 10.1109/ACCESS.2024.3489474.

Zhang, Y., Z. Zhang, J. Ji, M. Quiñones-Grueiro, W. Barbour, D. Gloudemans, G. Zachár, C. Weston, G. Biswas, and D. B. Work (2025). "Real-World Deployment and Assessment of a Multi-Agent Reinforcement Learning-Based Variable Speed Limit Control System". In: *IEEE Intelligent Transportation Systems Magazine*. DOI: 10.1109/MITS.2025.3592040. arXiv: 2503.01017.

Zhang, Y., Z. Zhang, M. Quiñones-Grueiro, W. Barbour, C. Weston, G. Biswas, and D. B. Work (2024). "Field Deployment of Multi-Agent Reinforcement Learning Based Variable Speed Limit Controllers". In: *2024 IEEE 27th International Conference on Intelligent Transportation Systems (ITSC)*, pp. 776–783. DOI: 10.1109/ITSC58415.2024.10919506. arXiv: 2407.08021.

Zhou, M., Z. Wan, H. Wang, M. Wen, R. Wu, Y. Wen, Y. Yang, Y. Yu, J. Wang, and W. Zhang (2023). "MALib: A Parallel Framework for Population-Based Multi-Agent Reinforcement Learning". In: *Journal of Machine Learning Research* 24.150, pp. 1–12. URL: https://www.jmlr.org/papers/v24/22-0169.html.

Zhu, D., B. Yang, Y. Liu, Z. Wang, K. Ma, and X. Guan (2022). "Energy Management Based on Multi-Agent Deep Reinforcement Learning for a Multi-Energy Industrial Park". In: *Applied Energy* 311, p. 118636. DOI: 10.1016/j.apenergy.2022.118636.

Zhu, Z., R. d. S. Braz, J. Bhandari, D. Jiang, Y. Wan, Y. Efroni, L. Wang, R. Xu, H. Guo, A. Nikulkov, D. Korenkevych, U. Dogan, F. Cheng, Z. Wu, and W. Xu (2024). "Pearl: A Production-Ready Reinforcement Learning Agent". In: *Journal of Machine Learning Research* 25.273, pp. 1–30. URL: https://www.jmlr.org/papers/v25/24-0196.html.

Appendix A

Reproducibility and Supplementary Material

This appendix organises the material required to reproduce and audit the principal findings without interrupting the main argument.

A.1 Claim and Evidence Inventory

A master inventory maps every retained claim to the relevant research question, method, repository and commit, scenario, dataset, experiment identifier, seeds, metrics, result table or figure, evidence level, publication, author contribution, and known limitation. This mapping defines the evidence path used throughout Chapters 4–9.

A.2 Scenario and Environment Manifests

Reconstruction of the principal experiments requires schemas or manifests that record temporal resolution, units, buildings, assets, data sources, event streams, constraints, observation and action interfaces, reward definitions, KPI configuration, software versions, and referenced data checksums.

A.3 Canonical Annual Dataset Contract

This section freezes the human-readable contract for the annual suite used in Chapters 4–6. The row-level record remains authoritative. Each family contains `dataset_manifest.json`, `settlement_contract.json`, variant schemas, Parquet catalogues for members, electrical services, chargers, EVs, sessions, and deferrables, event files, daily OMIE provenance, validation output, and `file_checksums.sha256`. The generator seed is 2023 and the contract version is `2023-q15-v1.9`. All families span 35,040 consecutive 900-second UTC intervals in 2023

and retain `Europe/Lisbon` local-calendar and daylight-saving attributes. Routine instants are first constructed in Lisbon civil time and only then converted to UTC; the missing spring hour is shifted forward by one hour and the first occurrence of the repeated autumn hour is selected deterministically. A complete annual controller run appends an observation-only terminal boundary at 1 January 2024 00:00, yielding 35,041 state points and exactly 35,040 action transitions without adding a physical energy or settlement row.

A.3.1 Calendar, price forecasts, and settlement

The current-price column is the official Portuguese OMIE 2023 day-ahead series converted from EUR/MWh to EUR/kWh. Each physical hourly value is repeated over four 15-minute steps; the 23- and 25-period daylight-saving days are preserved by the continuous UTC calendar. Forecast columns target 1, 6, and 24 hours. For a delivery interval whose day-ahead result is available by the declared conservative 13:00 local cut-off, the forecast contains that published value; before the cut-off it contains the corresponding previous-day value. The entity bundle additionally exposes 15-minute and 3-hour horizons. These use the shortest stored horizon capable of targeting the same delivery step, read at the corresponding earlier issue time. At an episode cold start with no such issue in the retained window, the current price is used. This rule is tested independently against the controller-visible observation and prevents the interface from reconstructing an unpublished realised price from the full annual table.

Settlement matches member surplus and demand within the same step, uses equal importer weights and a local price equal to 80% of the grid-import price, and assigns zero remuneration to residual grid export. The retained grid-only counterfactual disables local matching but preserves the same member demand, generation, actions, and OMIE price. Taxes, network tariffs, supplier margins, and regulated export compensation are outside this experimental accounting contract.

A.3.2 Demand, PV, and stationary storage

Demand profiles combine class- and routine-specific intraday structure, seasonal variation, correlated daily multipliers, local peak-time shifts, reduced-activity days, and correlated member-level short-term variation. The raw trajectory is scaled to an annual target sampled by rejection from the bounded log-normal ranges in Table A.1; this avoids an artificial point mass at either bound. The class centres are 2,490, 16,580, and 175,900 kWh per consumer-year and are anchored to Portuguese 2023 residential, non-domestic, and industrial averages (Direção-Geral de Energia e Geologia 2023; Pordata 2023). The ranges and routine frequencies create controlled scenario heterogeneity; they are not empirical confidence intervals or inferred population proportions.

Table A.2 lists the behaviour strata. The assignment is deterministic and family-independent, so the first 15 physical members remain identical between Core-15-Stripped and Core-30. Within an archetype, member-specific amplitudes, centres, activity reductions, annual targets, and short-term variation remain independently reproducible. Occupancy follows the routine rather than

Table A.1: Static-asset generation contract by member class. PV and BESS entries apply only to members that own the corresponding asset.

Member class	Annual demand target	PV contract	Stationary BESS contract
Residential	1.1–6.5 MWh; centre 2.49 MWh	1.5–8.0 kW; target annual coverage 0.58–1.32	5, 7.5, 9.6, 10, 13.5, or 15 kWh; 2.5–6 kW product classes
Small service/commercial	7–45 MWh; centre 16.58 MWh	8–40 kW; target annual coverage 0.42–1.02	20, 30, 40, 50, 60, or 75 kWh; 10–30 kW product classes
Larger consumption	75–350 MWh; centre 175.9 MWh	30–150 kW; target annual coverage 0.32–0.82	60, 80, 100, 120, 160, or 200 kWh; 30–80 kW product classes

remaining constant: commuter homes are largely unoccupied during the working day, home-working and home-day members retain daytime occupancy, and service occupancy follows its opening or process window.

Table A.2: Routine archetypes used to construct heterogeneous demand and flexibility.

Member class	Five stratified archetypes	Coupled effects
Residential	Early commuter; standard commuter; late commuter; home worker; predominantly home-day	Morning/evening/lunch balance, weekday absence, weekend profile, reduced-activity days, EV arrival/departure, and appliance request time
Small service/commercial	Office hours; six-day retail; split-day clinic; extended hospitality; early workshop	Opening days and hours, split or extended operation, occupancy, customer/employee charging windows, and type/timing of the deferrable service
Larger consumption	Day process; extended process; continuous process; morning-intensive process; evening-intensive process	Shift coverage, weekend reduction, maintenance days, shared-charger visits, and batch/process flexibility windows

PV capacity is obtained from the annual demand target, the common-profile specific yield, and the coverage range in Table A.1, subject to the stated class bounds. Orientation is sampled as east, south, or west with probabilities 0.25, 0.50, and 0.25; derating is sampled between 0.84 and 0.98 and combined with reproducible member-level shading. Generation is stored as absolute per-step energy. Stationary BESS capacity is selected from the product-like classes in the table by a stratified fraction of average daily demand, with a lower sizing floor tied to host PV. Nominal power is selected from the compatible product class and does not exceed 0.65 times energy capacity. Efficiency is sampled from 0.920–0.965, initial SoC from 0.25–0.60, depth of discharge from 0.80, 0.85, 0.90, or 0.95, standing-loss coefficient from 0.0005–0.0015, and capacity-loss coefficient from 0.5×10^{-5}–1.5×10^{-5}. A single-phase battery follows its member's assigned phase; a battery at a three-phase member is connected across all phases. These are benchmark parameter classes, not a market-share model.

A.3.3 Electrical connections and phase allocation

Table A.3 gives the rule used to construct member-level connection limits. The levels reproduce Portuguese connection classes at the resolution required by the simulator (E-REDES 2018; Entidade Reguladora dos Serviços Energéticos 2026a). CityLearn advances active energy, so each kVA value is mapped to the same numerical kW ceiling under an explicit unity-power-factor

assumption. For a balanced three-phase connection, each phase receives one third of the total ceiling; for a single-phase connection, the complete ceiling is assigned to one of L1–L3. Residential single-phase assignments rotate deterministically across members. The export ceiling is $\min(I_b^{\max}, P_b^{\mathrm{PV}} + P_b^{\mathrm{BESS}})$ and is divided using the same phase rule. This is a connection-headroom benchmark, not a voltage, reactive-power, protection, or feeder-power-flow model.

Table A.3: Electrical-service and charger contract. Charger powers are compatible with the charging classes described by ERSE (Entidade Reguladora dos Serviços Energéticos 2026b).

Configuration	**Contracted levels (kVA)**	**Installed charger power**	**Phase rule**
Residential, ordinary single-phase	3.45, 4.60, 5.75, 6.90, or 10.35	none, 3.7 kW, or 7.4 kW	Member and charger assigned wholly to a rotated L1, L2, or L3 community phase
Residential, one three-phase charger	13.80 or 17.25	one 11 kW charger	Uncontrollable demand and charger divided over L1–L3
Residential, two chargers	17.25 or 20.70	two independently controlled 7.4 kW chargers	Chargers connected to two rotated individual phases; shared total and per-phase limits
Small service/commercial	20.70, 27.60, 34.50, or 41.40	two 11 kW chargers where equipped; 5.5 kW V2G limit	Balanced three-phase connection; each charger spans all phases
Larger consumption	69.0, 103.5, or 138.0	four or six 22 kW chargers where equipped; 11 kW V2G limit	Balanced three-phase BTE-level surrogate; each charger spans all phases

The member-level catalogue records connection type, contracted kVA, the originally sampled class, active-power assumption, native demand and export peaks, total import/export limits, L1–L3 limits, default split, and asset phase connection. The charger catalogue separately records each physical charger and its phase. This permits two chargers at one member to receive independent actions while remaining jointly constrained by one connection. A rejection check increases the connection class when the uncontrolled native trace would otherwise exceed the sampled level before any flexible action is applied. This affects only two Premium members: Member 094 is raised from 69 to 138 kVA for native peaks of 111.5233 kW import and 97.5324 kW export, and Member 100 is raised from 69 to 103.5 kVA for a 99.6939 kW import peak. The original level and upgrade flag remain in the schema; no other family member requires this correction.

Electrical reporting distinguishes the pre-projection pressure of the requested action from the post-projection residual of the applied trajectory. Both are integrated as energy and counted as events. The residual calculation is reconstructed from total and phase power histories with a 10^{-5} kW numerical tolerance and is the hard safety criterion. Requested pressure remains a controller diagnostic and can be non-zero while the residual is zero. Robustness reporting follows an analogous identity rule: observation, forecast, and action corruptions retain field- or port-level counts, whereas asset-unavailable duration counts each physical asset once per affected time step, irrespective of the number of internal fields that expose its status.

A.3.4 EVs, sessions, and deferrable services

EV capacities are sampled from 40, 50, 58, 64, 75, and 82 kWh, with 0.90 depth of discharge and initial SoC between 0.30 and 0.60. Residential chargers use one persistent vehicle identity; shared service chargers use pools of four vehicles. Residential visit probability, arrival centre, following-morning departure centre, temporal spread, and trip-energy scale depend on the host routine and differ between weekdays and weekends. Service and larger-consumption chargers use two to four routine-dependent probabilistic windows on their eligible operating days. Independent charger-level draws allow two chargers at the same member to receive different sessions. All times are jittered and rounded to the 15-minute contract; cross-midnight normalisation prevents two sessions from occupying one physical charger simultaneously.

Required session energy is sampled from class-conditional log-normal scenario distributions with a 6 kWh residential base median multiplied by a 0.66–1.12 routine factor, a 9 kWh small-service median, and a 15 kWh larger-consumption median. It is limited to 85% of $\Delta t\, P^{\max} \eta$ for residential sessions, 72% for shared service sessions, and 72% of EV capacity before arrival and target SoC are written. The catalogue also records the minimum full-power steps and residual temporal slack. Thus every ordinary session is individually deliverable at 95% charger efficiency. Joint service can nevertheless become infeasible after total/phase limits, concurrent sessions, topology changes, demand-response obligations, or injected failures are introduced. Table A.4 freezes the generated catalogue counts and the zero-infeasibility audit result.

Every occupied charger row additionally stores the explicit session identity and an optional current-SoC boundary reference. The reference follows the neutral constant-rate trajectory from the declared arrival SoC to the departure target and is consulted only when an episode, member, charger, or new session becomes active inside an occupied interval. It prevents an arbitrary catalogue initial SoC from being substituted for omitted history; it does not overwrite the controller-driven SoC during an uninterrupted session. This distinction covers the 0, 3, 4, and 69 back-to-back session boundaries in Micro, Core-15, Core-30, and Premium, respectively, including the case in which the same residential EV begins the new session and charger occupancy remains continuous. The audit verifies that session identity agrees with the catalogue, the reference is finite and monotonic while connected, absent while disconnected, equal to arrival SoC at the session boundary, and leaves a deliverable remainder at every intermediate step. At runtime, a separate conservation check sums positive connected-session changes in stored EV energy and compares them with charger input. Any SoC gain not supported by metered input, beyond numerical tolerance, fails the controller-level audit.

All 47,258 distinct catalogue sessions across the four families are individually feasible before shared constraints and injected events are applied. For the dynamic variants, censoring is split by cause without double counting: Core-30 has 56 member-inactive and 28 charger-inactive sessions; Premium-100 has 13,048 and 17, respectively. The annual controller audit reconstructs these counts from the topology calendar and requires the recorded departure-event denominator to match. This separates an unavailable catalogue service from a failure to serve an obligation that

Table A.4: Generated EV identities, annual session counts, and topology-observable departure obligations. A censored session is absent from the controller's service denominator because its member or physical charger does not exist at the relevant boundary.

Family/variant	**EVs**	**Sessions**	**Observable**	**Censored**
Micro-4-Q	2	554	554	0
Core-15-Stripped	7	1,757	1,757	0
Core-30 static	15	3,774	3,774	0
Core-30 Dynamic/Combined	15	3,774	3,690	84
Premium-100 Clean/AllIn	212	41,173	28,108	13,065

the controller actually faced.

Each deferrable asset alternates between two complete cycle profiles and receives a member-specific energy multiplier from 0.82 to 1.22. Residential appliance types comprise washing machine, dishwasher, washer–dryer, domestic-hot-water cycle, and a mixed dishwasher/laundry class. Service types comprise cleaning or preconditioning, cold storage or cleaning, sterilisation or laundry, hospitality dishwashing or hot water, and compressor or pumping. Larger-member types comprise batch process, process auxiliaries, maintenance or pumping, thermal preparation, and late batch process. Requests occur probabilistically on routine-eligible days. Residential occurrence probabilities range from 0.52 to 0.70 and `must_run` probabilities from 0.66 to 0.76; the corresponding ranges are 0.82–0.90 and 0.88–0.94 for small services, and 0.90–0.94 and 0.94–0.97 for larger members. Earliest start, window length, completion margin, deadline, and priority vary reproducibly by asset and day. A new request is omitted if its earliest admissible start would overlap the preceding request's completion window for the same appliance. The schedule retains cycle identity, earliest/latest start, deadline, priority, and mandatory-service flag so completion, delay, and unserved energy can be evaluated.

The complete Micro, Core-15, Core-30, and Premium catalogues contain 214, 928, 1,892, and 7,360 deferrable cycles, respectively. All are observable in the static variants. Under the declared dynamic topology, Core-30 has 1,657 fully observable cycles, 234 whose start window occurs entirely while the member or appliance is absent, and one whose full start window is visible but whose completion interval crosses an asset-removal boundary. Premium has 5,950, 1,409, and one in the same categories. The last category can be completed before removal or interrupted by it and is therefore policy-dependent; the annual audit accepts the corresponding completed-plus-missed denominator only within the explicit one-cycle interval. No-start cases are topology-censored, not unserved requests.

A.3.5 Topology and robustness calendars

Topology events change whether an entity exists in the active scenario; robustness events leave it in the topology but alter data, commands, or availability. This distinction is preserved in the event logs and controller-facing masks. Reinstallation creates a new runtime instance from the declared template and initializes a stateful asset at its boundary state, while the pre-removal instance

remains in historical KPI aggregation. It does not imply unobserved state continuity through the interval in which the asset is absent. Tables A.5 and A.6 summarise the UTC topology calendar; exact time steps, target and source identifiers, and restoration templates remain in each variant schema.

An `add_asset` event constructs a new runtime instance from the immutable catalogue template. Paired removal/restoration entries therefore have replacement or recommissioning semantics: the restored capability starts from its declared initial state, and no hidden SoC, degradation, or unfinished-cycle continuity is inferred across the absence. Temporary loss of the same equipment is modelled through a robustness-unavailability event instead; the entity identity and accumulated state remain present while the selected telemetry or control channel is unavailable. This separation avoids attributing an undocumented state-reset convention to a controller result.

Table A.5: Core-30 dynamic topology calendar. Paired dates denote removal and restoration.

UTC date/time	**Target**	**Change**
01 Feb 06:00	Member 016	First entry
04 Mar 12:00 / 04 Apr 18:00	Member 006, charger 02	Removal / restoration
26 May 20:00 / 06 Jun 06:00	Member 003, BESS	Removal / restoration
16 Jun 16:00	Member 027	First entry
18 Aug 04:00 / 28 Aug 14:00	Member 017, deferrable	Removal / restoration
08 Sep 00:00	Member 029	First entry
28 Sep 20:00 / 09 Oct 06:00	Member 018, PV	Removal / restoration
19 Oct 16:00	Member 030	First entry
09 Nov 12:00	Member 019	Permanent exit
10 Dec 18:00	Member 008	Permanent exit

Table A.6: Premium-100 dynamic topology calendar. Member ranges denote simultaneous first-entry cohorts.

UTC date/time	**Target**	**Change**
31 Jan / 02 Mar / 01 Apr	Members 061–065 / 081–085 / 091–093	First-entry cohorts
15 Apr / 25 Apr	Member 025, charger 02	Removal / restoration
31 May / 30 Jun	Members 066–070 / 086–090	First-entry cohorts
07 Jul / 17 Jul	Member 012, PV	Removal / restoration
28 Jul / 02 Aug	Member 001, BESS	Removal / restoration
30 Jul / 29 Aug	Members 094–096 / 097–100	First-entry cohorts
07 Aug / 18 Aug	Member 071, charger 02	Removal / restoration
08 Sep / 18 Sep	Member 073, deferrable	Removal / restoration
28 Sep	Members 005, 006, 072	Permanent exits
27 Nov	Members 020–022	Permanent exits

Health and AllIn event files cover missing non-controllable-load observations, biased price forecasts, delayed storage actions, and temporary charger unavailability. Premium-100-AllIn additionally includes PV telemetry loss, charger-command dropout, a service-charger outage, a stationary-storage outage, and selected grid-outage intervals. Four non-overlapping demand-response requests cover upward and downward directions, different durations, tolerances, activation prices, and shortfall penalties; the Premium power targets are three times the Core-30 targets.

These events are scenario interventions, not empirical incident-frequency models.

The final structural audit verifies cadence and daylight saving, explicit civil-time localization policy, exact mapping between session timestamps and quarter-hour indices, the 365 official OMIE source records and their hashes, hourly-to-quarter-hour expansion, publication-aware price forecasts, all member and asset counts, referential integrity, explicit session identity, session occupancy and deliverability, mid-session SoC boundary initialization, non-overlapping deferrable deadlines, Portuguese contracted-power classes, phase sums, topology coverage, clean-twin equality, and every generated-file checksum. Across the suite, 1,817 structural and cross-family checks pass. The diversity audit additionally measures exact-duplicate traces and schedules, pairwise and cross-archetype profile correlations, peak-clock dispersion, annual demand, PV and BESS distributions, EV arrivals, duration, energy and slack, and deferrable-window dispersion; it reports no finding. Nine week-long schema rollouts complete 6,048 control transitions with finite observations and rewards. Across the regular and compressed rollouts, 31,565 controller-visible price-feature values match their causal inputs; the compressed dynamic timelines apply all 120 topology events and independently verify the corresponding member/asset state, the stationary-battery SoC at activation, and the physical PV balance. The full simulator regression contains 457 passing tests. The scientific audit classifies the suite as ready for benchmarking within the declared scope. The remaining limitations are the absence of a population-level goodness-of-fit claim and of feeder-level electrical physics.

A.3.6 Matched annual baseline scorecard

The final campaign contains two independently identified controller runs for each schema and binds every result to the same dataset-contract version, current family checksum, exact Simulator source digest, runtime attestation, and policy-pair fingerprint. Table A.7 gives the cost and service results. The cost column reports BAU followed by RBC Smart; the percentage column is the matched RBC change. EV values use the feasible-departure denominator, while deferrable values use the topology-valid completed-plus-missed denominator. Exact EV-target rates are omitted because both declared references stop charging within 0.04 SoC of the requirement; minimum and tolerance rates are the relevant service measures.

Table A.7: Annual matched cost and service scorecard. Paired entries are BAU/RBC Smart.

Variant	Cost (EUR) BAU/RBC	RBC Δ (%)	EV minimum BAU/RBC (%)	EV tolerance BAU/RBC (%)	Deferrable BAU/RBC (%)
Micro-4-Q	1229.62/1169.80	−4.86	100.00/100.00	100.00/100.00	100/100
Core-15-Stripped	4078.73/3964.39	−2.80	100.00/100.00	100.00/100.00	100/100
Core-30-Nominal	23497.82/23060.93	−1.86	100.00/100.00	100.00/100.00	100/100
Core-30-Safety	23496.46/23080.49	−1.77	100.00/100.00	100.00/100.00	100/100
Core-30-Health	23498.14/23061.00	−1.86	100.00/100.00	100.00/100.00	100/100
Core-30-Dynamic	12972.56/12676.37	−2.28	100.00/100.00	100.00/100.00	100/100
Core-30-Combined	12971.27/12695.37	−2.13	100.00/100.00	100.00/100.00	100/100
Premium-100-Clean	113639.38/112209.55	−1.26	99.67/99.73	99.66/99.72	100/100
Premium-100-AllIn	113610.37/112170.26	−1.27	99.63/99.65	99.62/99.64	100/100

Premium Clean records 93/77 BAU/RBC minimum misses among 28,108 observable departures. These occur only at Members 091, 095, and 097, which host four, four, and six shared 22 kW chargers: the BAU distribution is 76, 13, and 4, and RBC Smart yields 62, 14, and 1. AllIn records 107/103 misses and spreads the additional cases over fault-affected service members. This diagnostic is important because the catalogue audit independently found every session to be individually feasible. The annual misses therefore measure simultaneous service, connection, and event pressure rather than an impossible request inserted by the generator.

Table A.8 retains the stress-case evidence. Requested pressure is integrated before the feasibility projection; residual violation is reconstructed from the applied total and per-phase trajectories. Robustness counts are identical for the matched policies and record physical asset–time-step unavailability, not the number of internal status fields. Four demand-response events have valid baselines in every declared run.

Table A.8: Annual safety, demand-response, and robustness scorecard for the stress variants. Paired entries are BAU/RBC Smart.

Variant	Requested pressure BAU/RBC (kWh)	Residual BAU/RBC (kWh)	DR compliance BAU/RBC	DR shortfall BAU/RBC (kWh)	Health events/ unavailable steps
Core-30-Safety	6.281/0	0/0	—	—	—
Core-30-Health	0/0	0/0	—	—	4/48
Core-30-Combined	4.696/0	0/0	0.340/0.281	228.923/264.090	4/48
Premium-100-Clean	4235.047/1465.524	0/0	0.469/0.437	705.288/765.391	—
Premium-100-AllIn	4267.160/1494.777	0/0	0.640/0.627	394.175/427.110	8/120

All 18 runs have zero EV energy-accounting shortfall, zero residual electrical violation, and complete every runtime-visible deferrable cycle with zero unserved energy. These integrity results do not erase controller limitations. RBC Smart increases ramping relative to BAU in six schemas and reduces demand-response compliance in Combined and both Premium variants. The detailed machine-readable scorecard retains these outcomes as controller watches rather than relabelling them as dataset defects.

The member and asset ranges in this appendix are complemented by the row-level supplementary inventory: the family manifests and Parquet catalogues enumerate every member, physical charger, EV, session, PV system, stationary battery, deferrable appliance, phase assignment, connection level, limit, routine, event target, and source checksum. This machine-readable inventory is authoritative where a 100-member row listing would otherwise duplicate the released schema without adding a scientific distinction.

A.4 Simulator Audits and Demonstrative Results

This section retains the detailed evidence that supports, but would interrupt, the architectural and methodological account in Chapter 4. The results are bound to the archived simulator checkout and declared workloads. They establish implementation consistency and demonstrate that the corresponding mechanisms are executable and observable; they do not by themselves establish external

validity or controller optimality. The dataset-specific structural audit and annual BAU/RBC Smart scorecards remain in Appendix A.3.

A.4.1 Software and contract verification

Table A.9: Verification and validation evidence reported for the simulator manuscript checkout.

Check	**Failure mode protected against**	**Reported evidence**
Full regression suite	General simulator behaviour and compatibility across the extended codebase	457 passing tests; 18 warnings
Targeted REC requirements	Incorrect KPIs, event semantics, topology propagation, entity interfaces, and multi-community behaviour	103 passing tests; 8 warnings
Physics and time-unit audit	Energy/power inconsistencies across temporal resolutions	16 of 16 scenarios passed at 60, 300, 900, and 3600 s
Entity/change audit	Unstable identities, relations, masks, action rows, or configuration versions	Strict contract pass
Canonical annual-suite audit	Misaligned 2023 clocks or civil-time routines, incorrect OMIE expansion or publication boundary, duplicate profiles or identities, overlapping services, infeasible EV sessions, invalid Portuguese connection classes, incomplete checksums, unmatched scenario twins, or topology events without physical effect	1,817 structural and cross-family checks pass; diversity and deterministic-generation audits pass without findings; zero individually infeasible EV sessions; nine week-long rollouts, 31,565 causal-price checks, and 120 physically verified topology-event applications pass; Core-15 is the declared physical subset of Core-30
BAU/KPI sanity audit	Non-finite values, duplicate keys, or inconsistent reference deltas	3317 rows per export; 1401 BAU-derived rows; no duplicate keys
Thermal/HVAC compatibility	Regressions in inherited thermal and heat-pump behaviour	12 passing tests

These counts characterise the evaluated evidence bundle and should be regenerated if the implementation or dataset contract changes. They complement independently calculable golden cases, which remain necessary to detect systematic balance or service errors that could be concealed by plausible annual trajectories.

A.4.2 Selected capability demonstrations

The demonstrations below retain the observed values beside the precise interpretation that each case can support.

Table A.10: Selected application evidence and the bounded claim supported by each example.

Example	**Observed evidence**	**Supported interpretation**
Historical 17-member REC/EV reference	BAU: 28.7 kEUR, 166.8 MWh import, 30.0 tCO_2; community RBC: 22.0 kEUR, 133.3 MWh, 22.8 tCO_2, approximately 91% EV within-tolerance service and 100% flexible-load service	Service, settlement, and constraint indicators prevent a cost-only reading; transparent RBCs are references, not proof of optimality.
Temporal resolution	The same annual trace yields peaks of 96.8, 188.2, and 281.5 kW at 1 h, 15 min, and 15 s	A declared timestep changes the observable capacity problem even when annual energy is preserved.
Historical 17-member settlement	17.0 MWh shared; cost changes from 24.8 to 22.0 kEUR; all 17 members save under the configured rule	Participant and accounting outcomes are observable; equity and legal applicability remain separate questions.
Demand response	Shortfall changes from 101.1 to 51.6 kWh; compliance from 0.43 to 0.70; cost after settlement from 863.6 to 790.6 EUR	Requests, delivery, shortfall, and settlement can be audited without claiming profitability or controller optimality.
Dynamic topology	Four member/asset events propagate to active counts, identified rows, relations, actions, and logs	Runtime change is represented consistently; controller adaptation must be evaluated separately.
Controlled failures	Missing observations, forecast bias, command dropout, and asset unavailability appear in their respective channels and exports	Disturbances are repeatable and measurable; robustness outside the declared classes is not inferred.
Multiple communities	Four independent RECs retain local and combined results; 2.45 MWh of simultaneous surplus/demand is identified	Portfolio evidence is preserved without claiming physical coupling or a market-clearing mechanism.

A.4.3 Performance and scalability smoke benchmarks

The smoke benchmarks compare matched configurations and isolate the computational effect of interface detail, community multiplicity, storage format, and action-feedback retention.

Table A.11: Computational-cost evidence from matched simulator configurations.

Comparison	Observed change	Interpretation
Flat versus basic entity input	5.30 to 5.51 ms/step, approximately +4.1%	Stable identities, relations, masks, and action feedback add a small measured overhead in the matched case.
Flat versus detailed entity input	4.88 to 10.18 ms/step; 165 flat values versus 2538 table values and 302 feature names	The richest interface more than doubles payload processing but remains quantitatively characterised.
One, two, and four communities	5.02, 7.08, and 11.41 ms/step; four communities produce 5227 KPI rows, including 107 combined rows	Four communities take 2.27 times the single-community step time while retaining local and portfolio evidence.
15-second CSV versus Parquet	Approximately 2.4–2.6 GB becomes 12–13 MB; initialisation falls by 34–38%	Columnar storage addresses data loading and storage; it does not materially change post-load physics.
Entity-feedback memory regression	Approximately +2071 MiB reduced to approximately +5 MiB over 700 targeted steps	The audit exposed and bounded a major implementation leak in the exercised configuration.

The basic entity contract remains close to flat execution in the matched case, while the detailed bundle deliberately exchanges a much richer payload for approximately twice the step time. The four-community configuration corresponds to approximately 88 steps/s. These observations make the computational cost visible for the reported studies; they do not imply linear scaling to arbitrary population sizes, detailed feeder models, or distributed deployment targets.

A.5 Simulator KPI Catalogue

The core controller scorecard in Chapter 4 is a deliberately small interpretation layer over a broader CityLearn v3 KPI surface. The complete machine-readable output retains the entity or aggregation level, controller condition, baseline condition, unit, and raw value. It is produced through the primary `evaluate_v2()` interface. Table A.12 records the implemented families and the principal quantities used to interpret them. Exact code-level names are versioned with the simulator documentation and exported result, which remain authoritative when the catalogue evolves.

Table A.12: Expanded CityLearn v3 KPI catalogue.

Family	Principal quantities	Typical scope and unit
Energy and grid exchange	Net electricity consumption; total grid import and export; net grid energy; average daily and maximum import/export peaks; ramping; load factor; zero-net-energy indicators	Building, community, or portfolio; kWh, kW, or ratio
Cost and settlement	Grid import cost; export revenue; local import/export payment; cost after local and demand-response settlement; grid-only counterfactual; raw, BAU-difference, and valid BAU-ratio rows	Member, community, or portfolio; EUR or ratio

Family	Principal quantities	Typical scope and unit
Participant benefit distribution	Members with lower cost than the matched reference; benefit-coverage fraction; minimum, maximum, median, and dispersion of member savings	Member distribution; count, fraction, or EUR
Emissions	Imported-energy emissions; average carbon intensity; raw and BAU-relative emissions	Building, community, or portfolio; $kgCO_2$ or $kgCO_2$/kWh
PV and local sharing	PV generation, self-consumption, self-sufficiency, local import/export, matched local exchange, community surplus and deficit, sharing ratios	Member or community; kWh or ratio
Stationary storage	Charged/discharged energy, losses, state of charge, cycling and degradation-related outputs where enabled, requested/applied power difference	Asset, building, or community; kWh, kW, ratio, or state
EV and charger service	Connected steps, arrivals, topology-valid departures, target/minimum/tolerance success, departure deficit, charged and V2G energy, required average power, slack, charger clipping, and EV energy-accounting shortfall	EV, charger, member, or community; count, ratio, kWh, kW, or time
Deferrable service	Requested, started, completed, missed, topology-censored, and interrupted cycles; served/unserved energy; mean and maximum start delay; deadline violations	Asset, member, or community; count, ratio, kWh, or time
Electrical feasibility	Requested total/per-phase pressure, rejected or reduced power, residual applied violation, violation duration, state-bound violations, and feasible headroom	Asset, member, phase, or community; kW, kWh, count, or time
Demand response	Request and active-step counts; requested, delivered, credited, and shortfall energy; compliance; invalid baselines; payment, penalty, and net revenue; up/down delivery	Request, community, or portfolio; count, kWh, ratio, or EUR
Topology and health	Active member/asset counts and periods; additions/removals; altered observation and forecast values; missing data; changed/lost actions; equipment-unavailable asset–step counts	Entity, member, community, or portfolio; count, time, or mixed
Controller interface	Entity-table and row counts; relations; feature and action identities; masks; feasible action ranges; requested, clipped, applied, and rejected actions with reasons	Controller, entity, or action port; count or native action unit
Computational effort	Training wall time, inference latency, simulated steps per second, initialisation, reset and export time, peak memory, and the associated hardware, software, population, interface, and training-budget context	Controller or experiment; ms, s, h, steps/s, or MiB
Multiple communities	Per-community KPI tables; summed energy, cost, emissions, payments, requests, and events; declared weighted ratios; combined portfolio results	Community or portfolio; inherited unit
Inherited thermal, HVAC, and comfort	Heat-pump electricity, unmet thermal demand, comfort violations, occupant overrides, thermal resilience, and HVAC clipping where the selected scenario enables those inherited models	Building or district; kWh, count, time, or ratio

Three interpretation rules apply across this catalogue. First, reward and KPI are distinct: reward is controller-facing and may combine selected objectives, whereas KPIs remain evaluative outputs. Second, additive quantities may be summed across members or communities, but ratios require a declared denominator or reporting weight. Third, requested-pressure and applied-residual quantities must not be conflated: the former describes controller intent before feasibility projection, while the latter establishes whether the executed trajectory violated the configured physical boundary. The archived time series and event logs remain necessary whenever an aggregate KPI cannot explain when or why an outcome occurred.

A.6 Controller, TI-RL, and TI-MARL Configurations

Complete controller definitions cover the passive case, the five RBC levels, MILP variants, PPO, SAC, TD3, MADDPG, MATD3, hierarchical controllers, learning enhancements, and the TI-RL/TI-MARL variants. The configuration record includes schema, compiler, entity/interface graph, local policy, grouped decoder, local feasibility construction, critic, typed constraint graph, shared hard-monitor, runtime-assurance, and backbone versions and distinguishes default settings, tuned settings, oracle information, operational information, and target-specific adaptations.

A.7 Hyperparameters, Seeds, and Statistical Procedures

The supplementary record contains hyperparameter tables, search ranges, selected values, random seeds, training budgets, checkpoint-selection rules, evaluation windows, statistical tests, confidence intervals, and the treatment of failed runs.

A.8 Supplementary and Negative Results

Supplementary evidence includes additional learning curves, participant-level results, scale curves, disturbance-severity curves, ablations, failure cases, methods that did not improve the selected objective, and configurations rejected before deployment. Negative results are retained where they explain a design choice in Chapters 5 or 6.

A.9 Lifecycle and Deployment Records

Lifecycle records include experiment-manifest examples, policy-set manifests, compatibility rules, validation reports, package specifications, target profiles, deployment sequences, mixed-version tests, rollback records, monitoring schemas, soak-test summaries, and failure-recovery traces.

A.10 Software, Data, and Archival Record

The archival record lists repositories, releases, licences, archived commits or DOIs, container images, external dependencies, datasets and access conditions, partner-restricted components, hardware specifications, and commands used to reproduce the principal figures and tables. Restricted artefacts are accompanied by a stated limitation and the closest reproducible substitute.